thermofluids-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `thermofluids_ai999x.about()` for the same answer.

Every capability thermofluids-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import thermofluids_ai999x

result = thermofluids_ai999x.solve('colebrook_friction_factor', Re=100000, eD=0.0002)
result.show()
# f = 0.0190054 1
```

By topic

topic	capabilities
Aerodynamics	Lift coefficient of a thin symmetric airfoil
Boundary layers	Displacement thickness of a laminar flat-plate boundary layer, Momentum thickness of a laminar flat-plate boundary layer, Local skin-friction coefficient of a laminar flat plate, Average skin-friction coefficient of a laminar flat plate
Combustion	Adiabatic flame temperature with constant product specific heat
Combustion / mixture strength	Equivalence ratio of a fuel-air mixture
Combustion / stoichiometry	Stoichiometric air-fuel ratio of a gaseous fuel
Compressible flow	Mach number, Speed of sound in an ideal gas, Stagnation temperature from the Mach number, Isentropic stagnation pressure of air from the Mach number, Mach number behind a normal shock in air, Static pressure behind a normal shock in air
Conduction	Fourier's law: heat conduction through a plane wall, Radial heat conduction through a cylindrical wall, Radial heat conduction through a spherical shell, Overall heat transfer coefficient of a plane wall with convection on both sides, Heat transfer rate from the overall heat transfer coefficient, Critical radius of insulation on a cylinder, Maximum temperature of a plane wall with uniform heat generation, Thermal diffusivity
Convection	Newton's law of cooling (convection heat transfer), Heat transfer coefficient from the Nusselt number, Prandtl number, Dittus-Boelter correlation, fluid being heated, Dittus-Boelter correlation, fluid being cooled, Average Nusselt number for laminar flow over a flat plate, Local Nusselt number for laminar flow over a flat plate
Convection / external flow	Churchill-Bernstein correlation for a cylinder in cross flow

topic	capabilities
Convection / internal flow	Gnielinski correlation for turbulent pipe flow
Convection / natural convection	Churchill-Chu correlation for natural convection on a vertical plate
External flow	Drag force from the drag coefficient, Lift force from the lift coefficient, Stokes drag on a small sphere, Terminal settling velocity of a small sphere (Stokes)
Fins	Heat transfer rate of a very long fin
Fins / extended surfaces	Efficiency of a pin fin with an insulated tip
Fluid dynamics	Continuity: speed after a change of pipe diameter, Bernoulli's equation along a horizontal streamline, Speed of efflux from a hole (Torricelli's law), Dynamic pressure of a moving fluid
Fluid flow / Bernoulli's equation	Bernoulli's equation with a change of height
Fluid statics	Gauge pressure at a depth in a liquid, Absolute pressure at a depth in a liquid, Buoyant force (Archimedes' principle), Hydrostatic force on a submerged plane surface
Fluid statics / Pascal's principle	Hydraulic press (Pascal's principle)
Gas processes	Temperature after an isentropic pressure change of air, Temperature after an isentropic volume change of air, Actual outlet temperature of an adiabatic air compressor, Ideal gas law, Work done by an ideal gas in a reversible isothermal process, Specific entropy change of an ideal gas, Entropy change of an incompressible substance
Heat exchangers	Number of transfer units of a heat exchanger, Effectiveness of a parallel-flow heat exchanger, Effectiveness of a heat exchanger with one stream changing phase, Heat transfer rate from the effectiveness, Heat added to a flowing stream (steady-flow energy balance)
Heat exchangers / effectiveness-NTU	Heat exchanger rating by the effectiveness-NTU method
Heat exchangers / LMTD method	Log-mean temperature difference of a heat exchanger
Natural convection	Grashof number, Rayleigh number
Open channel	Froude number of an open-channel flow
Pipe flow	Reynolds number from density, velocity, diameter and viscosity, Reynolds number from velocity, diameter and kinematic viscosity, Kinematic viscosity from dynamic viscosity and density, Darcy friction factor in laminar pipe flow, Blasius friction factor for turbulent flow in a smooth pipe, Swamee-Jain explicit friction factor for turbulent pipe flow, Haaland explicit friction factor for turbulent pipe flow, Darcy-Weisbach head loss in a pipe, Darcy-Weisbach pressure drop in a pipe, Hagen-Poiseuille flow rate in laminar pipe flow, Minor head loss in a fitting
Pipe flow / friction factor	Darcy friction factor from the Colebrook equation
Proofs and derivations	Derive bernoulli horizontal, Derive blasius local skin friction, Derive blasius momentum thickness, Derive brayton efficiency air, Derive buoyant force, Derive continuity speed, Derive darcy weisbach head loss, Derive darcy weisbach pressure drop, Derive diesel efficiency air, Derive dynamic pressure, Derive effectiveness phase change, Derive humidity ratio vapour pressure, Derive hydraulic power, Derive hydrostatic force plane, Derive minor loss head, Derive nusselt heat transfer coefficient, Derive otto efficiency air, Derive overall heat rate, Derive overall u plane wall, Derive plane wall conduction, Derive pump shaft power, Derive radiation net exchange, Derive radiation parallel plates, Derive reynolds number, Derive reynolds number kinematic, Derive speed of sound ideal gas, Derive sphere conduction

topic	capabilities
Psychrometrics	Humidity ratio from the partial pressure of water vapour, Specific enthalpy of moist air, Specific volume of moist air per kilogram of dry air
Psychrometrics / dew point	Dew-point temperature from dry-bulb temperature and relative humidity
Psychrometrics / humidity ratio	Humidity ratio from temperature, relative humidity and pressure
Psychrometrics / saturation	Saturation pressure of water vapour
Psychrometrics / wet bulb	Thermodynamic wet-bulb temperature
Pumps	Hydraulic power delivered to a fluid, Shaft power required by a pump
Radiation	Emissive power of a blackbody (Stefan-Boltzmann law), Net radiation exchange of a small grey surface with large surroundings, Net radiation between two large parallel grey plates, Wien's displacement law, Linearised radiation heat transfer coefficient
Thermodynamic cycles	Carnot efficiency of a reversible heat engine, Coefficient of performance of a Carnot refrigerator, Coefficient of performance of a Carnot heat pump, Thermal efficiency of the air-standard Otto cycle, Thermal efficiency of the air-standard Diesel cycle, Thermal efficiency of the air-standard Brayton cycle
Thermodynamics / boundary work	Boundary work of a constant-pressure (isobaric) process
Thermodynamics / entropy	Entropy change for heat exchanged at a constant temperature
Thermodynamics / entropy and the second law	Entropy generated when heat flows between two reservoirs
Thermodynamics / first law	First law of thermodynamics: change of internal energy
Thermodynamics / properties of pure substances	Wet steam: quality and mixture properties from saturated values
Transient conduction	Biot number, Fourier number, Temperature of a lumped body cooling or heating in a fluid, Thermal time constant of a lumped body

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below

outcome	meaning	what to do
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Giving a unit of the wrong kind (a mass in m): INVALID_INPUT names the expected unit.
- Leaving out an input: NEEDS_INPUT names it; declared constants (g, k, h ...) are filled and stated.
- Giving every quantity of an equation: say which one to compute with target=.
- Asking several quantities in one question: ask one at a time (or call solve() once per quantity).
- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.
- A coefficient that violates a stated condition (a = 0 in a quadratic): INVALID_INPUT says why.
- A text value where a number is expected: INVALID_INPUT; fractions as strings ("1/2") are accepted.
- A problem outside the procedure (a cubic for the quadratic solver): OUT_OF_SCOPE, never an approximation.

Catalogue

id	operation	family	inputs
hydrostatic_gauge_pressure	Gauge pressure at a depth in a liquid	fluid_statics	P, rho, g, h
absolute_pressure_depth	Absolute pressure at a depth in a liquid	fluid_statics	P, P0, rho, g, h
buoyant_force	Buoyant force (Archimedes' principle)	fluid_statics	F, rho, g, V
continuity_speed	Continuity: speed after a change of pipe diameter	fluid_dynamics	v2, v1, D1, D2
bernoulli_horizontal	Bernoulli's equation along a horizontal streamline	fluid_dynamics	P2, P1, rho, v1, v2
torricelli_efflux	Speed of efflux from a hole (Torricelli's law)	fluid_dynamics	v, g, h
dynamic_pressure	Dynamic pressure of a moving fluid	fluid_dynamics	q, rho, v

id	operation	family	inputs
reynolds_number	Reynolds number from density, velocity, diameter and viscosity	pipe_flow	Re, rho, v, D, mu
reynolds_number_kinematic	Reynolds number from velocity, diameter and kinematic viscosity	pipe_flow	Re, v, D, nu
kinematic_viscosity	Kinematic viscosity from dynamic viscosity and density	pipe_flow	nu, mu, rho
laminar_friction_factor	Darcy friction factor in laminar pipe flow	pipe_flow	f, Re
blasius_friction_factor	Blasius friction factor for turbulent flow in a smooth pipe	pipe_flow	f, Re
swamee_jain_friction_factor	Swamee-Jain explicit friction factor for turbulent pipe flow	pipe_flow	f, eps, D, Re
haaland_friction_factor	Haaland explicit friction factor for turbulent pipe flow	pipe_flow	f, eps, D, Re
darcy_weisbach_head_loss	Darcy-Weisbach head loss in a pipe	pipe_flow	hL, f, L, v, g, D
darcy_weisbach_pressure_drop	Darcy-Weisbach pressure drop in a pipe	pipe_flow	dP, f, L, rho, v, D
hagen_poiseuille	Hagen-Poiseuille flow rate in laminar pipe flow	pipe_flow	Q, D, dP, mu, L
minor_loss_head	Minor head loss in a fitting	pipe_flow	hL, K, v, g
hydraulic_power	Hydraulic power delivered to a fluid	pumps	P, rho, g, Q, H
pump_shaft_power	Shaft power required by a pump	pumps	P, rho, g, Q, H, eta
hydrostatic_force_plane	Hydrostatic force on a submerged plane surface	fluid_statics	F, rho, g, hc, A
drag_force	Drag force from the drag coefficient	external_flow	F, Cd, rho, v, A
lift_force	Lift force from the lift coefficient	external_flow	F, CL, rho, v, A
stokes_drag	Stokes drag on a small sphere	external_flow	F, mu, D, v
stokes_terminal_velocity	Terminal settling velocity of a small sphere (Stokes)	external_flow	v, g, D, rho_p, rho, mu
thin_airfoil_lift_coefficient	Lift coefficient of a thin symmetric airfoil	aerodynamics	CL, alpha
mach_number	Mach number	compressible_flow	M, v, a
speed_of_sound_ideal_gas	Speed of sound in an ideal gas	compressible_flow	a, gamma, R, T
stagnation_temperature_mach	Stagnation temperature from the Mach number	compressible_flow	T0, T, gamma, M
stagnation_pressure_air	Isentropic stagnation pressure of air from the Mach number	compressible_flow	P0, P, M
normal_shock_mach_air	Mach number behind a normal shock in air	compressible_flow	M2, M1
normal_shock_pressure_air	Static pressure behind a normal shock in air	compressible_flow	P2, P1, M1
blasius_displacement_thickness	Displacement thickness of a laminar flat-plate boundary layer	boundary_layers	ds, x, Rex
blasius_momentum_thickness	Momentum thickness of a laminar flat-plate boundary layer	boundary_layers	th, x, Rex

id	operation	family	inputs
blasius_local_skin_friction	Local skin-friction coefficient of a laminar flat plate	boundary_layers	cf, Rex
blasius_average_skin_friction	Average skin-friction coefficient of a laminar flat plate	boundary_layers	cfm, ReL
froude_number	Froude number of an open-channel flow	open_channel	Fr, v, g, h
plane_wall_conduction	Fourier's law: heat conduction through a plane wall	conduction	q, k, A, T1, T2, L
cylinder_conduction	Radial heat conduction through a cylindrical wall	conduction	q, k, L, T1, T2, r1, r2
sphere_conduction	Radial heat conduction through a spherical shell	conduction	q, k, T1, T2, r1, r2
overall_u_plane_wall	Overall heat transfer coefficient of a plane wall with convection on both sides	conduction	U, h1, L, k, h2
overall_heat_rate	Heat transfer rate from the overall heat transfer coefficient	conduction	q, U, A, Ti, To
critical_insulation_radius	Critical radius of insulation on a cylinder	conduction	r, k, h
wall_generation_max_temperature	Maximum temperature of a plane wall with uniform heat generation	conduction	Tmax, Ts, q, L, k
thermal_diffusivity	Thermal diffusivity	conduction	alpha, k, rho, cp
biot_number	Biot number	transient_conduction	Bi, h, Lc, k
fourier_number	Fourier number	transient_conduction	Fo, alpha, t, Lc
newton_cooling	Newton's law of cooling (convection heat transfer)	convection	q, h, A, Ts, Tinf
nusselt_heat_transfer_coefficient	Heat transfer coefficient from the Nusselt number	convection	h, Nu, k, Lc
prandtl_number	Prandtl number	convection	Pr, cp, mu, k
dittus_boelter_heating	Dittus-Boelter correlation, fluid being heated	convection	Nu, Re, Pr
dittus_boelter_cooling	Dittus-Boelter correlation, fluid being cooled	convection	Nu, Re, Pr
flat_plate_laminar_average_nusselt	Average Nusselt number for laminar flow over a flat plate	convection	Nu, ReL, Pr
flat_plate_laminar_local_nusselt	Local Nusselt number for laminar flow over a flat plate	convection	Nux, Rex, Pr
grashof_number	Grashof number	natural_convection	Gr, g, beta, Ts, Tinf, L, nu
rayleigh_number	Rayleigh number	natural_convection	Ra, Gr, Pr
blackbody_emissive_power	Emissive power of a blackbody (Stefan-Boltzmann law)	radiation	E, sigma, T
radiation_net_exchange	Net radiation exchange of a small grey surface with large surroundings	radiation	q, eps, sigma, A, Ts, Tsur
radiation_parallel_plates	Net radiation between two large parallel grey plates	radiation	qf, sigma, T1, T2, e1, e2
wien_displacement	Wien's displacement law	radiation	lam, T, b

id	operation	family	inputs
radiation_heat_transfer_coefficient	Linearised radiation heat transfer coefficient	radiation	hr, eps, sigma, Ts, Tsur
infinite_fin_heat_rate	Heat transfer rate of a very long fin	fins	q, h, P, k, Ac, Tb, Tinf
lumped_capacitance_temperature	Temperature of a lumped body cooling or heating in a fluid	transient_conduction	T, Tinf, Ti, h, A, t, rho, V, c
lumped_time_constant	Thermal time constant of a lumped body	transient_conduction	tau, rho, V, c, h, A
ntu_definition	Number of transfer units of a heat exchanger	heat_exchangers	N, U, A, Cmin
effectiveness_parallel_flow	Effectiveness of a parallel-flow heat exchanger	heat_exchangers	eps, N, Cr
effectiveness_phase_change	Effectiveness of a heat exchanger with one stream changing phase	heat_exchangers	eps, N
heat_rate_effectiveness	Heat transfer rate from the effectiveness	heat_exchangers	Q, eps, Cmin, Thi, Tci
sensible_heat_rate	Heat added to a flowing stream (steady-flow energy balance)	heat_exchangers	Q, mdot, cp, Tout, Tin
carnot_efficiency	Carnot efficiency of a reversible heat engine	thermodynamic_cycles	eta, Tc, Th
carnot_cop_refrigerator	Coefficient of performance of a Carnot refrigerator	thermodynamic_cycles	cop, Tc, Th
carnot_cop_heat_pump	Coefficient of performance of a Carnot heat pump	thermodynamic_cycles	cop, Tc, Th
otto_efficiency_air	Thermal efficiency of the air-standard Otto cycle	thermodynamic_cycles	eta, r
diesel_efficiency_air	Thermal efficiency of the air-standard Diesel cycle	thermodynamic_cycles	eta, r, rc
brayton_efficiency_air	Thermal efficiency of the air-standard Brayton cycle	thermodynamic_cycles	eta, rp
isentropic_temperature_pressure_air	Temperature after an isentropic pressure change of air	gas_processes	T2, T1, P2, P1
isentropic_temperature_volume_air	Temperature after an isentropic volume change of air	gas_processes	T2, T1, V1, V2
compressor_outlet_temperature_air	Actual outlet temperature of an adiabatic air compressor	gas_processes	T2, T1, P2, P1, eta
ideal_gas_law	Ideal gas law	gas_processes	P, V, n, R, T
isothermal_work_ideal_gas	Work done by an ideal gas in a reversible isothermal process	gas_processes	W, n, R, T, V2, V1
entropy_change_ideal_gas	Specific entropy change of an ideal gas	gas_processes	ds, cp, R, T1, T2, P1, P2
entropy_change_incompressible	Entropy change of an incompressible substance	gas_processes	dS, m, c, T2, T1
humidity_ratio_vapour_pressure	Humidity ratio from the partial pressure of water vapour	psychrometrics	W, pv, P

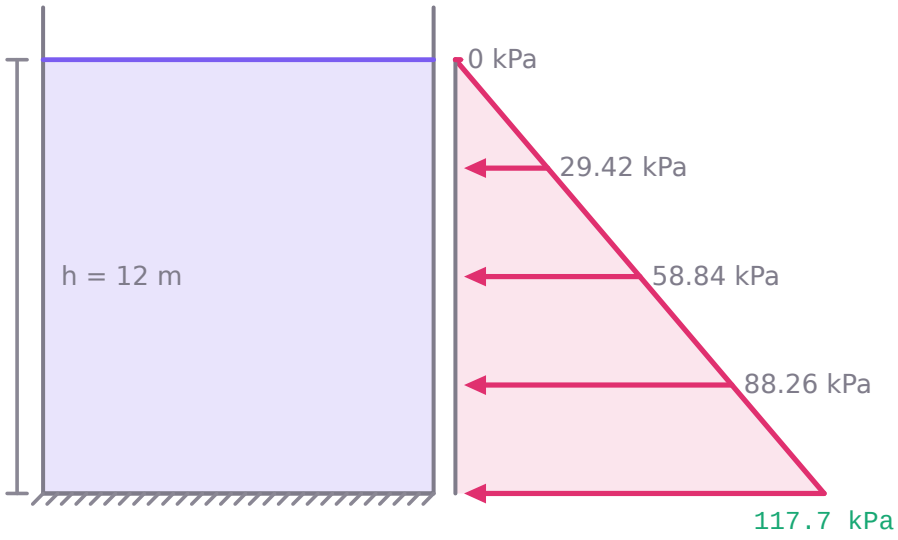
id	operation	family	inputs
moist_air_enthalpy	Specific enthalpy of moist air	psychrometrics	h, T, W, cpa, cpv, hg, T0
moist_air_specific_volume	Specific volume of moist air per kilogram of dry air	psychrometrics	v, T, W, P, Ra
adiabatic_flame_temperature_simple	Adiabatic flame temperature with constant product specific heat	combustion	Tad, T0, LHV, AFR, cp

Gauge pressure at a depth in a liquid, `hydrostatic_gauge_pressure`

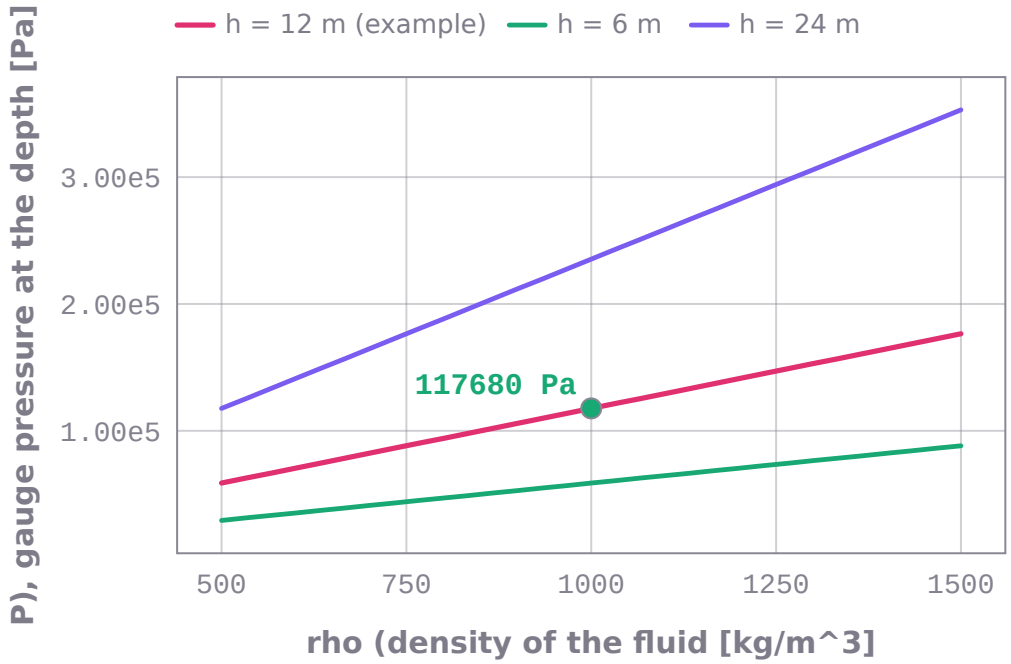
$$P = \rho g h$$

input	meaning	kind / role	unit	default
`P`	gauge pressure at the depth	pressure / gauge	`Pa`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	depth below the free surface	length / depth	`m`	

- **Conditions:** hydrostatic
- **Applies when:** a point at a depth below the free surface of a liquid at rest
- **Fails when:** compressible fluids over large heights; accelerating containers
- **Assumes:** incompressible liquid at rest; uniform gravity
- **Evidence:** theorem



A tank of liquid (ρ = 1000 kg/m^3) to depth h = 12 m with the gauge pressure on its wall: the package's pressure at h/4, h/2, 3h/4 and h (at the surface the gauge pressure is zero), drawn as arrows to a common scale. Marked: 117.7 kPa at the bottom (the worked example), 0 kPa at 0 m, 29.42 kPa at 3 m, 58.84 kPa at 6 m, 88.26 kPa at 9 m.



P (gauge pressure at the depth) against rho (density of the fluid) from 500 kg/m^3 to 1500 kg/m^3. One curve for each value of h (depth below the free surface): 12 m, 6 m, 24 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, rho = 1000 kg/m^3, P = 117680 Pa.

```
thermofluids_ai999x.solve('hydrostatic_gauge_pressure', rho='1000 kg/m^3', h='12 m',
target='P')
# -> 117680 Pa   (SOLVED)
```

$$P = \rho g h$$
$$P = g h \rho = 117680\ \mathrm{Pa}$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	hydrostatic_gauge_pressure needs rho (density of the fluid, kg/m^3). Example: solve('hydrostatic_gauge_pressure', rho='... kg/m^3', g='... m
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

- "What is the gauge pressure at a depth of 12 m in a lake of water with density 1000 kg/m^3?" → 117680 Pa
- "Find the gauge pressure 3.5 m below the surface of an oil of density 850 kg/m^3." → 29174.8 Pa

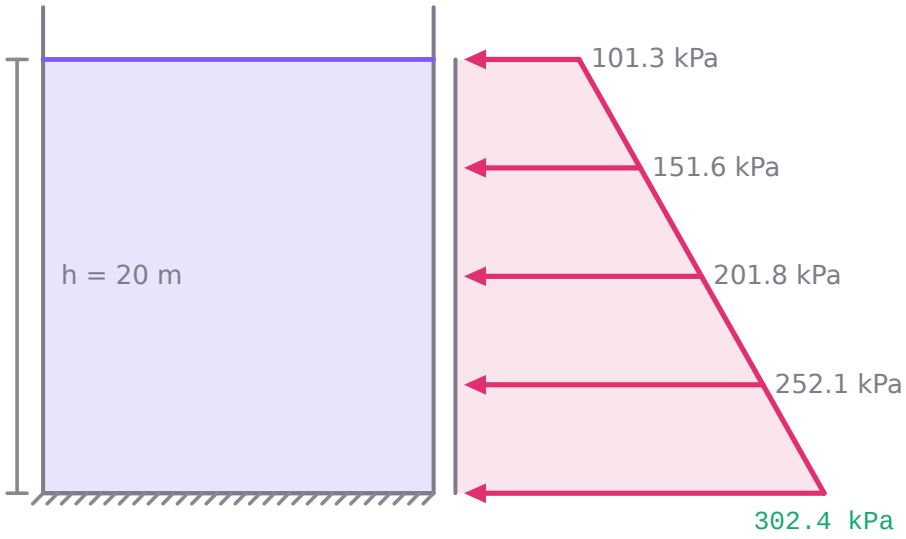
Absolute pressure at a depth in a liquid, `absolute_pressure_depth`

$$P = P_0 + \rho g h$$

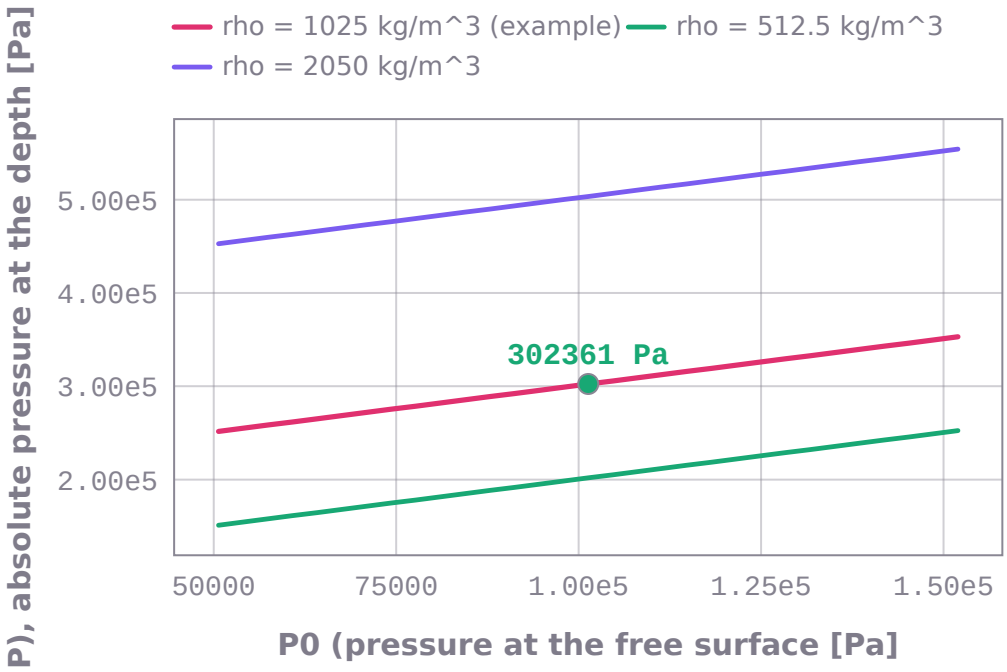
input	meaning	kind / role	unit	default
`P`	absolute pressure at the depth	pressure / absolute	`Pa`	
`P0`	pressure at the free surface	pressure / atmospheric	`Pa`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	

input	meaning	kind / role	unit	default
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	depth below the free surface	length / depth	`m`	

- **Conditions:** hydrostatic
- **Applies when:** a point at a depth below a free surface with a known surface pressure
- **Fails when:** compressible fluids over large heights; accelerating containers
- **Assumes:** incompressible liquid at rest; uniform gravity
- **Evidence:** theorem



A tank of liquid (ρ = 1025 kg/m^3, surface pressure 101325 Pa) to depth h = 20 m with the absolute pressure on its wall: the package's pressure at h/4, h/2, 3h/4 and h (at the surface the stated surface pressure), drawn as arrows to a common scale. Marked: 302.4 kPa at the bottom (the worked example), 101.3 kPa at 0 m, 151.6 kPa at 5 m, 201.8 kPa at 10 m, 252.1 kPa at 15 m.



P (absolute pressure at the depth) against P0 (pressure at the free surface) from 50662.5 Pa to 151988 Pa. One curve for each value of rho (density of the fluid): 1025 kg/m^3, 512.5 kg/m^3, 2050 kg/m^3. Other inputs as in the worked example: h = 20 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, P0 = 101325 Pa, P = 302361 Pa.

```
thermofluids_ai999x.solve('absolute_pressure_depth', P0='101325 Pa', rho='1025 kg/m^3',
h='20 m', target='P')
# -> 302361 Pa   (SOLVED)
```

$$P = P_0 + \rho g h$$
$$P = P_0 + g h \rho = 302361 \text{ Pa}$$

invalid call	outcome	message
without `P0`	NEEDS_INPUT	absolute_pressure_depth needs P0 (pressure at the free surface, Pa). Example: solve('absolute_pressure_depth', P0='... Pa', rho='... kg/m^3'
`P0` in a wrong unit	INVALID_INPUT	P0 is a pressure: its unit must be like Pa, not 'kg'.

Questions in words that work:

- "A diver is at a depth of 20 m in seawater of density 1025 kg/m^3. The atmospheric pressure is 101325 Pa. What is the absolute pressure on the diver?" → 302361 Pa
- "Find the absolute pressure 4 m below the surface of water with density 1000 kg/m^3 if the pressure at the surface is 100 kPa." → 139227 Pa

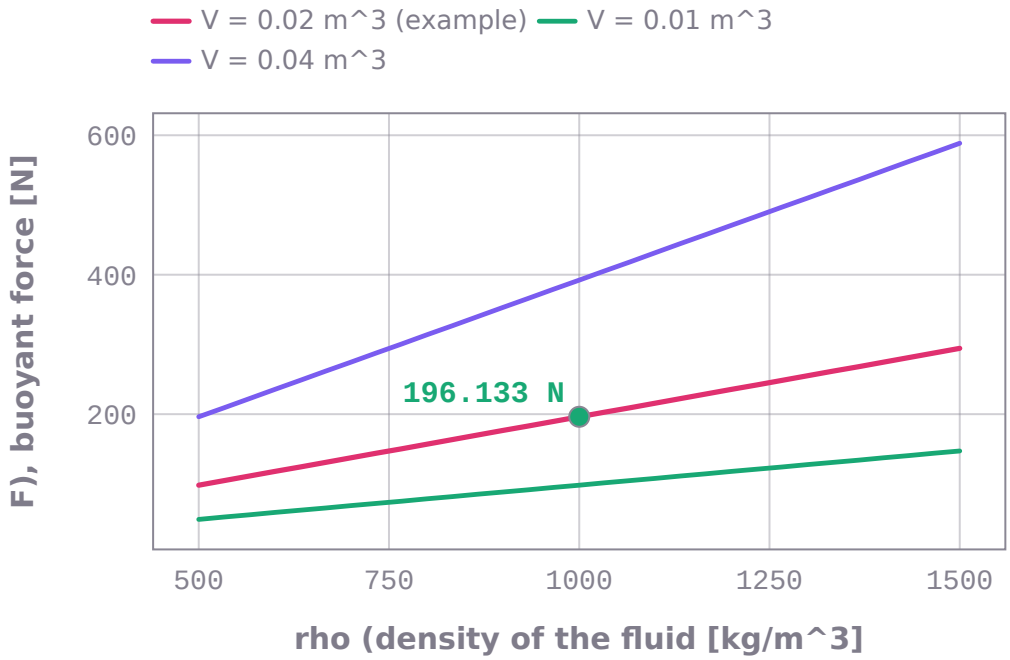
Buoyant force (Archimedes' principle), `buoyant_force`

$$F = \rho g V$$

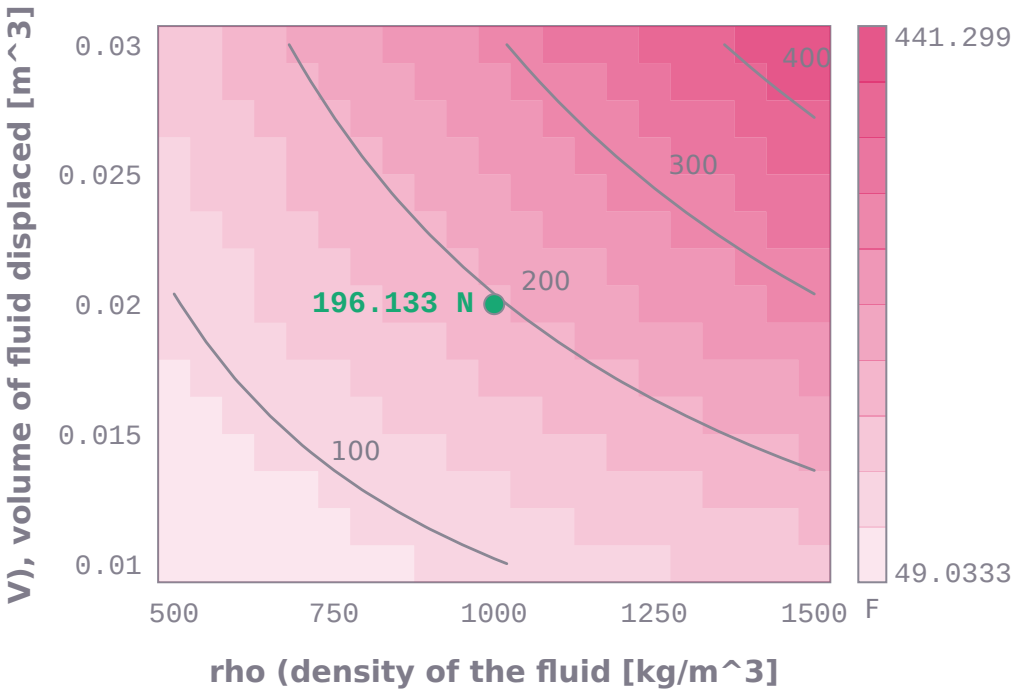
input	meaning	kind / role	unit	default
`F`	buoyant force	force / buoyant	`N`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	

input	meaning	kind / role	unit	default
g		acceleration / gravity	m/s^2	9.80665 m/s^2
V	volume of fluid displaced	volume / submerged	m^3	

- **Conditions:** buoyancy
- **Applies when:** a body wholly or partly immersed in a fluid at rest
- **Fails when:** accelerating fluid; stratified fluid
- **Assumes:** fluid at rest; uniform gravity; uniform fluid density
- **Evidence:** theorem



F (buoyant force) against rho (density of the fluid) from 500 kg/m^3 to 1500 kg/m^3. One curve for each value of V (volume of fluid displaced): 0.02 m^3, 0.01 m^3, 0.04 m^3. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, rho = 1000 kg/m^3, F = 196.133 N.



F (buoyant force) over rho (density of the fluid) from 500 kg/m^3 to 1500 kg/m^3 and V (volume of fluid displaced) from 0.01 m^3 to 0.03 m^3: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 49.0333 N to dark 441.299 N); iso-lines at F = 100, 200, 300, 400 N. Marked: the worked example, rho = 1000 kg/m^3, V = 0.02 m^3, F = 196.133 N.

```
thermofluids_ai999x.solve('buoyant_force', rho='1000 kg/m^3', V='0.02 m^3', target='F')
# -> 196.133 N   (SOLVED)
```

$$F = \rho g V$$
$$F = V g \rho = 196.133\ \mathrm{N}$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	buoyant_force needs rho (density of the fluid, kg/m^3). Example: solve('buoyant_force', rho='... kg/m^3', g='... m/s^2', V='... m^3').
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

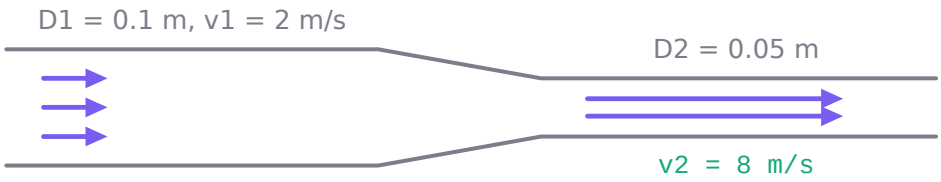
- "A solid block with a submerged volume of 0.02 m^3 is fully immersed in water of density 1000 kg/m^3. What is the buoyant force on it?" → 196.133 N
- "What buoyant force acts on a balloon that displaces a volume of 3 m^3 of air with density 1.2 kg/m^3?" → 35.3039 N

Continuity: speed after a change of pipe diameter, `continuity_speed`

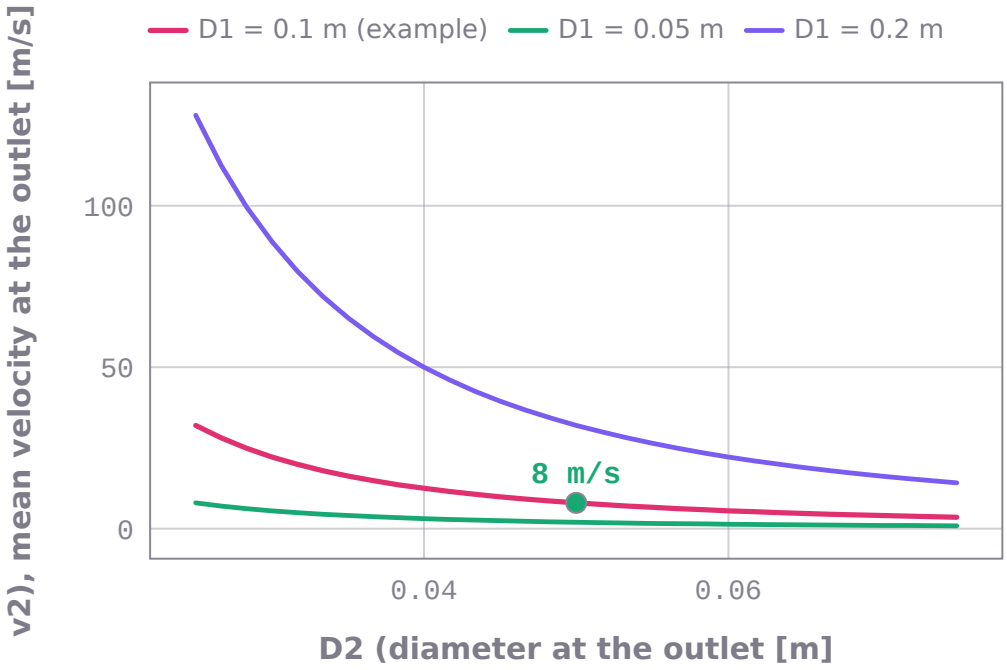
$$v_2 = v_1 \left(\frac{D_1}{D_2} \right)^2$$

input	meaning	kind / role	unit	default
`v2`	mean velocity at the outlet	speed / outlet	`m/s`	
`v1`	mean velocity at the inlet	speed / inlet	`m/s`	
`D1`	diameter at the inlet	length / inlet_diameter	`m`	
`D2`	diameter at the outlet	length / outlet_diameter	`m`	

- **Conditions:** continuity
- **Applies when:** liquid flowing through a reducer, nozzle or expansion
- **Fails when:** compressible gas flow with large density change
- **Assumes:** incompressible steady flow; circular cross sections; uniform velocity over each section
- **Evidence:** theorem



Flow through a contraction drawn to scale: diameter 0.1 m narrowing to 0.05 m, inlet speed 2 m/s (the worked example); arrow lengths are proportional to the mean speed. Marked: the package's outlet speed v2 = 8 m/s.



v2 (mean velocity at the outlet) against D2 (diameter at the outlet) from 0.025 m to 0.075 m. One curve for each value of D1 (diameter at the inlet): 0.1 m, 0.05 m, 0.2 m. Other inputs as in the worked example: v1 = 2 m/s. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, D2 = 0.05 m, v2 = 8 m/s.

```
thermofluids_ai999x.solve('continuity_speed', v1='2 m/s', D1='0.1 m', D2='0.05 m',
target='v2')
# -> 8 m/s   (SOLVED)
```

$$v_2 = v_1\left(\frac{D_1}{D_2}\right)^2$$
$$v_2 = \frac{D_1^2 v_1}{D_2^2} = 8\ \mathrm{m/s}$$

invalid call	outcome	message
without `v1`	NEEDS_INPUT	continuity_speed needs v1 (mean velocity at the inlet, m/s). Example: solve('continuity_speed', v1='... m/s', D1='... m', D2='... m').
`v1` in a wrong unit	INVALID_INPUT	v1 is a speed: its unit must be like m/s, not 'kg'.

Questions in words that work:

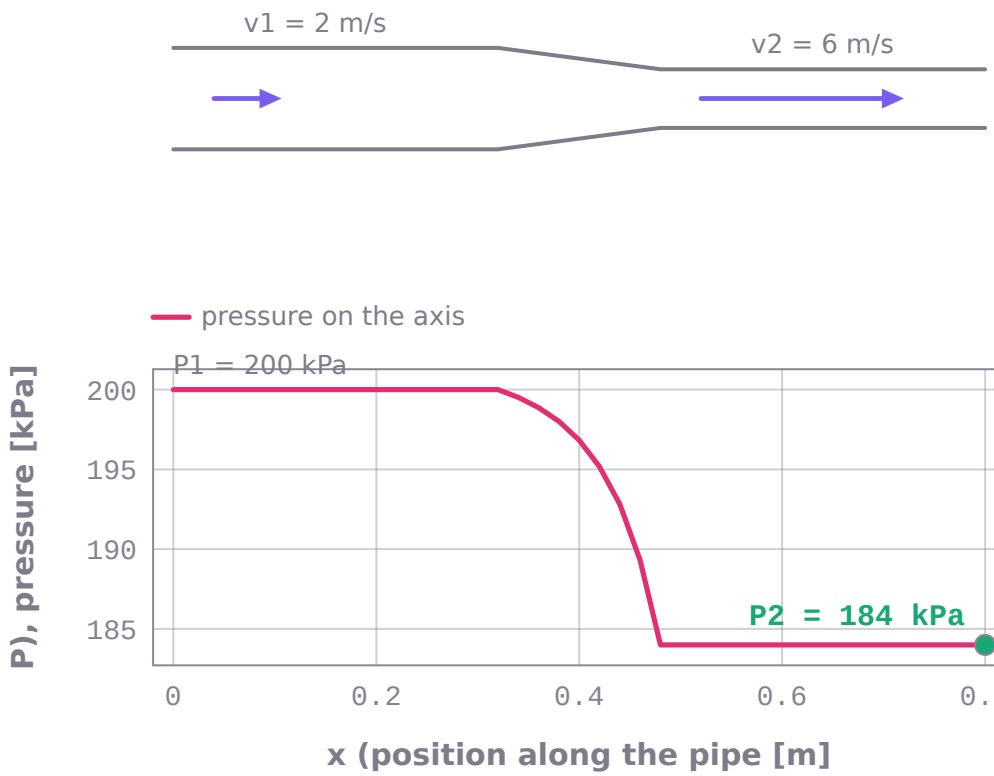
- "Water enters a reducer with an inlet diameter of 0.1 m at an inlet velocity of 2 m/s. The outlet diameter is 0.05 m. What is the outlet velocity?" → 8 m/s
- "A nozzle narrows from an inlet diameter of 40 mm to an outlet diameter of 20 mm. If the inlet speed is 1.5 m/s, find the exit speed." → 6 m/s

Bernoulli's equation along a horizontal streamline, `bernoulli_horizontal`

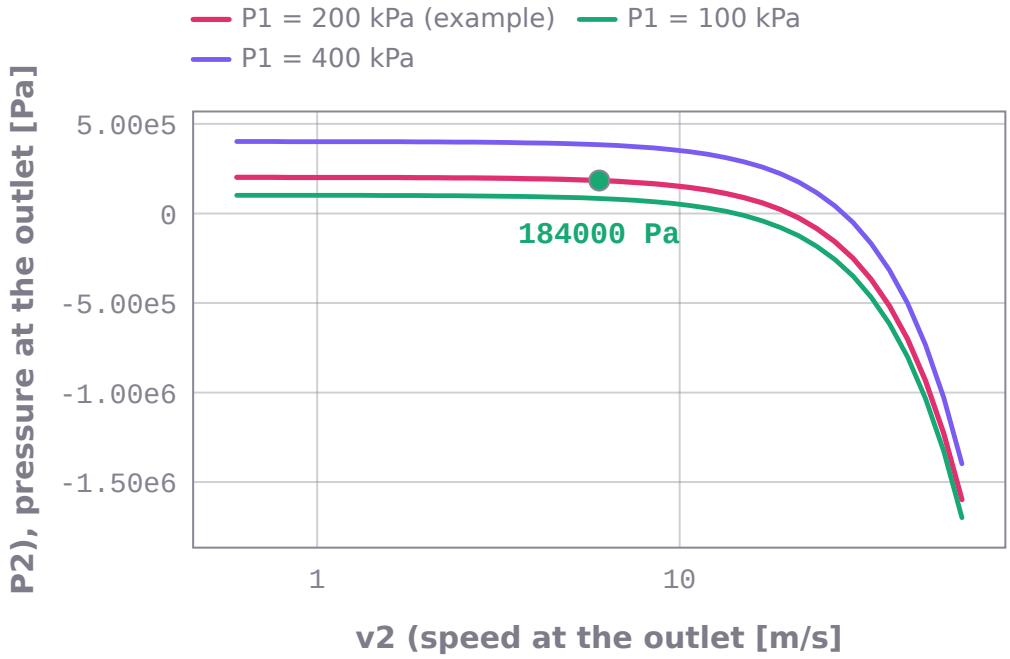
$$P_2 = P_1 + \frac{\rho\left(v_1^2 - v_2^2\right)}{2}$$

input	meaning	kind / role	unit	default
`P2`	pressure at the outlet	pressure / outlet	`Pa`	
`P1`	pressure at the inlet	pressure / inlet	`Pa`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`v1`	speed at the inlet	speed / inlet	`m/s`	
`v2`	speed at the outlet	speed / outlet	`m/s`	

- **Conditions:** bernoulli, horizontal
- **Applies when:** frictionless flow between two points at the same height
- **Fails when:** viscous losses; pumps or turbines between the points
- **Assumes:** steady, incompressible, inviscid flow along a streamline; no height change
- **Evidence:** theorem



Bernoulli's equation along a horizontal contraction (ρ = 1000 kg/m^3, P1 = 200 kPa, v1 = 2 m/s, v2 = 6 m/s; the worked example). The pipe is drawn for a stated inlet diameter 0.1 m, the outlet 0.057735 m from the package's continuity relation; the pressure at 41 positions is the package's Bernoulli pressure at the local speed (from continuity). Marked: P2 = 184 kPa.



P2 (pressure at the outlet) against v2 (speed at the outlet) from 0.6 m/s to 60 m/s on a logarithmic axis. One curve for each value of P1 (pressure at the inlet): 200 kPa, 100 kPa, 400 kPa. Other inputs as in the worked example: rho = 1000 kg/m^3, v1 = 2 m/s. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, v2 = 6 m/s, P2 = 184000 Pa.

```
thermofluids_ai999x.solve('bernoulli_horizontal', P1='200 kPa', rho='1000 kg/m^3', v1='2
m/s', v2='6 m/s', target='P2')
# -> 184000 Pa   (SOLVED)
```

$$P_2 = P_1 + \frac{\rho\left(v_1^2 - v_2^2\right)}{2}$$

$$P_2 = P_1 + \frac{\rho v_1^2}{2} - \frac{\rho v_2^2}{2} = 184000\ \text{Pa}$$

invalid call	outcome	message
without `P1`	NEEDS_INPUT	bernoulli_horizontal needs P1 (pressure at the inlet, Pa). Example: solve('bernoulli_horizontal', P1='... Pa', rho='... kg/m^3', v1='... m/s
`P1` in a wrong unit	INVALID_INPUT	P1 is a pressure: its unit must be like Pa, not 'kg'.

Questions in words that work:

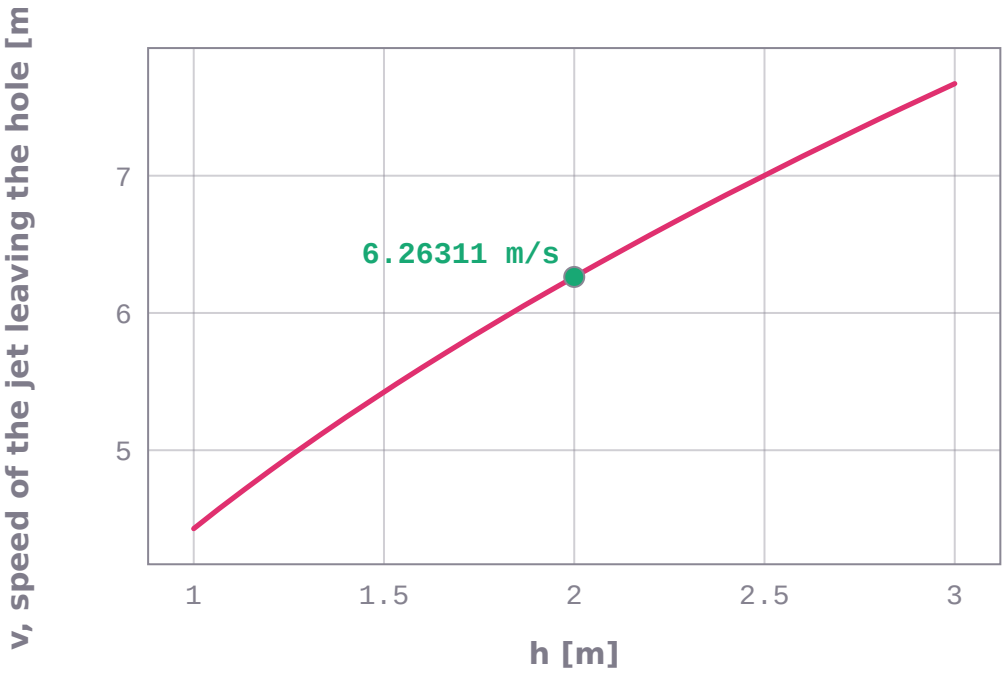
- "Water of density 1000 kg/m^3 flows without losses through a horizontal pipe. The inlet pressure is 200 kPa and the inlet velocity is 2 m/s. The outlet velocity is 6 m/s. What is the outlet pressure?" → 184000 Pa
- "Using Bernoulli's equation for a horizontal pipe, find the outlet pressure if the pressure at the inlet is 150000 Pa, the speed at the inlet is 1 m/s, the speed at the outlet is 4 m/s and the fluid density is 800 kg/m^3." → 144000 Pa

Speed of efflux from a hole (Torricelli's law), `torricelli_efflux`

$$v = \sqrt{2gh}$$

input	meaning	kind / role	unit	default
`v`	speed of the jet leaving the hole	speed / efflux	`m/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	depth of the hole below the free surface	length / depth	`m`	

- **Conditions:** torricelli
- **Applies when:** liquid draining from a small hole in a large open tank
- **Fails when:** viscous losses (use a discharge coefficient); pressurized tanks
- **Assumes:** large tank (surface speed negligible); no losses at the hole; open to the atmosphere
- **Evidence:** theorem



v (speed of the jet leaving the hole) against h (depth of the hole below the free surface) from 1 m to 3 m. 31 points computed by thermofluids-ai999x; marked: the worked example, h = 2 m, v = 6.26311 m/s.

```
thermofluids_ai999x.solve('torricelli_efflux', h='2 m', target='v')
# -> 6.26311 m/s   (SOLVED)
```

$$v = \sqrt{2gh}$$
$$v = \sqrt{2}\sqrt{g}\sqrt{h} = 6.26311\ \mathrm{m/s}$$

invalid call	outcome	message
without `h`	NEEDS_INPUT	torricelli_efflux needs h (depth of the hole below the free surface, m). Example: solve('torricelli_efflux', g='... m/s^2', h='... m').
`h` in a wrong unit	INVALID_INPUT	h is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "Water leaves a small hole in the side of a tank 2 m below the surface of the water. What is the speed of efflux?" → 6.26311 m/s
- "A large open tank has a small orifice at a depth of 5 m below the water level. Using Torricelli's law, find the efflux velocity." → 9.90285 m/s

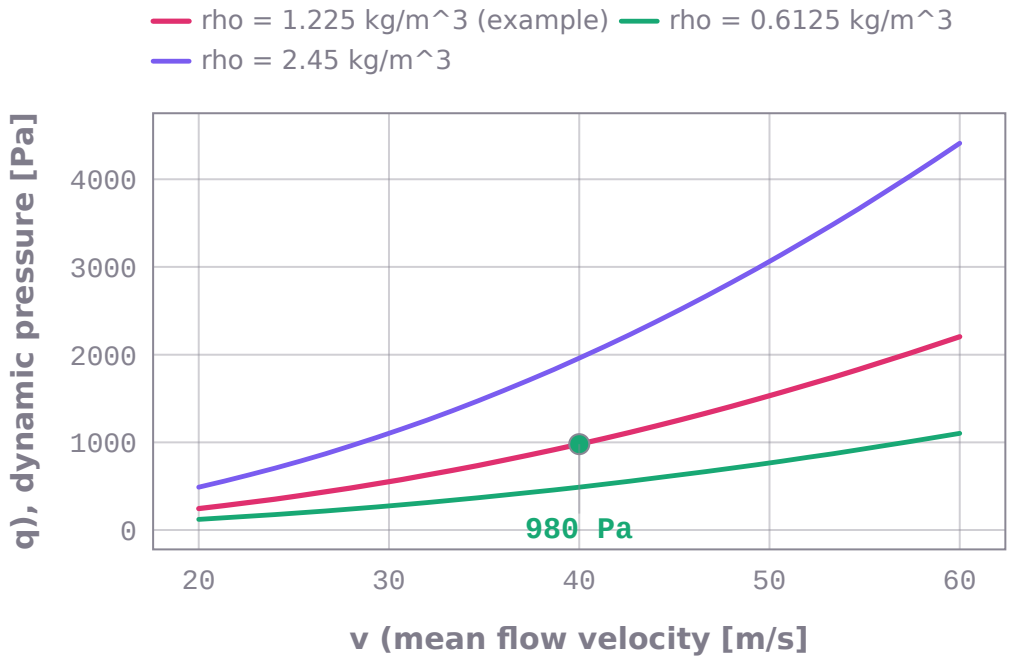
Dynamic pressure of a moving fluid, `dynamic_pressure`

$$q = \frac{\rho v^2}{2}$$

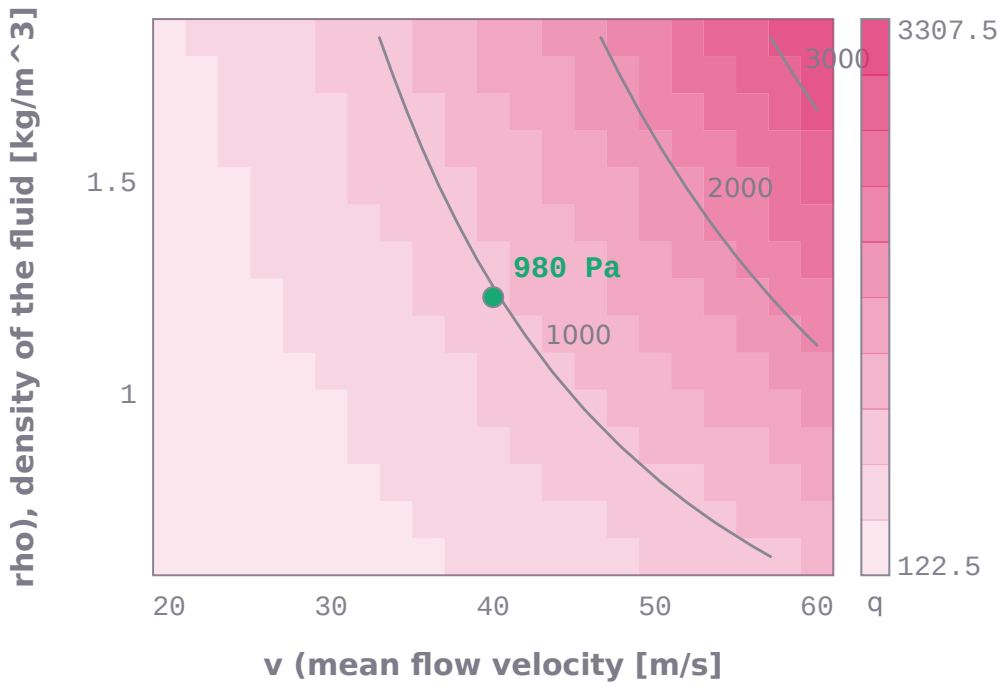
input	meaning	kind / role	unit	default
`q`	dynamic pressure	pressure / dynamic	`Pa`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	

input	meaning	kind / role	unit	default
`v`	mean flow velocity	speed / flow	`m/s`	

- **Conditions:** dynamic_pressure
- **Applies when:** a pitot-static measurement or the kinetic pressure of a stream
- **Fails when:** compressible flow above Mach 0.3
- **Assumes:** incompressible flow (low Mach number)
- **Evidence:** theorem



q (dynamic pressure) against v (mean flow velocity) from 20 m/s to 60 m/s. One curve for each value of rho (density of the fluid): 1.225 kg/m^3, 0.6125 kg/m^3, 2.45 kg/m^3. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, v = 40 m/s, q = 980 Pa.



q (dynamic pressure) over v (mean flow velocity) from 20 m/s to 60 m/s and rho (density of the fluid) from 0.6125 kg/m^3 to 1.8375 kg/m^3: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 122.5 Pa to dark 3307.5 Pa); iso-lines at q = 1000, 2000, 3000 Pa. Marked: the worked example, v = 40 m/s, rho = 1.225 kg/m^3, q = 980 Pa.

```
thermofluids_ai999x.solve('dynamic_pressure', rho='1.225 kg/m^3', v='40 m/s', target='q')
# -> 980 Pa   (SOLVED)
```

$$q = \frac{\rho v^2}{2}$$

$$q = \frac{\rho v^2}{2} = 980\ \text{Pa}$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	dynamic_pressure needs rho (density of the fluid, kg/m^3). Example: solve('dynamic_pressure', rho='... kg/m^3', v='... m/s').
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

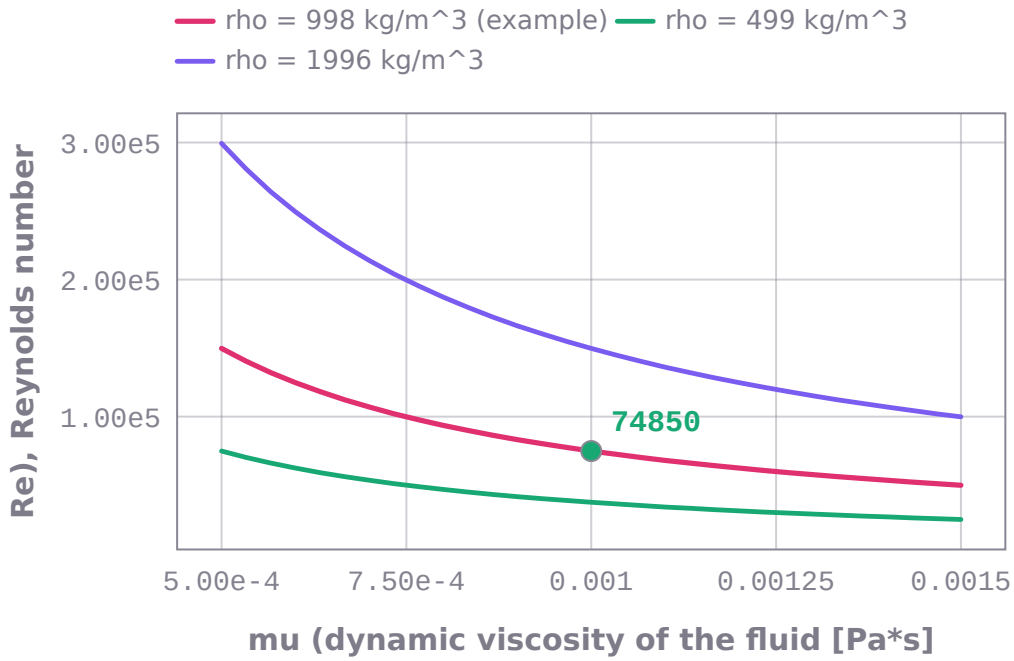
- "What is the dynamic pressure of air with density 1.225 kg/m^3 moving at 40 m/s?" → 980 Pa
- "A pitot tube in a water stream of density 1000 kg/m^3 measures a dynamic pressure of 4500 Pa. What is the flow speed?" → 3 m/s

Reynolds number from density, velocity, diameter and viscosity, `reynolds_number`

$$\mathrm{Re} = \frac{\rho v D}{\mu}$$

input	meaning	kind / role	unit	default
`Re`	Reynolds number	reynolds_number / plain	`1`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`v`	mean flow velocity	speed / flow	`m/s`	
`D`	pipe or body diameter	length / diameter	`m`	
`mu`	dynamic viscosity of the fluid	dynamic_viscosity / plain	`Pa*s`	

- **Conditions:** fluid_flow
- **Applies when:** flow in a circular pipe or past a sphere or cylinder of that diameter
- **Fails when:** non-Newtonian fluids
- **Assumes:** Newtonian fluid; the diameter as the characteristic length
- **Evidence:** theorem



Re (Reynolds number) against mu (dynamic viscosity of the fluid) from 0.0005 Pa*s to 0.0015 Pa*s. One curve for each value of rho (density of the fluid): 998 kg/m^3, 499 kg/m^3, 1996 kg/m^3. Other inputs as in the worked example: v = 1.5 m/s, D = 0.05 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, mu = 0.001 Pa*s, Re = 74850.

```
thermofluids_ai999x.solve('reynolds_number', rho='998 kg/m^3', v='1.5 m/s', D='0.05 m',
mu='0.001 Pa*s', target='Re')
# -> 74850 1   (SOLVED)
```

$$\mathrm{Re} = \frac{\rho v D}{\mu}$$

$$\mathrm{Re} = \frac{D \rho v}{\mu} = 74850\ 1$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	reynolds_number needs rho (density of the fluid, kg/m^3). Example: solve('reynolds_number', rho='... kg/m^3', v='... m/s', D='... m', mu='..
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

- "Water with density 998 kg/m^3 and dynamic viscosity 0.001 Pa*s flows at an average velocity of 1.5 m/s in a pipe of diameter 0.05 m. What is the Reynolds number?" → 74850 1
- "Find the Reynolds number for oil of density 900 kg/m^3 and viscosity 0.1 Pa*s flowing through a tube with an inner diameter of 0.02 m at a mean velocity of 0.5 m/s." → 90 1

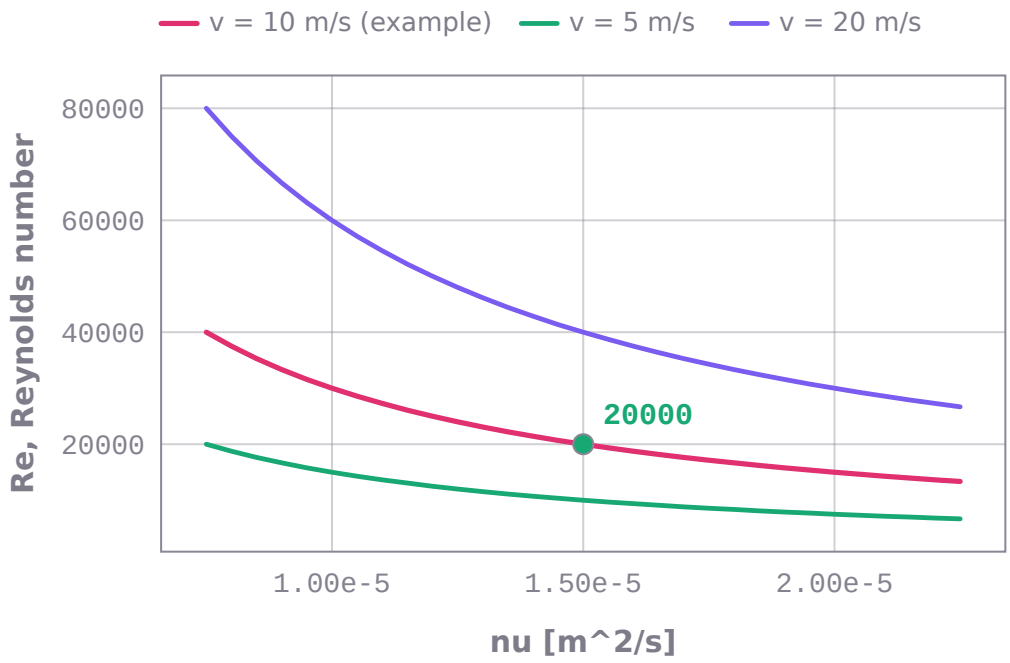
Reynolds number from velocity, diameter and kinematic viscosity,

`reynolds_number_kinematic`

$$\mathrm{Re} = \frac{vD}{\nu}$$

input	meaning	kind / role	unit	default
`Re`	Reynolds number	reynolds_number / plain	`1`	
`v`	mean flow velocity	speed / flow	`m/s`	
`D`	pipe or body diameter	length / diameter	`m`	
`nu`	kinematic viscosity of the fluid	kinematic_viscosity / plain	`m^2/s`	

- **Conditions:** fluid_flow
- **Applies when:** flow in a circular pipe or past a sphere or cylinder of that diameter
- **Fails when:** non-Newtonian fluids
- **Assumes:** Newtonian fluid; the diameter as the characteristic length
- **Evidence:** theorem



Re (Reynolds number) against nu (kinematic viscosity of the fluid) from 7.5000e-06 m^2/s to 2.2500e-05 m^2/s. One curve for each value of v (mean flow velocity): 10 m/s, 5 m/s, 20 m/s. Other inputs as in the worked example: D = 0.03 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, nu = 1.5000e-05 m^2/s, Re = 20000.

```
thermofluids_ai999x.solve('reynolds_number_kinematic', v='10 m/s', D='0.03 m', nu='1.5e-05 m^2/s', target='Re')
# -> 20000 1   (SOLVED)
```

$$\mathrm{Re} = \frac{vD}{\nu}$$

$$\mathrm{Re} = \frac{Dv}{\nu} = 20000\ 1$$

invalid call	outcome	message
without `v`	NEEDS_INPUT	reynolds_number_kinematic needs v (mean flow velocity, m/s). Example: solve('reynolds_number_kinematic', v='... m/s', D='... m', nu='... m^2
`v` in a wrong unit	INVALID_INPUT	v is a speed: its unit must be like m/s, not 'kg'.

Questions in words that work:

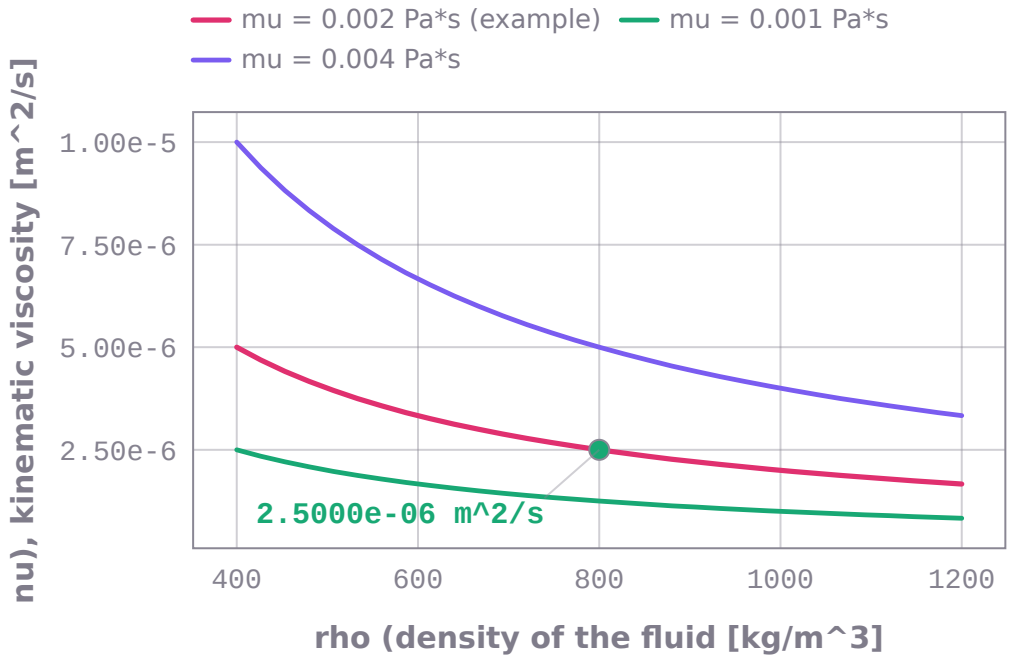
- "Air with a kinematic viscosity of 0.000015 m^2/s flows at 10 m/s past a cylinder of diameter 0.03 m. What is the Reynolds number?" → 20000 1
- "What is the Reynolds number of water with kinematic viscosity 0.000001 m^2/s flowing at a mean velocity of 2 m/s in a pipe of diameter 0.1 m?" → 200000 1

Kinematic viscosity from dynamic viscosity and density, `kinematic_viscosity`

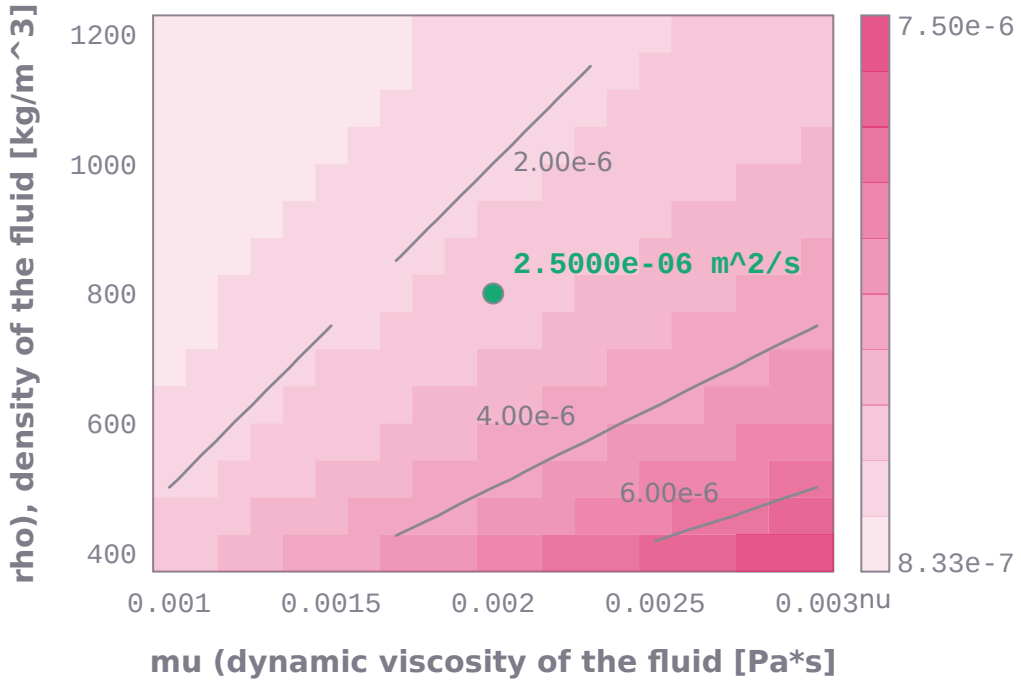
$$\nu = \frac{\mu}{\rho}$$

input	meaning	kind / role	unit	default
`nu`	kinematic viscosity	kinematic_viscosity / plain	`m^2/s`	
`mu`	dynamic viscosity of the fluid	dynamic_viscosity / plain	`Pa*s`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	

- **Conditions:** kinematic
- **Applies when:** converting between dynamic and kinematic viscosity
- **Fails when:** non-Newtonian fluids (no single viscosity)
- **Assumes:** Newtonian fluid
- **Evidence:** theorem



nu (kinematic viscosity) against rho (density of the fluid) from 400 kg/m^3 to 1200 kg/m^3. One curve for each value of mu (dynamic viscosity of the fluid): 0.002 Pa*s, 0.001 Pa*s, 0.004 Pa*s. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, rho = 800 kg/m^3, nu = 2.5000e-06 m^2/s.



nu (kinematic viscosity) over mu (dynamic viscosity of the fluid) from 0.001 Pa*s to 0.003 Pa*s and rho (density of the fluid) from 400 kg/m^3 to 1200 kg/m^3: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 8.3333e-07 m^2/s to dark 7.5000e-06 m^2/s); iso-lines at nu = 2.00e-6, 4.00e-6, 6.00e-6 m^2/s. Marked: the worked example, mu = 0.002 Pa*s, rho = 800 kg/m^3, nu = 2.5000e-06 m^2/s.

```
thermofluids_ai999x.solve('kinematic_viscosity', mu='0.002 Pa*s', rho='800 kg/m^3',
target='nu')
# -> 2.5e-06 m^2/s   (SOLVED)
```

$$\nu = \frac{\mu}{\rho}$$

$$\nu = \frac{\mu}{\rho} = 2.5e-06\ \mathrm{m^2/s}$$

invalid call	outcome	message
without `mu`	NEEDS_INPUT	kinematic_viscosity needs mu (dynamic viscosity of the fluid, Pa*s). Example: solve('kinematic_viscosity', mu='... Pa*s', rho='... kg/m^3').
`mu` in a wrong unit	INVALID_INPUT	mu is a dynamic viscosity: its unit must be like Pa*s, not 'kg'.

Questions in words that work:

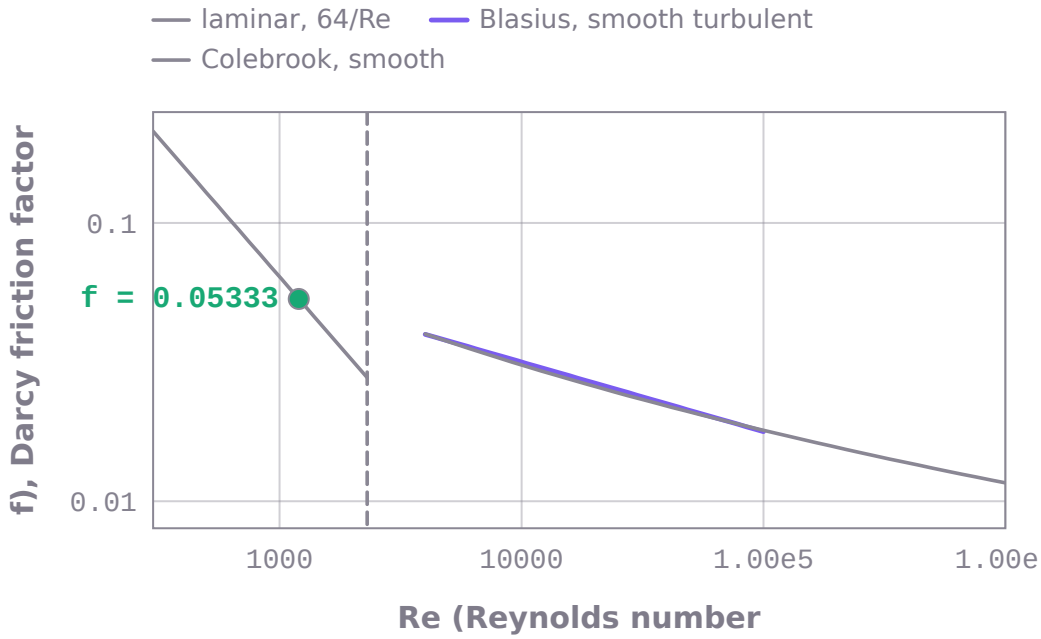
- "A liquid has a dynamic viscosity of 0.002 Pa*s and a density of 800 kg/m^3. What is its kinematic viscosity?" → 2.5e-06 m^2/s
- "Find the kinematic viscosity of glycerin with density 1260 kg/m^3 and viscosity 1.41 Pa*s." → 0.00111905 m^2/s

Darcy friction factor in laminar pipe flow, `laminar_friction_factor`

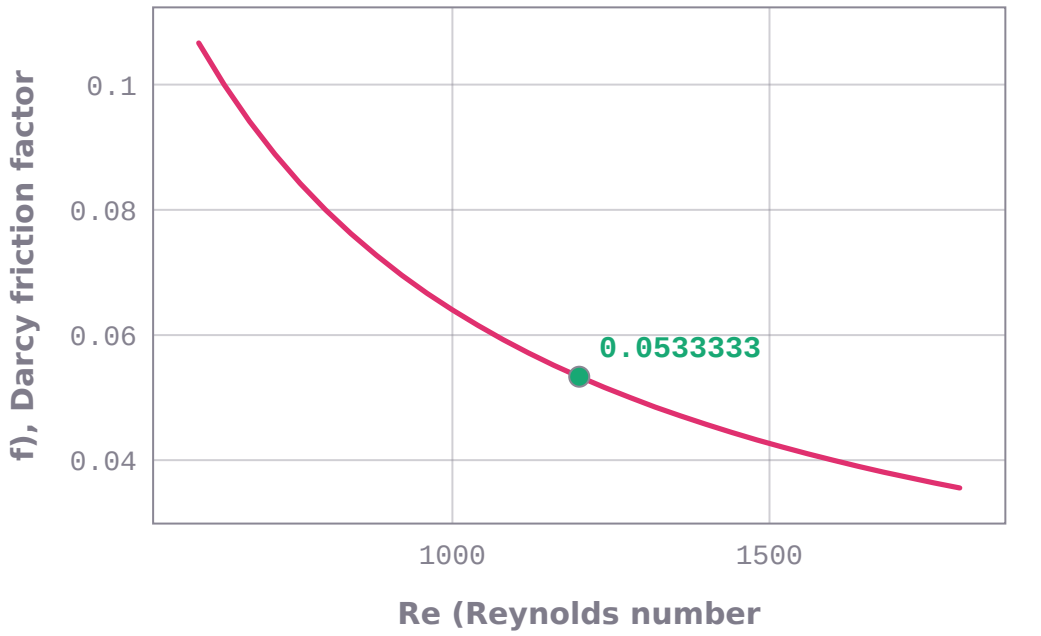
$$f = \frac{64}{\mathrm{Re}}$$

input	meaning	kind / role	unit	default
`f`	Darcy friction factor	darcy_friction_factor / plain	`1`	
`Re`	Reynolds number	reynolds_number / plain	`1`	

- **Conditions:** pipe, laminar
- **Not when:** turbulent, fanning
- **Applies when:** Re below about 2300 in a circular pipe
- **Fails when:** turbulent or transitional flow; non-circular ducts
- **Assumes:** fully developed laminar flow in a circular pipe; Darcy (not Fanning) friction factor
- **Evidence:** theorem



Friction factor against Reynolds number (log–log): the package's laminar line 64/Re to Re = 2300, its Blasius smooth-pipe formula from 4000 to 10^5 and, for comparison, its Colebrook factor of a smooth pipe to 10^6, each at up to 41 Reynolds numbers. Marked: the worked example on the laminar 64/Re line, Re = 1200, f = 0.0533333.



f (Darcy friction factor) against Re (Reynolds number) from 600 to 1800. 31 points computed by thermofluids-ai999x; marked: the worked example, Re = 1200, f = 0.0533333.

```
thermofluids_ai999x.solve('laminar_friction_factor', Re='1200 1', target='f')
# -> 0.0533333 1   (SOLVED)
```

$$f = \frac{64}{\mathrm{Re}}$$

$$f = \frac{64}{\mathrm{Re}} = 0.0533333\ 1$$

invalid call	outcome	message
without `Re`	NEEDS_INPUT	laminar_friction_factor needs Re (Reynolds number, 1). Example: solve('laminar_friction_factor', Re='... 1').
`Re` in a wrong unit	INVALID_INPUT	Re is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

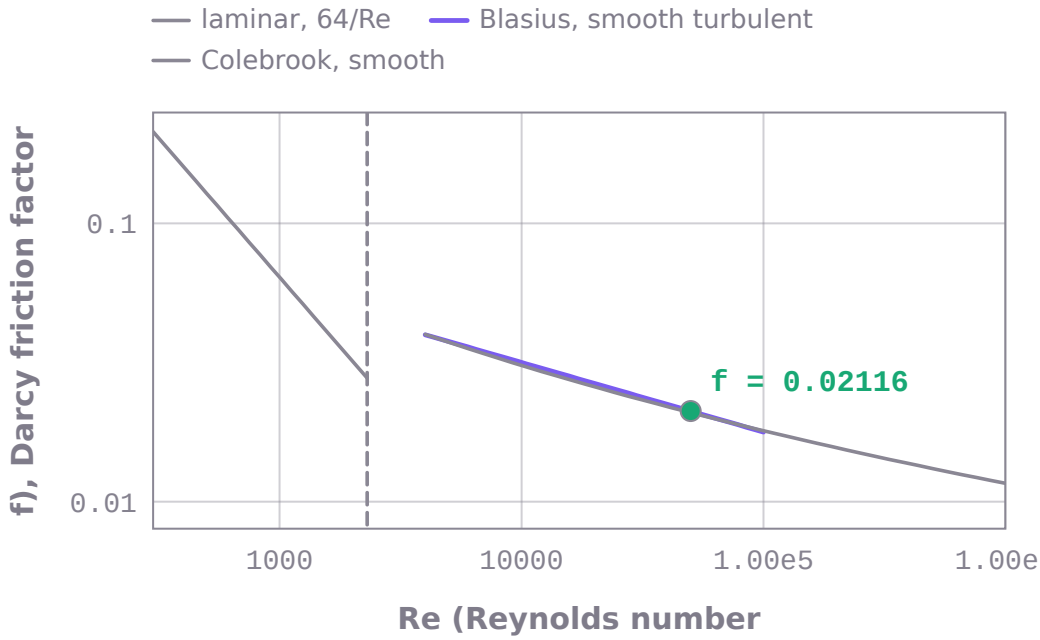
- "Oil flows in a pipe with a Reynolds number of 1200. The flow is laminar. What is the Darcy friction factor?" → 0.0533333 1
- "Find the friction factor for laminar flow in a tube at a Reynolds number of 800." → 0.08 1

Blasius friction factor for turbulent flow in a smooth pipe, `blasius_friction_factor`

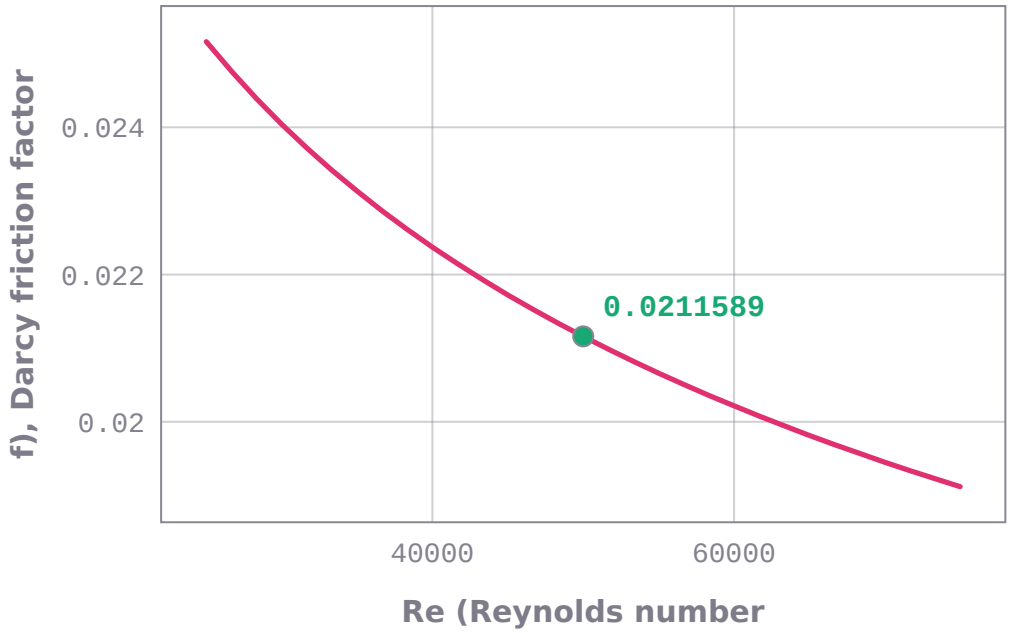
$$f = 0.3164 \mathrm{Re}^{\frac{-1}{4}}$$

input	meaning	kind / role	unit	default
`f`	Darcy friction factor	darcy_friction_factor / plain	`1`	
`Re`	Reynolds number	reynolds_number / plain	`1`	

- **Conditions:** pipe, turbulent, smooth_pipe
- **Not when:** laminar, fanning
- **Applies when:** $4000 < Re < 100000$ in a smooth pipe
- **Fails when:** rough pipes; Re above about 100000
- **Assumes:** fully developed turbulent flow in a hydraulically smooth circular pipe; Darcy friction factor
- **Evidence:** theorem



Friction factor against Reynolds number (log–log): the package's laminar line 64/Re to Re = 2300, its Blasius smooth-pipe formula from 4000 to 10^5 and, for comparison, its Colebrook factor of a smooth pipe to 10^6, each at up to 41 Reynolds numbers. Marked: the worked example on the Blasius line, Re = 50000, f = 0.0211589.



f (Darcy friction factor) against Re (Reynolds number) from 25000 to 75000. 31 points computed by thermofluids-ai999x; marked: the worked example, Re = 50000, f = 0.0211589.

```
thermofluids_ai999x.solve('blasius_friction_factor', Re='50000 1', target='f')
# -> 0.0211589 1   (SOLVED)
```

$$f = 0.3164 \mathrm{Re}^{\frac{-1}{4}}$$
$$f = \frac{791}{2500 \sqrt[4]{\mathrm{Re}}} = 0.0211589\ 1$$

invalid call	outcome	message
without `Re`	NEEDS_INPUT	blasius_friction_factor needs Re (Reynolds number, 1). Example: solve('blasius_friction_factor', Re='... 1').
`Re` in a wrong unit	INVALID_INPUT	Re is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

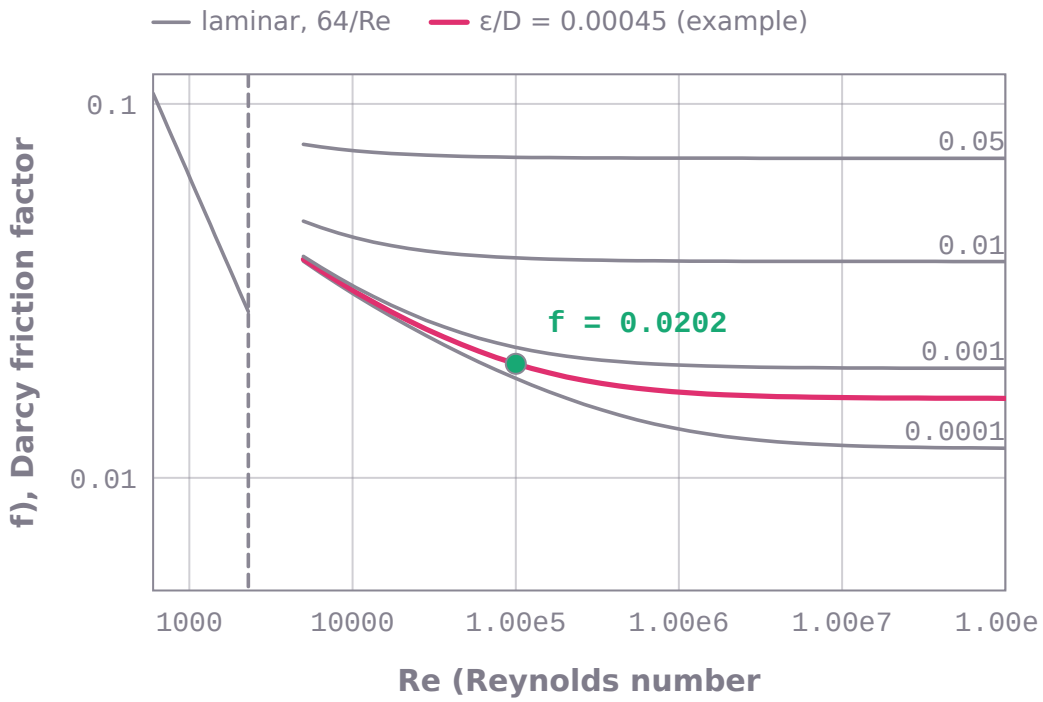
- "Water flows turbulently through a smooth pipe at a Reynolds number of 50000. Using the Blasius correlation, what is the Darcy friction factor?" → 0.0211589 1
- "Find the friction factor for turbulent flow in a smooth tube with a Reynolds number of 20000." → 0.026606 1

Swamee-Jain explicit friction factor for turbulent pipe flow, `swamee_jain_friction_factor`

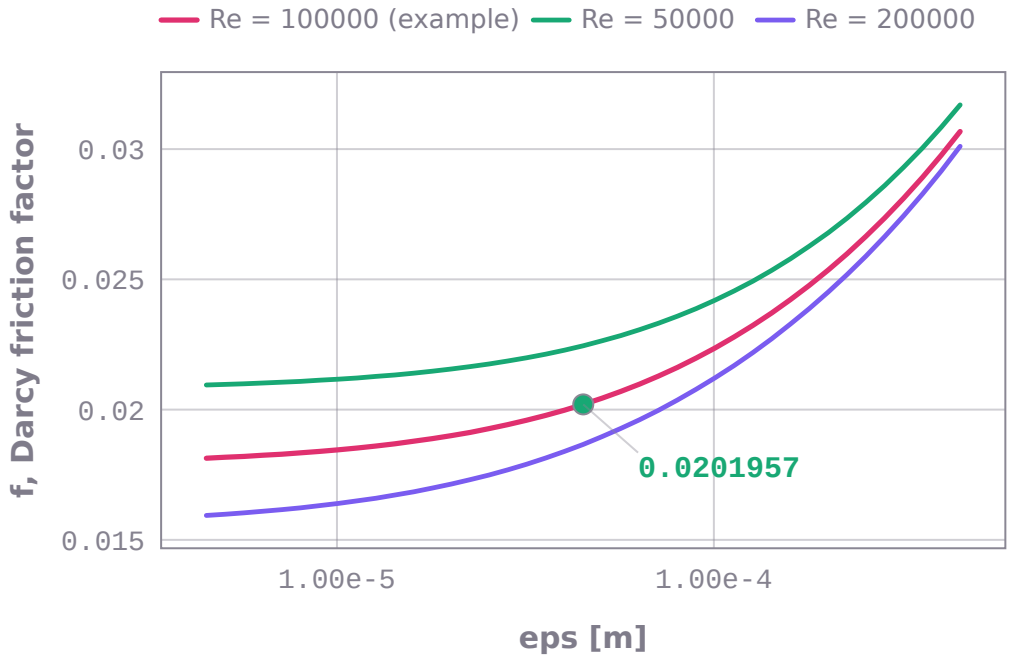
$$f = \frac{0.25}{\left(\frac{\ln\left(\frac{\varepsilon}{3.7D} + \left(\frac{6.97}{\mathrm{Re}} \right)^{\frac{9}{10}} \right)}{\ln(10)} \right)^2}$$

input	meaning	kind / role	unit	default
`f`	Darcy friction factor	darcy_friction_factor / plain	`1`	
`eps`	absolute roughness of the pipe wall	length / roughness	`m`	
`D`	inner diameter of the pipe	length / diameter	`m`	
`Re`	Reynolds number	reynolds_number / plain	`1`	

- **Conditions:** pipe, swamee_jain
- **Not when:** laminar, fanning
- **Applies when:** $5000 < Re < 1e8$ and 1e-6 < eps/D < 0.01
- **Fails when:** laminar flow; eps/D outside the fitted range
- **Assumes:** fully developed turbulent flow in a circular pipe; Darcy friction factor
- **Evidence:** theorem



Friction-factor chart from the package's Swamee–Jain formula at 41 Reynolds numbers from 5000 to 10^8, pipe diameter D = 0.1 m, relative roughness ε/D = 0.0001, 0.00045, 0.001, 0.01, 0.05 (labelled at the right), with its laminar 64/Re line below Re = 2300; log–log axes. Marked: the worked example, ε = 4.5000e-05 m (ε/D = 0.00045), Re = 100000, f = 0.0201957.



f (Darcy friction factor) against eps (absolute roughness of the pipe wall) from 4.5000e-06 m to 0.00045 m on a logarithmic axis. One curve for each value of Re (Reynolds number): 100000, 50000, 200000. Other inputs as in the worked example: D = 0.1 m. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, eps = 4.5000e-05 m, f = 0.0201957.

```
thermofluids_ai999x.solve('swamee_jain_friction_factor', eps='4.5e-05 m', D='0.1 m',
Re='100000 1', target='f')
# -> 0.0201957 1   (SOLVED)
```

$$f = \frac{0.25}{\left(\frac{\ln\left(\frac{\varepsilon}{3.7D} + \left(\frac{6.97}{\mathrm{Re}}\right)^{\frac{9}{10}}\right)}{\ln(10)}\right)^2}$$

$$f = \frac{\ln(10)^2}{4\ln\left(\frac{\sqrt[5]{10}\cdot 697^{\frac{9}{10}}}{100\mathrm{Re}^{\frac{9}{10}}} + \frac{10\varepsilon}{37D}\right)^2} = 0.0201957\ 1$$

invalid call	outcome	message
without `eps`	NEEDS_INPUT	swamee_jain_friction_factor needs eps (absolute roughness of the pipe wall, m). Example: solve('swamee_jain_friction_factor', eps='... m', D
`eps` in a wrong unit	INVALID_INPUT	eps is a length: its unit must be like m, not 'kg'.

Questions in words that work:

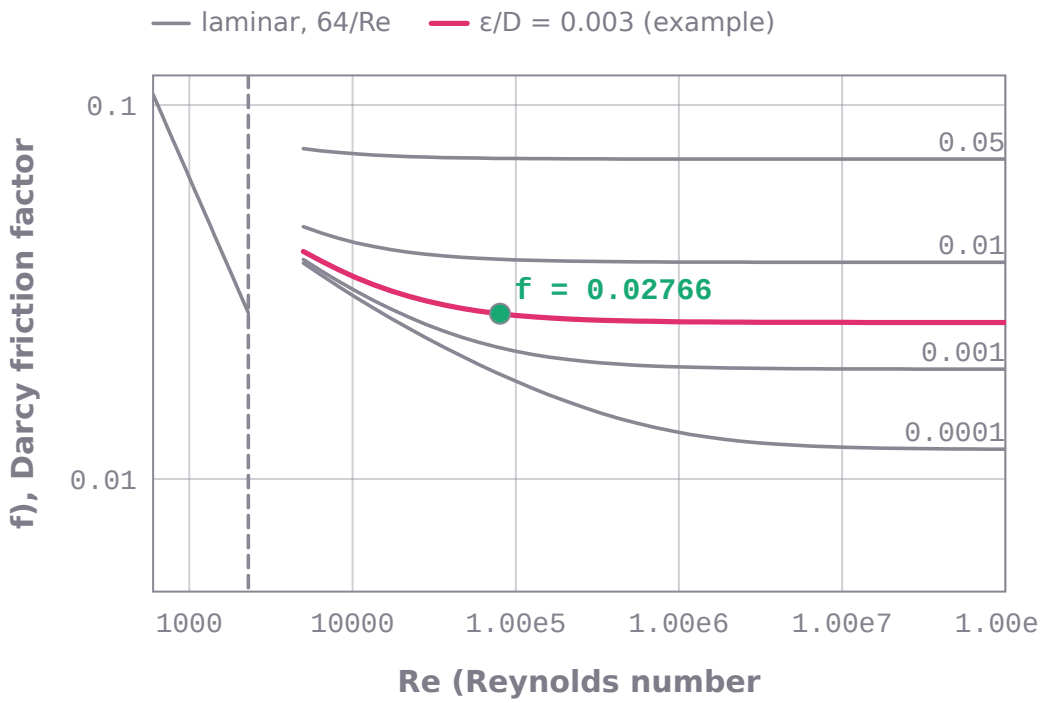
- "Water flows in a commercial steel pipe of diameter 0.1 m with an absolute roughness of 0.000045 m at a Reynolds number of 100000. Use the Swamee-Jain equation to find the Darcy friction factor." → 0.0201957 1
- "Using the Swamee-Jain formula, estimate the friction factor in a cast iron pipe with roughness 0.00026 m and diameter 0.2 m when the Reynolds number is 500000." → 0.021569 1

Haaland explicit friction factor for turbulent pipe flow, `haaland_friction_factor`

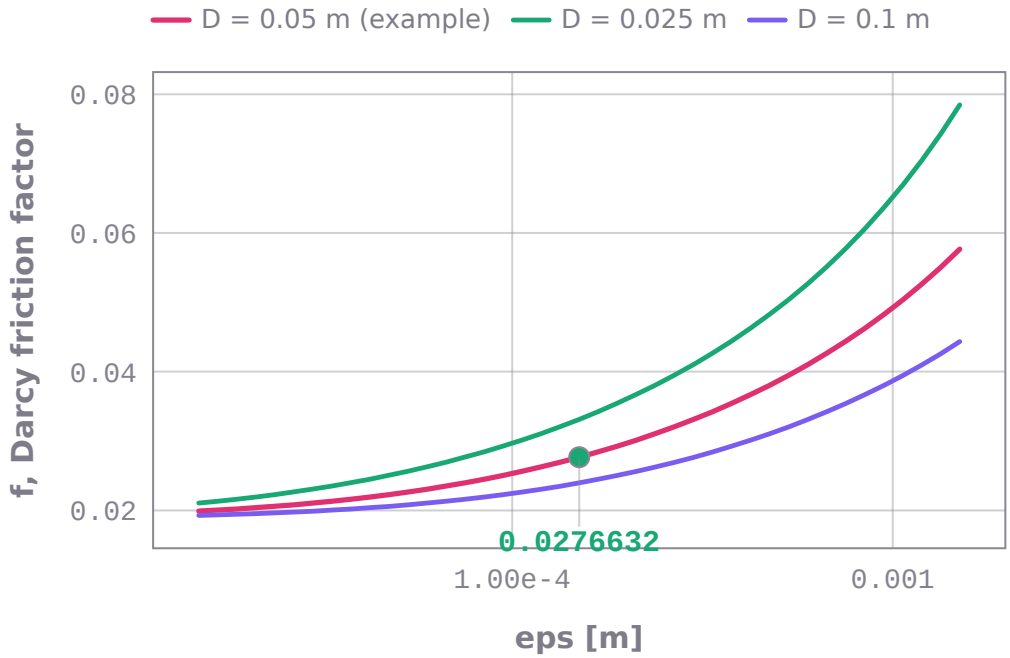
$$f = \frac{1}{\left(\frac{1.8 \ln\left(\left(\frac{\varepsilon}{3.7D} \right)^{\frac{111}{100}} + \frac{6.9}{Re} \right)}{\ln(10)} \right)^2}$$

input	meaning	kind / role	unit	default
`f`	Darcy friction factor	darcy_friction_factor / plain	`1`	
`eps`	absolute roughness of the pipe wall	length / roughness	`m`	
`D`	inner diameter of the pipe	length / diameter	`m`	
`Re`	Reynolds number	reynolds_number / plain	`1`	

- **Conditions:** pipe, haaland
- **Not when:** laminar, fanning
- **Applies when:** $4000 < Re < 1e8$ and eps/D < 0.05
- **Fails when:** laminar flow
- **Assumes:** fully developed turbulent flow in a circular pipe; Darcy friction factor
- **Evidence:** theorem



Friction-factor chart from the package's Haaland formula at 41 Reynolds numbers from 5000 to 10^8, pipe diameter D = 0.05 m, relative roughness ε/D = 0.0001, 0.001, 0.003, 0.01, 0.05 (labelled at the right), with its laminar 64/Re line below Re = 2300; log–log axes. Marked: the worked example, ε = 0.00015 m (ε/D = 0.003), Re = 80000, f = 0.0276632.



f (Darcy friction factor) against eps (absolute roughness of the pipe wall) from 1.5000e-05 m to 0.0015 m on a logarithmic axis. One curve for each value of D (inner diameter of the pipe): 0.05 m, 0.025 m, 0.1 m. Other inputs as in the worked example: Re = 80000 1. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, eps = 0.00015 m, f = 0.0276632.

```
thermofluids_ai999x.solve('haaland_friction_factor', eps='0.00015 m', D='0.05 m', Re='80000
1', target='f')
# -> 0.0276632 1   (SOLVED)
```

$$f = \frac{1}{\left(\frac{1.8\ln\left(\left(\frac{\varepsilon}{3.7D}\right)^{\frac{111}{100}}+\frac{6.9}{\mathrm{Re}}\right)}{\ln(10)}\right)^2}$$

$$f = \frac{25\ln(10)^2}{81\ln\left(\frac{69}{10\mathrm{Re}}+\frac{10\cdot 10^{\frac{11}{100}}\cdot 37^{\frac{89}{100}}\varepsilon^{\frac{111}{100}}}{1369D^{\frac{111}{100}}}\right)^2} = 0.0276632\ 1$$

invalid call	outcome	message
without `eps`	NEEDS_INPUT	haaland_friction_factor needs eps (absolute roughness of the pipe wall, m). Example: solve('haaland_friction_factor', eps='... m', D='... m'
`eps` in a wrong unit	INVALID_INPUT	eps is a length: its unit must be like m, not 'kg'.

Questions in words that work:

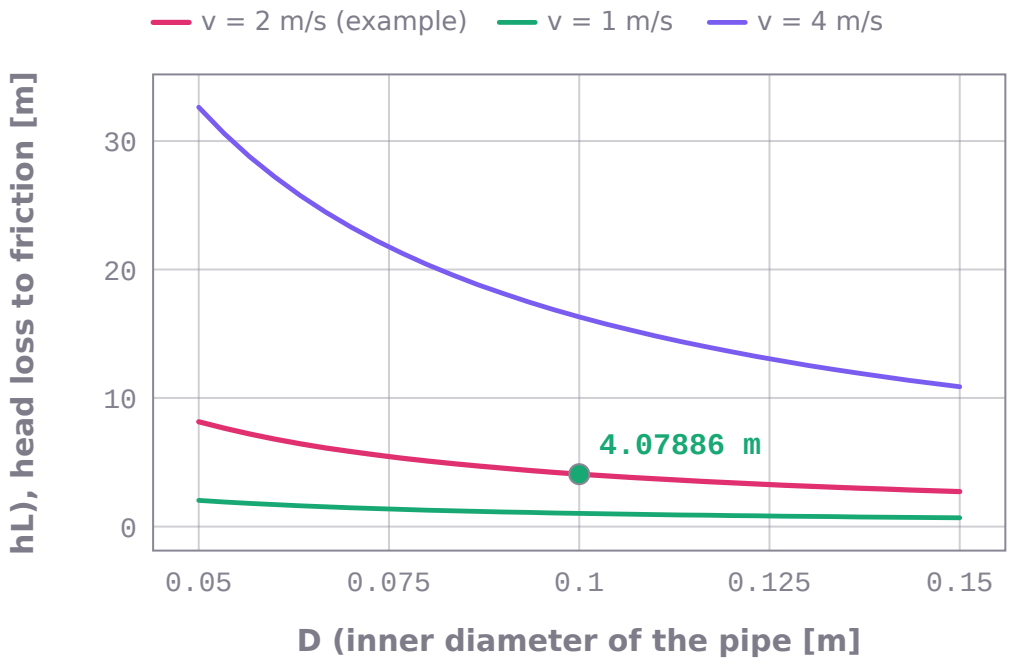
- "Use the Haaland equation to find the Darcy friction factor for turbulent flow in a pipe of diameter 0.05 m with an absolute roughness of 0.00015 m at a Reynolds number of 80000." → 0.0276632 1
- "With the Haaland formula, what is the friction factor in a steel pipe of diameter 0.3 m and roughness 0.000045 m at a Reynolds number of 1000000?" → 0.0140075 1

Darcy-Weisbach head loss in a pipe, `darcy_weisbach_head_loss`

$$h_L = \frac{fLv^2}{2gD}$$

input	meaning	kind / role	unit	default
`hL`	head loss to friction	length / head_loss	`m`	
`f`	Darcy friction factor	darcy_friction_factor / plain	`1`	
`L`	length of the pipe	length / pipe_length	`m`	
`v`	mean flow velocity	speed / flow	`m/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`D`	inner diameter of the pipe	length / diameter	`m`	

- **Conditions:** pipe
- **Not when:** fanning
- **Applies when:** frictional loss along a straight pipe of constant diameter
- **Fails when:** Fanning friction factor given; fittings (minor losses)
- **Assumes:** fully developed flow; Darcy friction factor; constant diameter
- **Evidence:** theorem



hL (head loss to friction) against D (inner diameter of the pipe) from 0.05 m to 0.15 m. One curve for each value of v (mean flow velocity): 2 m/s, 1 m/s, 4 m/s. Other inputs as in the worked example: f = 0.02 1, L = 100 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, D = 0.1 m, hL = 4.07886 m.

```
thermofluids_ai999x.solve('darcy_weisbach_head_loss', f='0.02 1', L='100 m', v='2 m/s',
D='0.1 m', target='hL')
# -> 4.07886 m   (SOLVED)
```

$$h_L = \frac{fLv^2}{2gD}$$

$$h_L = \frac{Lfv^2}{2Dg} = 4.07886\ \mathrm{m}$$

invalid call	outcome	message
without `f`	NEEDS_INPUT	darcy_weisbach_head_loss needs f (Darcy friction factor, 1). Example: solve('darcy_weisbach_head_loss', f='... 1', L='... m', v='... m/s', g
`f` in a wrong unit	INVALID_INPUT	f is a darcy friction factor: its unit must be like 1, not 'kg'.

Questions in words that work:

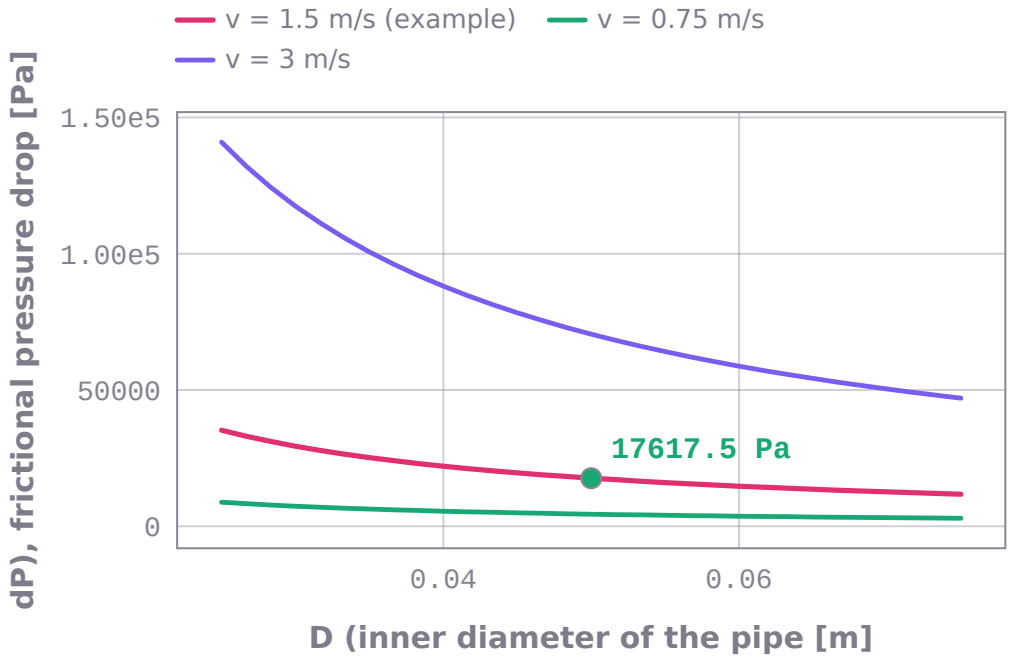
- "Water flows at an average velocity of 2 m/s through a 100 m long pipe of diameter 0.1 m. The Darcy friction factor is 0.02. What is the head loss?" → 4.07886 m
- "Find the frictional head loss in a pipe of length 250 m and diameter 0.15 m carrying water at a mean velocity of 1.2 m/s, if the friction factor is 0.018." → 2.20259 m

Darcy-Weisbach pressure drop in a pipe, `darcy_weisbach_pressure_drop`

$$d_P = \frac{fL\rho v^2}{2D}$$

input	meaning	kind / role	unit	default
`dP`	frictional pressure drop	pressure / drop	`Pa`	
`f`	Darcy friction factor	darcy_friction_factor / plain	`1`	
`L`	length of the pipe	length / pipe_length	`m`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`v`	mean flow velocity	speed / flow	`m/s`	
`D`	inner diameter of the pipe	length / diameter	`m`	

- **Conditions:** pipe
- **Not when:** fanning
- **Applies when:** frictional pressure loss along a straight pipe of constant diameter
- **Fails when:** Fanning friction factor given
- **Assumes:** fully developed flow; Darcy friction factor; constant diameter
- **Evidence:** theorem



dP (frictional pressure drop) against D (inner diameter of the pipe) from 0.025 m to 0.075 m. One curve for each value of v (mean flow velocity): 1.5 m/s, 0.75 m/s, 3 m/s. Other inputs as in the worked example: f = 0.03 1, L = 30 m, rho = 870 kg/m^3. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, D = 0.05 m, dP = 17617.5 Pa.

```
thermofluids_ai999x.solve('darcy_weisbach_pressure_drop', f='0.03 1', L='30 m', rho='870
kg/m^3', v='1.5 m/s', D='0.05 m', target='dP')
# -> 17617.5 Pa   (SOLVED)
```

$$d_P = \frac{fL\rho v^2}{2D}$$
$$d_P = \frac{Lf\rho v^2}{2D} = 17617.5\ \mathrm{Pa}$$

invalid call	outcome	message
without `f`	NEEDS_INPUT	darcy_weisbach_pressure_drop needs f (Darcy friction factor, 1). Example: solve('darcy_weisbach_pressure_drop', f='... 1', L='... m', rho='.
`f` in a wrong unit	INVALID_INPUT	f is a darcy friction factor: its unit must be like 1, not 'kg'.

Questions in words that work:

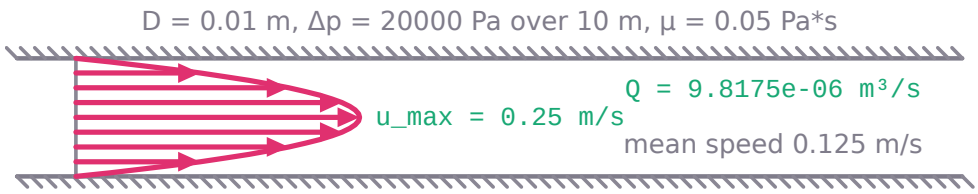
- "Oil of density 870 kg/m^3 flows at a mean velocity of 1.5 m/s through a 30 m long pipe of diameter 0.05 m with a friction factor of 0.03. What is the pressure drop?" → 17617.5 Pa
- "Find the pressure drop due to friction for water of density 1000 kg/m^3 flowing at an average velocity of 3 m/s in a pipe of length 50 m and diameter 0.1 m if the Darcy friction factor is 0.02." → 45000 Pa

Hagen-Poiseuille flow rate in laminar pipe flow, `hagen_poiseuille`

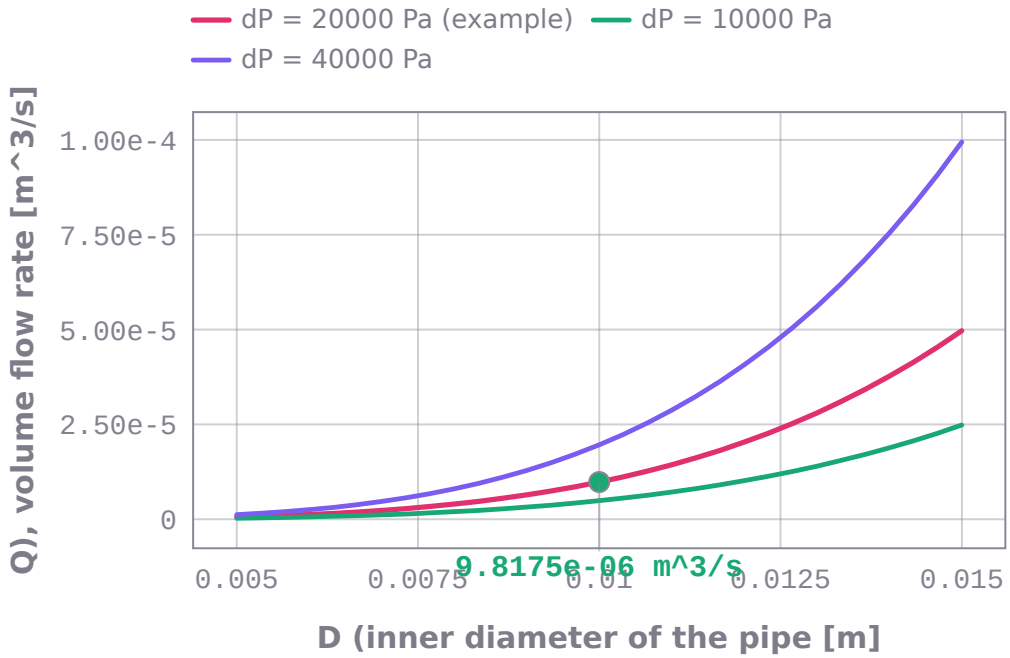
$$Q = \frac{\pi D^4 d_P}{128 \mu L}$$

input	meaning	kind / role	unit	default
`Q`	volume flow rate	flow_rate / plain	`m^3/s`	
`D`	inner diameter of the pipe	length / diameter	`m`	
`dP`	pressure drop along the pipe	pressure / drop	`Pa`	
`mu`	dynamic viscosity of the fluid	dynamic_viscosity / plain	`Pa*s`	
`L`	length of the pipe	length / pipe_length	`m`	

- **Conditions:** pipe, laminar
- **Not when:** turbulent
- **Applies when:** Re below about 2300
- **Fails when:** turbulent flow; entrance region
- **Assumes:** fully developed laminar flow; Newtonian fluid; circular pipe
- **Evidence:** theorem



Laminar flow in a pipe (D = 0.01 m, pressure drop 20000 Pa over 10 m, μ = 0.05 Pa*s; the worked example). Marked: the package's volume flow Q = 9.8175e-06 m³/s; the mean speed Q/A = 0.125 m/s and the parabolic velocity profile u = 2·(Q/A)·(1 − r²/R²) with its centre-line maximum 0.25 m/s are derived from it.



Q (volume flow rate) against D (inner diameter of the pipe) from 0.005 m to 0.015 m. One curve for each value of dP (pressure drop along the pipe): 20000 Pa, 10000 Pa, 40000 Pa. Other inputs as in the worked example: mu = 0.05 Pa*s, L = 10 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, D = 0.01 m, Q = 9.8175e-06 m^3/s.

```
thermofluids_ai999x.solve('hagen_poiseuille', D='0.01 m', dP='20000 Pa', mu='0.05 Pa*s',
L='10 m', target='Q')
# -> 9.81748e-06 m^3/s   (SOLVED)
```

$$Q = \frac{\pi D^4 d_P}{128 \mu L}$$

$$Q = \frac{\pi D^4 d_P}{128 L \mu} = 9.81748e-06\ \mathrm{m^3/s}$$

invalid call	outcome	message
without `D`	NEEDS_INPUT	hagen_poiseuille needs D (inner diameter of the pipe, m). Example: solve('hagen_poiseuille', D='... m', dP='... Pa', mu='... Pa*s', L='... m
`D` in a wrong unit	INVALID_INPUT	D is a length: its unit must be like m, not 'kg'.

Questions in words that work:

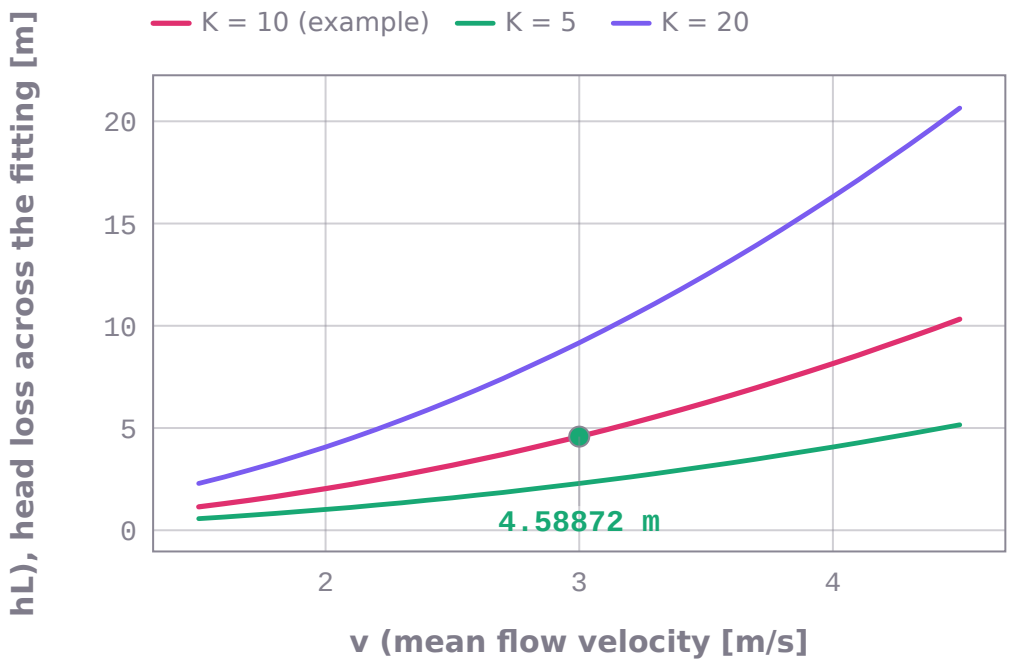
- "Oil with a viscosity of 0.05 Pa*s flows in laminar flow through a 10 m long tube of diameter 0.01 m under a pressure drop of 20000 Pa. What is the volume flow rate?" → 9.81748e-06 m^3/s
- "Find the flow rate of glycerin with viscosity 1.4 Pa*s through a pipe of length 2 m and diameter 0.02 m in laminar flow when the pressure drop is 50000 Pa." → 7.01248e-05 m^3/s

Minor head loss in a fitting, `minor_loss_head`

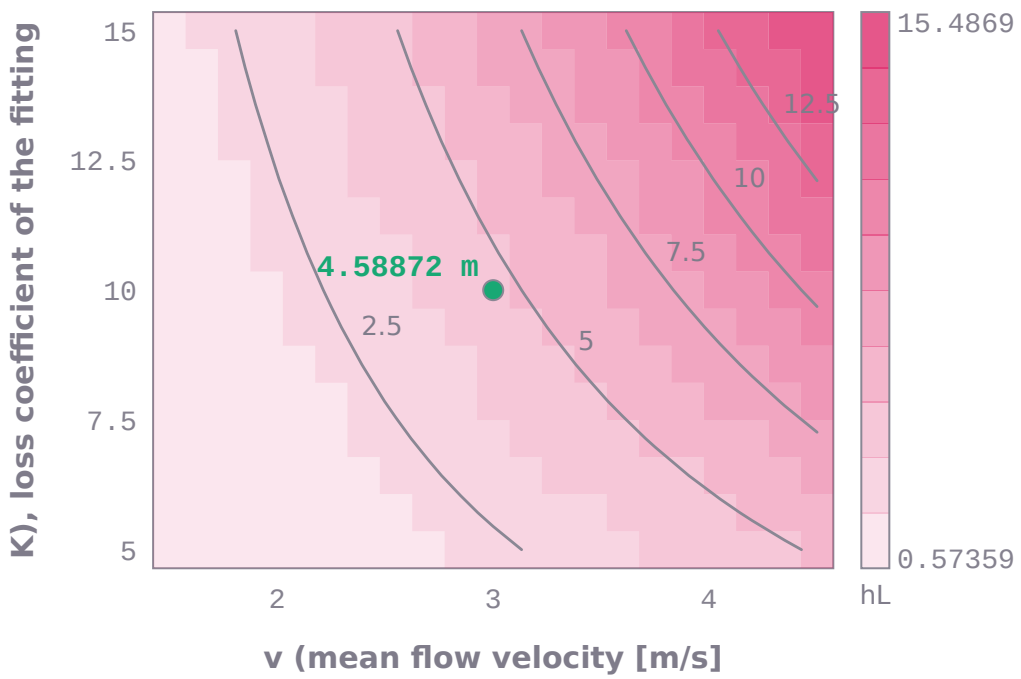
$$h_L = \frac{K v^2}{2g}$$

input	meaning	kind / role	unit	default
`hL`	head loss across the fitting	length / head_loss	`m`	
`K`	loss coefficient of the fitting	loss_coefficient / plain	`1`	
`v`	mean flow velocity	speed / flow	`m/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2

- **Conditions:** fitting
- **Applies when:** valves, elbows, entrances, exits and expansions
- **Fails when:** laminar flow (K depends on Re)
- **Assumes:** fully turbulent flow (K independent of Re); K based on the mean velocity in the pipe
- **Evidence:** theorem



hL (head loss across the fitting) against v (mean flow velocity) from 1.5 m/s to 4.5 m/s. One curve for each value of K (loss coefficient of the fitting): 10, 5, 20. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, v = 3 m/s, hL = 4.58872 m.



hL (head loss across the fitting) over v (mean flow velocity) from 1.5 m/s to 4.5 m/s and K (loss coefficient of the fitting) from 5 to 15: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 0.57359 m to dark 15.4869 m); iso-lines at hL = 2.5, 5, 7.5, 10, 12.5 m. Marked: the worked example, v = 3 m/s, K = 10, hL = 4.58872 m.

```
thermofluids_ai999x.solve('minor_loss_head', K='10 1', v='3 m/s', target='hL')
# -> 4.58872 m   (SOLVED)
```

$$h_L = \frac{K v^2}{2g}$$

$$h_L = \frac{K v^2}{2g} = 4.58872\ \mathrm{m}$$

invalid call	outcome	message
without `K`	NEEDS_INPUT	minor_loss_head needs K (loss coefficient of the fitting, 1). Example: solve('minor_loss_head', K='... 1', v='... m/s', g='... m/s^2').
`K` in a wrong unit	INVALID_INPUT	K is a loss coefficient: its unit must be like 1, not 'kg'.

Questions in words that work:

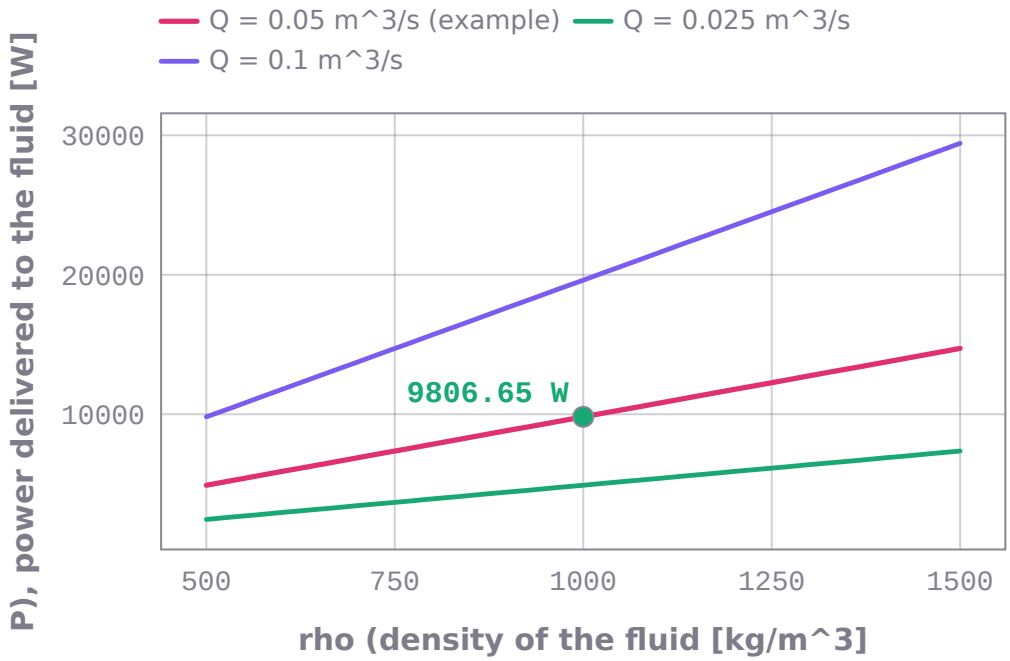
- "Water flows at 3 m/s through a fully open globe valve with a loss coefficient of 10. What is the head loss across the valve?" → 4.58872 m
- "Find the minor head loss in an elbow with a loss coefficient of 0.9 when the mean velocity is 2 m/s." → 0.183549 m

Hydraulic power delivered to a fluid, `hydraulic_power`

$$P = \rho g Q H$$

input	meaning	kind / role	unit	default
`P`	power delivered to the fluid	power / hydraulic	`W`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`Q`	volume flow rate	flow_rate / plain	`m^3/s`	
`H`	head added by the pump	length / pump_head	`m`	

- **Conditions:** pump
- **Applies when:** a pump adding head H to a liquid stream
- **Fails when:** compressible gases with large pressure ratios
- **Assumes:** incompressible liquid; steady flow
- **Evidence:** theorem



P (power delivered to the fluid) against rho (density of the fluid) from 500 kg/m^3 to 1500 kg/m^3. One curve for each value of Q (volume flow rate): 0.05 m^3/s, 0.025 m^3/s, 0.1 m^3/s. Other inputs as in the worked example: H = 20 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, rho = 1000 kg/m^3, P = 9806.65 W.

```
thermofluids_ai999x.solve('hydraulic_power', rho='1000 kg/m^3', Q='0.05 m^3/s', H='20 m',
target='P')
# -> 9806.65 W   (SOLVED)
```

$$P = \rho g Q H$$
$$P = H Q g \rho = 9806.65\ \mathrm{W}$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	hydraulic_power needs rho (density of the fluid, kg/m^3). Example: solve('hydraulic_power', rho='... kg/m^3', g='... m/s^2', Q='... m^3/s',
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

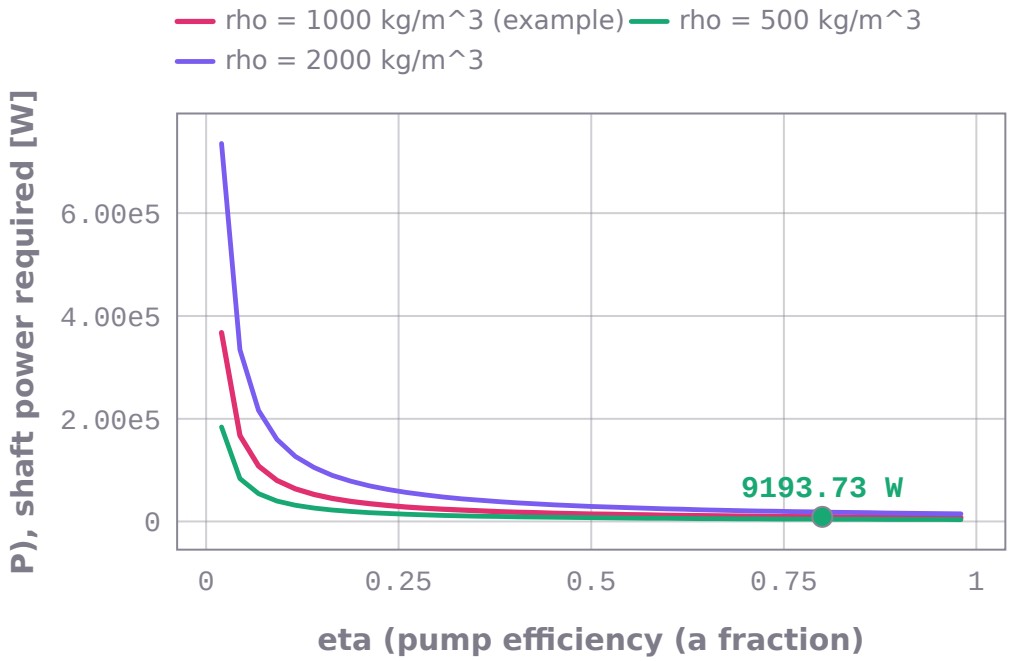
- "A pump delivers water of density 1000 kg/m^3 at a flow rate of 0.05 m^3/s against a total head of 20 m. What is the hydraulic power?" → 9806.65 W
- "Find the power added to the water by a pump that moves 0.02 m^3/s of water with density 998 kg/m^3 through a head of 35 m." → 6850.93 W

Shaft power required by a pump, `pump_shaft_power`

$$P = \frac{\rho g Q H}{\eta}$$

input	meaning	kind / role	unit	default
`P`	shaft power required	power / shaft	`W`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`Q`	volume flow rate	flow_rate / plain	`m^3/s`	
`H`	head added by the pump	length / pump_head	`m`	
`eta`	pump efficiency (a fraction)	efficiency / pump	`1`	

- **Conditions:** pump
- **Applies when:** sizing the drive of a pump
- **Fails when:** efficiency stated as a percentage (give it as a fraction)
- **Assumes:** incompressible liquid; steady flow; efficiency given as a fraction
- **Evidence:** theorem



P (shaft power required) against eta (pump efficiency (a fraction)) from 0.02 to 0.98. One curve for each value of rho (density of the fluid): 1000 kg/m^3, 500 kg/m^3, 2000 kg/m^3. Other inputs as in the worked example: Q = 0.03 m^3/s, H = 25 m. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, eta = 0.8, P = 9193.73 W.

```
thermofluids_ai999x.solve('pump_shaft_power', rho='1000 kg/m^3', Q='0.03 m^3/s', H='25 m',
eta='0.8 1', target='P')
# -> 9193.73 W   (SOLVED)
```

$$P = \frac{\rho g Q H}{\eta}$$
$$P = \frac{H Q g \rho}{\eta} = 9193.73 \text{ W}$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	pump_shaft_power needs rho (density of the fluid, kg/m^3). Example: solve('pump_shaft_power', rho='... kg/m^3', g='... m/s^2', Q='... m^3/s'
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

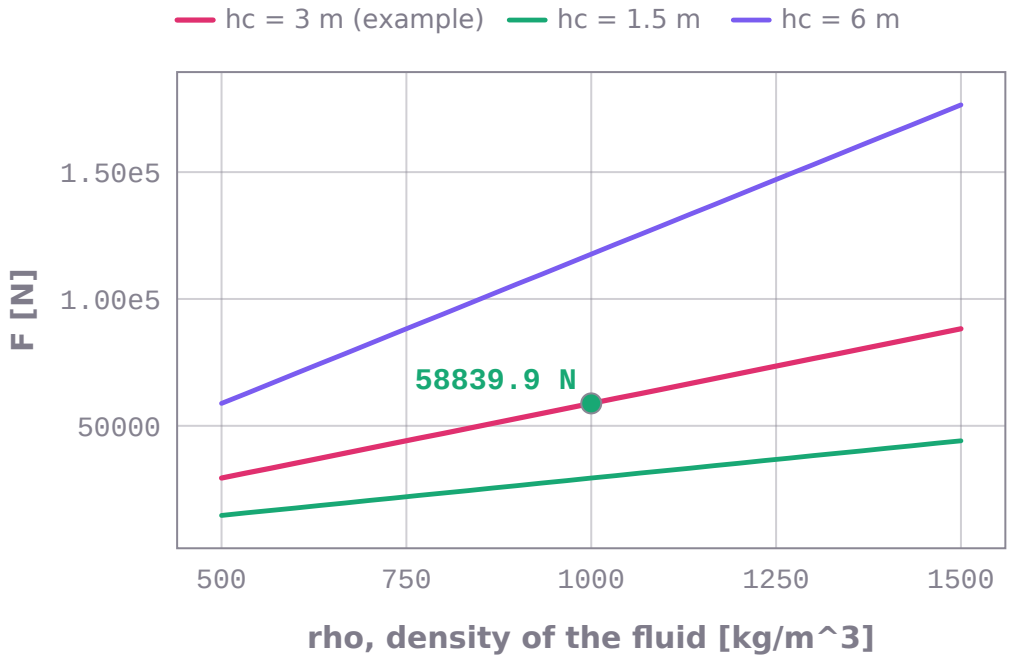
- "A pump with an efficiency of 0.8 lifts water of density 1000 kg/m^3 at a flow rate of 0.03 m^3/s through a total head of 25 m. What shaft power is required?" → 9193.73 W
- "What is the power input to a pump of efficiency 0.75 that delivers 0.01 m^3/s of oil with density 850 kg/m^3 against a head of 40 m?" → 4445.68 W

Hydrostatic force on a submerged plane surface, `hydrostatic_force_plane`

$$F = \rho g h_c A$$

input	meaning	kind / role	unit	default
`F`	resultant hydrostatic force on the surface	force / hydrostatic	`N`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`hc`	depth of the surface's centroid	length / centroid_depth	`m`	
`A`	area of the surface	area / plain	`m^2`	

- **Conditions:** submerged_surface
- **Applies when:** a plane gate, wall or plate wholly submerged in a liquid
- **Fails when:** surfaces partly above the free surface
- **Assumes:** liquid at rest; gauge pressure (atmosphere acts on both sides)
- **Evidence:** theorem



F (resultant hydrostatic force on the surface) against rho (density of the fluid) from 500 kg/m^3 to 1500 kg/m^3. One curve for each value of hc (depth of the surface's centroid): 3 m, 1.5 m, 6 m. Other inputs as in the worked example: A = 2 m^2. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, rho = 1000 kg/m^3, F = 58839.9 N.

```
thermofluids_ai999x.solve('hydrostatic_force_plane', rho='1000 kg/m^3', hc='3 m', A='2
m^2', target='F')
# -> 58839.9 N   (SOLVED)
```

$$F = \rho g h_c A$$
$$F = A g h_c \rho = 58839.9\ \mathrm{N}$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	hydrostatic_force_plane needs rho (density of the fluid, kg/m^3). Example: solve('hydrostatic_force_plane', rho='... kg/m^3', g='... m/s^2',
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

- "A vertical rectangular gate of area 2 m^2 is submerged in water of density 1000 kg/m^3 with the depth of the centroid at 3 m. What is the hydrostatic force on the gate?" → 58839.9 N
- "Find the resultant force of seawater with density 1025 kg/m^3 on a submerged plate of area 1.5 m^2 whose centroid depth is 4 m." → 60310.9 N

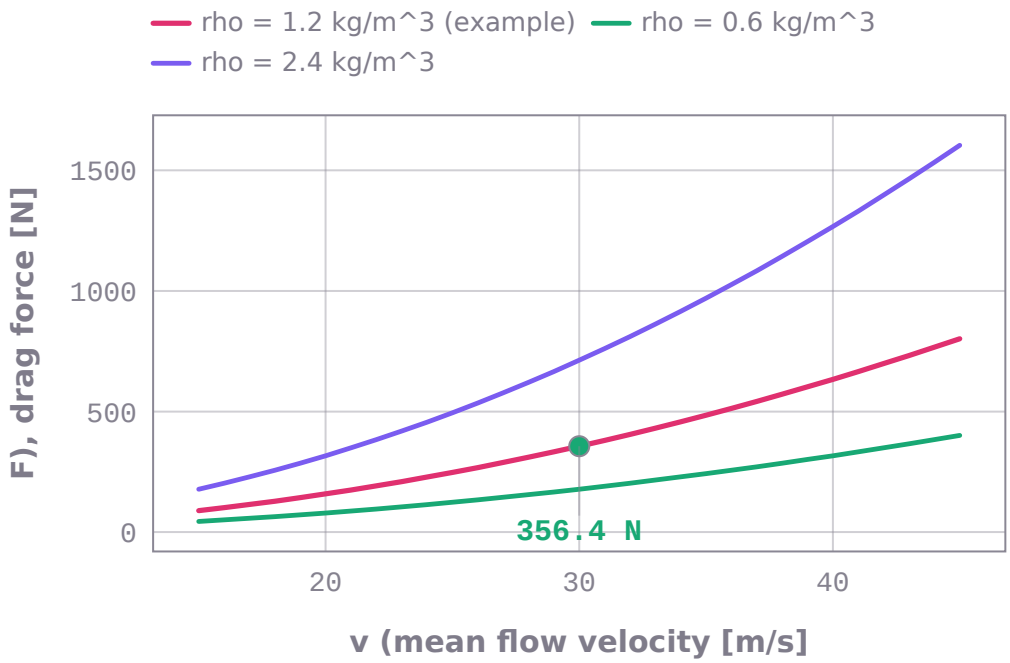
Drag force from the drag coefficient, `drag_force`

$$F = \frac{C_d \rho v^2 A}{2}$$

input	meaning	kind / role	unit	default
`F`	drag force	force / drag	`N`	

input	meaning	kind / role	unit	default
`Cd`	drag coefficient	drag_coefficient / plain	`1`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`v`	mean flow velocity	speed / flow	`m/s`	
`A`	frontal (reference) area	area / frontal	`m^2`	

- **Conditions:** drag
- **Applies when:** a body moving through a fluid with a known drag coefficient
- **Fails when:** compressible flow near Mach 1
- **Assumes:** drag coefficient based on the stated reference area; incompressible flow
- **Evidence:** theorem



F (drag force) against v (mean flow velocity) from 15 m/s to 45 m/s. One curve for each value of rho (density of the fluid): 1.2 kg/m^3, 0.6 kg/m^3, 2.4 kg/m^3. Other inputs as in the worked example: Cd = 0.3 1, A = 2.2 m^2. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, v = 30 m/s, F = 356.4 N.

```
thermofluids_ai999x.solve('drag_force', Cd='0.3 1', rho='1.2 kg/m^3', v='30 m/s', A='2.2
m^2', target='F')
# -> 356.4 N   (SOLVED)
```

$$F = \frac{C_d \rho v^2 A}{2}$$

$$F = \frac{A C_d \rho v^2}{2} = 356.4\ \mathrm{N}$$

invalid call	outcome	message
without `Cd`	NEEDS_INPUT	drag_force needs Cd (drag coefficient, 1). Example: solve('drag_force', Cd='... 1', rho='... kg/m^3', v='... m/s', A='... m^2').
`Cd` in a wrong unit	INVALID_INPUT	Cd is a drag coefficient: its unit must be like 1, not 'kg'.

Questions in words that work:

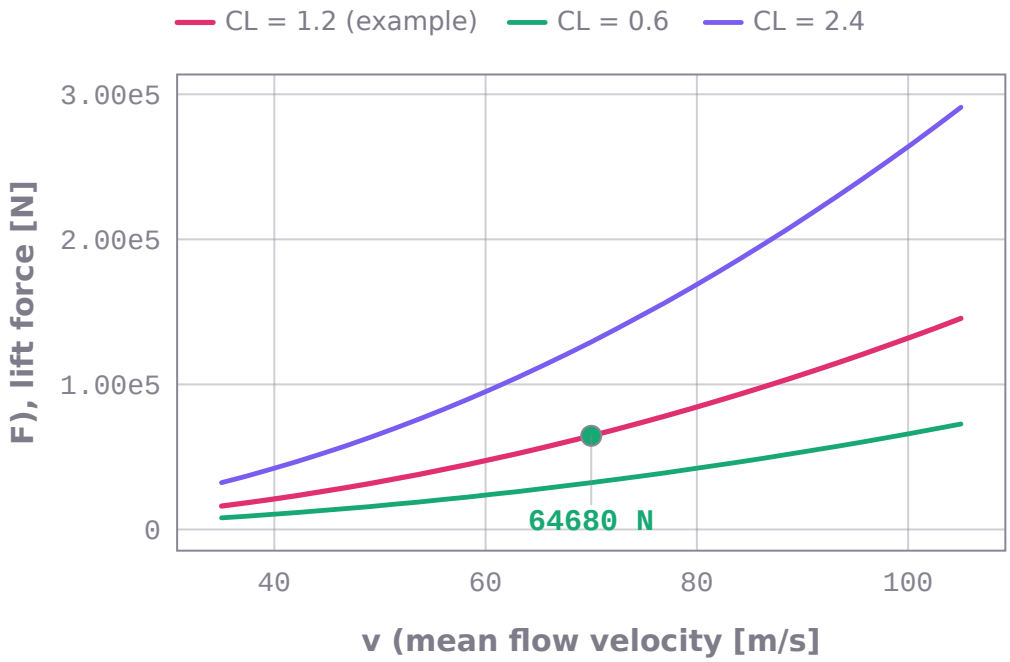
- "A car with a drag coefficient of 0.3 and a frontal area of 2.2 m^2 travels at 30 m/s through air of density 1.2 kg/m^3. What is the drag force?" → 356.4 N
- "Find the aerodynamic drag on a cyclist with drag coefficient 0.9 and frontal area 0.5 m^2 moving at 10 m/s in air with density 1.225 kg/m^3." → 27.5625 N

Lift force from the lift coefficient, `lift_force`

$$F = \frac{C_L \rho v^2 A}{2}$$

input	meaning	kind / role	unit	default
`F`	lift force	force / lift	`N`	
`CL`	lift coefficient	lift_coefficient / plain	`1`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`v`	mean flow velocity	speed / flow	`m/s`	
`A`	wing planform area	area / wing	`m^2`	

- **Conditions:** lift
- **Applies when:** a wing or airfoil with a known lift coefficient
- **Fails when:** stall; compressible flow near Mach 1
- **Assumes:** lift coefficient based on the planform area; incompressible flow
- **Evidence:** theorem



F (lift force) against v (mean flow velocity) from 35 m/s to 105 m/s. One curve for each value of CL (lift coefficient): 1.2, 0.6, 2.4. Other inputs as in the worked example: rho = 1.1 kg/m^3, A = 20 m^2. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, v = 70 m/s, F = 64680 N.

```
thermofluids_ai999x.solve('lift_force', CL='1.2 1', rho='1.1 kg/m^3', v='70 m/s', A='20
m^2', target='F')
# -> 64680 N   (SOLVED)
```

$$F = \frac{C_L \rho v^2 A}{2}$$
$$F = \frac{A C_L \rho v^2}{2} = 64680 \text{ N}$$

invalid call	outcome	message
without `CL`	NEEDS_INPUT	lift_force needs CL (lift coefficient, 1). Example: solve('lift_force', CL='... 1', rho='... kg/m^3', v='... m/s', A='... m^2').
`CL` in a wrong unit	INVALID_INPUT	CL is a lift coefficient: its unit must be like 1, not 'kg'.

Questions in words that work:

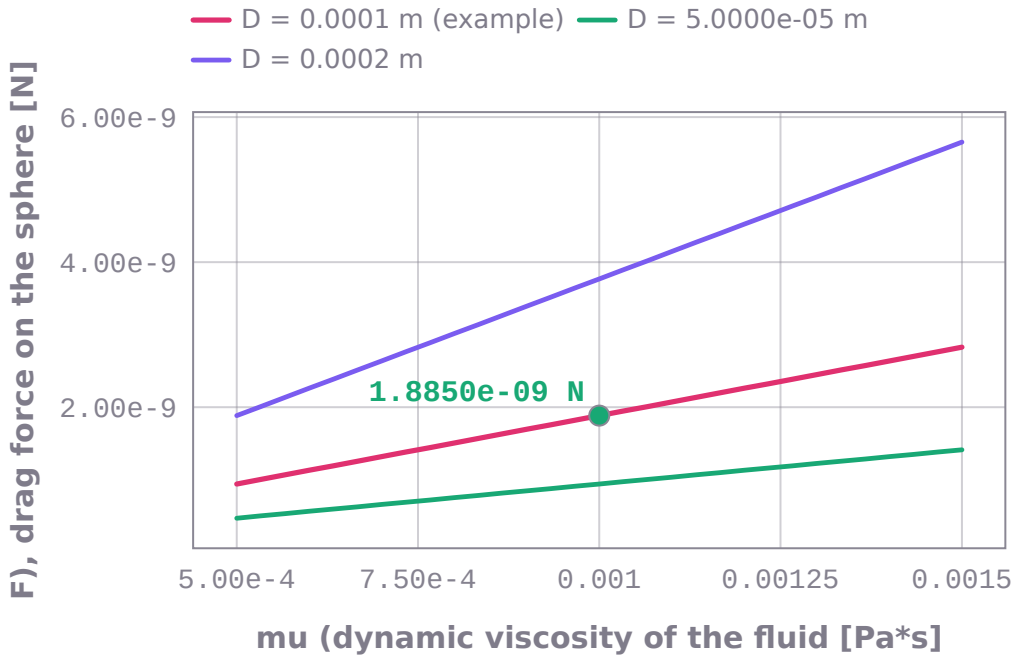
- "An aircraft wing with a wing area of 20 m^2 has a lift coefficient of 1.2 and moves at 70 m/s through air of density 1.1 kg/m^3. What is the lift force?" → 64680 N
- "Find the lift on an airfoil with lift coefficient 0.8 and planform area 1.5 m^2 at a speed of 50 m/s in air of density 1.225 kg/m^3." → 1837.5 N

Stokes drag on a small sphere, `stokes_drag`

$$F = 3\pi\mu D v$$

input	meaning	kind / role	unit	default
`F`	drag force on the sphere	force / drag	`N`	
`mu`	dynamic viscosity of the fluid	dynamic_viscosity / plain	`Pa*s`	
`D`	sphere diameter	length / diameter	`m`	
`v`	mean flow velocity	speed / flow	`m/s`	

- **Conditions:** stokes
- **Applies when:** a small particle or droplet moving slowly through a viscous fluid
- **Fails when:** Re above about 1
- **Assumes:** creeping flow $(Re < 1)$; rigid sphere far from walls
- **Evidence:** theorem



F (drag force on the sphere) against mu (dynamic viscosity of the fluid) from 0.0005 Pa*s to 0.0015 Pa*s. One curve for each value of D (sphere diameter): 0.0001 m, 5.0000e-05 m, 0.0002 m. Other inputs as in the worked example: v = 0.002 m/s. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, mu = 0.001 Pa*s, F = 1.8850e-09 N.

```
thermofluids_ai999x.solve('stokes_drag', mu='0.001 Pa*s', D='0.0001 m', v='0.002 m/s',
target='F')
# -> 1.88496e-09 N   (SOLVED)
```

$$F = 3\pi\mu D v$$
$$F = 3\pi D\mu v = 1.88496e-09\ \mathrm{N}$$

invalid call	outcome	message
without `mu`	NEEDS_INPUT	stokes_drag needs mu (dynamic viscosity of the fluid, Pa*s). Example: solve('stokes_drag', mu='... Pa*s', D='... m', v='... m/s').
`mu` in a wrong unit	INVALID_INPUT	mu is a dynamic viscosity: its unit must be like Pa*s, not 'kg'.

Questions in words that work:

- "Using Stokes' law, what is the drag force on a sphere of diameter 0.0001 m moving at 0.002 m/s through water with viscosity 0.001 Pa*s?" → 1.88496e-09 N
- "A small droplet of diameter 0.00002 m moves at 0.01 m/s through air of viscosity 0.000018 Pa*s in creeping flow. Find the Stokes drag force." → 3.39292e-11 N

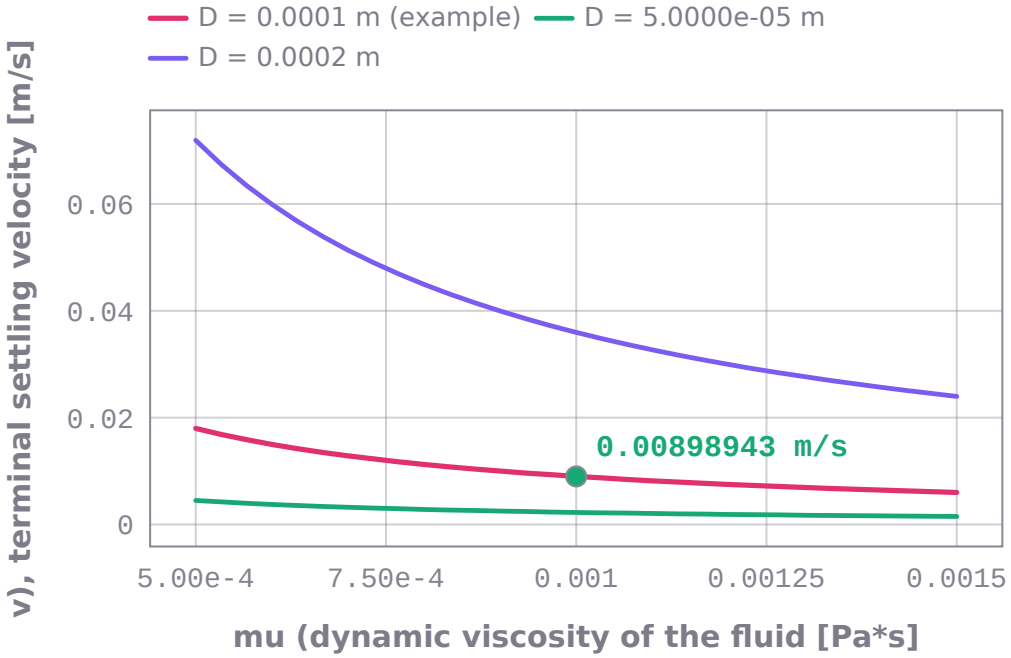
Terminal settling velocity of a small sphere (Stokes), `stokes_terminal_velocity`

$$v = \frac{gD^2\left(\rho_p - \rho\right)}{18\mu}$$

input	meaning	kind / role	unit	default
`v`	terminal settling velocity	speed / terminal	`m/s`	

input	meaning	kind / role	unit	default
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`D`	sphere diameter	length / diameter	`m`	
`rho_p`	density of the particle	density / particle	`kg/m^3`	
`rho`	density of the fluid	density / fluid	`kg/m^3`	
`mu`	dynamic viscosity of the fluid	dynamic_viscosity / plain	`Pa*s`	

- **Conditions:** stokes
- **Applies when:** fine particles or droplets settling in a viscous fluid
- **Fails when:** Re above about 1; hindered settling in dense suspensions
- **Assumes:** creeping flow $(Re < 1)$; particle denser than the fluid; rigid sphere
- **Evidence:** theorem



v (terminal settling velocity) against mu (dynamic viscosity of the fluid) from 0.0005 Pa*s to 0.0015 Pa*s. One curve for each value of D (sphere diameter): 0.0001 m, 5.0000e-05 m, 0.0002 m. Other inputs as in the worked example: rho_p = 2650 kg/m^3, rho = 1000 kg/m^3. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, mu = 0.001 Pa*s, v = 0.00898943 m/s.

```
thermofluids_ai999x.solve('stokes_terminal_velocity', D='0.0001 m', rho_p='2650 kg/m^3',
rho='1000 kg/m^3', mu='0.001 Pa*s', target='v')
# -> 0.00898943 m/s   (SOLVED)
```

$$v = \frac{gD^2\left(\rho_p - \rho\right)}{18\mu}$$

$$v = \frac{D^2 g\left(-\rho + \rho_p\right)}{18\mu} = 0.00898943\ \mathrm{m/s}$$

invalid call	outcome	message
without `D`	NEEDS_INPUT	stokes_terminal_velocity needs D (sphere diameter, m). Example: solve('stokes_terminal_velocity', g='... m/s^2', D='... m', rho_p='... kg/m^

invalid call	outcome	message
`D` in a wrong unit	INVALID_INPUT	D is a length: its unit must be like m, not 'kg'.

Questions in words that work:

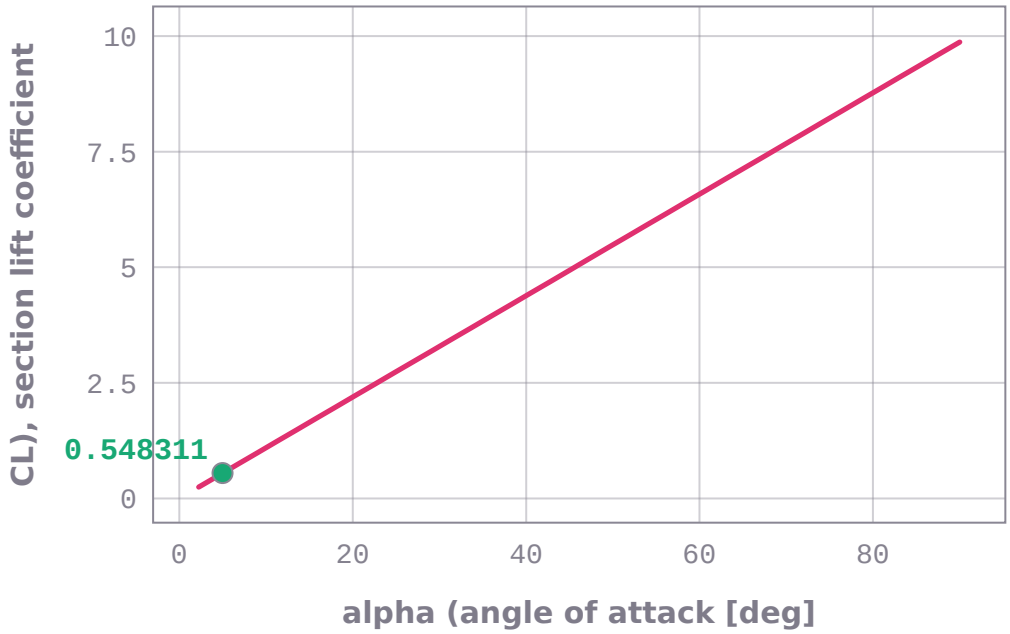
- "A sand grain of diameter 0.0001 m and particle density 2650 kg/m^3 settles in water of density 1000 kg/m^3 and viscosity 0.001 Pa*s. Assuming Stokes flow, what is its terminal velocity?" → 0.00898943 m/s
- "Find the settling velocity of a steel ball of diameter 0.002 m and density of the ball 7800 kg/m^3 in glycerin of density 1260 kg/m^3 and viscosity 1.4 Pa*s, using Stokes' law." → 0.0101802 m/s

Lift coefficient of a thin symmetric airfoil, `thin_airfoil_lift_coefficient`

$$C_L = 2\pi\alpha$$

input	meaning	kind / role	unit	default
`CL`	section lift coefficient	lift_coefficient / plain	`1`	
`alpha`	angle of attack	angle / attack	`rad`	

- **Conditions:** thin_airfoil
- **Applies when:** small angles of attack below stall
- **Fails when:** near or beyond stall; cambered airfoils (add the zero-lift angle)
- **Assumes:** thin symmetric airfoil; small angle of attack; inviscid incompressible flow
- **Evidence:** theorem



CL (section lift coefficient) against alpha (angle of attack) from 0 deg to 90 deg. 40 points computed by thermofluids-ai999x; marked: the worked example, alpha = 5 deg, CL = 0.548311.

```
thermofluids_ai999x.solve('thin_airfoil_lift_coefficient', alpha='5 deg', target='CL')
# -> 0.548311 1   (SOLVED)
```

$$C_L = 2\pi\alpha$$
$$C_L = 2\pi\alpha = 0.548311\ 1$$

invalid call	outcome	message
without `alpha`	NEEDS_INPUT	thin_airfoil_lift_coefficient needs alpha (angle of attack, rad). Example: solve('thin_airfoil_lift_coefficient', alpha='... rad').
`alpha` in a wrong unit	INVALID_INPUT	alpha is a angle: its unit must be like rad, not 'kg'.

Questions in words that work:

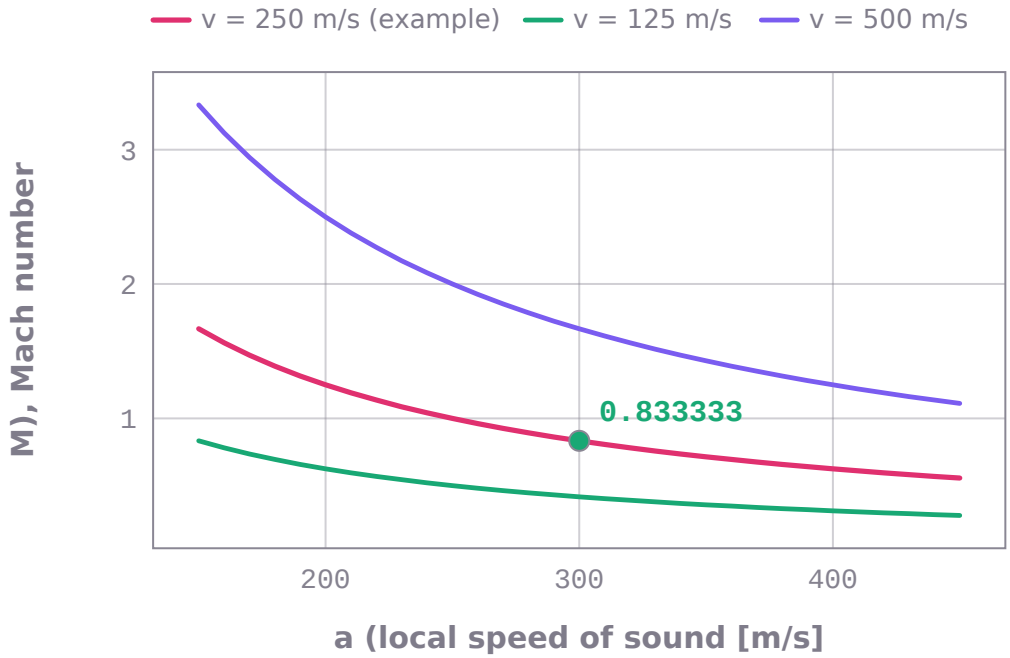
- "Using thin airfoil theory, what is the lift coefficient of a symmetric airfoil at an angle of attack of 5 deg?" → 0.548311 1
- "A thin aerofoil is at an angle of attack of 0.1 rad. Find its lift coefficient." → 0.628319 1

Mach number, `mach_number`

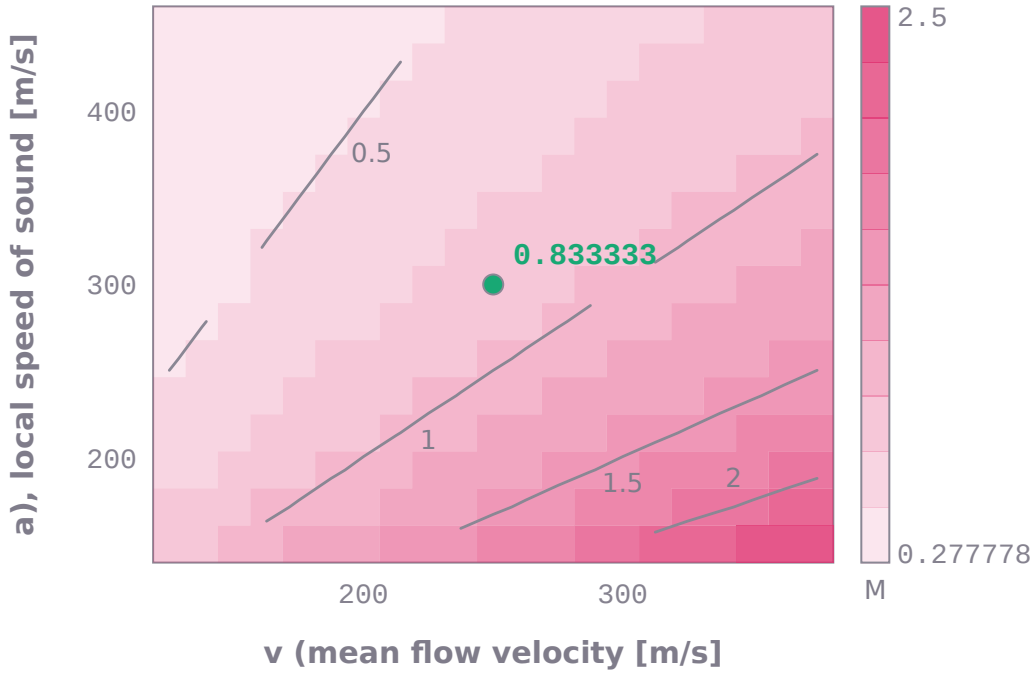
$$M = \frac{v}{a}$$

input	meaning	kind / role	unit	default
`M`	Mach number	mach_number / plain	`1`	
`v`	mean flow velocity	speed / flow	`m/s`	
`a`	local speed of sound	speed / sound	`m/s`	

- **Conditions:** compressible
- **Applies when:** any flow or flight speed compared with the local speed of sound
- **Fails when:** a taken at a different temperature
- **Assumes:** v and a measured in the same frame and state
- **Evidence:** theorem



M (Mach number) against a (local speed of sound) from 150 m/s to 450 m/s. One curve for each value of v (mean flow velocity): 250 m/s, 125 m/s, 500 m/s. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, a = 300 m/s, M = 0.833333.



M (Mach number) over v (mean flow velocity) from 125 m/s to 375 m/s and a (local speed of sound) from 150 m/s to 450 m/s: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 0.277778 to dark 2.5); iso-lines at M = 0.5, 1, 1.5, 2. Marked: the worked example, v = 250 m/s, a = 300 m/s, M = 0.833333.

```
thermofluids_ai999x.solve('mach_number', v='250 m/s', a='300 m/s', target='M')
# -> 0.833333 1   (SOLVED)
```

$$M = \frac{v}{a}$$

$$M = \frac{v}{a} = 0.833333\,1$$

invalid call	outcome	message
without `v`	NEEDS_INPUT	mach_number needs v (mean flow velocity, m/s). Example: solve('mach_number', v='... m/s', a='... m/s').
`v` in a wrong unit	INVALID_INPUT	v is a speed: its unit must be like m/s, not 'kg'.

Questions in words that work:

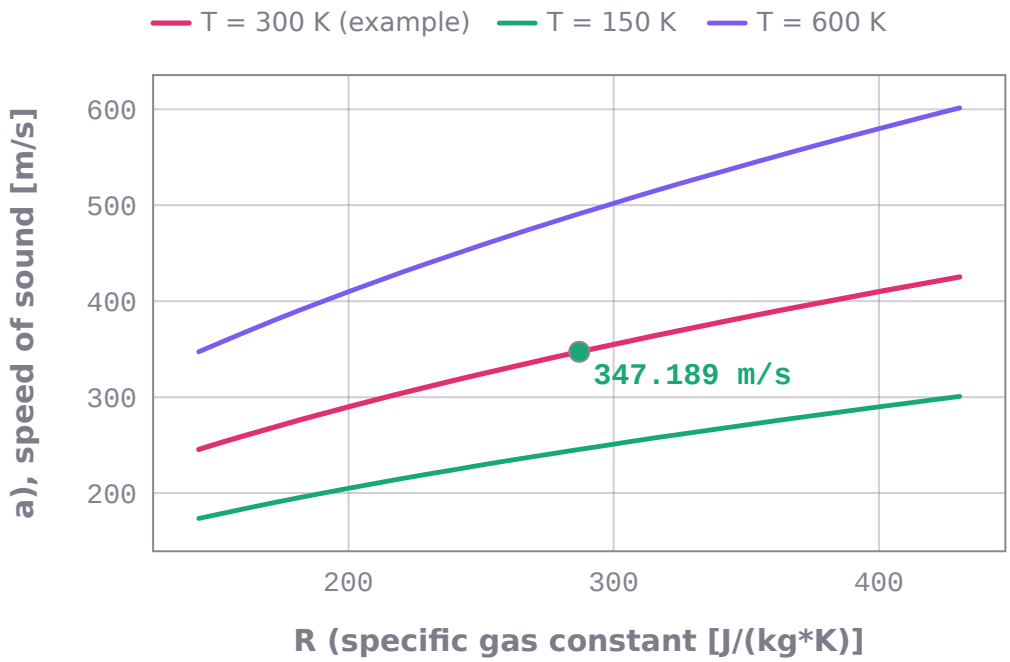
- "An aircraft flies at 250 m/s where the speed of sound is 300 m/s. What is its Mach number?" → 0.833333 1
- "Find the Mach number of a projectile moving at 680 m/s through air in which the speed of sound is 340 m/s." → 2 1

Speed of sound in an ideal gas, `speed_of_sound_ideal_gas`

$$a = \sqrt{\gamma R T}$$

input	meaning	kind / role	unit	default
`a`	speed of sound	speed / sound	`m/s`	
`gamma`	ratio of specific heats	heat_capacity_ratio / plain	`1`	
`R`	specific gas constant	specific_gas_constant / plain	`J/(kg*K)`	
`T`	absolute temperature of the gas	temperature / plain	`K`	

- **Conditions:** compressible
- **Applies when:** gases at moderate pressure
- **Fails when:** dense or real gases near saturation; liquids
- **Assumes:** ideal gas; small-amplitude (acoustic) waves
- **Evidence:** theorem



a (speed of sound) against R (specific gas constant) from 143.5 J/(kg*K) to 430.5 J/(kg*K). One curve for each value of T (absolute temperature of the gas): 300 K, 150 K, 600 K. Other inputs as in the worked example: gamma = 1.4 1. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, R = 287 J/(kg*K), a = 347.189 m/s.

```
thermofluids_ai999x.solve('speed_of_sound_ideal_gas', gamma='1.4 1', R='287 J/(kg*K)',
T='300 K', target='a')
# -> 347.189 m/s   (SOLVED)
```

$$a = \sqrt{\gamma R T}$$
$$a = \sqrt{R}\sqrt{T}\sqrt{\gamma} = 347.189\ \mathrm{m/s}$$

invalid call	outcome	message
without `gamma`	NEEDS_INPUT	speed_of_sound_ideal_gas needs gamma (ratio of specific heats, 1). Example: solve('speed_of_sound_ideal_gas', gamma='... 1', R='... J/(kg*K)
`gamma` in a wrong unit	INVALID_INPUT	gamma is a heat capacity ratio: its unit must be like 1, not 'kg'.

Questions in words that work:

- "What is the speed of sound in air at a temperature of 300 K, with a specific heat ratio of 1.4 and a specific gas constant of 287 J/(kg*K)?" → 347.189 m/s
- "Find the speed of sound in helium at 273 K if its ratio of specific heats is 1.667 and its gas constant is 2077 J/(kg*K)." → 972.226 m/s

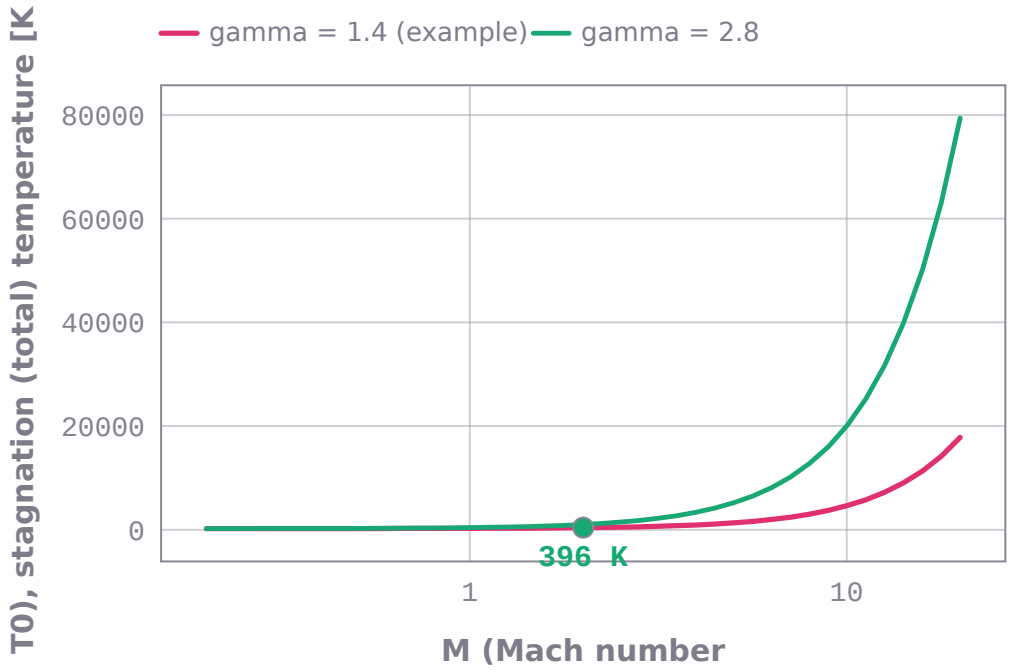
Stagnation temperature from the Mach number, `stagnation_temperature_mach`

$$T_0 = T\left(1 + \frac{(\gamma - 1)M^2}{2}\right)$$

input	meaning	kind / role	unit	default
`T0`	stagnation (total) temperature	temperature / stagnation	`K`	

input	meaning	kind / role	unit	default
`T`	static temperature	temperature / static	`K`	
`gamma`	ratio of specific heats	heat_capacity_ratio / plain	`1`	
`M`	Mach number	mach_number / plain	`1`	

- **Conditions:** compressible
- **Applies when:** any adiabatic flow of an ideal gas
- **Fails when:** heat transfer to or from the stream; high-temperature real-gas effects
- **Assumes:** ideal gas with constant specific heats; adiabatic deceleration
- **Evidence:** theorem



T0 (stagnation (total) temperature) against M (Mach number) from 0.2 to 20 on a logarithmic axis. One curve for each value of gamma (ratio of specific heats): 1.4, 2.8. Other inputs as in the worked example: T = 220 K. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, M = 2, T0 = 396 K.

```
thermofluids_ai999x.solve('stagnation_temperature_mach', T='220 K', gamma='1.4 1', M='2 1',
target='T0')
# -> 396 K   (SOLVED)
```

$$T_0 = T\left(1 + \frac{(\gamma - 1)M^2}{2}\right)$$

$$T_0 = \frac{T\left(M^2\gamma - M^2 + 2\right)}{2} = 396\ \mathrm{K}$$

invalid call	outcome	message
without `T`	NEEDS_INPUT	stagnation_temperature_mach needs T (static temperature, K). Example: solve('stagnation_temperature_mach', T='... K', gamma='... 1', M='...
`T` in a wrong unit	INVALID_INPUT	T is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

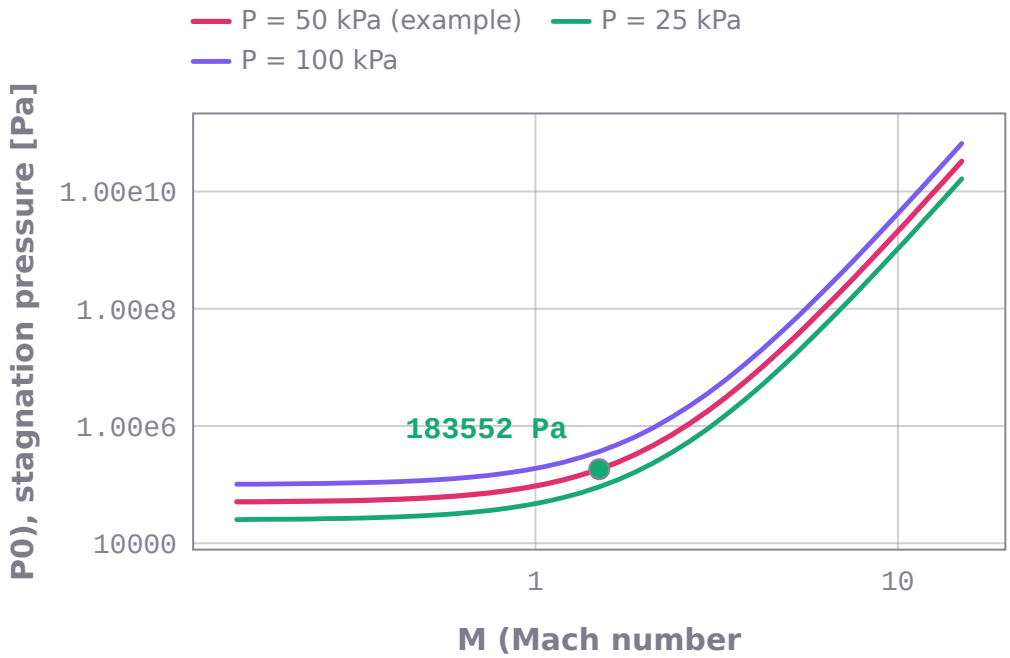
- "A gas with a specific heat ratio of 1.4 flows at a Mach number of 2 with a static temperature of 220 K. What is the stagnation temperature?" → 396 K
- "Find the total temperature of a gas stream with static temperature 300 K, Mach number 0.8 and ratio of specific heats 1.3." → 328.8 K

Isentropic stagnation pressure of air from the Mach number, `stagnation_pressure_air`

$$P_0 = P\left(1 + \frac{M^2}{5}\right)^{\frac{7}{2}}$$

input	meaning	kind / role	unit	default
`P0`	stagnation pressure	pressure / stagnation	`Pa`	
`P`	static pressure	pressure / static	`Pa`	
`M`	Mach number	mach_number / plain	`1`	

- **Conditions:** compressible, air
- **Not when:** non_air
- **Applies when:** isentropic flow of air
- **Fails when:** across shock waves; gases other than air
- **Assumes:** air as an ideal gas with $\gamma = 1.4$; isentropic deceleration
- **Evidence:** theorem



P0 (stagnation pressure) against M (Mach number) from 0.15 to 15 on a logarithmic axis, logarithmic output axis. One curve for each value of P (static pressure): 50 kPa, 25 kPa, 100 kPa. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, M = 1.5, P0 = 183552 Pa.

```
thermofluids_ai999x.solve('stagnation_pressure_air', P='50 kPa', M='1.5 1', target='P0')
# -> 183552 Pa   (SOLVED)
```

$$P_0 = P\left(1 + \frac{M^2}{5}\right)^{\frac{7}{2}}$$

$$P_0 = \frac{\sqrt{5}P\left(M^2 + 5\right)^{\frac{7}{2}}}{625} = 183552\ \mathrm{Pa}$$

invalid call	outcome	message
without `P`	NEEDS_INPUT	stagnation_pressure_air needs P (static pressure, Pa). Example: solve('stagnation_pressure_air', P='... Pa', M='... 1').
`P` in a wrong unit	INVALID_INPUT	P is a pressure: its unit must be like Pa, not 'kg'.

Questions in words that work:

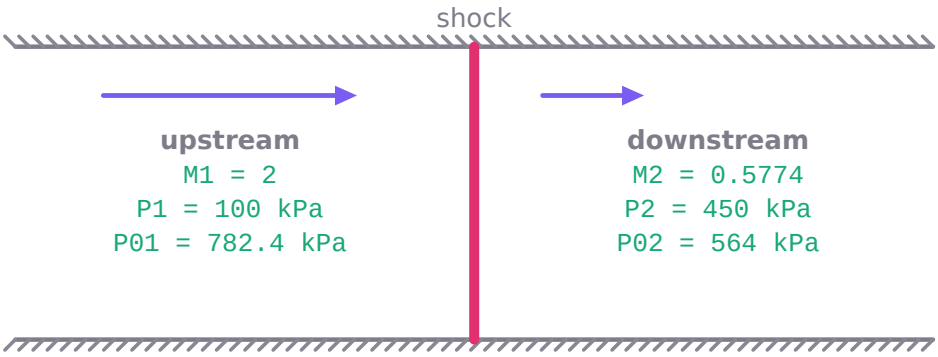
- "Air flows isentropically at a Mach number of 1.5 with a static pressure of 50 kPa. What is the stagnation pressure?" → 183552 Pa
- "Find the stagnation pressure of an air stream with static pressure 101325 Pa moving at a Mach number of 0.6." → 129240 Pa

Mach number behind a normal shock in air, `normal_shock_mach_air`

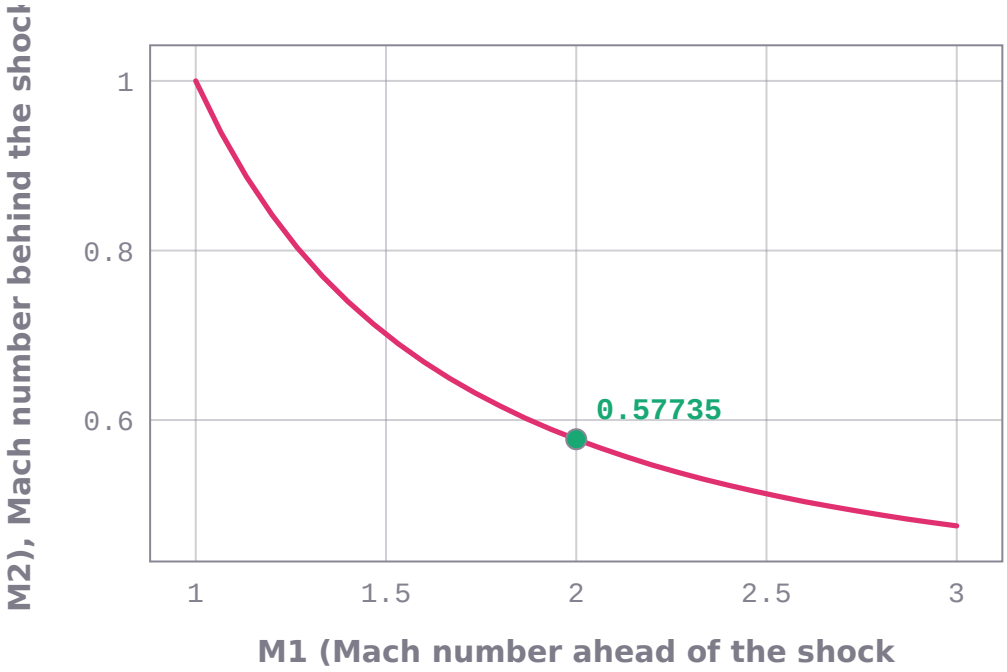
$$M_2 = \sqrt{\frac{M_1^2 + 5}{7M_1^2 - 1}}$$

input	meaning	kind / role	unit	default
`M2`	Mach number behind the shock	mach_number / downstream	`1`	
`M1`	Mach number ahead of the shock	mach_number / upstream	`1`	

- **Conditions:** shock, air
- **Not when:** non_air
- **Applies when:** a stationary normal shock with supersonic upstream flow
- **Fails when:** oblique shocks; $M_1 \leq 1$ (no shock)
- **Assumes:** air as an ideal gas with $\gamma = 1.4$; steady normal shock; adiabatic
- **Evidence:** theorem



A normal shock in air (γ = 1.4) at M1 = 2 with an upstream static pressure of 100 kPa (the worked example): the package gives M2 = 0.57735 and P2 = 450 kPa; the stagnation pressures from its stagnation-pressure relation, 782.445 kPa before and 564.044 kPa after (the loss across the shock). Marked: M2, P2 and both stagnation pressures.



M2 (Mach number behind the shock) against M1 (Mach number ahead of the shock) from 1 to 3. 31 points computed by thermofluids-ai999x; marked: the worked example, M1 = 2, M2 = 0.57735.

```
thermofluids_ai999x.solve('normal_shock_mach_air', M1='2 1', target='M2')
# -> 0.57735 1   (SOLVED)
```

$$M_2 = \sqrt{\frac{M_1^2 + 5}{7M_1^2 - 1}}$$

$$M_2 = \sqrt{M_1^2 + 5}\sqrt{\frac{1}{7M_1^2 - 1}} = 0.57735\ 1$$

invalid call	outcome	message
without `M1`	NEEDS_INPUT	normal_shock_mach_air needs M1 (Mach number ahead of the shock, 1). Example: solve('normal_shock_mach_air', M1='... 1').

invalid call	outcome	message
`M1` in a wrong unit	INVALID_INPUT	M1 is a mach number: its unit must be like 1, not 'kg'.

Questions in words that work:

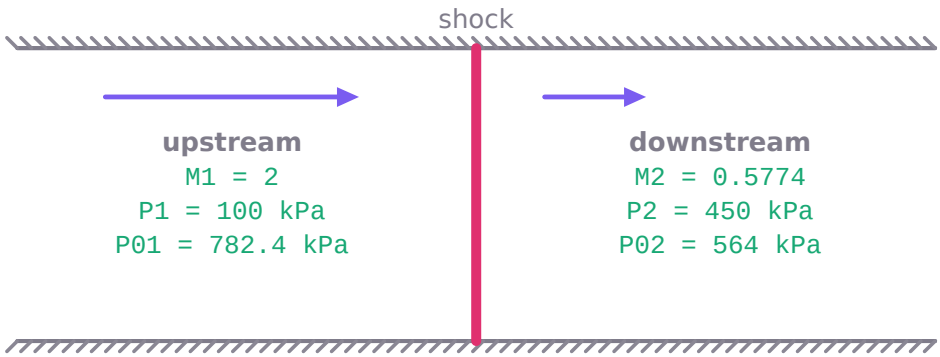
- "Air with an upstream Mach number of 2 passes through a normal shock. What is the downstream Mach number?" → 0.57735 1
- "A normal shock wave forms in air where the upstream Mach number is 3. Find the Mach number behind the shock." → 0.475191 1

Static pressure behind a normal shock in air, `normal_shock_pressure_air`

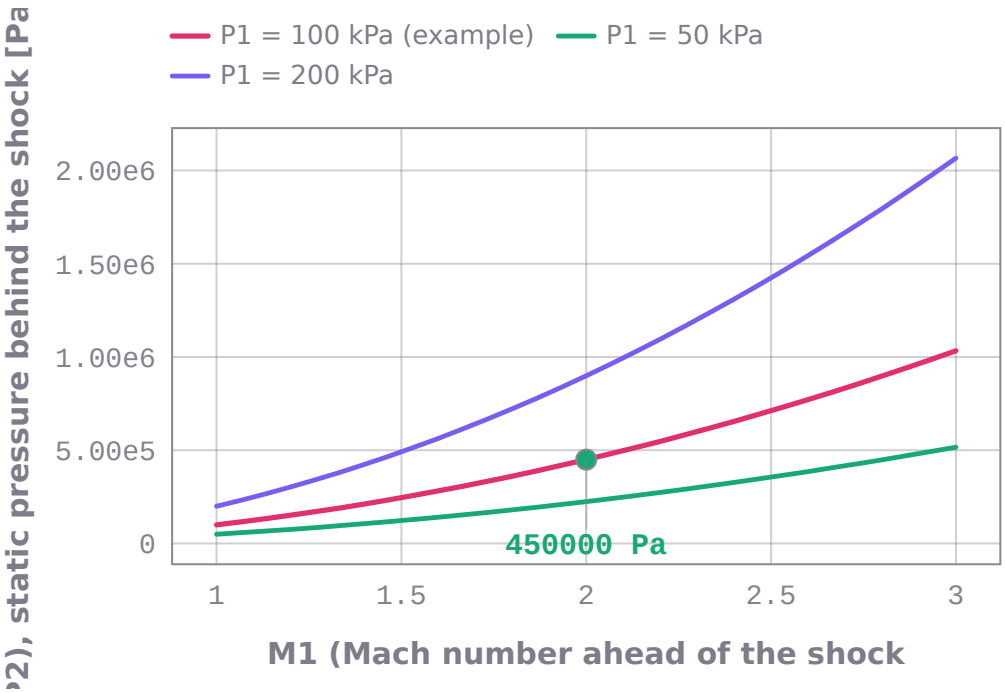
$$P_2 = \frac{P_1\left(7M_1^2 - 1\right)}{6}$$

input	meaning	kind / role	unit	default
`P2`	static pressure behind the shock	pressure / downstream	`Pa`	
`P1`	static pressure ahead of the shock	pressure / upstream	`Pa`	
`M1`	Mach number ahead of the shock	mach_number / upstream	`1`	

- **Conditions:** shock, air
- **Not when:** non_air
- **Applies when:** a stationary normal shock with supersonic upstream flow
- **Fails when:** oblique shocks; $M_1 \leq 1$ (no shock)
- **Assumes:** air as an ideal gas with $\gamma = 1.4$; steady normal shock; adiabatic
- **Evidence:** theorem



A normal shock in air (γ = 1.4) at M1 = 2 with an upstream static pressure of 100 kPa (the worked example): the package gives M2 = 0.57735 and P2 = 450 kPa; the stagnation pressures from its stagnation-pressure relation, 782.445 kPa before and 564.044 kPa after (the loss across the shock). Marked: M2, P2 and both stagnation pressures.



P2 (static pressure behind the shock) against M1 (Mach number ahead of the shock) from 1 to 3. One curve for each value of P1 (static pressure ahead of the shock): 100 kPa, 50 kPa, 200 kPa. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, M1 = 2, P2 = 450000 Pa.

```
thermofluids_ai999x.solve('normal_shock_pressure_air', P1='100 kPa', M1='2 1', target='P2')
# -> 450000 Pa   (SOLVED)
```

$$P_2 = \frac{P_1 \left(7 M_1^2 - 1\right)}{6}$$

$$P_2 = \frac{P_1 \left(7 M_1^2 - 1\right)}{6} = 450000 \text{ Pa}$$

invalid call	outcome	message
without `P1`	NEEDS_INPUT	normal_shock_pressure_air needs P1 (static pressure ahead of the shock, Pa). Example: solve('normal_shock_pressure_air', P1='... Pa', M1='..
`P1` in a wrong unit	INVALID_INPUT	P1 is a pressure: its unit must be like Pa, not 'kg'.

Questions in words that work:

- "Air with an upstream pressure of 100 kPa and an upstream Mach number of 2 passes through a normal shock. What is the downstream pressure?" → 450000 Pa
- "Find the pressure behind the shock for a normal shock in air with an upstream Mach number of 1.5 and a pressure ahead of the shock of 80000 Pa." → 196667 Pa

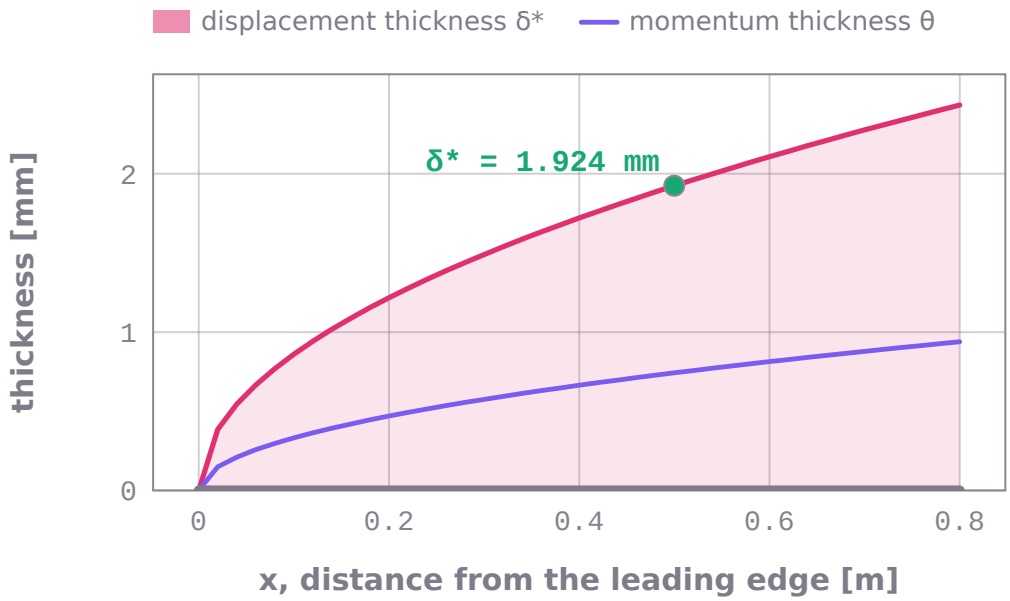
Displacement thickness of a laminar flat-plate boundary layer,

`blasius_displacement_thickness`

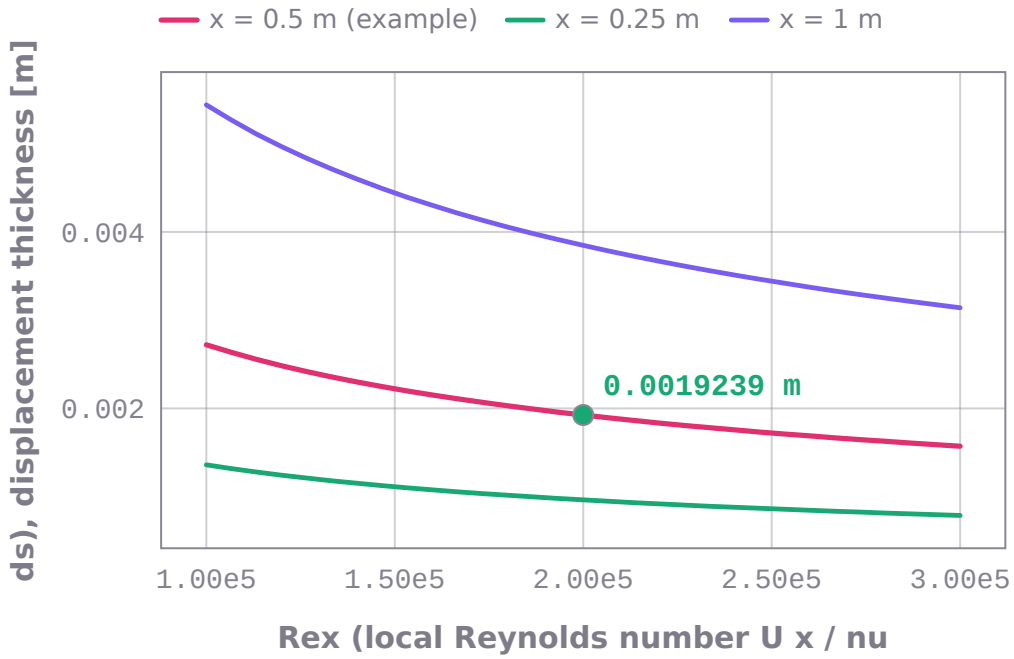
$$d_s = \frac{1.7207876575205 x}{\sqrt{\mathrm{Rex}}}$$

input	meaning	kind / role	unit	default
`ds`	displacement thickness	length / displacement_thickness	`m`	
`x`	distance from the leading edge	length / plate_distance	`m`	
`Rex`	local Reynolds number U x / nu	reynolds_number / local	`1`	

- **Conditions:** flat_plate, laminar
- **Not when:** turbulent
- **Applies when:** Re_x below about 500000
- **Fails when:** turbulent boundary layers; pressure gradients
- **Assumes:** steady laminar flow over a flat plate; zero pressure gradient
- **Evidence:** theorem



Laminar boundary layer on a flat plate: displacement and momentum thickness from the package's Blasius relations at 40 positions, with the local Reynolds number growing in proportion to x (Rex = 200000 at x = 0.5 m, the worked example). Marked: the example point, δ* = 1.9239 mm at x = 0.5 m.



ds (displacement thickness) against Rex (local Reynolds number U x / nu) from 100000 to 300000. One curve for each value of x (distance from the leading edge): 0.5 m, 0.25 m, 1 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Rex = 200000, ds = 0.0019239 m.

```
thermofluids_ai999x.solve('blasius_displacement_thickness', x='0.5 m', Rex='200000 1',
target='ds')
# -> 0.0019239 m   (SOLVED)
```

$$d_s = \frac{1.7207876575205x}{\sqrt{\mathrm{Rex}}}$$

$$d_s = \frac{3441575315041x}{2000000000000\sqrt{\mathrm{Rex}}} = 0.0019239\ \mathrm{m}$$

invalid call	outcome	message
without `x`	NEEDS_INPUT	blasius_displacement_thickness needs x (distance from the leading edge, m). Example: solve('blasius_displacement_thickness', x='... m', Rex=
`x` in a wrong unit	INVALID_INPUT	x is a length: its unit must be like m, not 'kg'.

Questions in words that work:

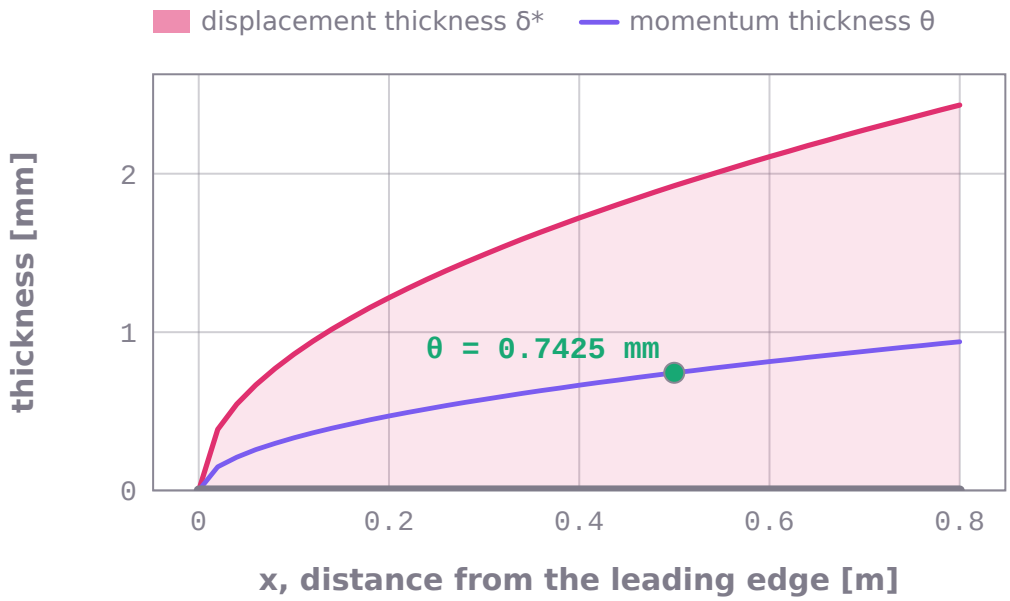
- "Air flows over a flat plate with a laminar boundary layer. At a distance from the leading edge of 0.5 m the local Reynolds number is 200000. What is the displacement thickness?" → 0.0019239 m
- "Find the displacement thickness of the laminar boundary layer on a flat plate 0.2 m from the leading edge where the local Reynolds number is 50000." → 0.00153912 m

Momentum thickness of a laminar flat-plate boundary layer, `blasius_momentum_thickness`

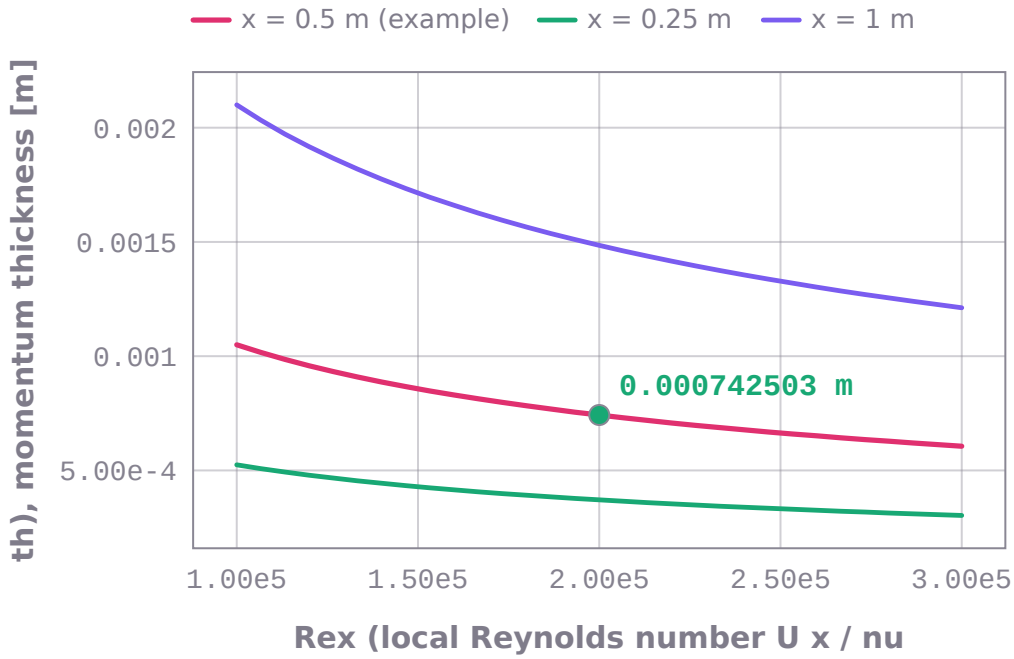
$$t_h = \frac{0.6641146724303926x}{\sqrt{\mathrm{Rex}}}$$

input	meaning	kind / role	unit	default
`th`	momentum thickness	length / momentum_thickness	`m`	
`x`	distance from the leading edge	length / plate_distance	`m`	
`Rex`	local Reynolds number U x / nu	reynolds_number / local	`1`	

- **Conditions:** flat_plate, laminar
- **Not when:** turbulent
- **Applies when:** Re_x below about 500000
- **Fails when:** turbulent boundary layers; pressure gradients
- **Assumes:** steady laminar flow over a flat plate; zero pressure gradient
- **Evidence:** theorem



Laminar boundary layer on a flat plate: displacement and momentum thickness from the package's Blasius relations at 40 positions, with the local Reynolds number growing in proportion to x (Rex = 200000 at x = 0.5 m, the worked example). Marked: the example point, θ = 0.742503 mm at x = 0.5 m.



th (momentum thickness) against Rex (local Reynolds number U x / nu) from 100000 to 300000. One curve for each value of x (distance from the leading edge): 0.5 m, 0.25 m, 1 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Rex = 200000, th = 0.000742503 m.

```
thermofluids_ai999x.solve('blasius_momentum_thickness', x='0.5 m', Rex='200000 1',
target='th')
# -> 0.000742503 m   (SOLVED)
```

$$t_h = \frac{0.6641146724303926\,x}{\sqrt{\mathrm{Rex}}}$$

$$t_h = \frac{3320573362151963\,x}{5000000000000000\sqrt{\mathrm{Rex}}} = 0.000742503\ \mathrm{m}$$

invalid call	outcome	message
without `x`	NEEDS_INPUT	blasius_momentum_thickness needs x (distance from the leading edge, m). Example: solve('blasius_momentum_thickness', x='... m', Rex='... 1')
`x` in a wrong unit	INVALID_INPUT	x is a length: its unit must be like m, not 'kg'.

Questions in words that work:

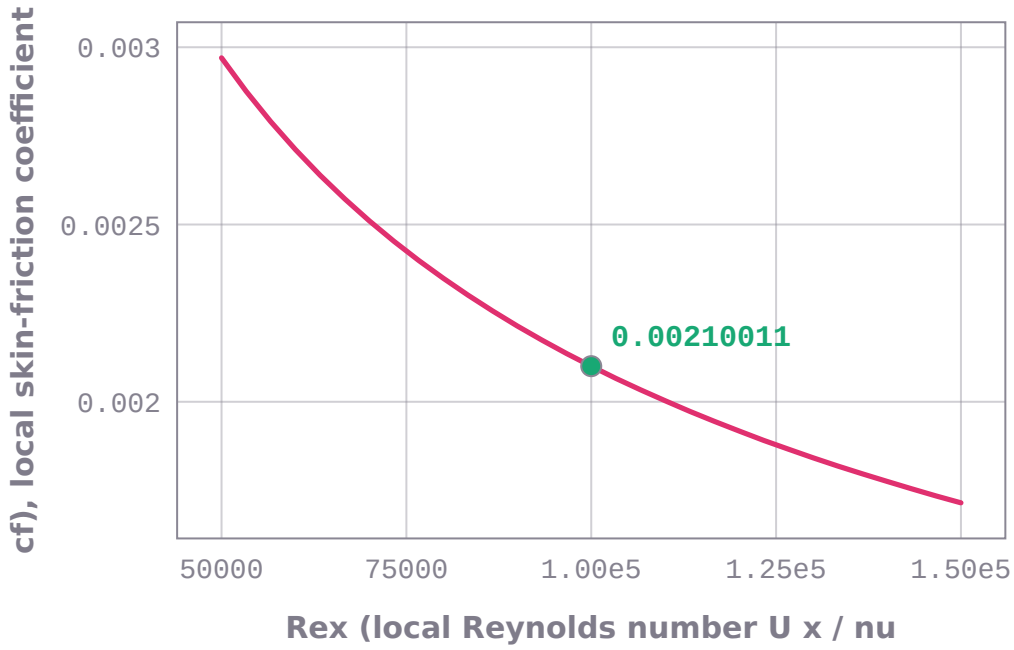
- "A laminar boundary layer grows on a flat plate. At a distance from the leading edge of 0.5 m the local Reynolds number is 200000. What is the momentum thickness?" → 0.000742503 m
- "Find the momentum thickness of a laminar flat plate boundary layer 0.3 m from the leading edge if the local Reynolds number is 90000." → 0.000664115 m

Local skin-friction coefficient of a laminar flat plate, `blasius_local_skin_friction`

$$c_f = \frac{0.6641146724303926}{\sqrt{\mathrm{Rex}}}$$

input	meaning	kind / role	unit	default
`cf`	local skin-friction coefficient	skin_friction_coefficient / local	`1`	
`Rex`	local Reynolds number U x / nu	reynolds_number / local	`1`	

- **Conditions:** flat_plate, laminar
- **Not when:** turbulent
- **Applies when:** Re_x below about 500000
- **Fails when:** turbulent boundary layers; pressure gradients
- **Assumes:** steady laminar flow over a flat plate; zero pressure gradient
- **Evidence:** theorem



cf (local skin-friction coefficient) against Rex (local Reynolds number U x / nu) from 50000 to 150000. 31 points computed by thermofluids-ai999x; marked: the worked example, Rex = 100000, cf = 0.00210011.

```
thermofluids_ai999x.solve('blasius_local_skin_friction', Rex='100000 1', target='cf')
# -> 0.00210011 1   (SOLVED)
```

$$c_f = \frac{0.6641146724303926}{\sqrt{\mathrm{Rex}}}$$

$$c_f = \frac{3320573362151963}{5000000000000000\sqrt{\mathrm{Rex}}} = 0.00210011\ 1$$

invalid call	outcome	message
without `Rex`	NEEDS_INPUT	blasius_local_skin_friction needs Rex (local Reynolds number U x / nu, 1). Example: solve('blasius_local_skin_friction', Rex='... 1').
`Rex` in a wrong unit	INVALID_INPUT	Rex is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

- "For laminar flow over a flat plate, what is the local skin friction coefficient where the local Reynolds number is 100000?" → 0.00210011 1

- "Find the local skin-friction coefficient in a laminar boundary layer on a flat plate at a local Reynolds number of 40000." → 0.00332057 1

Average skin-friction coefficient of a laminar flat plate, `blasius_average_skin_friction`

$$\mathrm{cfm} = \frac{1.3282293448607851}{\sqrt{\mathrm{ReL}}}$$

input	meaning	kind / role	unit	default
`cfm`	average skin-friction coefficient over the plate	skin_friction_coefficient / average	`1`	
`ReL`	Reynolds number based on the plate length	reynolds_number / plate	`1`	

- **Conditions:** flat_plate, laminar
- **Not when:** turbulent
- **Applies when:** Re_L below about 500000
- **Fails when:** transition on the plate; pressure gradients
- **Assumes:** laminar boundary layer over the whole plate; zero pressure gradient
- **Evidence:** theorem



cfm (average skin-friction coefficient over the plate) against ReL (Reynolds number based on the plate length) from 125000 to 375000. 31 points computed by thermofluids-ai999x; marked: the worked example, ReL = 250000, cfm = 0.00265646.

```
thermofluids_ai999x.solve('blasius_average_skin_friction', ReL='250000 1', target='cfm')
# -> 0.00265646 1   (SOLVED)
```

$$\mathrm{cfm} = \frac{1.3282293448607851}{\sqrt{\mathrm{ReL}}}$$

$$\mathrm{cfm} = \frac{13282293448607851}{10000000000000000\sqrt{\mathrm{ReL}}} = 0.00265646\ 1$$

invalid call	outcome	message
without `ReL`	NEEDS_INPUT	blasius_average_skin_friction needs ReL (Reynolds number based on the plate length, 1). Example: solve('blasius_average_skin_friction', ReL=
`ReL` in a wrong unit	INVALID_INPUT	ReL is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

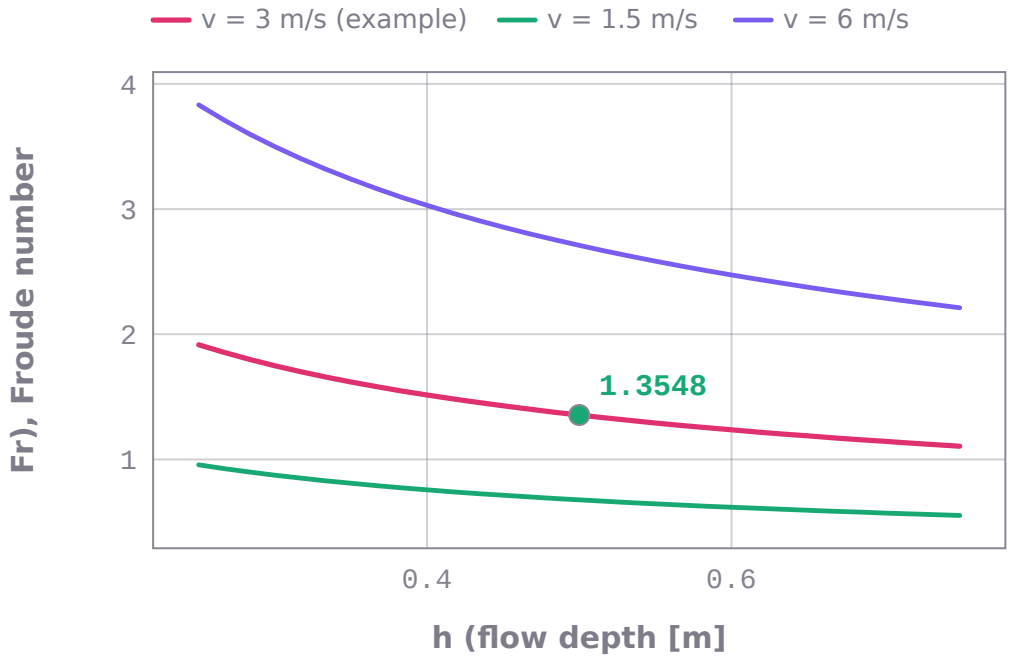
- "Air flows over a flat plate with a laminar boundary layer and a Reynolds number of 250000 at the trailing edge. What is the average skin friction coefficient?" → 0.00265646 1
- "Find the average friction coefficient for laminar flow along a flat plate with a Reynolds number of 10000." → 0.0132823 1

Froude number of an open-channel flow, `froude_number`

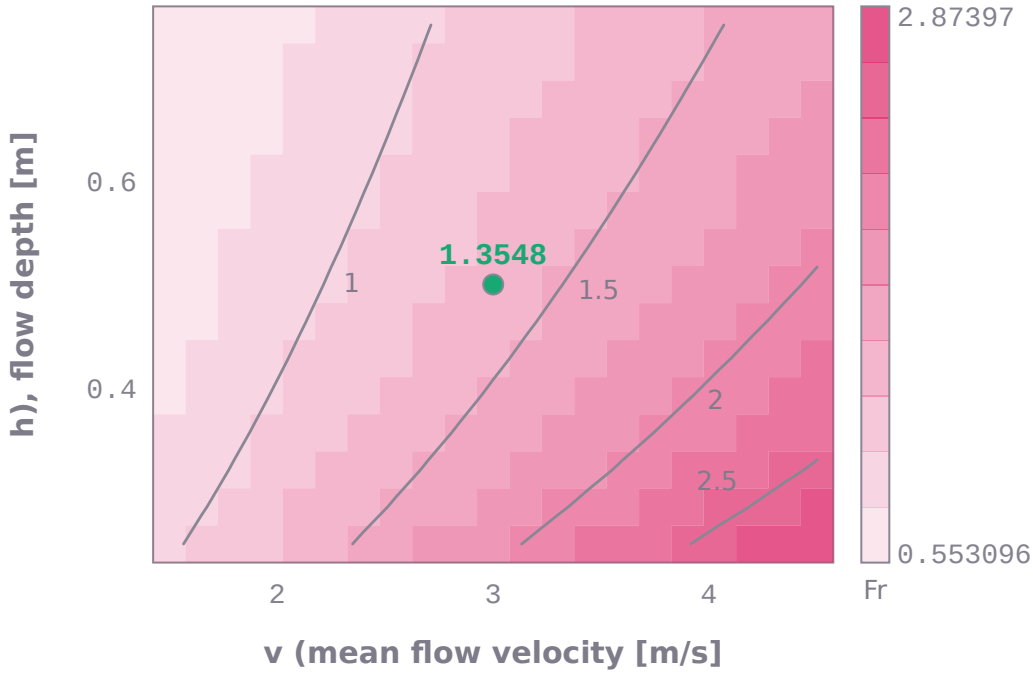
$$\mathrm{Fr} = \frac{v}{\sqrt{gh}}$$

input	meaning	kind / role	unit	default
`Fr`	Froude number	froude_number / plain	`1`	
`v`	mean flow velocity	speed / flow	`m/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`h`	flow depth	length / depth	`m`	

- **Conditions:** open_channel
- **Applies when:** free-surface flow in a channel or river
- **Fails when:** closed conduits flowing full
- **Assumes:** rectangular channel (hydraulic depth = flow depth)
- **Evidence:** theorem



Fr (Froude number) against h (flow depth) from 0.25 m to 0.75 m. One curve for each value of v (mean flow velocity): 3 m/s, 1.5 m/s, 6 m/s. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, h = 0.5 m, Fr = 1.3548.



Fr (Froude number) over v (mean flow velocity) from 1.5 m/s to 4.5 m/s and h (flow depth) from 0.25 m to 0.75 m: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 0.553096 to dark 2.87397); iso-lines at Fr = 1, 1.5, 2, 2.5. Marked: the worked example, v = 3 m/s, h = 0.5 m, Fr = 1.3548.

```
thermofluids_ai999x.solve('froude_number', v='3 m/s', h='0.5 m', target='Fr')
# -> 1.3548 1   (SOLVED)
```

$$\mathrm{Fr} = \frac{v}{\sqrt{gh}}$$

$$\mathrm{Fr} = \frac{v}{\sqrt{g}\sqrt{h}} = 1.3548\ 1$$

invalid call	outcome	message
without `v`	NEEDS_INPUT	froude_number needs v (mean flow velocity, m/s). Example: solve('froude_number', v='... m/s', g='... m/s^2', h='... m').
`v` in a wrong unit	INVALID_INPUT	v is a speed: its unit must be like m/s, not 'kg'.

Questions in words that work:

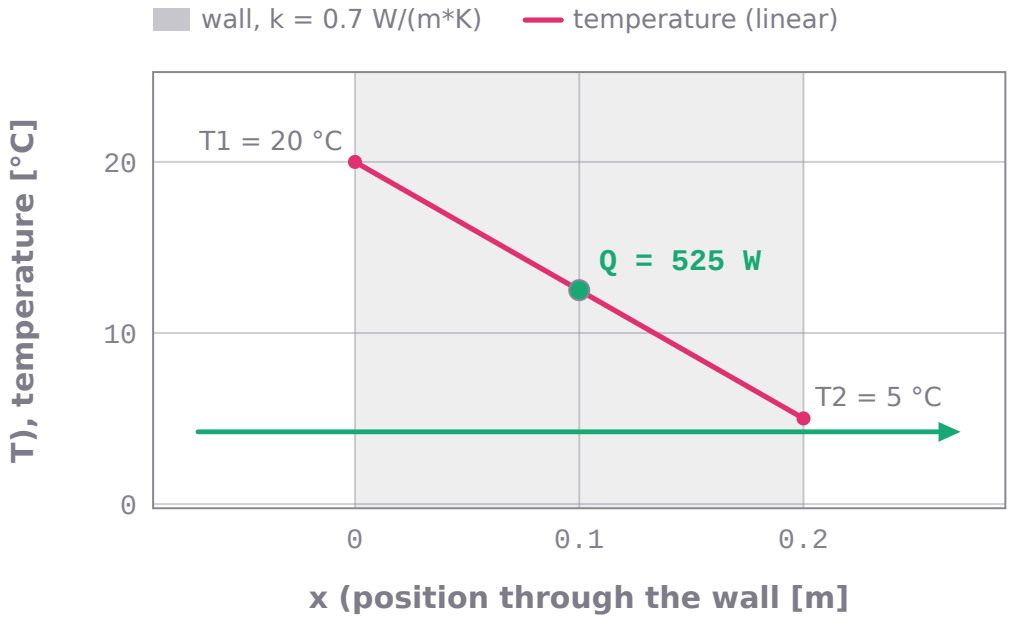
- "Water flows in an open channel at 3 m/s with a depth of 0.5 m. What is the Froude number?" → 1.3548 1
- "Find the Froude number for a river flowing at a mean velocity of 1.2 m/s where the water depth is 2 m." → 0.27096 1

Fourier's law: heat conduction through a plane wall, `plane_wall_conduction`

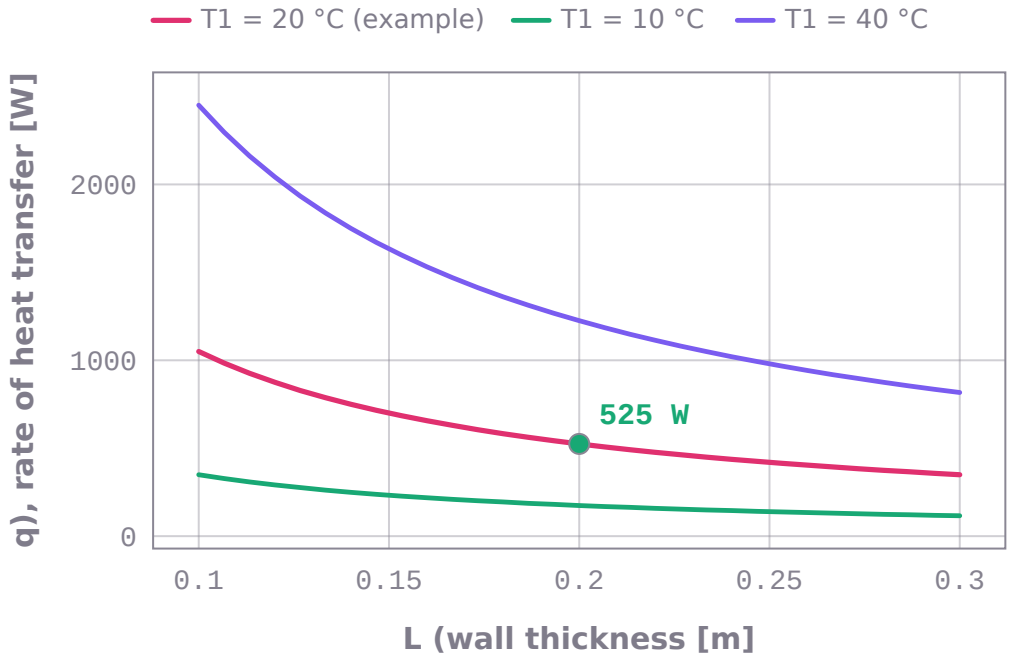
$$q = \frac{kA\,(T_1 - T_2)}{L}$$

input	meaning	kind / role	unit	default
`q`	rate of heat transfer	power / heat_rate	`W`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`A`	wall area	area / plain	`m^2`	
`T1`	inner surface temperature	temperature / inner_surface	`K`	
`T2`	outer surface temperature	temperature / outer_surface	`K`	
`L`	wall thickness	length / thickness	`m`	

- **Conditions:** conduction, plane_wall
- **Applies when:** a flat wall, slab or window with known surface temperatures
- **Fails when:** transient conduction; multidimensional heat flow
- **Assumes:** steady one-dimensional conduction; constant thermal conductivity; no heat generation
- **Evidence:** theorem



Steady conduction through a plane wall of thickness L = 0.2 m, area 10 m^2, k = 0.7 W/(m*K), faces at 20 °C and 5 °C (the worked example); the temperature falls linearly through the wall. Marked: the package's heat rate Q = 525 W (flux 52.5 W/m², Q divided by the area).



q (rate of heat transfer) against L (wall thickness) from 0.1 m to 0.3 m. One curve for each value of T1 (inner surface temperature): 20 °C, 10 °C, 40 °C. Other inputs as in the worked example: k = 0.7 W/(m*K), A = 10 m^2, T2 = 5 °C. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, L = 0.2 m, q = 525 W.

```
thermofluids_ai999x.solve('plane_wall_conduction', k='0.7 W/(m*K)', A='10 m^2', T1='20 °C',
T2='5 °C', L='0.2 m', target='q')
# -> 525 W   (SOLVED)
```

$$q = \frac{kA\,(T_1 - T_2)}{L}$$

$$q = \frac{Ak\,(T_1 - T_2)}{L} = 525\ \mathrm{W}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	plane_wall_conduction needs k (thermal conductivity, W/(m*K)). Example: solve('plane_wall_conduction', k='... W/(m*K)', A='... m^2', T1='...
`k` in a wrong unit	INVALID_INPUT	k is a thermal conductivity: its unit must be like W/(m*K), not 'kg'.

Questions in words that work:

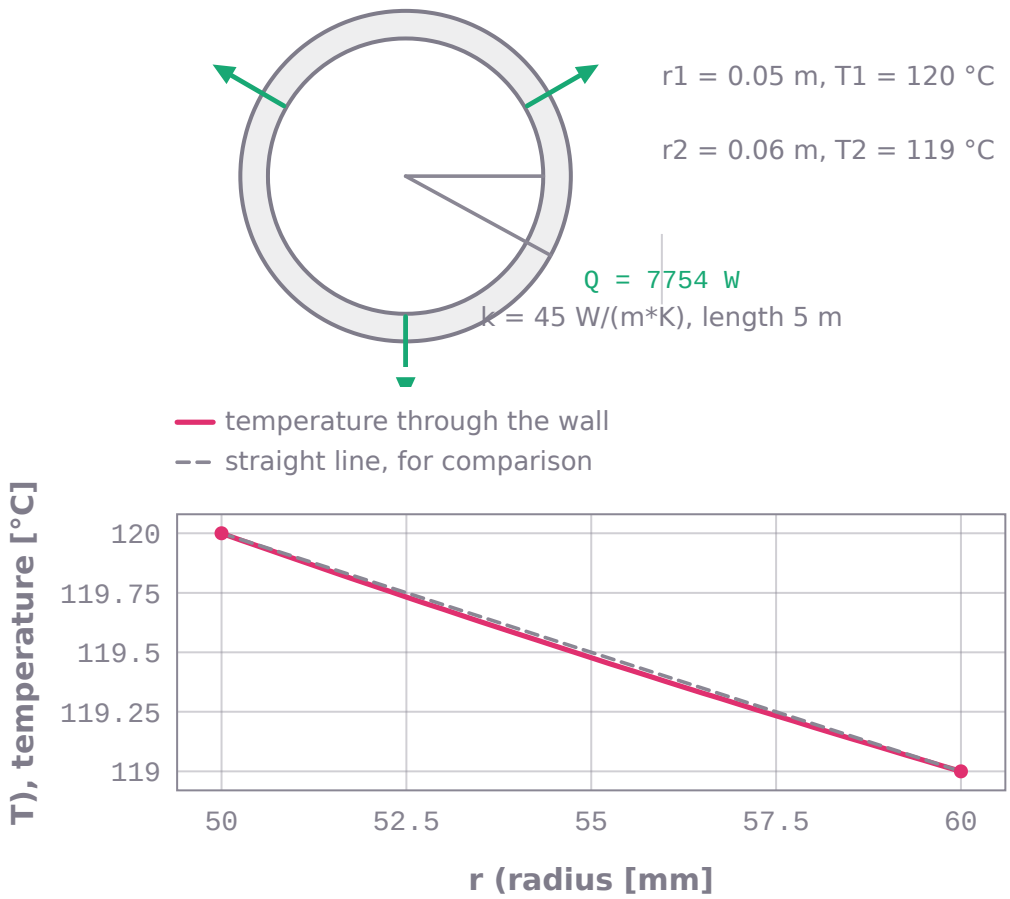
- "A brick wall 0.2 m thick has an area of 10 m^2 and a thermal conductivity of 0.7 W/(m*K). The inner surface temperature is 20 °C and the outer surface temperature is 5 °C. What is the rate of heat transfer through the wall?" → 525 W
- "A glass window of area 1.5 m^2 and thickness 0.005 m has a thermal conductivity of 0.8 W/(m*K). If the inner surface temperature is 15 °C and the outer surface temperature is 10 °C, find the rate of heat conduction through it." → 1200 W

Radial heat conduction through a cylindrical wall, `cylinder_conduction`

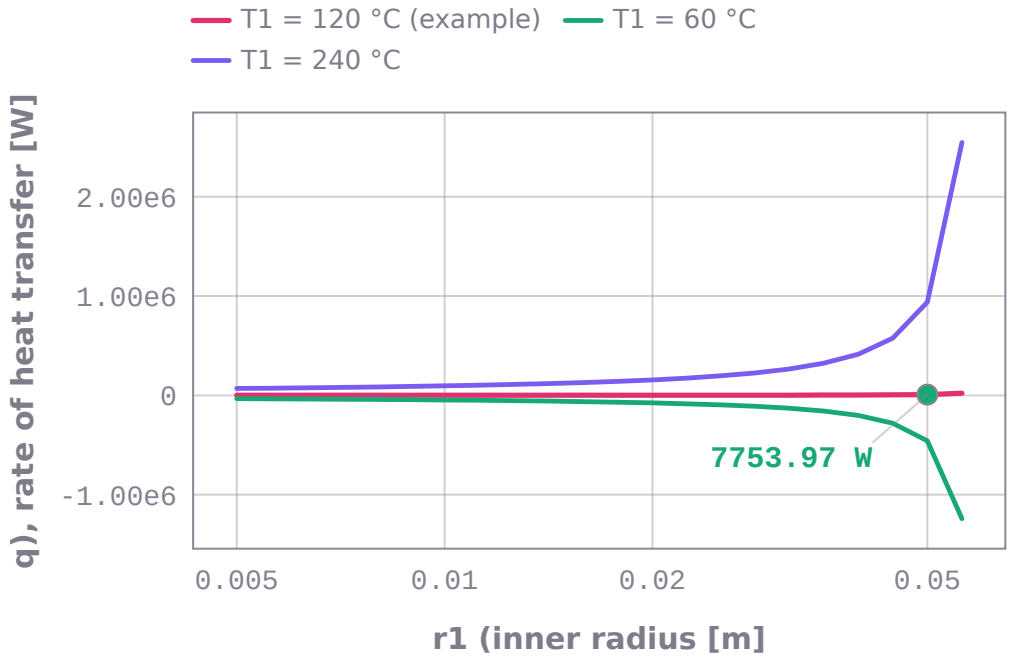
$$q = \frac{2\pi k L\,(T_1 - T_2)}{\ln\left(\frac{r_2}{r_1}\right)}$$

input	meaning	kind / role	unit	default
`q`	rate of heat transfer	power / heat_rate	`W`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`L`	length of the cylinder	length / pipe_length	`m`	
`T1`	inner surface temperature	temperature / inner_surface	`K`	
`T2`	outer surface temperature	temperature / outer_surface	`K`	
`r1`	inner radius	length / inner_radius	`m`	
`r2`	outer radius	length / outer_radius	`m`	

- **Conditions:** conduction, cylindrical
- **Applies when:** pipe walls and pipe insulation
- **Fails when:** axial heat flow; transient conduction
- **Assumes:** steady radial conduction; constant thermal conductivity; no heat generation
- **Evidence:** theorem



Steady radial conduction through a cylinder wall (the worked example): r1 = 0.05 m at 120 °C, r2 = 0.06 m at 119 °C, k = 45 W/(m*K), length 5 m. Top: the cross-section with the radial heat flow; bottom: the temperature through the wall, T(r) = T1 − (T1 − T2)·ln(r/r1)/ln(r2/r1), derived from the stated surface temperatures. Marked: the package's heat rate Q = 7753.97 W.



q (rate of heat transfer) against r1 (inner radius) from 0.005 m to 0.0561009 m on a logarithmic axis. One curve for each value of T1 (inner surface temperature): 120 °C, 60 °C, 240 °C. Other inputs as in the worked example: k = 45 W/(m*K), L = 5 m, T2 = 119 °C, r2 = 0.06 m. 22 points per curve computed by thermofluids-ai999x; marked: the worked example, r1 = 0.05 m, q = 7753.97 W.

```
thermofluids_ai999x.solve('cylinder_conduction', k='45 W/(m*K)', L='5 m', T1='120 °C',
T2='119 °C', r1='0.05 m', r2='0.06 m', target='q')
# -> 7753.97 W   (SOLVED)
```

$$q = \frac{2\pi k L\,(T_1 - T_2)}{\ln\left(\frac{r_2}{r_1}\right)}$$

$$q = \frac{2\pi L k\,(T_1 - T_2)}{\ln\left(\frac{r_2}{r_1}\right)} = 7753.97\ \mathrm{W}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	cylinder_conduction needs k (thermal conductivity, W/(m*K)). Example: solve('cylinder_conduction', k='... W/(m*K)', L='... m', T1='... K', T
`k` in a wrong unit	INVALID_INPUT	k is a thermal conductivity: its unit must be like W/(m*K), not 'kg'.

Questions in words that work:

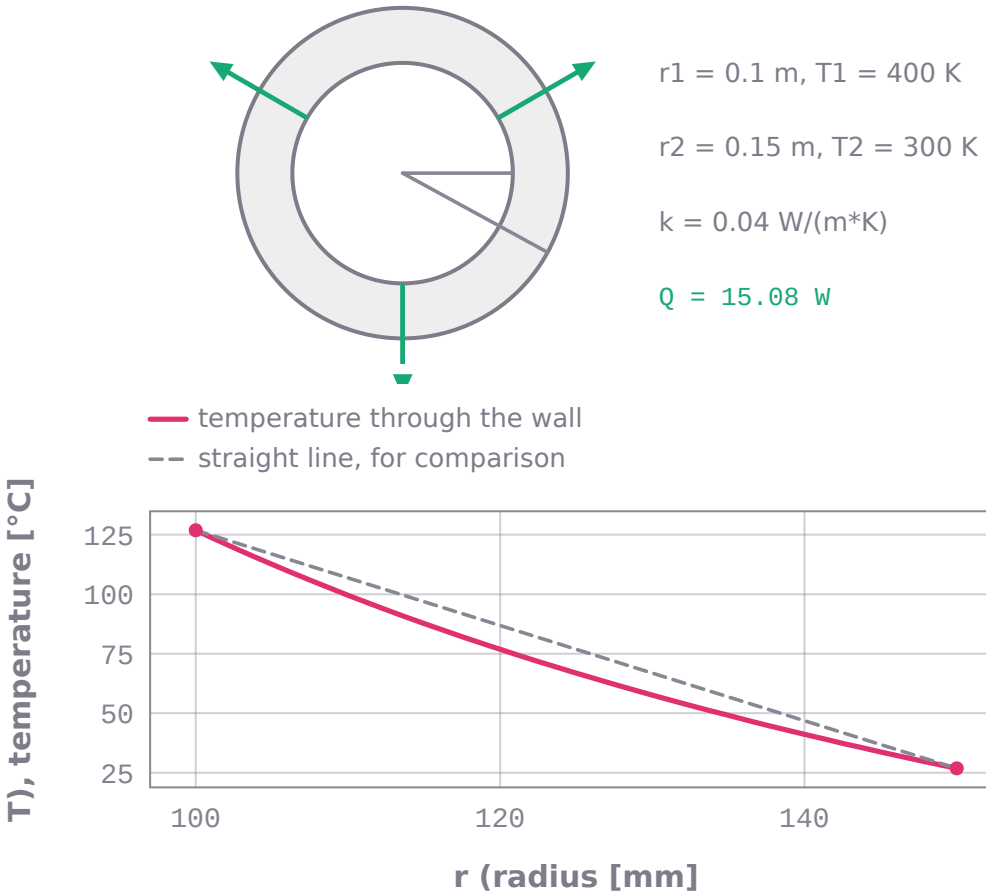
- "A 5 m long pipe made of steel has an inner radius of 0.05 m and an outer radius of 0.06 m, with thermal conductivity 45 W/(m*K). The inner surface temperature is 120 °C and the outer surface temperature is 119 °C. What is the rate of heat conduction through the pipe wall?" → 7753.97 W
- "An insulated tube of length 10 m has an inner radius of 0.03 m and an outer radius of 0.08 m. The insulation has a thermal conductivity of 0.05 W/(m*K), its inner surface temperature is 200 °C and its outer surface temperature is 40 °C. Find the rate of heat conduction through the insulation." → 512.479 W

Radial heat conduction through a spherical shell, `sphere_conduction`

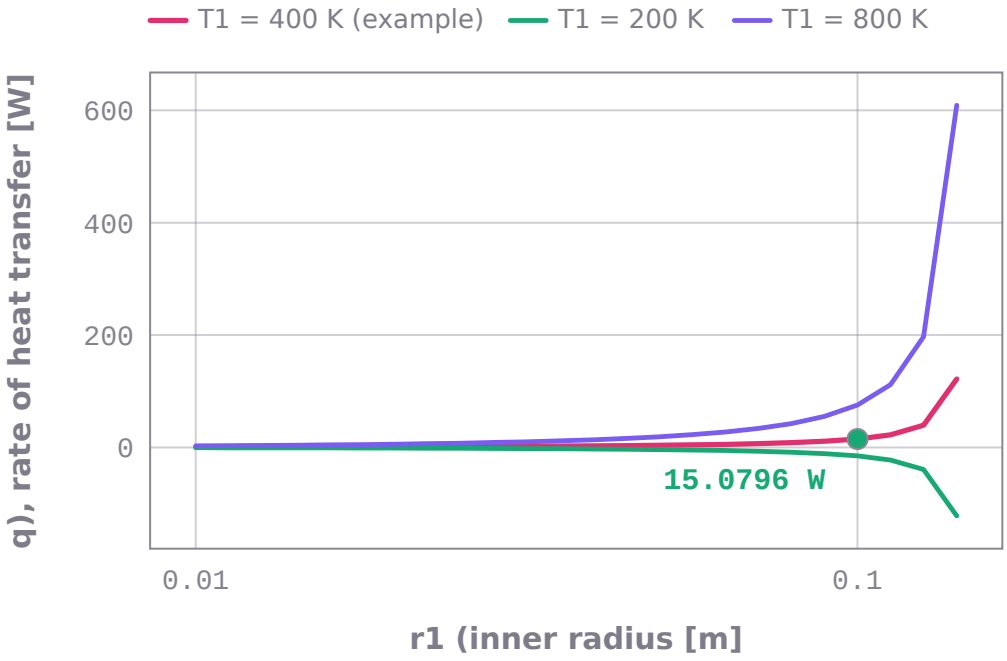
$$q = \frac{4\pi k\,(T_1 - T_2)}{\frac{1}{r_1} - \frac{1}{r_2}}$$

input	meaning	kind / role	unit	default
`q`	rate of heat transfer	power / heat_rate	`W`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`T1`	inner surface temperature	temperature / inner_surface	`K`	
`T2`	outer surface temperature	temperature / outer_surface	`K`	
`r1`	inner radius	length / inner_radius	`m`	
`r2`	outer radius	length / outer_radius	`m`	

- **Conditions:** conduction, spherical
- **Applies when:** spherical tanks and their insulation
- **Fails when:** transient conduction
- **Assumes:** steady radial conduction; constant thermal conductivity
- **Evidence:** theorem



Steady radial conduction through a sphere wall (the worked example): r1 = 0.1 m at 400 K, r2 = 0.15 m at 300 K, k = 0.04 W/(m*K). Top: the cross-section with the radial heat flow; bottom: the temperature through the wall, T(r) = T1 − (T1 − T2)·(1/r1 − 1/r)/(1/r1 − 1/r2), derived from the stated surface temperatures. Marked: the package's heat rate Q = 15.0796 W.



q (rate of heat transfer) against r1 (inner radius) from 0.01 m to 0.141254 m on a logarithmic axis. One curve for each value of T1 (inner surface temperature): 400 K, 200 K, 800 K. Other inputs as in the worked example: k = 0.04 W/(m*K), T2 = 300 K, r2 = 0.15 m. 24 points per curve computed by thermofluids-ai999x; marked: the worked example, r1 = 0.1 m, q = 15.0796 W.

```
thermofluids_ai999x.solve('sphere_conduction', k='0.04 W/(m*K)', T1='400 K', T2='300 K',
r1='0.1 m', r2='0.15 m', target='q')
# -> 15.0796 W   (SOLVED)
```

$$q = \frac{4\pi k\,(T_1 - T_2)}{\frac{1}{r_1} - \frac{1}{r_2}}$$

$$q = \frac{4\pi k r_1 r_2\,(-T_1 + T_2)}{r_1 - r_2} = 15.0796\ \mathrm{W}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	sphere_conduction needs k (thermal conductivity, W/(m*K)). Example: solve('sphere_conduction', k='... W/(m*K)', T1='... K', T2='... K', r1='
`k` in a wrong unit	INVALID_INPUT	k is a thermal conductivity: its unit must be like W/(m*K), not 'kg'.

Questions in words that work:

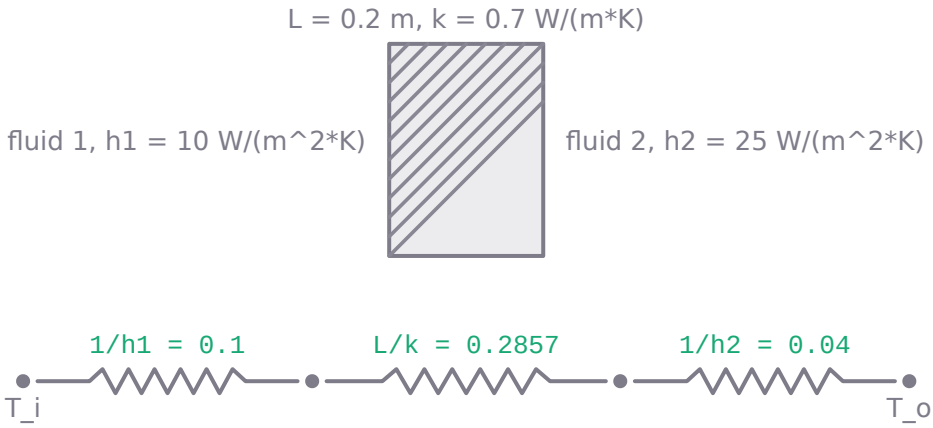
- "A spherical shell of insulation with thermal conductivity 0.04 W/(m*K) has an inner radius of 0.1 m and an outer radius of 0.15 m. The inner surface temperature is 400 K and the outer surface temperature is 300 K. What is the rate of heat conduction through the shell?" → 15.0796 W
- "Find the rate of heat conduction through a spherical steel shell of thermal conductivity 15 W/(m*K), inner radius 0.05 m and outer radius 0.06 m, if the inner surface temperature is 90 °C and the outer surface temperature is 70 °C." → 1130.97 W

Overall heat transfer coefficient of a plane wall with convection on both sides, `overall_u_plane_wall`

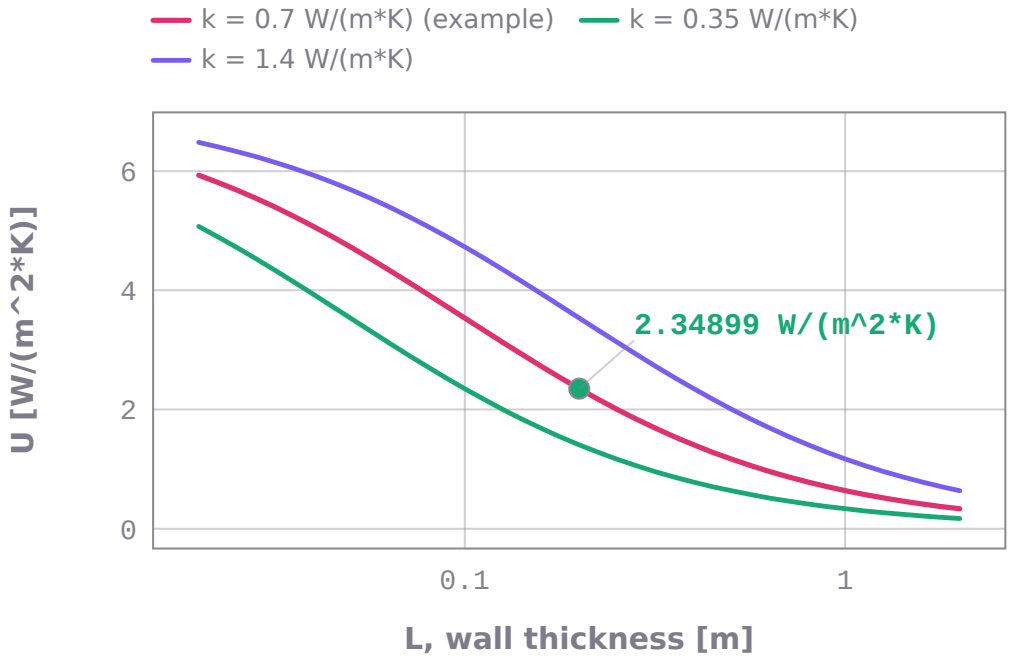
$$U = \frac{1}{\frac{1}{h_1} + \frac{L}{k} + \frac{1}{h_2}}$$

input	meaning	kind / role	unit	default
`U`	overall heat transfer coefficient	heat_transfer_coefficient / overall	`W/(m^2*K)`	
`h1`	inside heat transfer coefficient	heat_transfer_coefficient / inner	`W/(m^2*K)`	
`L`	wall thickness	length / thickness	`m`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`h2`	outside heat transfer coefficient	heat_transfer_coefficient / outer	`W/(m^2*K)`	

- **Conditions:** overall
- **Applies when:** walls and windows between two fluids
- **Fails when:** cylindrical walls (areas differ); several layers (add each L/k)
- **Assumes:** steady one-dimensional heat flow; a single plane layer; no radiation
- **Evidence:** theorem



A plane wall between two fluids (h1 = 10 W/(m^2*K), wall L = 0.2 m, k = 0.7 W/(m*K), h2 = 25 W/(m^2*K); the worked example) and its thermal-resistance chain per square metre: the three resistances 1/h1, L/k and 1/h2 in m²·K/W are the inputs inverted (derived); they add in series. Marked: the package's overall coefficient U = 2.34899 W/(m²·K) (1/U = 0.425714 m²·K/W).



U (overall heat transfer coefficient) against L (wall thickness) from 0.02 m to 2 m on a logarithmic axis. One curve for each value of k (thermal conductivity): 0.7 W/(m*K), 0.35 W/(m*K), 1.4 W/(m*K). Other inputs as in the worked example: h1 = 10 W/(m^2*K), h2 = 25 W/(m^2*K). 41 points per curve computed by thermofluids-ai999x; marked: the worked example, L = 0.2 m, U = 2.34899 W/(m^2*K).

```
thermofluids_ai999x.solve('overall_u_plane_wall', h1='10 W/(m^2*K)', L='0.2 m', k='0.7
W/(m*K)', h2='25 W/(m^2*K)', target='U')
# -> 2.34899 W/(m^2*K)   (SOLVED)
```

$$U = \frac{1}{\frac{1}{h_1} + \frac{L}{k} + \frac{1}{h_2}}$$

$$U = \frac{h_1 h_2 k}{L h_1 h_2 + h_1 k + h_2 k} = 2.34899\ \mathrm{W/(m^2\,K)}$$

invalid call	outcome	message
without `h1`	NEEDS_INPUT	overall_u_plane_wall needs h1 (inside heat transfer coefficient, W/(m^2*K)). Example: solve('overall_u_plane_wall', h1='... W/(m^2*K)', L='.
`h1` in a wrong unit	INVALID_INPUT	h1 is a heat transfer coefficient: its unit must be like W/(m^2*K), not 'kg'.

Questions in words that work:

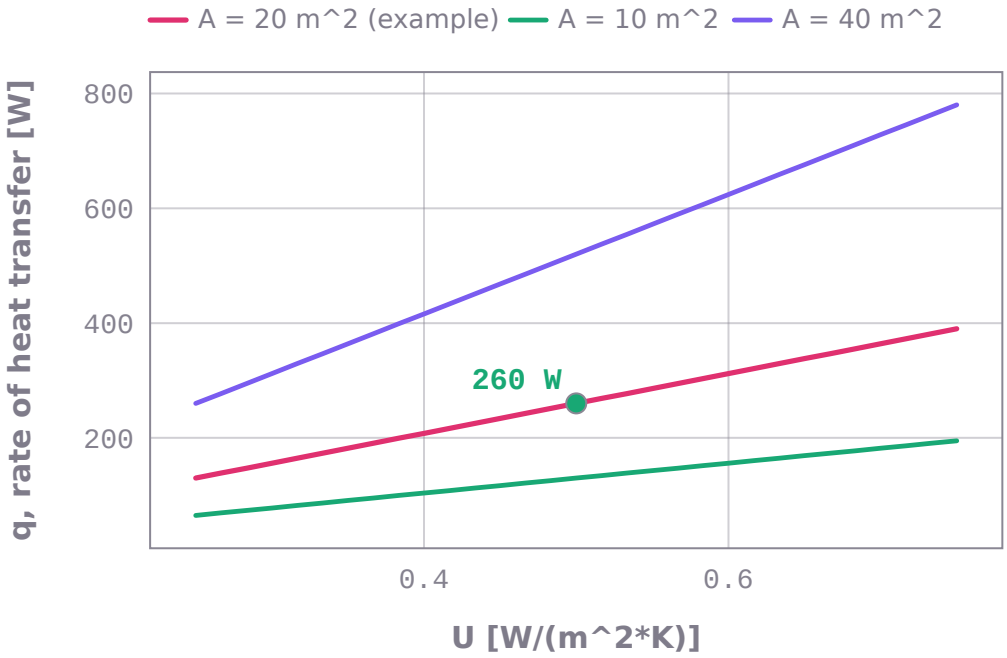
- "A brick wall 0.2 m thick has a thermal conductivity of 0.7 W/(m*K). The inside heat transfer coefficient is 10 W/(m^2*K) and the outside heat transfer coefficient is 25 W/(m^2*K). What is the overall heat transfer coefficient?" → 2.34899 W/(m^2*K)
- "Find the U-value of a glass pane of thickness 0.006 m and thermal conductivity 0.8 W/(m*K) with an inner heat transfer coefficient of 8 W/(m^2*K) and an outer heat transfer coefficient of 20 W/(m^2*K)." → 5.47945 W/(m^2*K)

Heat transfer rate from the overall heat transfer coefficient, `overall_heat_rate`

$$q = UA\left(T_i - T_o\right)$$

input	meaning	kind / role	unit	default
`q`	rate of heat transfer	power / heat_rate	`W`	
`U`	overall heat transfer coefficient	heat_transfer_coefficient / overall	`W/(m^2*K)`	
`A`	wall area	area / plain	`m^2`	
`Ti`	inside fluid temperature	temperature / inside_fluid	`K`	
`To`	outside fluid temperature	temperature / outside_fluid	`K`	

- **Conditions:** overall
- **Applies when:** building envelopes and walls between two fluids
- **Fails when:** transient conditions
- **Assumes:** steady heat flow; U based on the stated area
- **Evidence:** theorem



q (rate of heat transfer) against U (overall heat transfer coefficient) from 0.25 W/(m^2*K) to 0.75 W/(m^2*K). One curve for each value of A (wall area): 20 m^2, 10 m^2, 40 m^2. Other inputs as in the worked example: Ti = 21 °C, To = -5 °C. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, U = 0.5 W/(m^2*K), q = 260 W.

```
thermofluids_ai999x.solve('overall_heat_rate', U='0.5 W/(m^2*K)', A='20 m^2', Ti='21 °C',
To='-5 °C', target='q')
# -> 260 W   (SOLVED)
```

$$q = UA\left(T_i - T_o\right)$$
$$q = AU\left(T_i - T_o\right) = 260\ \mathrm{W}$$

invalid call	outcome	message
without `U`	NEEDS_INPUT	overall_heat_rate needs U (overall heat transfer coefficient, W/(m^2*K)). Example: solve('overall_heat_rate', U='... W/(m^2*K)', A='... m^2'
`U` in a wrong unit	INVALID_INPUT	U is a heat transfer coefficient: its unit must be like W/(m^2*K), not 'kg'.

Questions in words that work:

- "A wall of area 20 m^2 has an overall heat transfer coefficient of 0.5 W/(m^2*K). The inside air temperature is 21 °C and the outside air temperature is -5 °C. What is the heat loss through the wall?" → 260 W
- "Find the rate of heat loss through a window with a U-value of 2.8 W/(m^2*K) and area 3 m^2 when the indoor temperature is 293 K and the outdoor temperature is 273 K." → 168 W

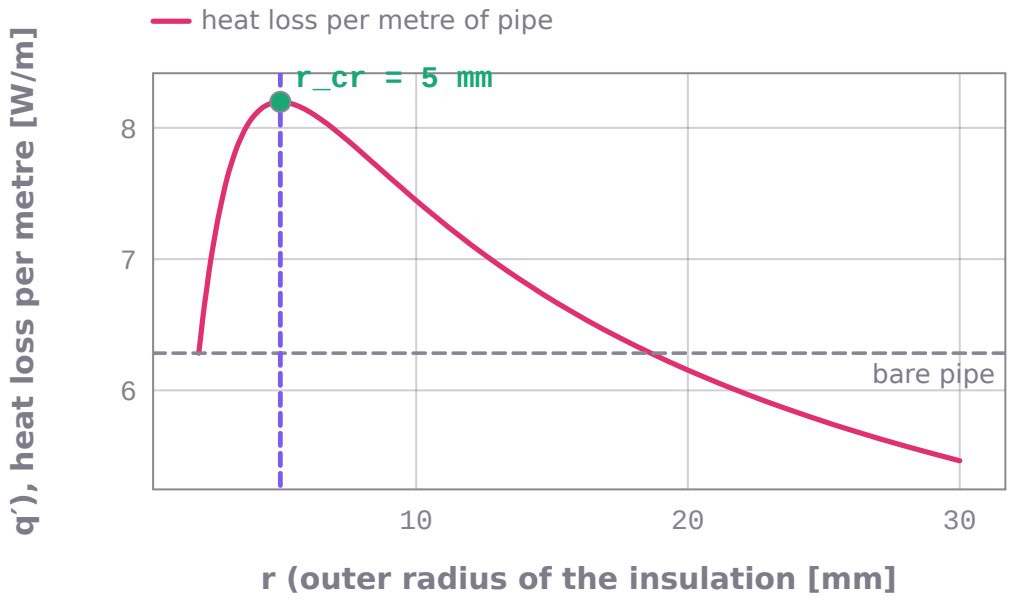
Critical radius of insulation on a cylinder, `critical_insulation_radius`

$$r = \frac{k}{h}$$

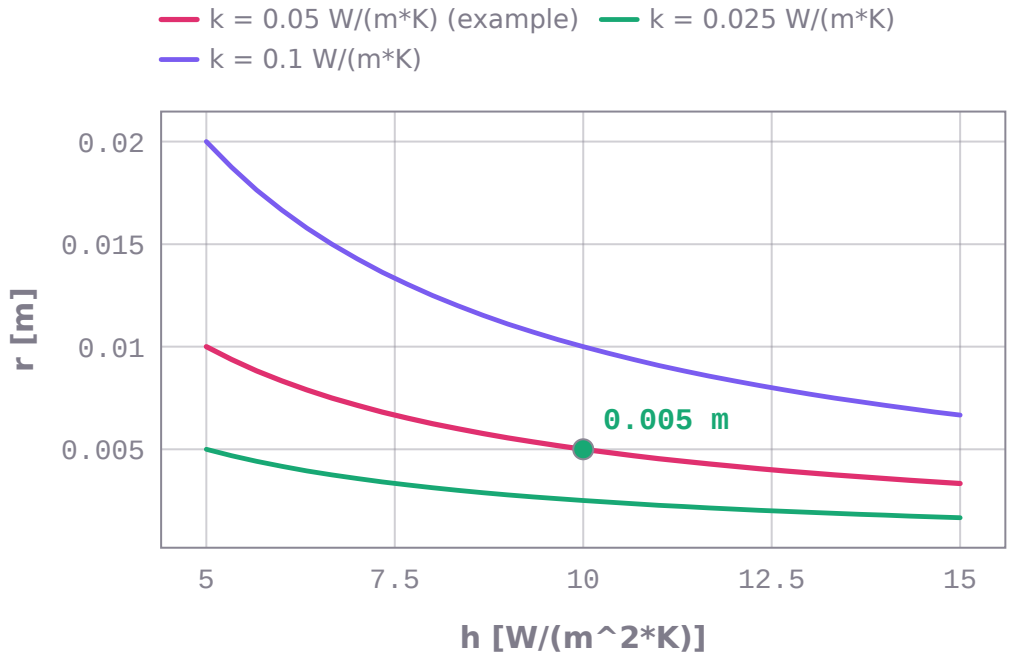
input	meaning	kind / role	unit	default
`r`	critical outer radius of the insulation	length / critical_radius	`m`	

input	meaning	kind / role	unit	default
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`h`	outside heat transfer coefficient	heat_transfer_coefficient / outer	`W/(m^2*K)`	

- **Conditions:** critical_insulation
- **Applies when:** deciding whether insulating a small pipe or wire increases heat loss
- **Fails when:** spheres $(r = 2k/h)$
- **Assumes:** cylindrical insulation; constant outside heat transfer coefficient; no radiation
- **Evidence:** theorem



Heat loss per metre from a pipe of radius 2 mm covered with insulation of k = 0.05 W/(m*K) in a fluid with h = 10 W/(m^2*K) (the worked example's k and h; the pipe radius and a 50 K temperature difference are stated), derived from the conduction and convection resistances in series. Adding insulation first raises the loss; the maximum lies at the package's critical radius, marked: r_cr = k/h = 5 mm.



r (critical outer radius of the insulation) against h (outside heat transfer coefficient) from 5 W/(m^2*K) to 15 W/(m^2*K). One curve for each value of k (thermal conductivity): 0.05 W/(m*K), 0.025 W/(m*K), 0.1 W/(m*K). 31 points per curve computed by thermofluids-ai999x; marked: the worked example, h = 10 W/(m^2*K), r = 0.005 m.

```
thermofluids_ai999x.solve('critical_insulation_radius', k='0.05 W/(m*K)', h='10 W/(m^2*K)',
target='r')
# -> 0.005 m   (SOLVED)
```

$$r = \frac{k}{h}$$

$$r = \frac{k}{h} = 0.005\ \mathrm{m}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	critical_insulation_radius needs k (thermal conductivity, W/(m*K)). Example: solve('critical_insulation_radius', k='... W/(m*K)', h='... W/(
`k` in a wrong unit	INVALID_INPUT	k is a thermal conductivity: its unit must be like W/(m*K), not 'kg'.

Questions in words that work:

- "Insulation with a thermal conductivity of 0.05 W/(m*K) covers a pipe, and the outside heat transfer coefficient is 10 W/(m^2*K). What is the critical radius of insulation?" → 0.005 m
- "Find the critical insulation radius for a wire covered with rubber of thermal conductivity 0.17 W/(m*K) in air with a heat transfer coefficient of 5 W/(m^2*K)." → 0.034 m

Maximum temperature of a plane wall with uniform heat generation, `wall_generation_max_temperature`

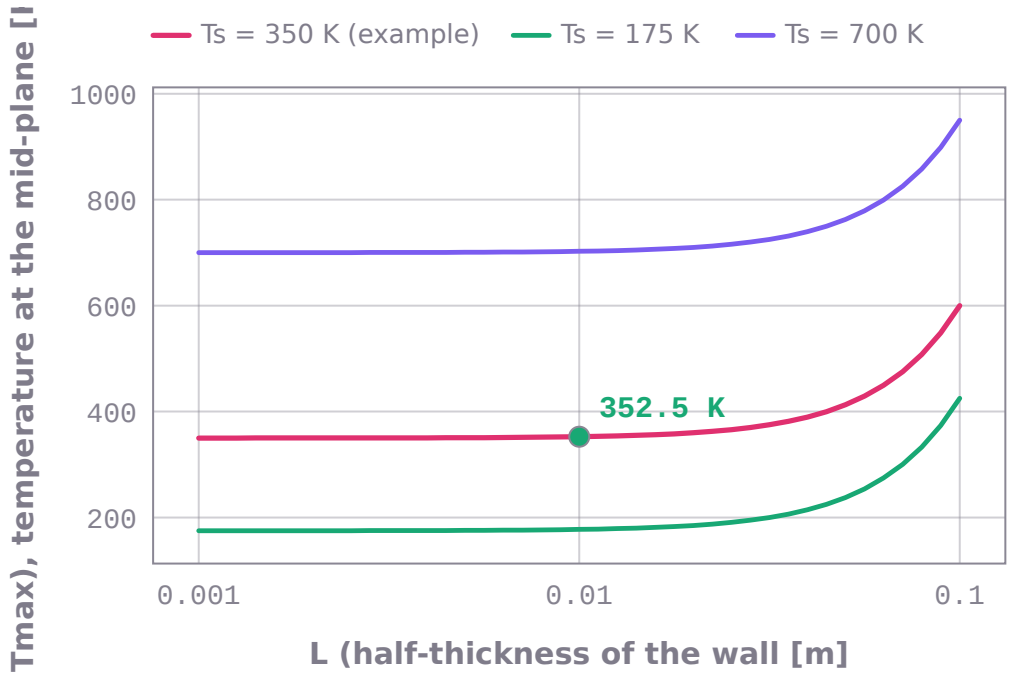
$$T_{\max} = T_s + \frac{qL^2}{2k}$$

input	meaning	kind / role	unit	default
`Tmax`	temperature at the mid-plane	temperature / max	`K`	
`Ts`	surface temperature	temperature / surface	`K`	
`q`	heat generated per unit volume	volumetric_heat_generation / plain	`W/m^3`	
`L`	half-thickness of the wall	length / half_thickness	`m`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	

- **Conditions:** heat_generation
- **Applies when:** electrically heated slabs and fuel plates
- **Fails when:** unequal surface temperatures; temperature-dependent conductivity
- **Assumes:** steady one-dimensional conduction; both faces at the same temperature; uniform generation
- **Evidence:** theorem



A plane wall of half-thickness L = 0.01 m (k = 20 W/(m*K)) generating heat uniformly at 1e+06 W/m^3, both faces held at 350 K (the worked example). The temperature is parabolic, T = Ts + (Tmax − Ts)(1 − x²/L²), drawn from the package's mid-plane maximum (derived shape). Marked: Tmax = 352.5 K.



Tmax (temperature at the mid-plane) against L (half-thickness of the wall) from 0.001 m to 0.1 m on a logarithmic axis. One curve for each value of Ts (surface temperature): 350 K, 175 K, 700 K. Other inputs as in the worked example: q = 1000000.0 W/m^3, k = 20 W/(m*K). 41 points per curve computed by thermofluids-ai999x; marked: the worked example, L = 0.01 m, Tmax = 352.5 K.

```
thermofluids_ai999x.solve('wall_generation_max_temperature', Ts='350 K', q='1e+06 W/m^3',
L='0.01 m', k='20 W/(m*K)', target='Tmax')
# -> 352.5 K   (SOLVED)
```

$$T_{\max} = T_s + \frac{qL^2}{2k}$$

$$T_{\max} = \frac{L^2 q}{2k} + T_s = 352.5\ \mathrm{K}$$

invalid call	outcome	message
without `Ts`	NEEDS_INPUT	wall_generation_max_temperature needs Ts (surface temperature, K). Example: solve('wall_generation_max_temperature', Ts='... K', q='... W/m^
`Ts` in a wrong unit	INVALID_INPUT	Ts is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

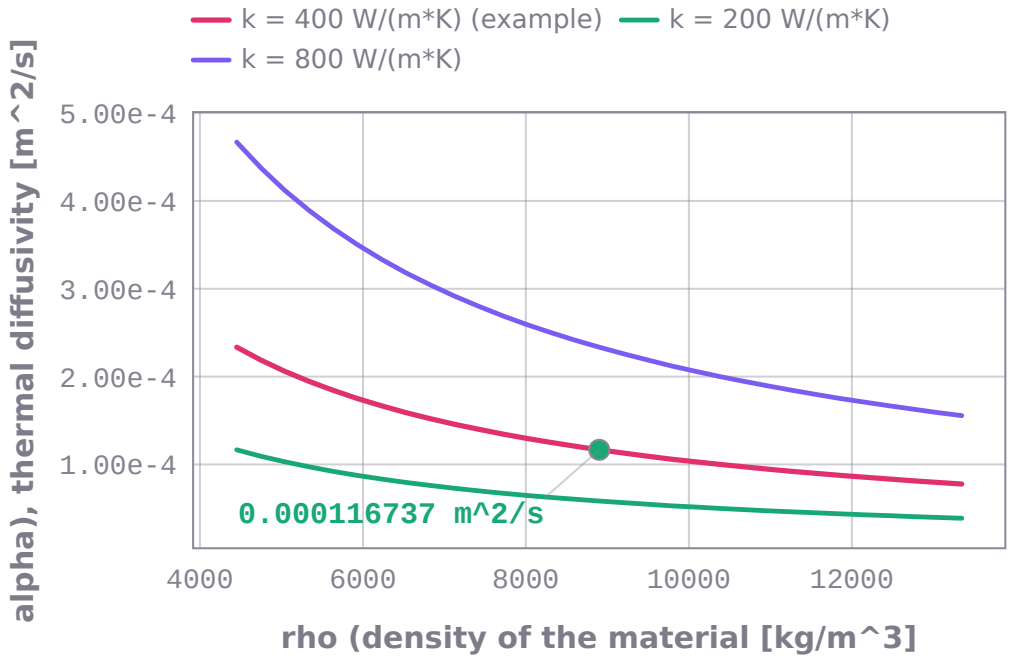
- "A plane wall with a half-thickness of 0.01 m and a thermal conductivity of 20 W/(m*K) has a uniform heat generation of 1000000 W/m^3. Both surfaces are held at a surface temperature of 350 K. What is the maximum temperature in the wall?" → 352.5 K
- "Heat is generated uniformly at a rate of heat generation of 500000 W/m^3 in a slab of half thickness 0.02 m and thermal conductivity 50 W/(m*K). The surface temperature is 80 °C on both faces. Find the center temperature." → 355.15 K

Thermal diffusivity, `thermal_diffusivity`

$$\alpha = \frac{k}{\rho c_p}$$

input	meaning	kind / role	unit	default
`alpha`	thermal diffusivity	thermal_diffusivity / plain	`m^2/s`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`rho`	density of the material	density / material	`kg/m^3`	
`cp`	specific heat capacity	specific_heat / plain	`J/(kg*K)`	

- **Conditions:** diffusivity
- **Applies when:** transient conduction in solids and fluids
- **Fails when:** phase change
- **Assumes:** constant properties
- **Evidence:** theorem



alpha (thermal diffusivity) against rho (density of the material) from 4450 kg/m^3 to 13350 kg/m^3. One curve for each value of k (thermal conductivity): 400 W/(m*K), 200 W/(m*K), 800 W/(m*K). Other inputs as in the worked example: cp = 385 J/(kg*K). 31 points per curve computed by thermofluids-ai999x; marked: the worked example, rho = 8900 kg/m^3, alpha = 0.000116737 m^2/s.

```
thermofluids_ai999x.solve('thermal_diffusivity', k='400 W/(m*K)', rho='8900 kg/m^3',
cp='385 J/(kg*K)', target='alpha')
# -> 0.000116737 m^2/s   (SOLVED)
```

$$\alpha = \frac{k}{\rho c_p}$$

$$\alpha = \frac{k}{c_p \rho} = 0.000116737\ \mathrm{m^2/s}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	thermal_diffusivity needs k (thermal conductivity, W/(m*K)). Example: solve('thermal_diffusivity', k='... W/(m*K)', rho='... kg/m^3', cp='..
`k` in a wrong unit	INVALID_INPUT	k is a thermal conductivity: its unit must be like W/(m*K), not 'kg'.

Questions in words that work:

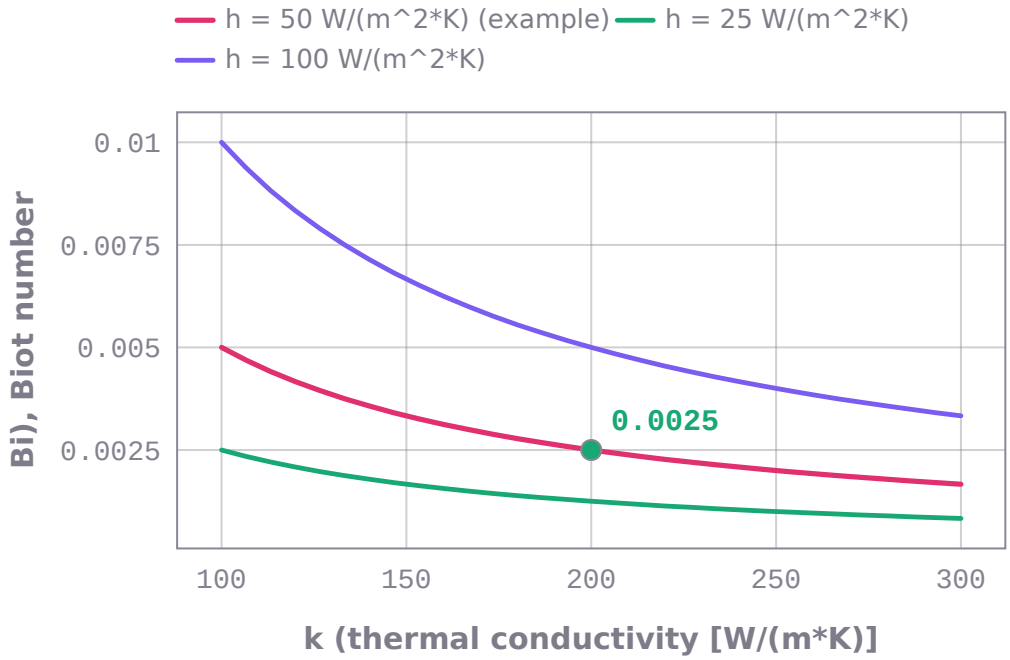
- "Copper has a thermal conductivity of 400 W/(m*K), a density of 8900 kg/m^3 and a specific heat of 385 J/(kg*K). What is its thermal diffusivity?" → 0.000116737 m^2/s
- "Find the thermal diffusivity of water with thermal conductivity 0.6 W/(m*K), density 1000 kg/m^3 and specific heat 4180 J/(kg*K)." → 1.43541e-07 m^2/s

Biot number, `biot_number`

$$\mathrm{Bi} = \frac{h L_c}{k}$$

input	meaning	kind / role	unit	default
`Bi`	Biot number	biot_number / plain	`1`	
`h`	convection heat transfer coefficient	heat_transfer_coefficient / plain	`W/(m^2*K)`	
`Lc`	characteristic length	length / characteristic	`m`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	

- **Conditions:** biot
- **Applies when:** judging whether a body's internal temperature is uniform $(Bi < 0.1)$
- **Fails when:** h varying strongly over the surface
- **Assumes:** Lc = volume / surface area for the lumped-capacitance test
- **Evidence:** theorem



Bi (Biot number) against k (thermal conductivity) from 100 W/(m*K) to 300 W/(m*K). One curve for each value of h (convection heat transfer coefficient): 50 W/(m^2*K), 25 W/(m^2*K), 100 W/(m^2*K). Other inputs as in the worked example: Lc = 0.01 m. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, k = 200 W/(m*K), Bi = 0.0025.

```
thermofluids_ai999x.solve('biot_number', h='50 W/(m^2*K)', Lc='0.01 m', k='200 W/(m*K)',
target='Bi')
# -> 0.0025 1   (SOLVED)
```

$$\mathrm{Bi} = \frac{h L_c}{k}$$

$$\mathrm{Bi} = \frac{L_c h}{k} = 0.0025\ 1$$

invalid call	outcome	message
without `h`	NEEDS_INPUT	biot_number needs h (convection heat transfer coefficient, W/(m^2*K)). Example: solve('biot_number', h='... W/(m^2*K)', Lc='... m', k='... W
`h` in a wrong unit	INVALID_INPUT	h is a heat transfer coefficient: its unit must be like W/(m^2*K), not 'kg'.

Questions in words that work:

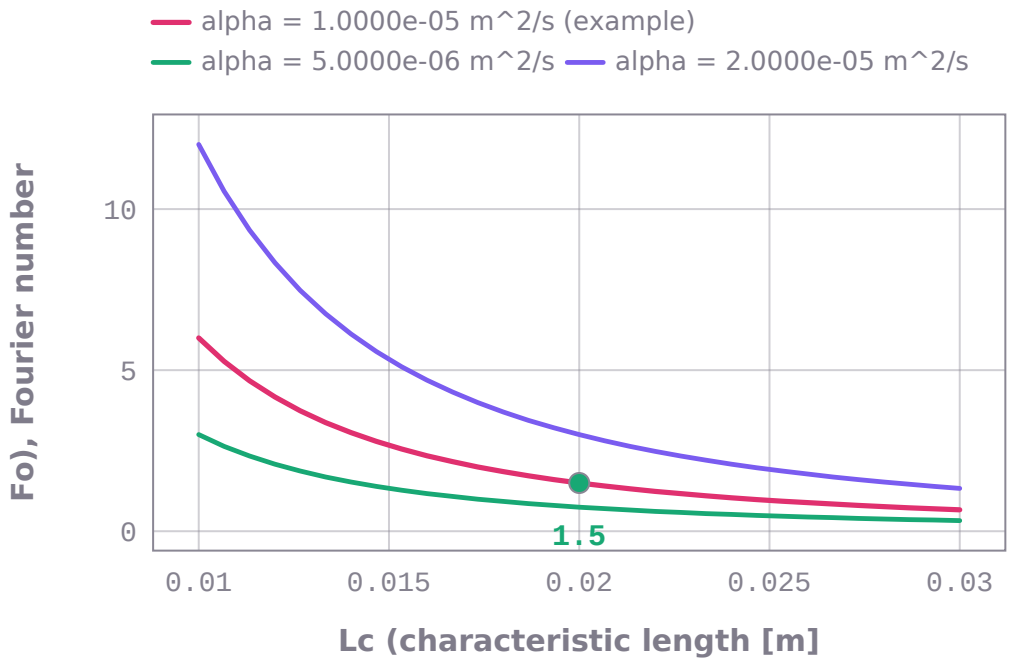
- "An aluminium part with a characteristic length of 0.01 m and thermal conductivity 200 W/(m*K) is cooled with a heat transfer coefficient of 50 W/(m^2*K). What is the Biot number?" → 0.0025 1
- "Find the Biot number for a steel ball with thermal conductivity 15 W/(m*K), characteristic length 0.005 m and heat transfer coefficient 120 W/(m^2*K)." → 0.04 1

Fourier number, `fourier_number`

$$F_o = \frac{\alpha t}{L_c^2}$$

input	meaning	kind / role	unit	default
`Fo`	Fourier number	fourier_number / plain	`1`	
`alpha`	thermal diffusivity	thermal_diffusivity / plain	`m^2/s`	
`t`	elapsed time	time / plain	`s`	
`Lc`	characteristic length	length / characteristic	`m`	

- **Conditions:** fourier_number
- **Applies when:** transient conduction charts and series solutions
- **Fails when:** phase change
- **Assumes:** constant properties
- **Evidence:** theorem



Fo (Fourier number) against Lc (characteristic length) from 0.01 m to 0.03 m. One curve for each value of alpha (thermal diffusivity): 1.0000e-05 m^2/s, 5.0000e-06 m^2/s, 2.0000e-05 m^2/s. Other inputs as in the worked example: t = 60 s. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Lc = 0.02 m, Fo = 1.5.

```
thermofluids_ai999x.solve('fourier_number', alpha='1e-05 m^2/s', t='60 s', Lc='0.02 m',
target='Fo')
# -> 1.5 1   (SOLVED)
```

$$F_o = \frac{\alpha t}{L_c^2}$$

$$F_o = \frac{\alpha t}{L_c^2} = 1.5\ 1$$

invalid call	outcome	message
without `alpha`	NEEDS_INPUT	fourier_number needs alpha (thermal diffusivity, m^2/s). Example: solve('fourier_number', alpha='... m^2/s', t='... s', Lc='... m').
`alpha` in a wrong unit	INVALID_INPUT	alpha is a thermal diffusivity: its unit must be like m^2/s, not 'kg'.

Questions in words that work:

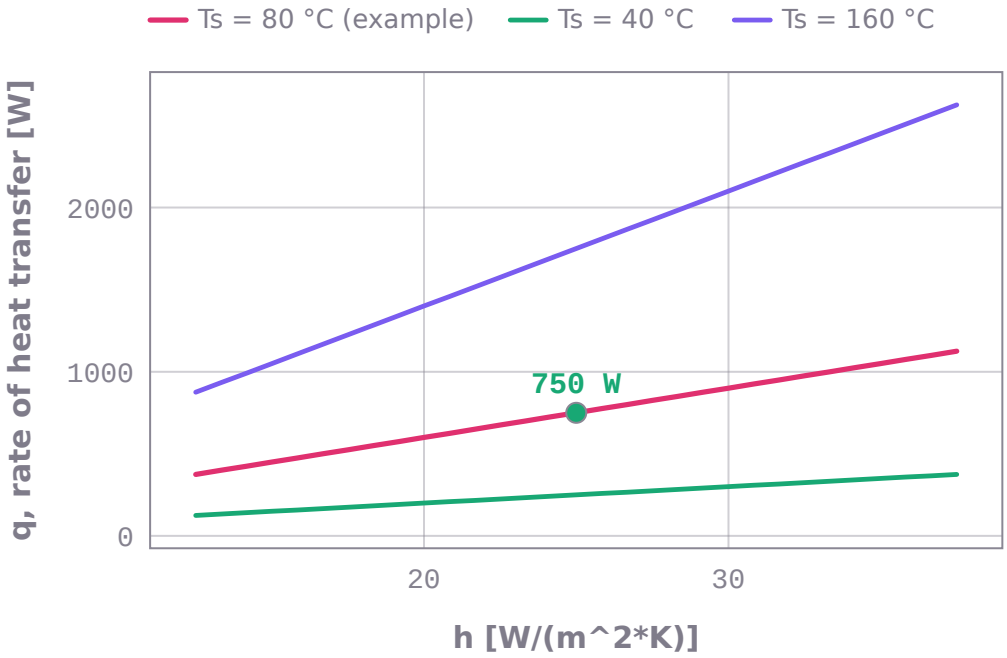
- "A steel plate with thermal diffusivity 0.00001 m^2/s and characteristic length 0.02 m is heated for a time of 60 s. What is the Fourier number?" → 1.5 1
- "Find the Fourier number after 600 s for a body of characteristic length 0.01 m whose thermal diffusivity is 0.00000014 m^2/s." → 0.84 1

Newton's law of cooling (convection heat transfer), `newton_cooling`

$$q = hA\left(T_s - \mathrm{Tinf}\right)$$

input	meaning	kind / role	unit	default
`q`	rate of heat transfer	power / heat_rate	`W`	
`h`	convection heat transfer coefficient	heat_transfer_coefficient / plain	`W/(m^2*K)`	
`A`	surface area in contact with the fluid	area / plain	`m^2`	
`Ts`	surface temperature	temperature / surface	`K`	
`Tinf`	temperature of the fluid far from the surface	temperature / fluid	`K`	

- **Conditions:** convection
- **Applies when:** heat exchange between a surface and a moving or still fluid
- **Fails when:** radiation not negligible
- **Assumes:** uniform surface temperature; h averaged over the surface
- **Evidence:** theorem



q (rate of heat transfer) against h (convection heat transfer coefficient) from 12.5 W/(m^2*K) to 37.5 W/(m^2*K). One curve for each value of Ts (surface temperature): 80 °C, 40 °C, 160 °C. Other inputs as in the worked example: A = 0.5 m^2, Tinf = 20 °C. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, h = 25 W/(m^2*K), q = 750 W.

```
thermofluids_ai999x.solve('newton_cooling', h='25 W/(m^2*K)', A='0.5 m^2', Ts='80 °C',
Tinf='20 °C', target='q')
# -> 750 W   (SOLVED)
```

$$q = hA\left(T_s - \mathrm{Tinf}\right)$$
$$q = Ah\left(-\mathrm{Tinf} + T_s\right) = 750\ \mathrm{W}$$

invalid call	outcome	message
without `h`	NEEDS_INPUT	newton_cooling needs h (convection heat transfer coefficient, W/(m^2*K)). Example: solve('newton_cooling', h='... W/(m^2*K)', A='... m^2', T
`h` in a wrong unit	INVALID_INPUT	h is a heat transfer coefficient: its unit must be like W/(m^2*K), not 'kg'.

Questions in words that work:

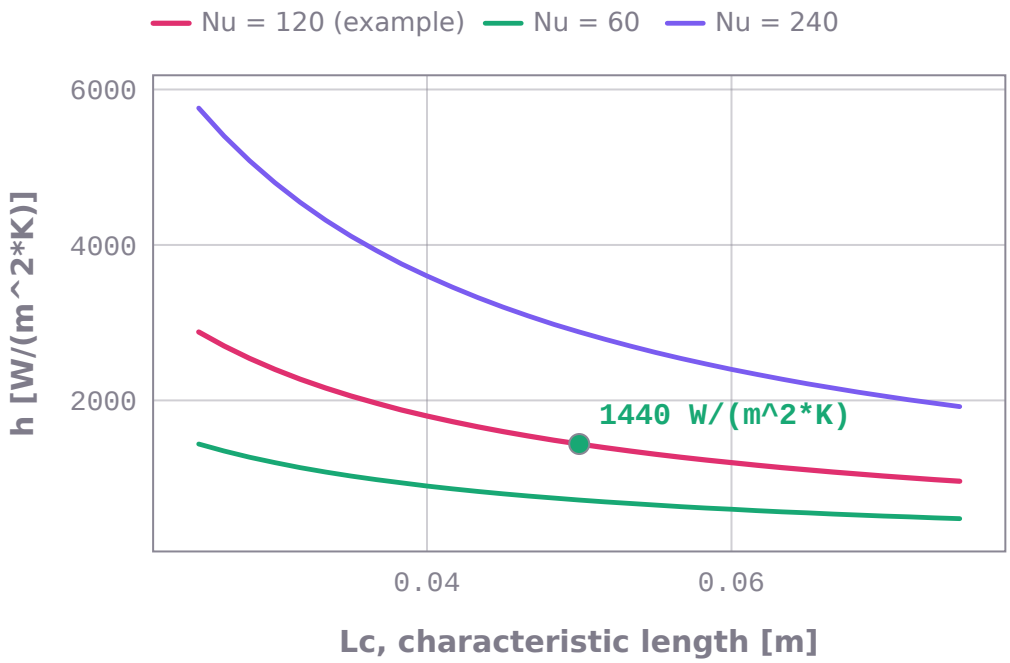
- "A hot plate with a surface area of 0.5 m^2 is cooled by convection with a heat transfer coefficient of 25 W/(m^2*K). The surface temperature is 80 °C and the air temperature is 20 °C. What is the rate of heat transfer?" → 750 W
- "Find the convective heat loss from a surface of area 2 m^2 at a surface temperature of 350 K to water at a fluid temperature of 300 K if the heat transfer coefficient is 100 W/(m^2*K)." → 10000 W

Heat transfer coefficient from the Nusselt number, `nusselt_heat_transfer_coefficient`

$$h = \frac{\mathrm{Nu}k}{L_c}$$

input	meaning	kind / role	unit	default
`h`	convection heat transfer coefficient	heat_transfer_coefficient / plain	`W/(m^2*K)`	
`Nu`	average Nusselt number	nusselt_number / plain	`1`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`Lc`	characteristic length (diameter or plate length)	length / characteristic	`m`	

- **Conditions:** nusselt
- **Applies when:** turning a convection correlation's Nusselt number into h
- **Fails when:** k of the solid used instead of the fluid
- **Assumes:** Nu based on the same characteristic length; k of the fluid
- **Evidence:** theorem



h (convection heat transfer coefficient) against Lc (characteristic length (diameter or plate length)) from 0.025 m to 0.075 m. One curve for each value of Nu (average Nusselt number): 120, 60, 240. Other inputs as in the worked example: k = 0.6 W/(m*K). 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Lc = 0.05 m, h = 1440 W/(m^2*K).

```
thermofluids_ai999x.solve('nusselt_heat_transfer_coefficient', Nu='120 1', k='0.6 W/(m*K)',
Lc='0.05 m', target='h')
# -> 1440 W/(m^2*K)   (SOLVED)
```

$$h = \frac{\mathrm{Nu} k}{L_c}$$

$$h = \frac{\mathrm{Nu} k}{L_c} = 1440\ \mathrm{W/(m^2\,K)}$$

invalid call	outcome	message
without `Nu`	NEEDS_INPUT	nusselt_heat_transfer_coefficient needs Nu (average Nusselt number, 1). Example: solve('nusselt_heat_transfer_coefficient', Nu='... 1', k='.
`Nu` in a wrong unit	INVALID_INPUT	Nu is a nusselt number: its unit must be like 1, not 'kg'.

Questions in words that work:

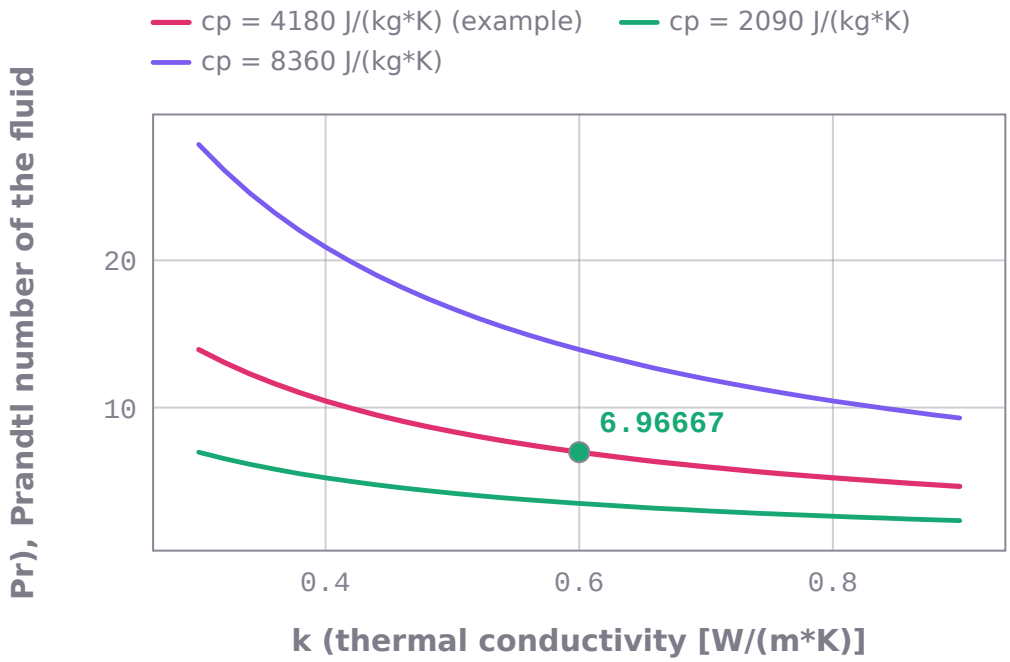
- "Water with thermal conductivity 0.6 W/(m*K) flows in a tube of diameter 0.05 m, and the Nusselt number is 120. What is the heat transfer coefficient?" → 1440 W/(m^2*K)
- "For air flow over a plate the average Nusselt number is 250. If the plate length is 1 m and the air has a thermal conductivity of 0.026 W/(m*K), find the heat transfer coefficient." → 6.5 W/(m^2*K)

Prandtl number, `prandtl_number`

$$\mathrm{Pr} = \frac{c_p \mu}{k}$$

input	meaning	kind / role	unit	default
`Pr`	Prandtl number of the fluid	prandtl_number / plain	`1`	
`cp`	specific heat of the fluid	specific_heat / plain	`J/(kg*K)`	
`mu`	dynamic viscosity of the fluid	dynamic_viscosity / plain	`Pa*s`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	

- **Conditions:** prandtl
- **Applies when:** any Newtonian fluid
- **Fails when:** non-Newtonian fluids
- **Assumes:** properties at the film temperature
- **Evidence:** theorem



Pr (Prandtl number of the fluid) against k (thermal conductivity) from 0.3 W/(m*K) to 0.9 W/(m*K). One curve for each value of cp (specific heat of the fluid): 4180 J/(kg*K), 2090 J/(kg*K), 8360 J/(kg*K). Other inputs as in the worked example: mu = 0.001 Pa*s. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, k = 0.6 W/(m*K), Pr = 6.96667.

```
thermofluids_ai999x.solve('prandtl_number', cp='4180 J/(kg*K)', mu='0.001 Pa*s', k='0.6
W/(m*K)', target='Pr')
# -> 6.96667 1   (SOLVED)
```

$$\mathrm{Pr} = \frac{c_p \mu}{k}$$

$$\mathrm{Pr} = \frac{c_p \mu}{k} = 6.96667\ 1$$

invalid call	outcome	message
without `cp`	NEEDS_INPUT	prandtl_number needs cp (specific heat of the fluid, J/(kg*K)). Example: solve('prandtl_number', cp='... J/(kg*K)', mu='... Pa*s', k='... W/
`cp` in a wrong unit	INVALID_INPUT	cp is a specific heat: its unit must be like J/(kg*K), not 's'.

Questions in words that work:

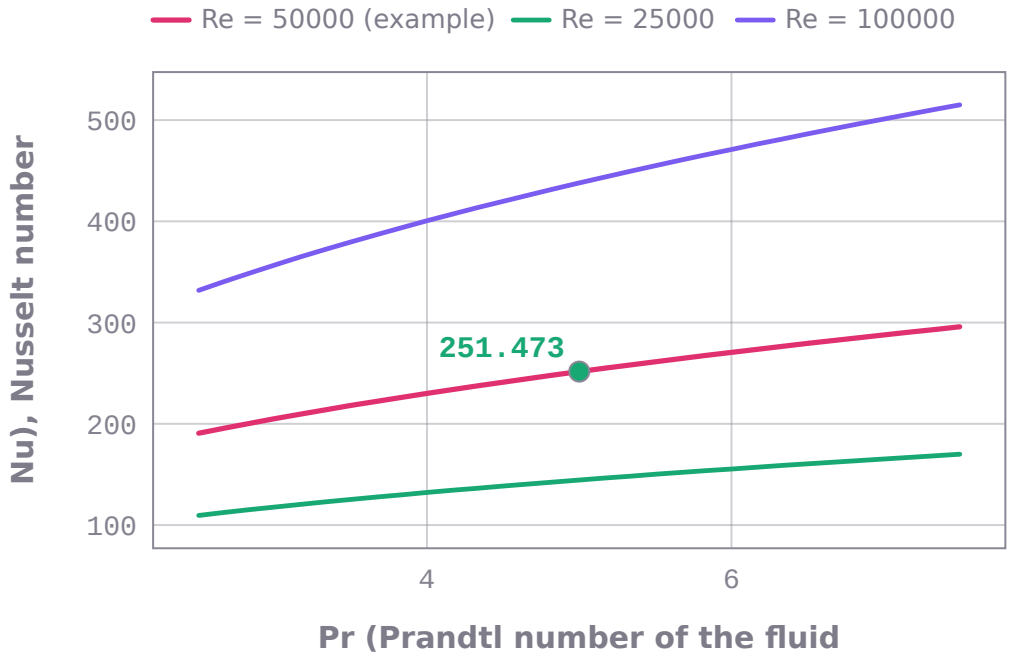
- "Water has a specific heat of 4180 J/(kg*K), a dynamic viscosity of 0.001 Pa*s and a thermal conductivity of 0.6 W/(m*K). What is its Prandtl number?" → 6.96667 1
- "Find the Prandtl number of air with specific heat 1007 J/(kg*K), viscosity 0.0000185 Pa*s and thermal conductivity 0.0263 W/(m*K)." → 0.708346 1

Dittus-Boelter correlation, fluid being heated, `dittus_boelter_heating`

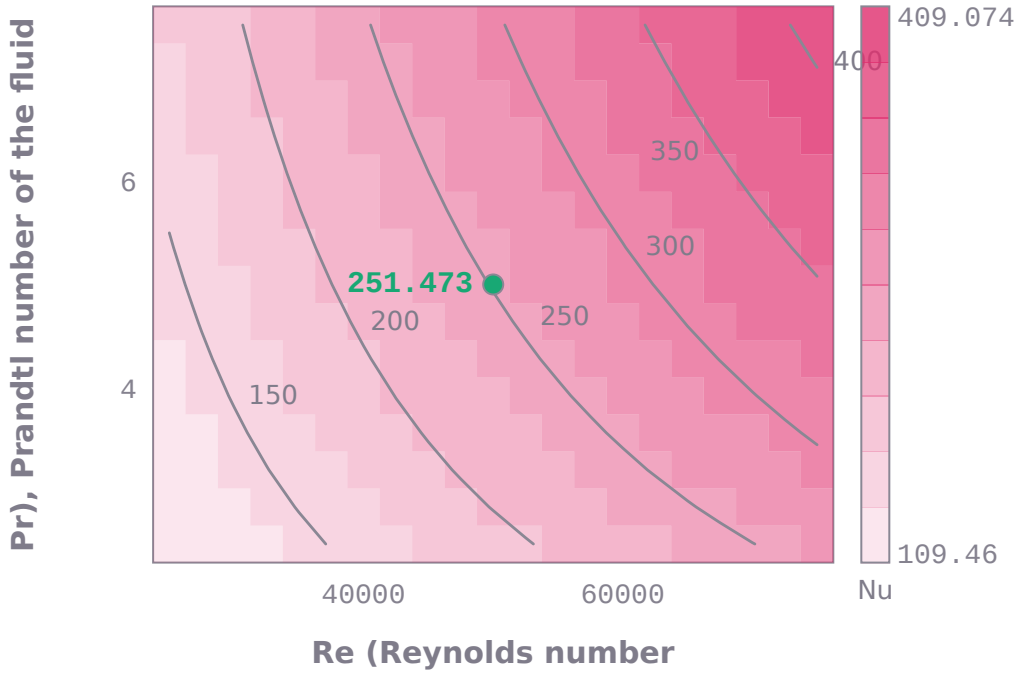
$$\mathrm{Nu} = 0.023 \mathrm{Re}^{\frac{4}{5}} \mathrm{Pr}^{\frac{2}{5}}$$

input	meaning	kind / role	unit	default
`Nu`	Nusselt number	nusselt_number / plain	`1`	
`Re`	Reynolds number	reynolds_number / plain	`1`	
`Pr`	Prandtl number of the fluid	prandtl_number / plain	`1`	

- **Conditions:** dittus_boelter, heated_fluid
- **Not when:** cooled_fluid, laminar
- **Applies when:** $Re > 10000$ and $0.6 < Pr < 160$
- **Fails when:** laminar or transitional flow; large wall-to-bulk temperature differences
- **Assumes:** fully developed turbulent flow in a smooth circular tube; properties at the bulk temperature
- **Evidence:** theorem



Nu (Nusselt number) against Pr (Prandtl number of the fluid) from 2.5 to 7.5. One curve for each value of Re (Reynolds number): 50000, 25000, 100000. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Pr = 5, Nu = 251.473.



Nu (Nusselt number) over Re (Reynolds number) from 25000 to 75000 and Pr (Prandtl number of the fluid) from 2.5 to 7.5: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 109.46 to dark 409.074); iso-lines at Nu = 150, 200, 250, 300, 350, 400. Marked: the worked example, Re = 50000, Pr = 5, Nu = 251.473.

```
thermofluids_ai999x.solve('dittus_boelter_heating', Re='50000 1', Pr='5 1', target='Nu')
# -> 251.473 1   (SOLVED)
```

$$\mathrm{Nu} = 0.023 \mathrm{Re}^{\frac{4}{5}} \mathrm{Pr}^{\frac{2}{5}}$$

$$\mathrm{Nu} = \frac{23 \mathrm{Pr}^{\frac{2}{5}} \mathrm{Re}^{\frac{4}{5}}}{1000} = 251.473\ 1$$

invalid call	outcome	message
without `Re`	NEEDS_INPUT	dittus_boelter_heating needs Re (Reynolds number, 1). Example: solve('dittus_boelter_heating', Re='... 1', Pr='... 1').
`Re` in a wrong unit	INVALID_INPUT	Re is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

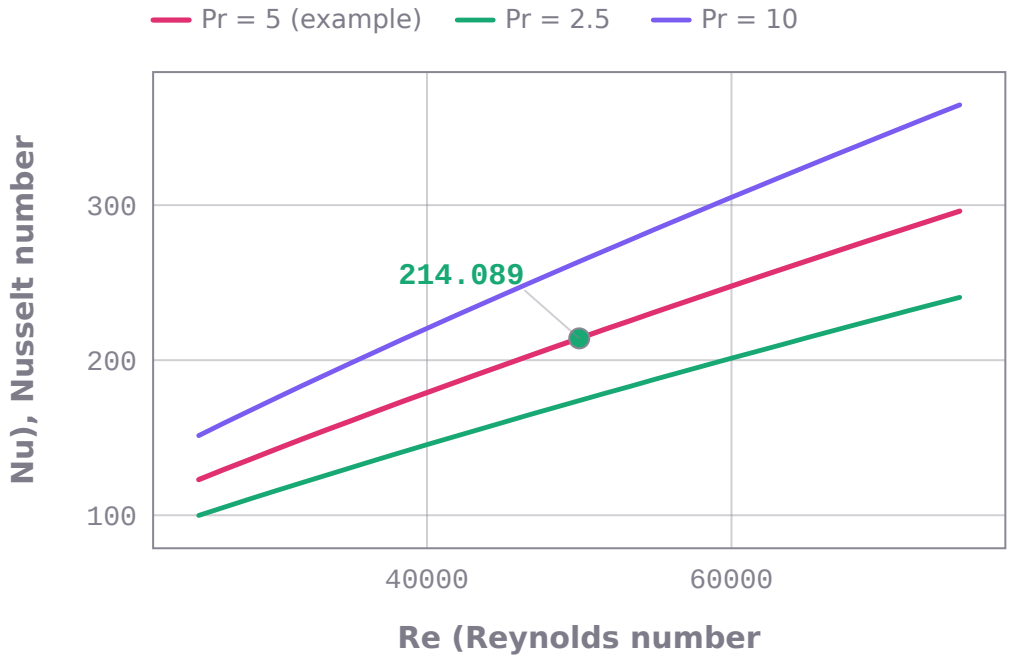
- "Water is heated in a pipe at a Reynolds number of 50000 and a Prandtl number of 5. Using the Dittus-Boelter correlation, what is the Nusselt number?" → 251.473 1
- "Air is heated in a tube with a Reynolds number of 20000 and a Prandtl number of 0.7. Find the Nusselt number from the Dittus-Boelter equation." → 55.0289 1

Dittus-Boelter correlation, fluid being cooled, `dittus_boelter_cooling`

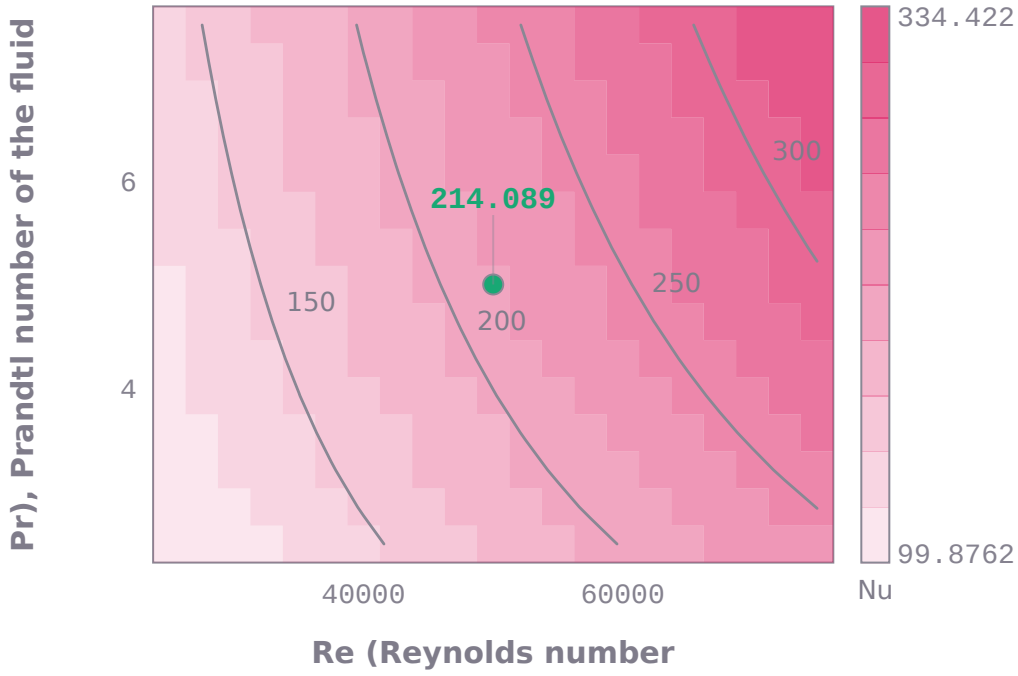
$$\mathrm{Nu} = 0.023 \mathrm{Re}^{\frac{4}{5}} \mathrm{Pr}^{\frac{3}{10}}$$

input	meaning	kind / role	unit	default
`Nu`	Nusselt number	nusselt_number / plain	`1`	
`Re`	Reynolds number	reynolds_number / plain	`1`	
`Pr`	Prandtl number of the fluid	prandtl_number / plain	`1`	

- **Conditions:** dittus_boelter, cooled_fluid
- **Not when:** heated_fluid, laminar
- **Applies when:** $Re > 10000$ and $0.6 < Pr < 160$
- **Fails when:** laminar or transitional flow; large wall-to-bulk temperature differences
- **Assumes:** fully developed turbulent flow in a smooth circular tube; properties at the bulk temperature
- **Evidence:** theorem



Nu (Nusselt number) against Re (Reynolds number) from 25000 to 75000. One curve for each value of Pr (Prandtl number of the fluid): 5, 2.5, 10. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Re = 50000, Nu = 214.089.



Nu (Nusselt number) over Re (Reynolds number) from 25000 to 75000 and Pr (Prandtl number of the fluid) from 2.5 to 7.5: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 99.8762 to dark 334.422); iso-lines at Nu = 150, 200, 250, 300. Marked: the worked example, Re = 50000, Pr = 5, Nu = 214.089.

```
thermofluids_ai999x.solve('dittus_boelter_cooling', Re='50000 1', Pr='5 1', target='Nu')
# -> 214.089 1   (SOLVED)
```

$$\mathrm{Nu} = 0.023 \mathrm{Re}^{\frac{4}{5}} \mathrm{Pr}^{\frac{3}{10}}$$

$$\mathrm{Nu} = \frac{23 \mathrm{Pr}^{\frac{3}{10}} \mathrm{Re}^{\frac{4}{5}}}{1000} = 214.089\ 1$$

invalid call	outcome	message
without `Re`	NEEDS_INPUT	dittus_boelter_cooling needs Re (Reynolds number, 1). Example: solve('dittus_boelter_cooling', Re='... 1', Pr='... 1').
`Re` in a wrong unit	INVALID_INPUT	Re is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

- "Water is cooled in a pipe at a Reynolds number of 50000 and a Prandtl number of 5. Using the Dittus-Boelter correlation, what is the Nusselt number?" → 214.089 1
- "Air is cooled in a tube with a Reynolds number of 20000 and a Prandtl number of 0.7. Find the Nusselt number from the Dittus-Boelter equation." → 57.0271 1

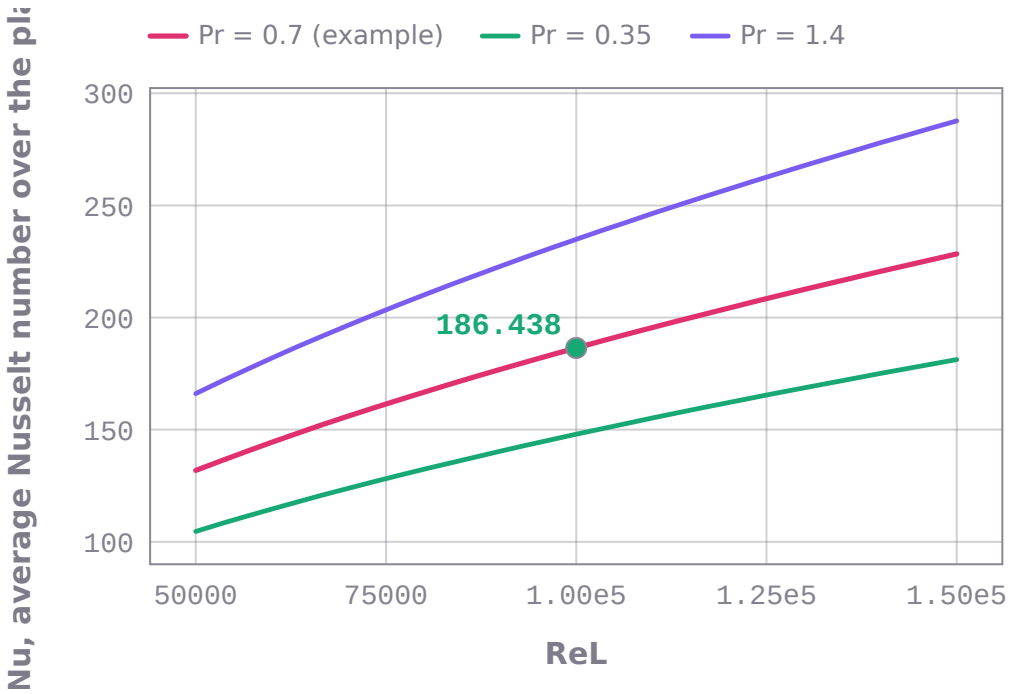
Average Nusselt number for laminar flow over a flat plate,

`flat_plate_laminar_average_nusselt`

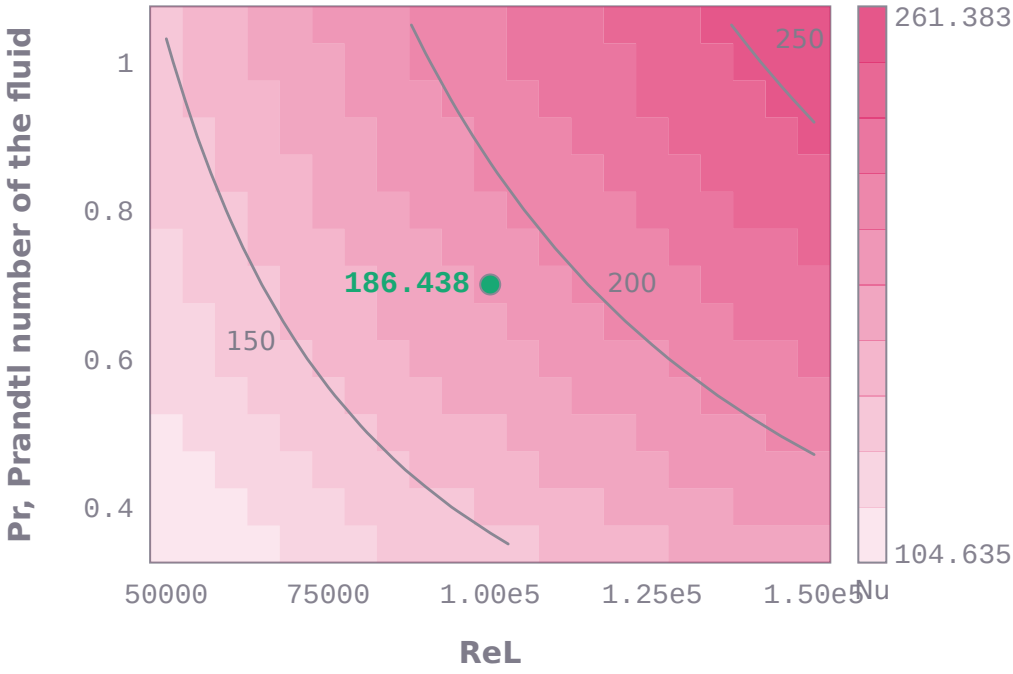
$$\mathrm{Nu} = 0.664 \mathrm{ReL}^{\frac{1}{2}} \mathrm{Pr}^{\frac{1}{3}}$$

input	meaning	kind / role	unit	default
`Nu`	average Nusselt number over the plate	nusselt_number / average	`1`	
`ReL`	Reynolds number based on the plate length	reynolds_number / plate	`1`	
`Pr`	Prandtl number of the fluid	prandtl_number / plain	`1`	

- **Conditions:** flat_plate, laminar
- **Not when:** turbulent
- **Applies when:** $Re_L < 500000$ and $0.6 < Pr < 10$
- **Fails when:** mixed laminar-turbulent plates; liquid metals
- **Assumes:** laminar boundary layer over the whole plate; isothermal plate
- **Evidence:** theorem



Nu (average Nusselt number over the plate) against ReL (Reynolds number based on the plate length) from 50000 to 150000. One curve for each value of Pr (Prandtl number of the fluid): 0.7, 0.35, 1.4. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, ReL = 100000, Nu = 186.438.



Nu (average Nusselt number over the plate) over ReL (Reynolds number based on the plate length) from 50000 to 150000 and Pr (Prandtl number of the fluid) from 0.35 to 1.05: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 104.635 to dark 261.383); iso-lines at Nu = 150, 200, 250. Marked: the worked example, ReL = 100000, Pr = 0.7, Nu = 186.438.

```
thermofluids_ai999x.solve('flat_plate_laminar_average_nusselt', ReL='100000 1', Pr='0.7 1',
target='Nu')
# -> 186.438 1   (SOLVED)
```

$$\mathrm{Nu} = 0.664 \mathrm{ReL}^{\frac{1}{2}} \mathrm{Pr}^{\frac{1}{3}}$$

$$\mathrm{Nu} = \frac{83 \sqrt[3]{\mathrm{Pr}} \sqrt{\mathrm{ReL}}}{125} = 186.438\ 1$$

invalid call	outcome	message
without `ReL`	NEEDS_INPUT	flat_plate_laminar_average_nusselt needs ReL (Reynolds number based on the plate length, 1). Example: solve('flat_plate_laminar_average_nuss
`ReL` in a wrong unit	INVALID_INPUT	ReL is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

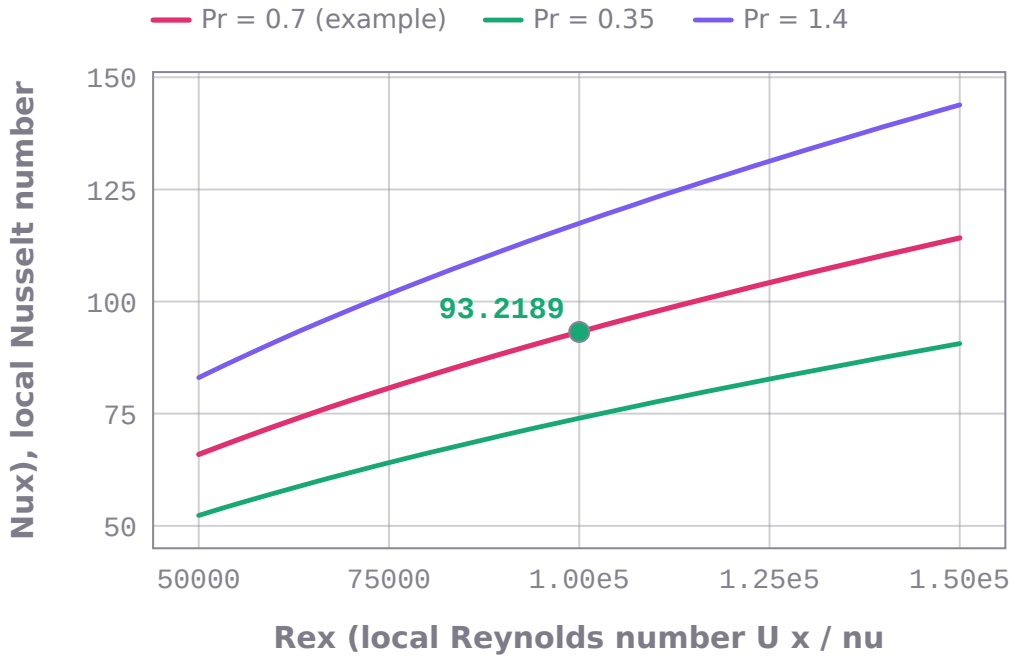
- "Air flows over a flat plate with a laminar boundary layer. At the end of the plate the Reynolds number is 100000 and the Prandtl number is 0.7. What is the average Nusselt number?" → 186.438 1
- "Find the average Nusselt number for laminar flow of water over a flat plate with a Reynolds number of 20000 and a Prandtl number of 5." → 160.573 1

Local Nusselt number for laminar flow over a flat plate, `flat_plate_laminar_local_nusselt`

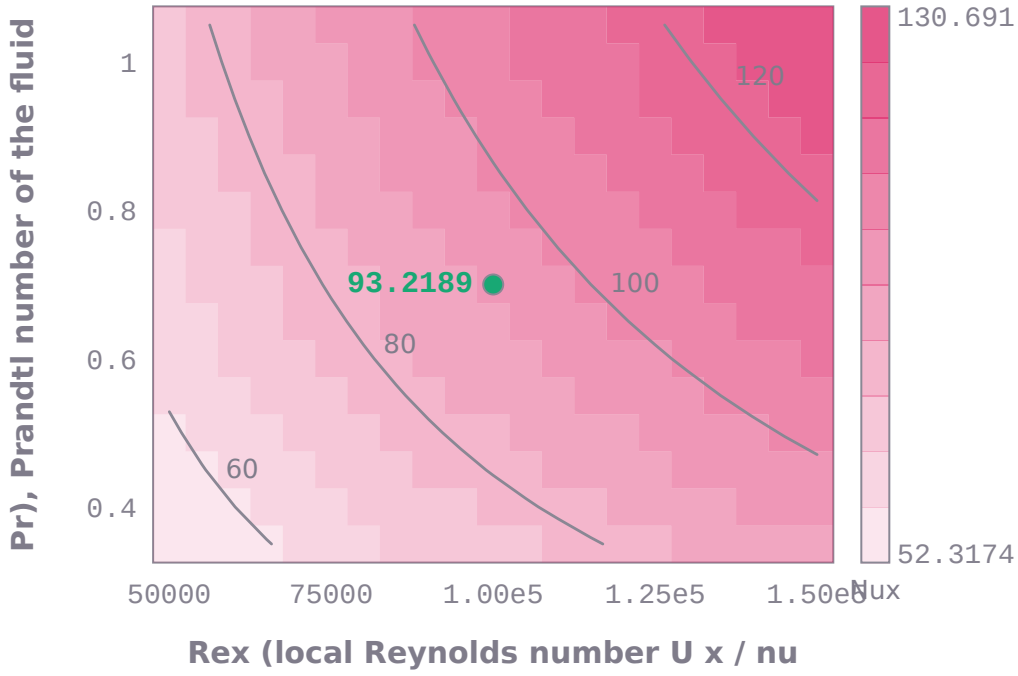
$$\mathrm{Nux} = 0.332 \mathrm{Rex}^{\frac{1}{2}} \mathrm{Pr}^{\frac{1}{3}}$$

input	meaning	kind / role	unit	default
`Nux`	local Nusselt number	nusselt_number / local	`1`	
`Rex`	local Reynolds number U x / nu	reynolds_number / local	`1`	
`Pr`	Prandtl number of the fluid	prandtl_number / plain	`1`	

- **Conditions:** flat_plate, laminar
- **Not when:** turbulent
- **Applies when:** $Re_x < 500000$ and $0.6 < Pr < 10$
- **Fails when:** turbulent boundary layers; liquid metals
- **Assumes:** laminar boundary layer; isothermal plate
- **Evidence:** theorem



Nux (local Nusselt number) against Rex (local Reynolds number U x / nu) from 50000 to 150000. One curve for each value of Pr (Prandtl number of the fluid): 0.7, 0.35, 1.4. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Rex = 100000, Nux = 93.2189.



Nux (local Nusselt number) over Rex (local Reynolds number U x / nu) from 50000 to 150000 and Pr (Prandtl number of the fluid) from 0.35 to 1.05: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 52.3174 to dark 130.691); iso-lines at Nux = 60, 80, 100, 120. Marked: the worked example, Rex = 100000, Pr = 0.7, Nux = 93.2189.

```
thermofluids_ai999x.solve('flat_plate_laminar_local_nusselt', Rex='100000 1', Pr='0.7 1',
target='Nux')
# -> 93.2189 1   (SOLVED)
```

$$\mathrm{Nux} = 0.332 \mathrm{Rex}^{\frac{1}{2}} \mathrm{Pr}^{\frac{1}{3}}$$

$$\mathrm{Nux} = \frac{83 \sqrt[3]{\mathrm{Pr}} \sqrt{\mathrm{Rex}}}{250} = 93.2189\ 1$$

invalid call	outcome	message
without `Rex`	NEEDS_INPUT	flat_plate_laminar_local_nusselt needs Rex (local Reynolds number U x / nu, 1). Example: solve('flat_plate_laminar_local_nusselt', Rex='...
`Rex` in a wrong unit	INVALID_INPUT	Rex is a reynolds number: its unit must be like 1, not 'kg'.

Questions in words that work:

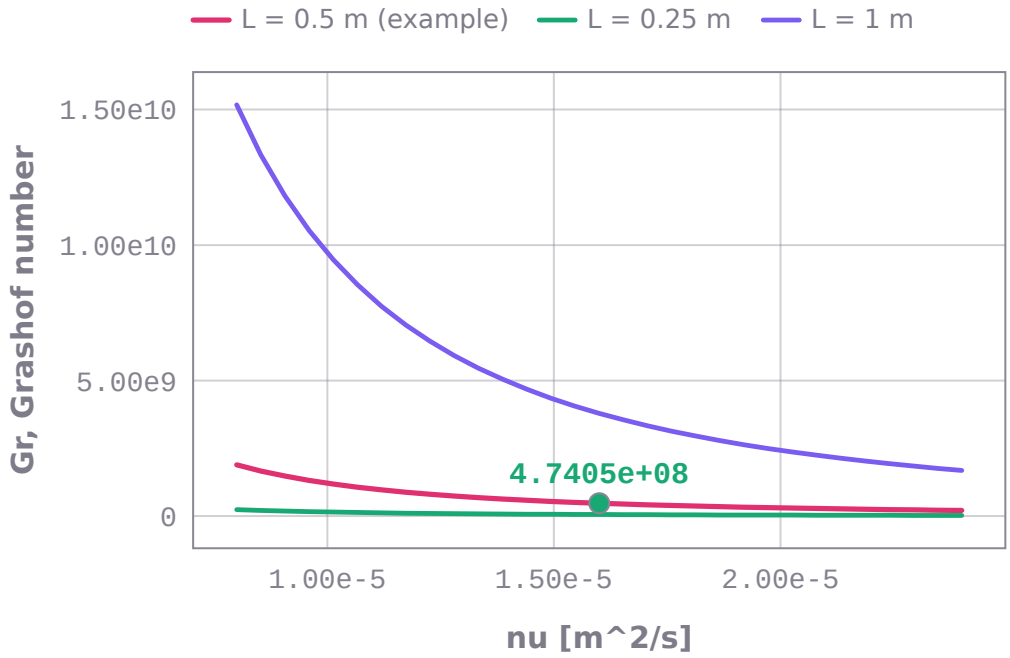
- "For laminar flow over a flat plate, what is the local Nusselt number where the local Reynolds number is 100000 and the Prandtl number is 0.7?" → 93.2189 1
- "Find the local Nusselt number in a laminar boundary layer on a flat plate at a local Reynolds number of 40000 with a Prandtl number of 2." → 83.6588 1

Grashof number, `grashof_number`

$$\mathrm{Gr} = \frac{g\beta\,(T_s - \mathrm{Tinf})L^3}{\nu^2}$$

input	meaning	kind / role	unit	default
`Gr`	Grashof number	grashof_number / plain	`1`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`beta`	volumetric thermal expansion coefficient of the fluid	expansion_coefficient / volume	`1/K`	
`Ts`	surface temperature	temperature / surface	`K`	
`Tinf`	temperature of the fluid far from the surface	temperature / fluid	`K`	
`L`	height of the plate (characteristic length)	length / plate_length	`m`	
`nu`	kinematic viscosity of the fluid	kinematic_viscosity / plain	`m^2/s`	

- **Conditions:** natural_convection
- **Applies when:** buoyancy-driven flow next to a heated or cooled surface
- **Fails when:** large temperature differences
- **Assumes:** Boussinesq approximation; surface hotter than the fluid
- **Evidence:** theorem



Gr (Grashof number) against nu (kinematic viscosity of the fluid) from 8.0000e-06 m^2/s to 2.4000e-05 m^2/s. One curve for each value of L (height of the plate (characteristic length)): 0.5 m, 0.25 m, 1 m. Other inputs as in the worked example: beta = 0.0033 1/K, Ts = 330 K, Tinf = 300 K. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, nu = 1.6000e-05 m^2/s, Gr = 4.7405e+08.

```
thermofluids_ai999x.solve('grashof_number', beta='0.0033 1/K', Ts='330 K', Tinf='300 K',
L='0.5 m', nu='1.6e-05 m^2/s', target='Gr')
# -> 4.74052e+08 1   (SOLVED)
```

$$\mathrm{Gr} = \frac{g\beta\left(T_s - \mathrm{Tinf}\right)L^3}{\nu^2}$$

$$\mathrm{Gr} = \frac{L^3\beta g\left(-\mathrm{Tinf} + T_s\right)}{\nu^2} = 4.74052e+08\ 1$$

invalid call	outcome	message
without `beta`	NEEDS_INPUT	grashof_number needs beta (volumetric thermal expansion coefficient of the fluid, 1/K). Example: solve('grashof_number', g='... m/s^2', beta
`beta` in a wrong unit	INVALID_INPUT	beta is a expansion coefficient: its unit must be like 1/K, not 'kg'.

Questions in words that work:

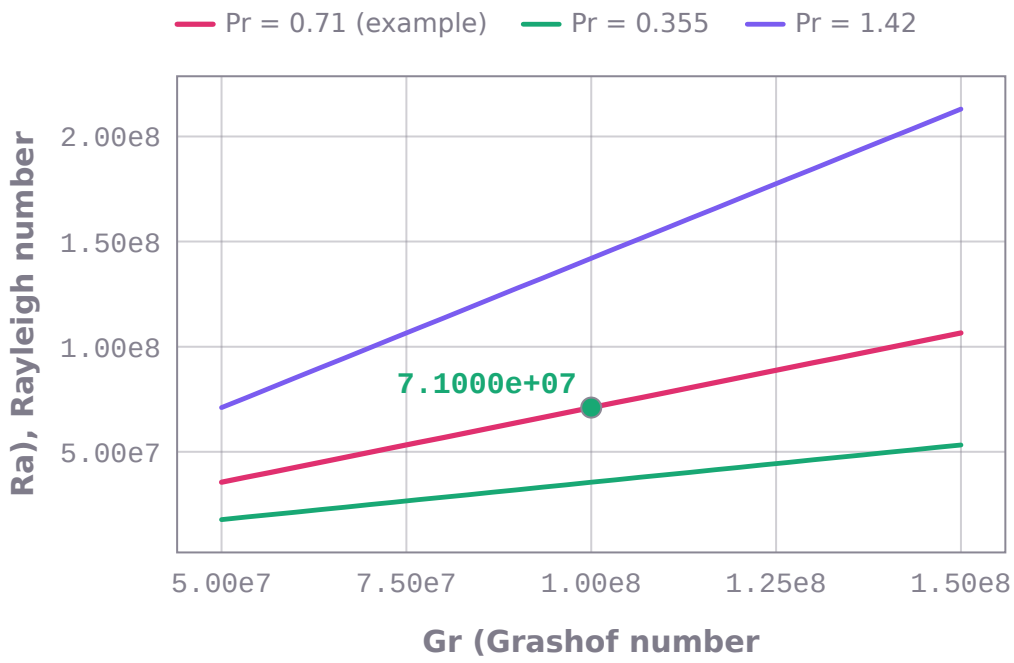
- "A vertical plate of height 0.5 m has a surface temperature of 330 K in still air at an air temperature of 300 K. The air has a coefficient of volume expansion of 0.0033 1/K and a kinematic viscosity of 0.000016 m^2/s. What is the Grashof number for natural convection?" → 4.74052e+08 1
- "Find the Grashof number for free convection from a plate of height 0.2 m with a surface temperature of 60 °C in water at a fluid temperature of 20 °C, if the water's volume expansion coefficient is 0.0002 1/K and its kinematic viscosity is 0.000001 m^2/s." → 6.27626e+08 1

Rayleigh number, `rayleigh_number`

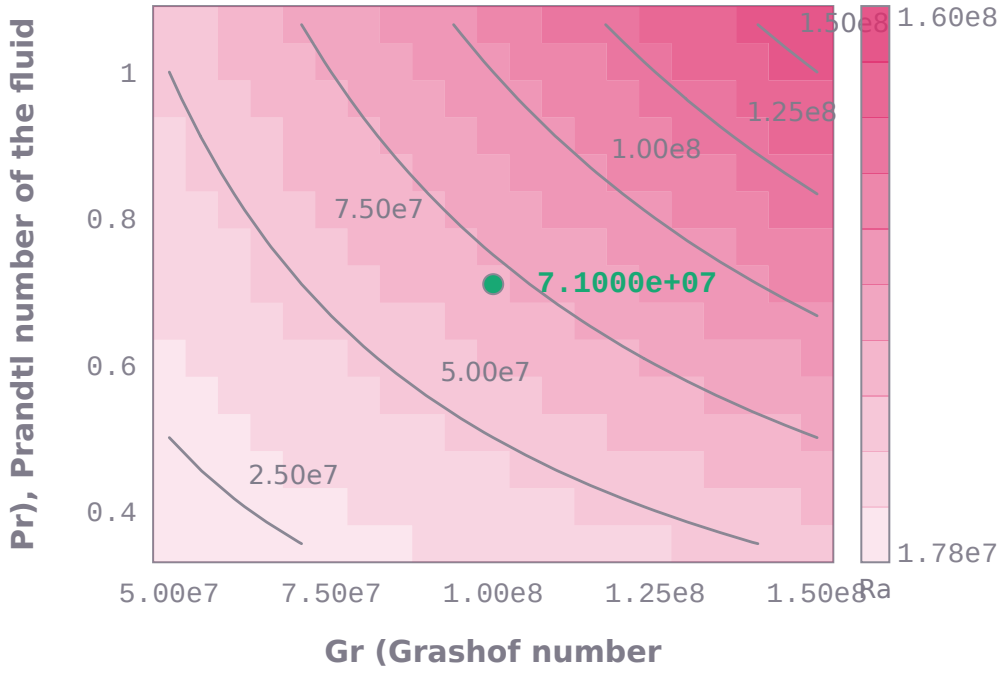
$$\mathrm{Ra} = \mathrm{GrPr}$$

input	meaning	kind / role	unit	default
`Ra`	Rayleigh number	rayleigh_number / plain	`1`	
`Gr`	Grashof number	grashof_number / plain	`1`	
`Pr`	Prandtl number of the fluid	prandtl_number / plain	`1`	

- **Conditions:** rayleigh
- **Applies when:** natural convection correlations
- **Fails when:** forced convection
- **Assumes:** properties at the film temperature
- **Evidence:** theorem



Ra (Rayleigh number) against Gr (Grashof number) from 5.0000e+07 to 1.5000e+08. One curve for each value of Pr (Prandtl number of the fluid): 0.71, 0.355, 1.42. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Gr = 1.0000e+08, Ra = 7.1000e+07.



Ra (Rayleigh number) over Gr (Grashof number) from 5.0000e+07 to 1.5000e+08 and Pr (Prandtl number of the fluid) from 0.355 to 1.065: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 1.7750e+07 to dark 1.5975e+08); iso-lines at Ra = 2.50e7, 5.00e7, 7.50e7, 1.00e8, 1.25e8, 1.50e8. Marked: the worked example, Gr = 1.0000e+08, Pr = 0.71, Ra = 7.1000e+07.

```
thermofluids_ai999x.solve('rayleigh_number', Gr='1e+08 1', Pr='0.71 1', target='Ra')
# -> 7.1e+07 1   (SOLVED)
```

$$\mathbf{Ra} = \mathbf{GrPr}$$
$$\mathbf{Ra} = \mathbf{GrPr} = 7.1e+07\ 1$$

invalid call	outcome	message
without `Gr`	NEEDS_INPUT	rayleigh_number needs Gr (Grashof number, 1). Example: solve('rayleigh_number', Gr='... 1', Pr='... 1').
`Gr` in a wrong unit	INVALID_INPUT	Gr is a grashof number: its unit must be like 1, not 'kg'.

Questions in words that work:

- "Natural convection from a heated wall has a Grashof number of 100000000 and a Prandtl number of 0.71. What is the Rayleigh number?" → 7.1e+07 1
- "Find the Rayleigh number for free convection in water with a Grashof number of 2500000 and a Prandtl number of 6." → 1.5e+07 1

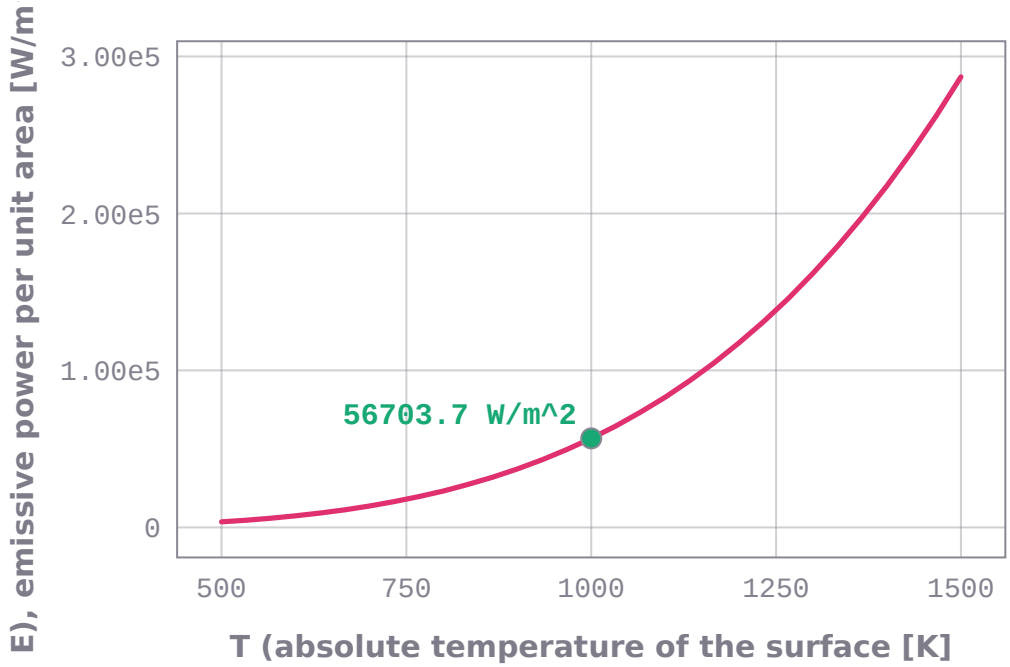
Emissive power of a blackbody (Stefan-Boltzmann law), `blackbody_emissive_power`

$$E = \sigma T^4$$

input	meaning	kind / role	unit	default
`E`	emissive power per unit area	heat_flux / emissive	`W/m^2`	

input	meaning	kind / role	unit	default
`sigma`		stefan_boltzmann_constant / constant	`W/(m^2*K^4)`	5.670374419e-08 W/(m^2*K^4)
`T`	absolute temperature of the surface	temperature / surface	`K`	

- **Conditions:** radiation, blackbody
- **Applies when:** total hemispherical emission of a black surface
- **Fails when:** real (grey) surfaces without an emissivity
- **Assumes:** an ideal blackbody; absolute temperature
- **Evidence:** theorem



E (emissive power per unit area) against T (absolute temperature of the surface) from 500 K to 1500 K. 31 points computed by thermofluids-ai999x; marked: the worked example, T = 1000 K, E = 56703.7 W/m^2.

```
thermofluids_ai999x.solve('blackbody_emissive_power', T='1000 K', target='E')
# -> 56703.7 W/m^2   (SOLVED)
```

$$E = \sigma T^4$$
$$E = T^4 \sigma = 56703.7\ \mathrm{W/m^2}$$

invalid call	outcome	message
without `T`	NEEDS_INPUT	blackbody_emissive_power needs T (absolute temperature of the surface, K). Example: solve('blackbody_emissive_power', sigma='... W/(m^2*K^4)
`T` in a wrong unit	INVALID_INPUT	T is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

- "What is the emissive power of a blackbody at 1000 K?" → 56703.7 W/m^2
- "The Sun radiates approximately as a black body with a surface temperature of 5800 K. Find its emissive power." → 6.41688e+07 W/m^2

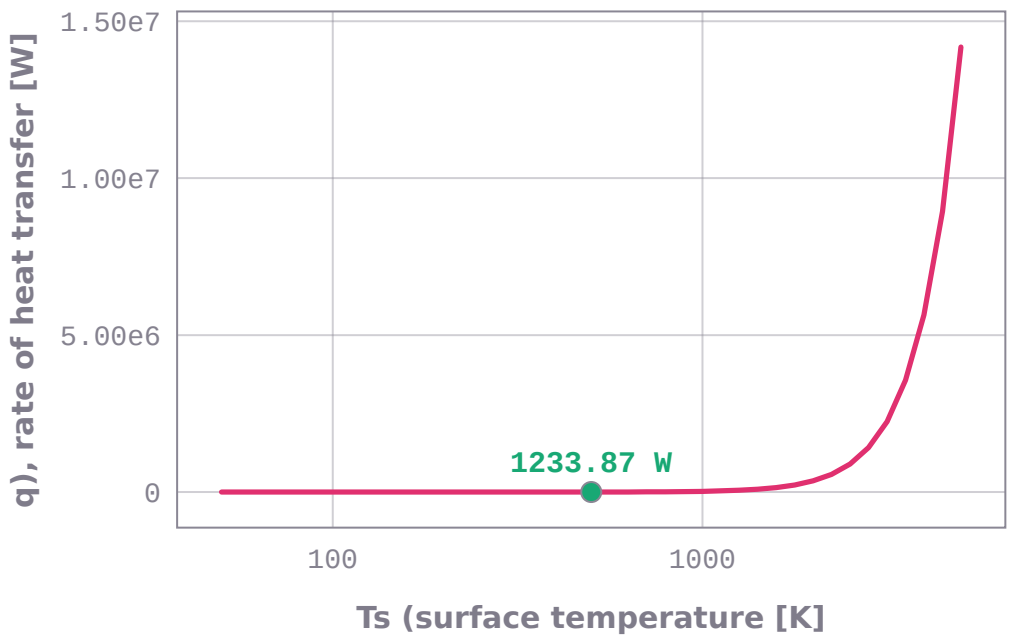
Net radiation exchange of a small grey surface with large surroundings,

`radiation_net_exchange`

$$q = \varepsilon \sigma A \left(T_s^4 - \mathrm{Tsur}^4\right)$$

input	meaning	kind / role	unit	default
`q`	rate of heat transfer	power / heat_rate	`W`	
`eps`	emissivity of the surface	emissivity / plain	`1`	
`sigma`		stefan_boltzmann_constant / constant	`W/(m^2*K^4)`	5.670374419e-08 W/(m^2*K^4)
`A`	surface area	area / plain	`m^2`	
`Ts`	surface temperature	temperature / surface	`K`	
`Tsur`	temperature of the surroundings	temperature / surroundings	`K`	

- **Conditions:** radiation
- **Not when:** parallel_plates
- **Applies when:** a small body inside a large enclosure
- **Fails when:** surfaces of comparable size
- **Assumes:** grey diffuse surface; surroundings large compared with the surface
- **Evidence:** theorem



q (rate of heat transfer) against Ts (surface temperature) from 50 K to 5000 K on a logarithmic axis. Other inputs as in the worked example: eps = 0.8 1, A = 0.5 m^2, Tsur = 300 K. 41 points computed by thermofluids-ai999x; marked: the worked example, Ts = 500 K, q = 1233.87 W.

```
thermofluids_ai999x.solve('radiation_net_exchange', eps='0.8 1', A='0.5 m^2', Ts='500 K',
Tsur='300 K', target='q')
# -> 1233.87 W   (SOLVED)
```

$$q = \varepsilon \sigma A \left(T_s^4 - \mathrm{Tsur}^4\right)$$
$$q = A \varepsilon \sigma \left(T_s^4 - \mathrm{Tsur}^4\right) = 1233.87\ \mathrm{W}$$

invalid call	outcome	message
without `eps`	NEEDS_INPUT	radiation_net_exchange needs eps (emissivity of the surface, 1). Example: solve('radiation_net_exchange', eps='... 1', sigma='... W/(m^2*K^4
`eps` in a wrong unit	INVALID_INPUT	eps is a emissivity: its unit must be like 1, not 'kg'.

Questions in words that work:

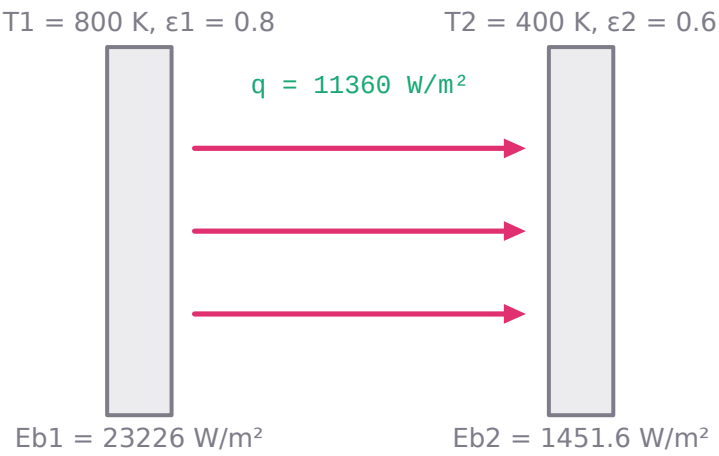
- "A surface of area 0.5 m^2 with an emissivity of 0.8 is at a surface temperature of 500 K in a large room whose surroundings are at 300 K. What is the net radiation heat transfer from the surface?" → 1233.87 W
- "A person with a surface area of 1.2 m^2 and emissivity 0.9 has a surface temperature of 35 °C, and the surroundings temperature is 20 °C. Find the radiation heat loss." → 99.9173 W

Net radiation between two large parallel grey plates, `radiation_parallel_plates`

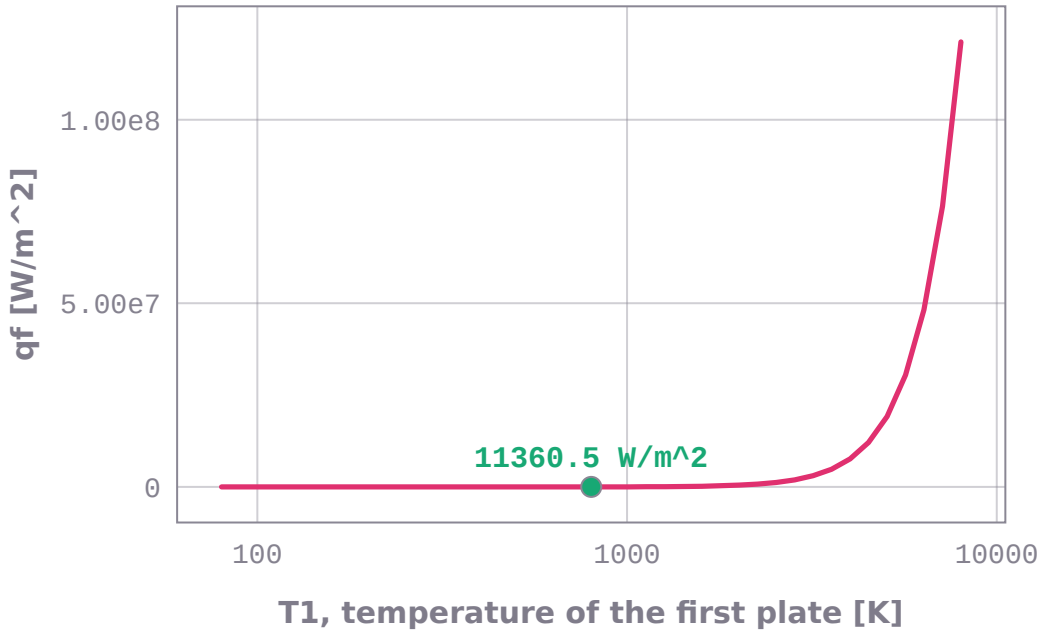
$$q_f = \frac{\sigma \left(T_1^4 - T_2^4\right)}{\frac{1}{e_1} + \frac{1}{e_2} - 1}$$

input	meaning	kind / role	unit	default
`qf`	net radiative heat flux from plate 1 to plate 2	heat_flux / net	`W/m^2`	
`sigma`		stefan_boltzmann_constant / constant	`W/(m^2*K^4)`	5.670374419e-08 W/(m^2*K^4)
`T1`	temperature of the first plate	temperature / first_plate	`K`	
`T2`	temperature of the second plate	temperature / second_plate	`K`	
`e1`	emissivity of the first plate	emissivity / first	`1`	
`e2`	emissivity of the second plate	emissivity / second	`1`	

- **Conditions:** radiation, parallel_plates
- **Applies when:** closely spaced large plates
- **Fails when:** finite plates far apart (view factor below 1)
- **Assumes:** infinite parallel grey diffuse plates; no participating medium
- **Evidence:** theorem



Radiation between two large parallel grey plates at 800 K (emissivity 0.8) and 400 K (emissivity 0.6), the worked example. Marked: the package's net flux q = 11360.5 W/m², and the blackbody emissive powers of the two surfaces from its blackbody capability, 23225.9 and 1451.62 W/m².



qf (net radiative heat flux from plate 1 to plate 2) against T1 (temperature of the first plate) from 80 K to 8000 K on a logarithmic axis. Other inputs as in the worked example: T2 = 400 K, e1 = 0.8 1, e2 = 0.6 1. 41 points computed by thermofluids-ai999x; marked: the worked example, T1 = 800 K, qf = 11360.5 W/m^2.

```
thermofluids_ai999x.solve('radiation_parallel_plates', T1='800 K', T2='400 K', e1='0.8 1',
e2='0.6 1', target='qf')
# -> 11360.5 W/m^2   (SOLVED)
```

$$q_f = \frac{\sigma\left(T_1^4 - T_2^4\right)}{\frac{1}{e_1} + \frac{1}{e_2} - 1}$$

$$q_f = \frac{e_1 e_2 \sigma\left(T_1^4 - T_2^4\right)}{-e_1 e_2 + e_1 + e_2} = 11360.5\ \mathrm{W/m^2}$$

invalid call	outcome	message
without `T1`	NEEDS_INPUT	radiation_parallel_plates needs T1 (temperature of the first plate, K). Example: solve('radiation_parallel_plates', sigma='... W/(m^2*K^4)',
`T1` in a wrong unit	INVALID_INPUT	T1 is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

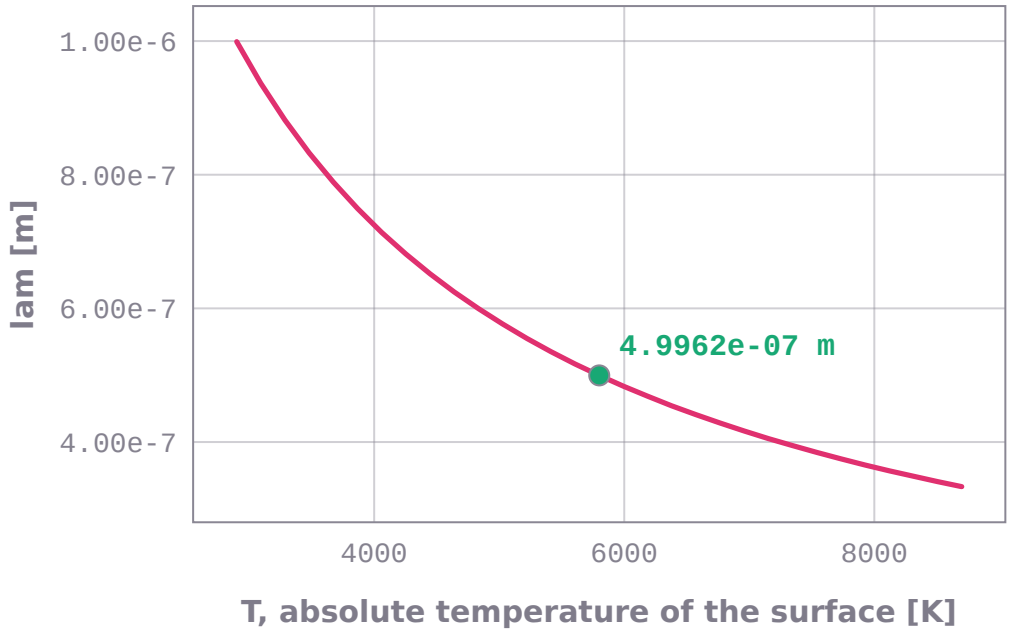
- "Two large parallel plates exchange radiation. The first plate is at 800 K and the second plate is at 400 K. The emissivity of the first plate is 0.8 and the emissivity of the second plate is 0.6. What is the net radiation heat flux between them?" → 11360.5 W/m^2
- "Find the net radiative heat flux between two parallel plates with a first plate temperature of 600 K and a second plate temperature of 300 K, if the emissivity of the first plate is 0.1 and the emissivity of the second plate is 0.1." → 362.606 W/m^2

Wien's displacement law, `wien_displacement`

$$\lambda = \frac{b}{T}$$

input	meaning	kind / role	unit	default
`lam`	wavelength of maximum spectral emission	length / peak_wavelength	`m`	
`T`	absolute temperature of the surface	temperature / surface	`K`	
`b`		wien_constant / constant	`m*K`	0.002897771955185172 m*K

- **Conditions:** peak_emission
- **Applies when:** the peak of the Planck spectrum
- **Fails when:** strongly selective emitters
- **Assumes:** blackbody (or grey body) emission
- **Evidence:** theorem



lam (wavelength of maximum spectral emission) against T (absolute temperature of the surface) from 2900 K to 8700 K. 31 points computed by thermofluids-ai999x; marked: the worked example, T = 5800 K, lam = 4.9962e-07 m.

```
thermofluids_ai999x.solve('wien_displacement', T='5800 K', target='lam')
# -> 4.99616e-07 m   (SOLVED)
```

$$\lambda = \frac{b}{T}$$

$$\lambda = \frac{b}{T} = 4.99616e-07\ \mathrm{m}$$

invalid call	outcome	message
without `T`	NEEDS_INPUT	wien_displacement needs T (absolute temperature of the surface, K). Example: solve('wien_displacement', T='... K', b='... m*K').
`T` in a wrong unit	INVALID_INPUT	T is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

- "A star has a surface temperature of 5800 K. At what peak wavelength does it emit most strongly?" → 4.99616e-07 m
- "Find the wavelength of maximum emission of a blackbody at a temperature of 300 K." → 9.65924e-06 m

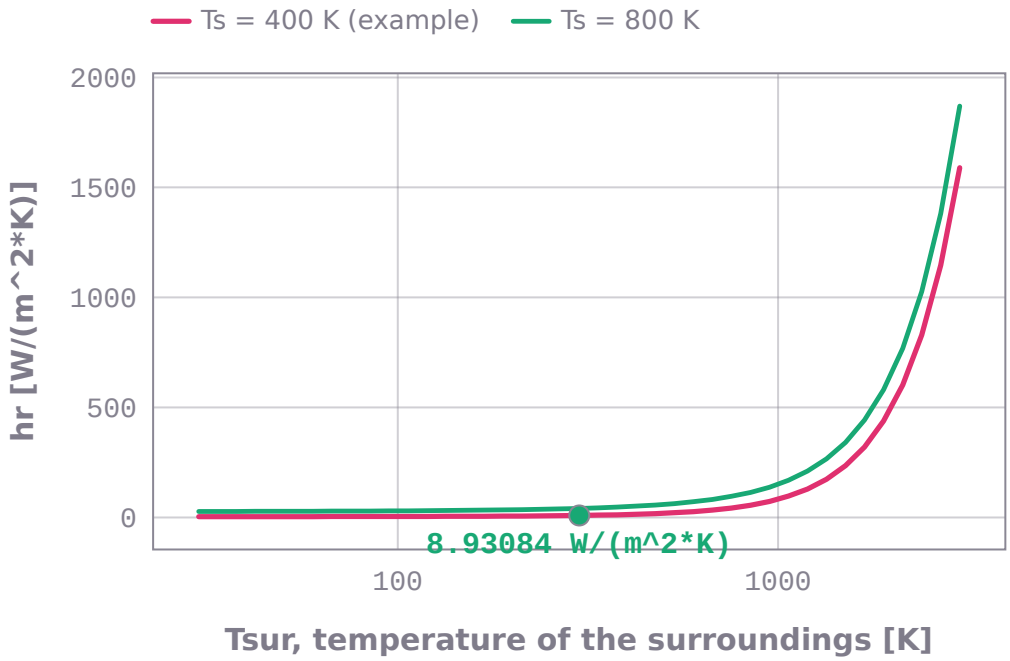
Linearised radiation heat transfer coefficient, `radiation_heat_transfer_coefficient`

$$h_r = \varepsilon\sigma\left(T_s + \mathrm{Tsur}\right)\left(T_s^2 + \mathrm{Tsur}^2\right)$$

input	meaning	kind / role	unit	default
`hr`	radiation heat transfer coefficient	heat_transfer_coefficient / radiation	`W/(m^2*K)`	

input	meaning	kind / role	unit	default
`eps`	surface emissivity	emissivity / plain	`1`	
`sigma`		stefan_boltzmann_constant / constant	`W/(m^2*K^4)`	5.670374419e-08 W/(m^2*K^4)
`Ts`	surface temperature	temperature / surface	`K`	
`Tsur`	temperature of the surroundings	temperature / surroundings	`K`	

- **Conditions:** radiation
- **Applies when:** combining radiation with convection
- **Fails when:** surfaces of comparable size
- **Assumes:** small grey surface in large surroundings
- **Evidence:** theorem



hr (radiation heat transfer coefficient) against Tsur (temperature of the surroundings) from 30 K to 3000 K on a logarithmic axis. One curve for each value of Ts (surface temperature): 400 K, 800 K. Other inputs as in the worked example: eps = 0.9 1. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, Tsur = 300 K, hr = 8.93084 W/(m^2*K).

```
thermofluids_ai999x.solve('radiation_heat_transfer_coefficient', eps='0.9 1', Ts='400 K',
Tsur='300 K', target='hr')
# -> 8.93084 W/(m^2*K)   (SOLVED)
```

$$h_r = \varepsilon\sigma\left(T_s + \mathrm{Tsur}\right)\left(T_s^2 + \mathrm{Tsur}^2\right)$$
$$h_r = \varepsilon\sigma\left(T_s^3 + T_s^2\mathrm{Tsur} + T_s\mathrm{Tsur}^2 + \mathrm{Tsur}^3\right) = 8.93084\ \mathrm{W/(m^2\,K)}$$

invalid call	outcome	message
without `eps`	NEEDS_INPUT	radiation_heat_transfer_coefficient needs eps (surface emissivity, 1). Example: solve('radiation_heat_transfer_coefficient', eps='... 1', si
`eps` in a wrong unit	INVALID_INPUT	eps is a emissivity: its unit must be like 1, not 'kg'.

Questions in words that work:

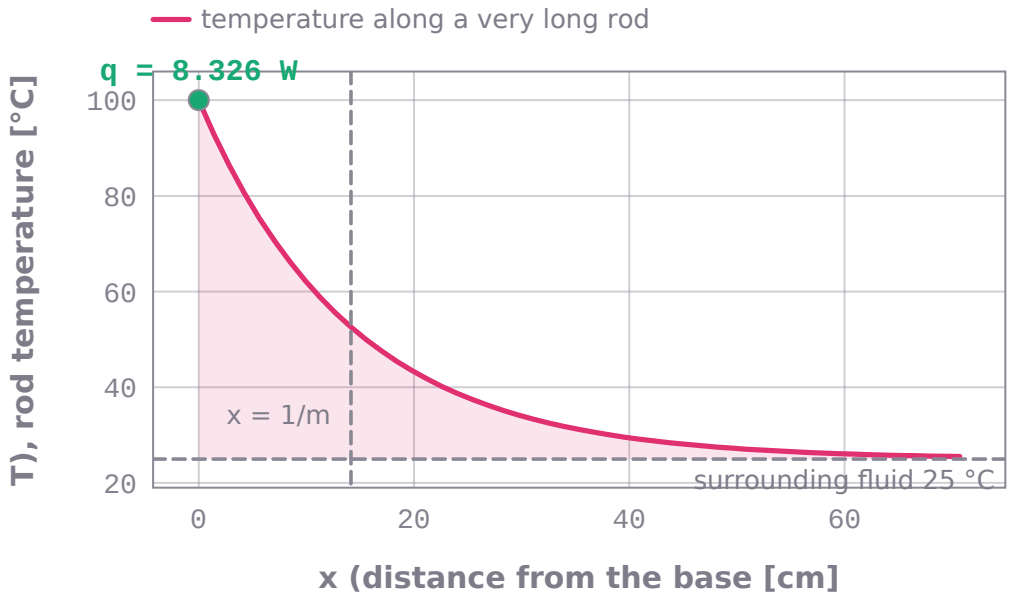
- "A surface with an emissivity of 0.9 is at a surface temperature of 400 K and its surroundings are at 300 K. What is the radiation heat transfer coefficient?" → 8.93084 W/(m^2*K)
- "Find the radiative heat transfer coefficient for a pipe with emissivity 0.5 at a surface temperature of 800 K when the surroundings temperature is 300 K." → 22.7666 W/(m^2*K)

Heat transfer rate of a very long fin, `infinite_fin_heat_rate`

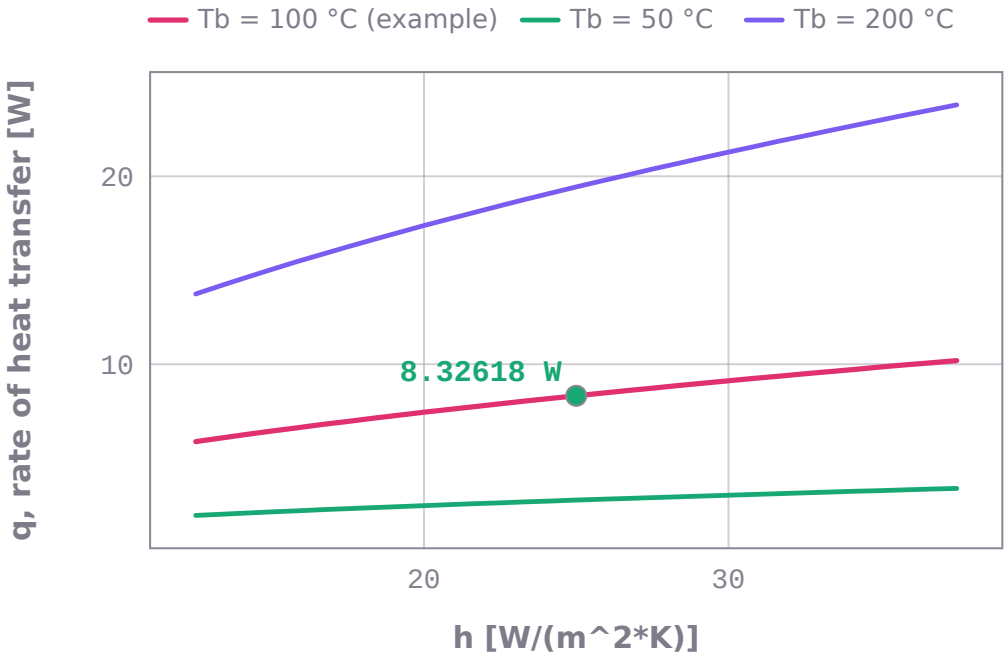
$$q = \sqrt{hPkA_c}\,(T_b - \mathrm{Tinf})$$

input	meaning	kind / role	unit	default
`q`	rate of heat transfer	power / heat_rate	`W`	
`h`	convection heat transfer coefficient	heat_transfer_coefficient / plain	`W/(m^2*K)`	
`P`	perimeter of the fin's cross section	length / perimeter	`m`	
`k`	thermal conductivity	thermal_conductivity / plain	`W/(m*K)`	
`Ac`	cross-sectional area of the fin	area / plain	`m^2`	
`Tb`	temperature at the fin base	temperature / base	`K`	
`Tinf`	temperature of the fluid far from the surface	temperature / fluid	`K`	

- **Conditions:** fin, long_fin
- **Applies when:** m L above about 2.65
- **Fails when:** short fins (use the adiabatic-tip result)
- **Assumes:** one-dimensional conduction along the fin; uniform h; constant cross section; very long fin
- **Evidence:** theorem



Temperature along a very long fin (h = 25 W/(m^2*K), perimeter 0.0314 m, k = 200 W/(m*K), cross-section 7.8500e-05 m^2, base 100 °C in fluid at 25 °C; the worked example): T = T∞ + (Tb − T∞)·e^(−mx) with m = √(hP/(kAc)) = 7.07107 1/m, derived from the inputs. Marked: the package's heat rate through the base, q = 8.32618 W; the excess falls to 1/e at x = 1/m = 14.1421 cm.



q (rate of heat transfer) against h (convection heat transfer coefficient) from 12.5 W/(m^2*K) to 37.5 W/(m^2*K). One curve for each value of Tb (temperature at the fin base): 100 °C, 50 °C, 200 °C. Other inputs as in the worked example: P = 0.0314 m, k = 200 W/(m*K), Ac = 7.85e-05 m^2, Tinf = 25 °C. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, h = 25 W/(m^2*K), q = 8.32618 W.

```
thermofluids_ai999x.solve('infinite_fin_heat_rate', h='25 W/(m^2*K)', P='0.0314 m', k='200
W/(m*K)', Ac='7.85e-05 m^2', Tb='100 °C', Tinf='25 °C', target='q')
# -> 8.32618 W   (SOLVED)
```

$$q = \sqrt{hPkA_c}\,(T_b - \mathrm{Tinf})$$
$$q = \sqrt{A_c}\sqrt{P}\sqrt{h}\sqrt{k}\,(T_b - \mathrm{Tinf}) = 8.32618\ \mathrm{W}$$

invalid call	outcome	message
without `h`	NEEDS_INPUT	infinite_fin_heat_rate needs h (convection heat transfer coefficient, W/(m^2*K)). Example: solve('infinite_fin_heat_rate', h='... W/(m^2*K)'
`h` in a wrong unit	INVALID_INPUT	h is a heat transfer coefficient: its unit must be like W/(m^2*K), not 'kg'.

Questions in words that work:

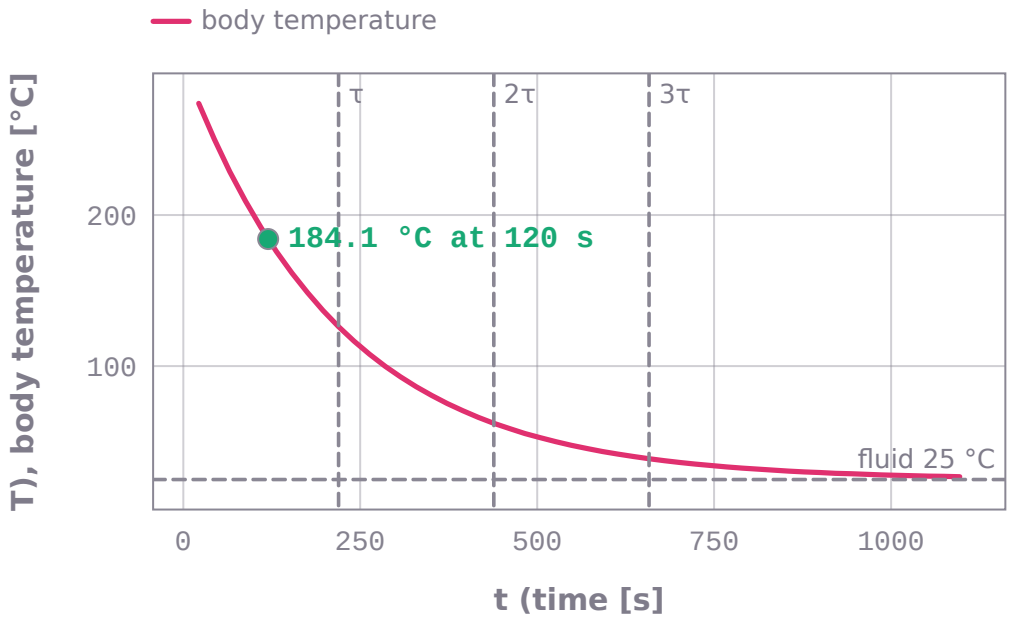
- "A very long aluminium pin fin has a perimeter of 0.0314 m, a cross-sectional area of 0.0000785 m^2 and a thermal conductivity of 200 W/(m*K). Its base temperature is 100 °C, the air temperature is 25 °C and the heat transfer coefficient is 25 W/(m^2*K). What is the rate of heat transfer from the fin?" → 8.32618 W
- "Find the heat loss from an infinitely long fin with a fin perimeter of 0.02 m, a cross-sectional area of 0.000025 m^2, thermal conductivity 50 W/(m*K) and heat transfer coefficient 100 W/(m^2*K), when the base temperature is 400 K and the fluid temperature is 300 K." → 5 W

Temperature of a lumped body cooling or heating in a fluid, `lumped_capacitance_temperature`

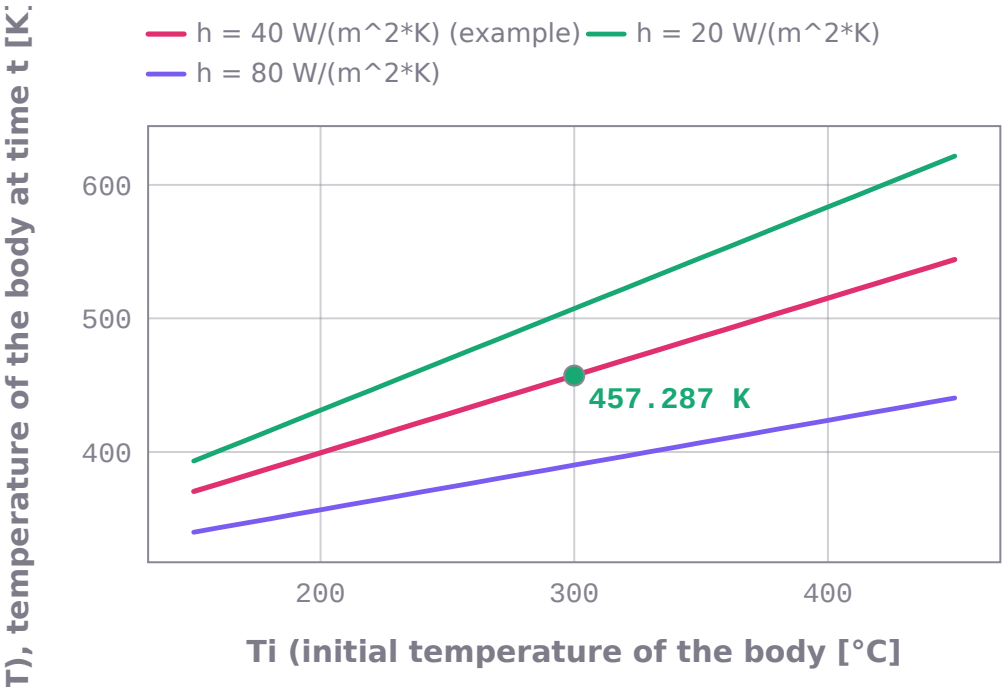
$$T = \mathrm{Tinf} + (T_i - \mathrm{Tinf})e^{\frac{-hAt}{\rho Vc}}$$

input	meaning	kind / role	unit	default
`T`	temperature of the body at time t	temperature / final	`K`	
`Tinf`	temperature of the fluid far from the surface	temperature / fluid	`K`	
`Ti`	initial temperature of the body	temperature / initial	`K`	
`h`	convection heat transfer coefficient	heat_transfer_coefficient / plain	`W/(m^2*K)`	
`A`	surface area of the body	area / plain	`m^2`	
`t`	elapsed time	time / plain	`s`	
`rho`	density of the body	density / material	`kg/m^3`	
`V`	volume of the body	volume / plain	`m^3`	
`c`	specific heat of the body	specific_heat / plain	`J/(kg*K)`	

- **Conditions:** lumped
- **Applies when:** small or highly conductive bodies
- **Fails when:** $Bi > 0.1$; radiation not negligible
- **Assumes:** Biot number below about 0.1 (uniform internal temperature); constant h and properties
- **Evidence:** theorem



Lumped cooling of a body (ρ = 7800 kg/m^3, V = 5.0000e-06 m^3, c = 450 J/(kg*K), A = 0.002 m^2, h = 40 W/(m^2*K)) from 300 °C in fluid at 25 °C: the package's temperature at 51 times over five time constants, the time constant τ = 219.375 s from its time-constant capability (dashed at τ, 2τ, 3τ). Marked: T = 184.137 °C at t = 120 s (the worked example).



T (temperature of the body at time t) against Ti (initial temperature of the body) from 150 °C to 450 °C. One curve for each value of h (convection heat transfer coefficient): 40 W/(m^2*K), 20 W/(m^2*K), 80 W/(m^2*K). Other inputs as in the worked example: Tinf = 25 °C, A = 0.002 m^2, t = 120 s, rho = 7800 kg/m^3, V = 5e-06 m^3, c = 450 J/(kg*K). 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Ti = 300 °C, T = 457.287 K.

```
thermofluids_ai999x.solve('lumped_capacitance_temperature', Tinf='25 °C', Ti='300 °C',
h='40 W/(m^2*K)', A='0.002 m^2', t='120 s', rho='7800 kg/m^3', V='5e-06 m^3', c='450
J/(kg*K)', target='T')
# -> 457.287 K   (SOLVED)
```

$$T = \mathrm{Tinf} + (T_i - \mathrm{Tinf})e^{\frac{-hAt}{\rho Vc}}$$
$$T = \left(T_i + \mathrm{Tinf}e^{\frac{Aht}{Vc\rho}} - \mathrm{Tinf}\right)e^{-\frac{Aht}{Vc\rho}} = 457.287\ \mathrm{K}$$

invalid call	outcome	message
without `Tinf`	NEEDS_INPUT	lumped_capacitance_temperature needs Tinf (temperature of the fluid far from the surface, K). Example: solve('lumped_capacitance_temperature
`Tinf` in a wrong unit	INVALID_INPUT	Tinf is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

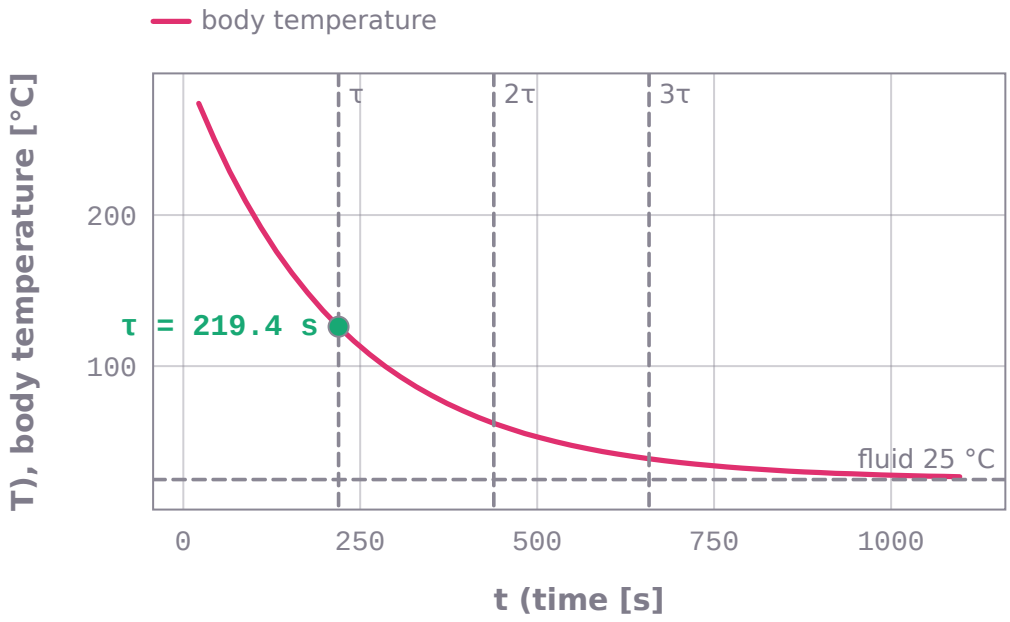
- "A small steel part with a volume of 0.000005 m^3, a surface area of 0.002 m^2, a density of 7800 kg/m^3 and a specific heat of 450 J/(kg*K) is initially at 300 °C. Treated as a lumped system, it cools in air at an ambient temperature of 25 °C with a heat transfer coefficient of 40 W/(m^2*K). What is its temperature after 120 s?" → 457.287 K
- "Using the lumped capacitance method, find the final temperature of an aluminium bead of volume 0.000001 m^3, surface area 0.0006 m^2, density 2700 kg/m^3 and specific heat 900 J/(kg*K) with an initial temperature of 500 K, cooled for 60 s by a fluid at a fluid temperature of 300 K with a heat transfer coefficient of 100 W/(m^2*K)." → 345.46 K

Thermal time constant of a lumped body, `lumped_time_constant`

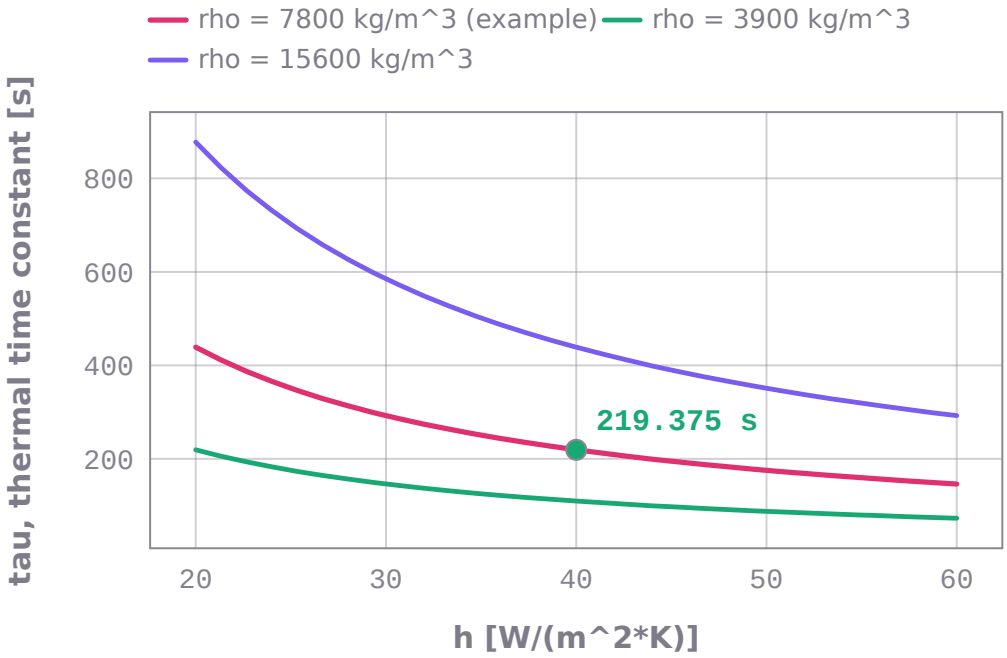
$$\tau = \frac{\rho V c}{h A}$$

input	meaning	kind / role	unit	default
`tau`	thermal time constant	time / time_constant	`s`	
`rho`	density of the body	density / material	`kg/m^3`	
`V`	volume of the body	volume / plain	`m^3`	
`c`	specific heat of the body	specific_heat / plain	`J/(kg*K)`	
`h`	convection heat transfer coefficient	heat_transfer_coefficient / plain	`W/(m^2*K)`	
`A`	surface area of the body	area / plain	`m^2`	

- **Conditions:** lumped
- **Applies when:** thermocouples and small bodies
- **Fails when:** $Bi > 0.1$
- **Assumes:** Biot number below about 0.1; constant h and properties
- **Evidence:** theorem



Lumped cooling of a body (ρ = 7800 kg/m^3, V = 5.0000e-06 m^3, c = 450 J/(kg*K), A = 0.002 m^2, h = 40 W/(m^2*K)) from 300 °C in fluid at 25 °C: the package's temperature at 51 times over five time constants, the time constant τ = 219.375 s from its time-constant capability (dashed at τ, 2τ, 3τ). Marked: τ = 219.375 s.



tau (thermal time constant) against h (convection heat transfer coefficient) from 20 W/(m^2*K) to 60 W/(m^2*K). One curve for each value of rho (density of the body): 7800 kg/m^3, 3900 kg/m^3, 15600 kg/m^3. Other inputs as in the worked example: V = 5e-06 m^3, c = 450 J/(kg*K), A = 0.002 m^2. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, h = 40 W/(m^2*K), tau = 219.375 s.

```
thermofluids_ai999x.solve('lumped_time_constant', rho='7800 kg/m^3', V='5e-06 m^3', c='450 J/(kg*K)', h='40 W/(m^2*K)', A='0.002 m^2', target='tau')
# -> 219.375 s   (SOLVED)
```

$$\tau = \frac{\rho V c}{h A}$$

$$\tau = \frac{V c \rho}{A h} = 219.375 \text{ s}$$

invalid call	outcome	message
without `rho`	NEEDS_INPUT	lumped_time_constant needs rho (density of the body, kg/m^3). Example: solve('lumped_time_constant', rho='... kg/m^3', V='... m^3', c='... J
`rho` in a wrong unit	INVALID_INPUT	rho is a density: its unit must be like kg/m^3, not 's'.

Questions in words that work:

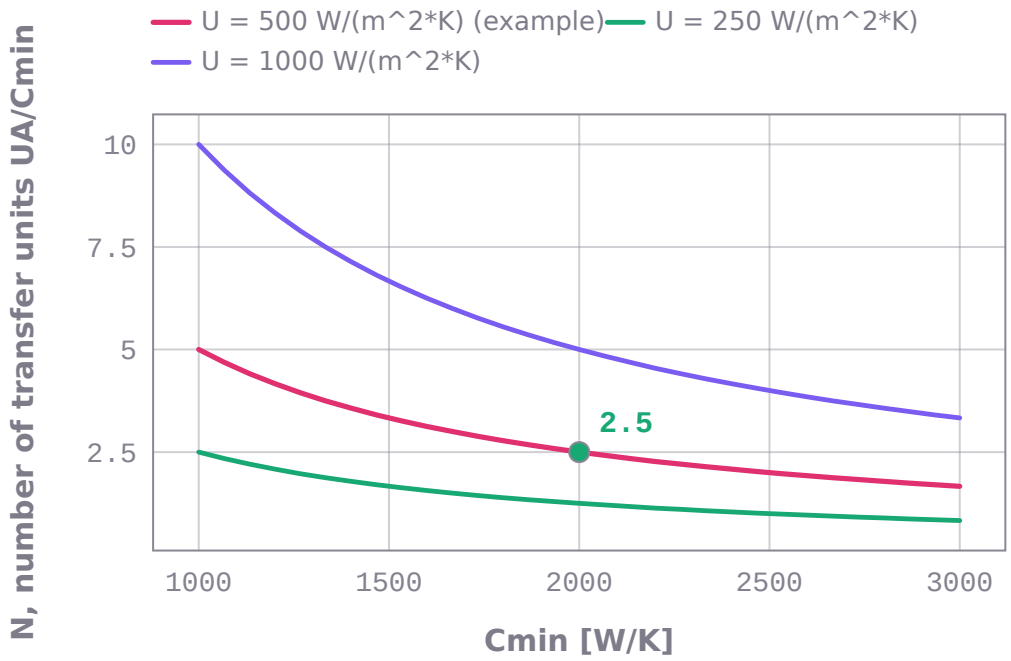
- "A lumped steel part has a volume of 0.000005 m^3, a surface area of 0.002 m^2, a density of 7800 kg/m^3, a specific heat of 450 J/(kg*K) and a heat transfer coefficient of 40 W/(m^2*K). What is its time constant?" → 219.375 s
- "Find the time constant of a thermocouple bead treated as a lumped system with volume 0.000001 m^3, surface area 0.0006 m^2, density 2700 kg/m^3, specific heat 900 J/(kg*K) and heat transfer coefficient 100 W/(m^2*K)." → 40.5 s

Number of transfer units of a heat exchanger, `ntu_definition`

$$N = \frac{UA}{C_{\min}}$$

input	meaning	kind / role	unit	default
`N`	number of transfer units UA/Cmin	number_of_transfer_units / plain	`1`	
`U`	overall heat transfer coefficient	heat_transfer_coefficient / overall	`W/(m^2*K)`	
`A`	heat transfer area	area / plain	`m^2`	
`Cmin`	smaller of the two heat capacity rates (m cp)	heat_capacity_rate / min	`W/K`	

- **Conditions:** heat_exchanger
- **Applies when:** the effectiveness-NTU method
- **Fails when:** U varying strongly along the exchanger
- **Assumes:** U uniform over the area
- **Evidence:** theorem



N (number of transfer units UA/Cmin) against Cmin (smaller of the two heat capacity rates (m cp)) from 1000 W/K to 3000 W/K. One curve for each value of U (overall heat transfer coefficient): 500 W/(m^2*K), 250 W/(m^2*K), 1000 W/(m^2*K). Other inputs as in the worked example: A = 10 m^2. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Cmin = 2000 W/K, N = 2.5.

```
thermofluids_ai999x.solve('ntu_definition', U='500 W/(m^2*K)', A='10 m^2', Cmin='2000 W/K',
target='N')
# -> 2.5 1   (SOLVED)
```

$$N = \frac{UA}{C_{\min}}$$

$$N = \frac{AU}{C_{\min}} = 2.5\ 1$$

invalid call	outcome	message
without `U`	NEEDS_INPUT	ntu_definition needs U (overall heat transfer coefficient, W/(m^2*K)). Example: solve('ntu_definition', U='... W/(m^2*K)', A='... m^2', Cmin
`U` in a wrong unit	INVALID_INPUT	U is a heat transfer coefficient: its unit must be like W/(m^2*K), not 'kg'.

Questions in words that work:

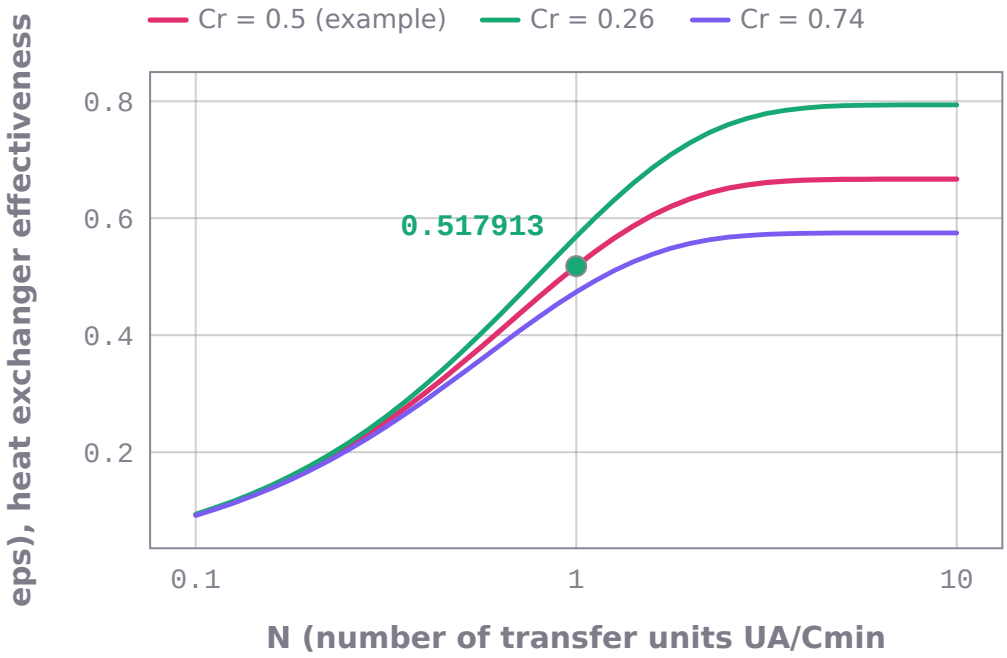
- "A heat exchanger has an overall heat transfer coefficient of 500 W/(m^2*K), an area of 10 m^2 and a minimum heat capacity rate of 2000 W/K. What is the number of transfer units?" → 2.5 1
- "Find the NTU of a heat exchanger with area 3 m^2, overall heat transfer coefficient 1200 W/(m^2*K) and minimum capacity rate 4500 W/K." → 0.8 1

Effectiveness of a parallel-flow heat exchanger, `effectiveness_parallel_flow`

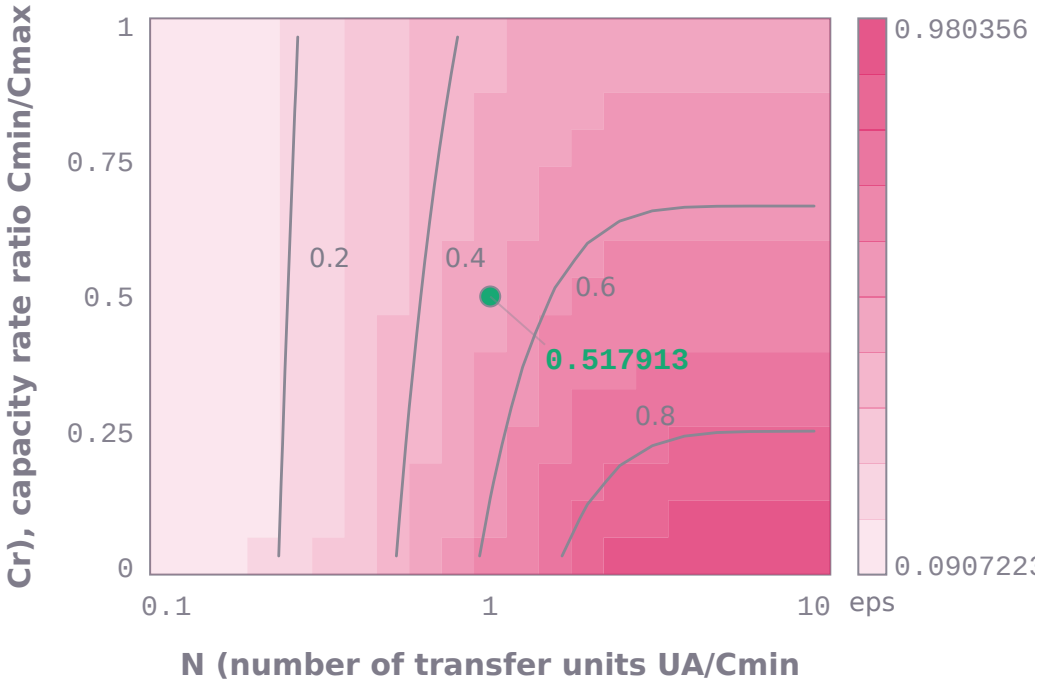
$$\varepsilon = \frac{1 - e^{-N(1+C_r)}}{1 + C_r}$$

input	meaning	kind / role	unit	default
`eps`	heat exchanger effectiveness	heat_exchanger_effectiveness / plain	`1`	
`N`	number of transfer units UA/Cmin	number_of_transfer_units / plain	`1`	
`Cr`	capacity rate ratio Cmin/Cmax	heat_capacity_rate_ratio / plain	`1`	

- **Conditions:** heat_exchanger, parallel_flow
- **Not when:** counterflow
- **Applies when:** pure parallel-flow (co-current) exchangers
- **Fails when:** counterflow or crossflow arrangements
- **Assumes:** steady flow, constant properties
- **Evidence:** theorem



eps (heat exchanger effectiveness) against N (number of transfer units UA/Cmin) from 0.1 to 10 on a logarithmic axis. One curve for each value of Cr (capacity rate ratio Cmin/Cmax): 0.5, 0.26, 0.74. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, N = 1, eps = 0.517913.



eps (heat exchanger effectiveness) over N (number of transfer units UA/Cmin) from 0.1 to 10 and Cr (capacity rate ratio Cmin/Cmax) from 0.02 to 0.98 (logarithmic horizontal axis): each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 0.0907223 to dark 0.980356); iso-lines at eps = 0.2, 0.4, 0.6, 0.8. Marked: the worked example, N = 1, Cr = 0.5, eps = 0.517913.

```
thermofluids_ai999x.solve('effectiveness_parallel_flow', N='1 1', Cr='0.5 1', target='eps')
# -> 0.517913 1   (SOLVED)
```

$$\varepsilon = \frac{1 - e^{-N(1+C_r)}}{1 + C_r}$$

$$\varepsilon = \frac{\left(e^{N(C_r+1)} - 1\right)e^{-N(C_r+1)}}{C_r + 1} = 0.517913\ 1$$

invalid call	outcome	message
without `N`	NEEDS_INPUT	effectiveness_parallel_flow needs N (number of transfer units UA/Cmin, 1). Example: solve('effectiveness_parallel_flow', N='... 1', Cr='...
`N` in a wrong unit	INVALID_INPUT	N is a number of transfer units: its unit must be like 1, not 'kg'.

Questions in words that work:

- "A parallel flow heat exchanger has a number of transfer units of 1 and a capacity rate ratio of 0.5. What is its effectiveness?" → 0.517913 1
- "Find the effectiveness of a parallel-flow heat exchanger with NTU 2 and capacity ratio 0.25." → 0.734332 1

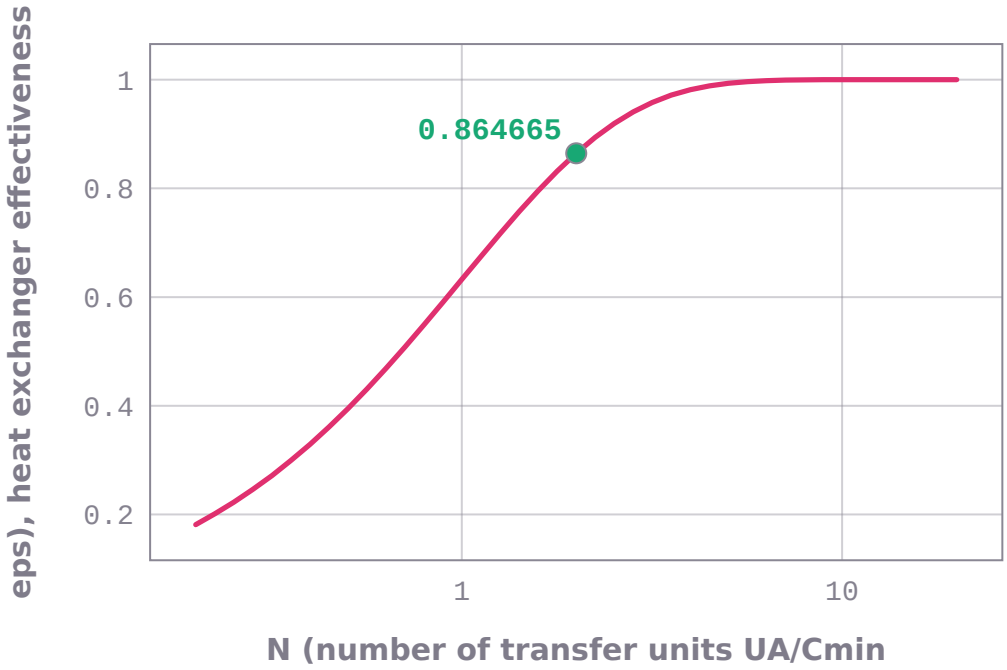
Effectiveness of a heat exchanger with one stream changing phase,

`effectiveness_phase_change`

$$\varepsilon = 1 - e^{-N}$$

input	meaning	kind / role	unit	default
`eps`	heat exchanger effectiveness	heat_exchanger_effectiveness / plain	`1`	
`N`	number of transfer units UA/Cmin	number_of_transfer_units / plain	`1`	

- **Conditions:** heat_exchanger, phase_change
- **Applies when:** condensers and evaporators, any flow arrangement
- **Fails when:** both streams changing temperature
- **Assumes:** one stream at constant temperature $(Cr = 0)$
- **Evidence:** theorem



eps (heat exchanger effectiveness) against N (number of transfer units UA/Cmin) from 0.2 to 20 on a logarithmic axis. 41 points computed by thermofluids-ai999x; marked: the worked example, N = 2, eps = 0.864665.

```
thermofluids_ai999x.solve('effectiveness_phase_change', N='2 1', target='eps')
# -> 0.864665 1   (SOLVED)
```

$$\varepsilon = 1 - e^{-N}$$
$$\varepsilon = 1 - e^{-N} = 0.864665\ 1$$

invalid call	outcome	message
without `N`	NEEDS_INPUT	effectiveness_phase_change needs N (number of transfer units UA/Cmin, 1). Example: solve('effectiveness_phase_change', N='... 1').
`N` in a wrong unit	INVALID_INPUT	N is a number of transfer units: its unit must be like 1, not 'kg'.

Questions in words that work:

- "Steam condenses in a condenser whose heat exchanger has a number of transfer units of 2. What is the effectiveness?" → 0.864665 1
- "Find the effectiveness of an evaporator heat exchanger with an NTU of 0.5." → 0.393469 1

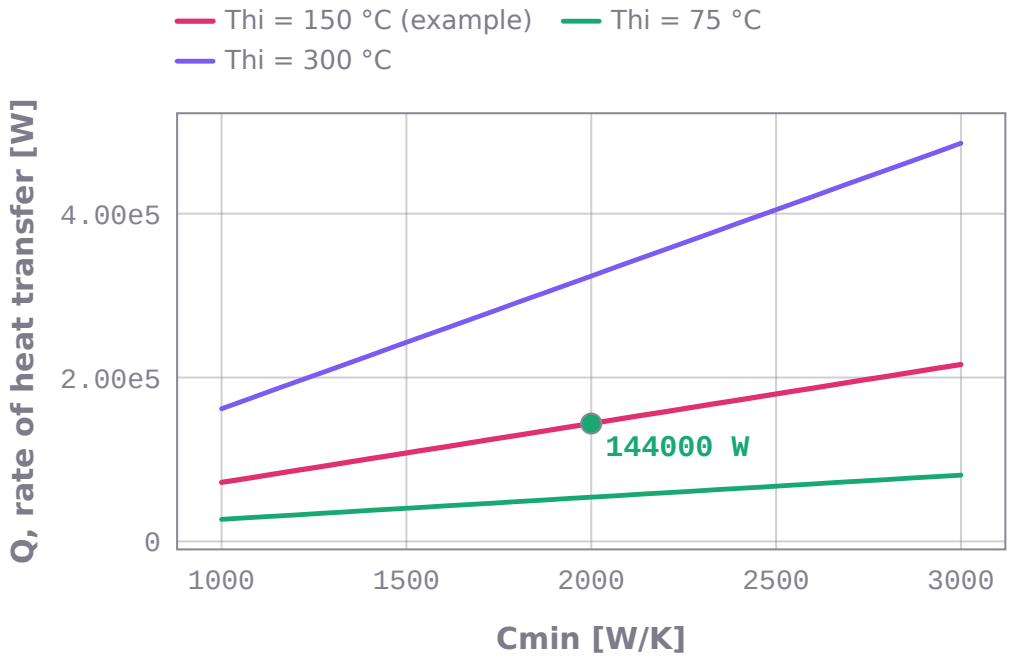
Heat transfer rate from the effectiveness, `heat_rate_effectiveness`

$$Q = \varepsilon C_{\min} \left(\mathrm{Thi} - \mathrm{Tci}\right)$$

input	meaning	kind / role	unit	default
`Q`	rate of heat transfer	power / heat_rate	`W`	
`eps`	heat exchanger effectiveness	heat_exchanger_effectiveness / plain	`1`	
`Cmin`	smaller of the two heat capacity rates (m cp)	heat_capacity_rate / min	`W/K`	
`Thi`	inlet temperature of the hot fluid	temperature / hot_inlet	`K`	

input	meaning	kind / role	unit	default
`Tci`	inlet temperature of the cold fluid	temperature / cold_inlet	`K`	

- **Conditions:** heat_exchanger
- **Applies when:** rating an exchanger whose effectiveness is known
- **Fails when:** effectiveness of another arrangement used
- **Assumes:** steady flow; no heat loss to the surroundings
- **Evidence:** theorem



Q (rate of heat transfer) against Cmin (smaller of the two heat capacity rates (m cp)) from 1000 W/K to 3000 W/K. One curve for each value of Thi (inlet temperature of the hot fluid): 150 °C, 75 °C, 300 °C. Other inputs as in the worked example: eps = 0.6 1, Tci = 30 °C. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Cmin = 2000 W/K, Q = 144000 W.

```
thermofluids_ai999x.solve('heat_rate_effectiveness', eps='0.6 1', Cmin='2000 W/K', Thi='150
°C', Tci='30 °C', target='Q')
# -> 144000 W   (SOLVED)
```

$$Q = \varepsilon C_{\min} \left(\mathrm{Thi} - \mathrm{Tci}\right)$$
$$Q = C_{\min} \varepsilon \left(-\mathrm{Tci} + \mathrm{Thi}\right) = 144000\ \mathrm{W}$$

invalid call	outcome	message
without `eps`	NEEDS_INPUT	heat_rate_effectiveness needs eps (heat exchanger effectiveness, 1). Example: solve('heat_rate_effectiveness', eps='... 1', Cmin='... W/K',
`eps` in a wrong unit	INVALID_INPUT	eps is a heat exchanger effectiveness: its unit must be like 1, not 'kg'.

Questions in words that work:

- "A heat exchanger has an effectiveness of 0.6 and a minimum heat capacity rate of 2000 W/K. The hot fluid inlet temperature is 150 °C and the cold fluid inlet temperature is 30 °C. What is the rate of heat transfer?" → 144000 W

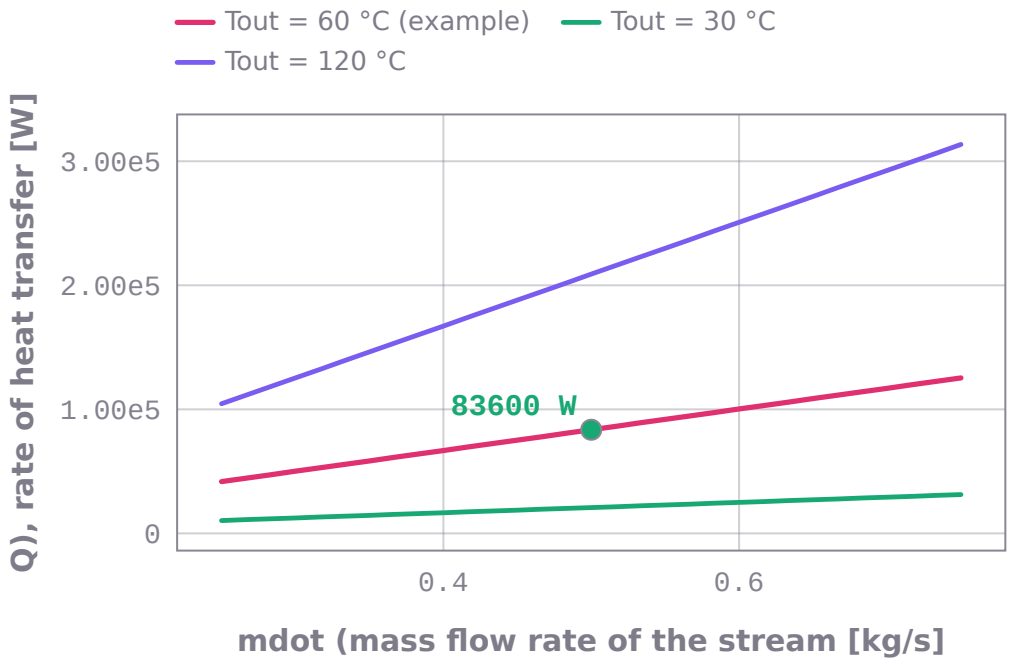
- "Find the heat transfer rate in an exchanger with effectiveness 0.75 and minimum capacity rate 1500 W/K when the hot inlet temperature is 400 K and the cold inlet temperature is 290 K." → 123750 W

Heat added to a flowing stream (steady-flow energy balance), `sensible_heat_rate`

$$Q = \mathrm{mdot} c_p \left(T_{\mathrm{out}} - T_{\mathrm{in}}\right)$$

input	meaning	kind / role	unit	default
`Q`	rate of heat transfer	power / heat_rate	`W`	
`mdot`	mass flow rate of the stream	mass_flow_rate / plain	`kg/s`	
`cp`	specific heat of the fluid	specific_heat / plain	`J/(kg*K)`	
`Tout`	outlet temperature	temperature / outlet	`K`	
`Tin`	inlet temperature	temperature / inlet	`K`	

- **Conditions:** stream_heating
- **Applies when:** heaters, coolers and each side of a heat exchanger
- **Fails when:** phase change in the stream
- **Assumes:** steady flow; constant cp; no phase change; kinetic and potential energy changes negligible
- **Evidence:** theorem



Q (rate of heat transfer) against mdot (mass flow rate of the stream) from 0.25 kg/s to 0.75 kg/s. One curve for each value of Tout (outlet temperature): 60 °C, 30 °C, 120 °C. Other inputs as in the worked example: cp = 4180 J/(kg*K), Tin = 20 °C. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, mdot = 0.5 kg/s, Q = 83600 W.

```
thermofluids_ai999x.solve('sensible_heat_rate', mdot='0.5 kg/s', cp='4180 J/(kg*K)',
Tout='60 °C', Tin='20 °C', target='Q')
# -> 83600 W   (SOLVED)
```

$$Q = \mathrm{mdot} c_p \left(T_{\mathrm{out}} - T_{\mathrm{in}}\right)$$
$$Q = c_p \mathrm{mdot} \left(-T_{\mathrm{in}} + T_{\mathrm{out}}\right) = 83600\ \mathrm{W}$$

invalid call	outcome	message
without `mdot`	NEEDS_INPUT	sensible_heat_rate needs mdot (mass flow rate of the stream, kg/s). Example: solve('sensible_heat_rate', mdot='... kg/s', cp='... J/(kg*K)',
`mdot` in a wrong unit	INVALID_INPUT	mdot is a mass flow rate: its unit must be like kg/s, not 's'.

Questions in words that work:

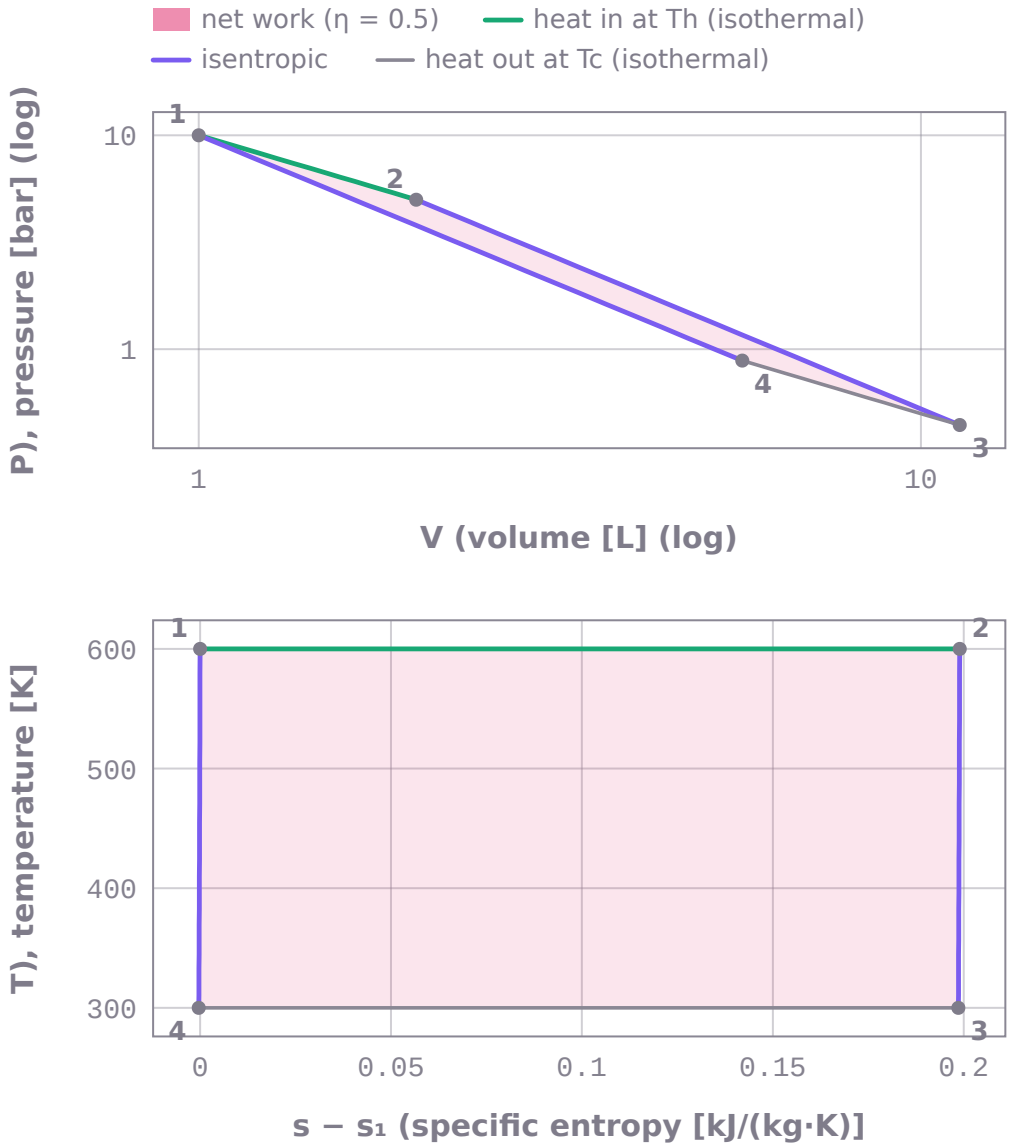
- "Water with a mass flow rate of 0.5 kg/s and a specific heat of 4180 J/(kg*K) is heated from an inlet temperature of 20 °C to an outlet temperature of 60 °C. What is the rate of heat transfer?" → 83600 W
- "Air flowing at a mass flow rate of 2 kg/s with specific heat 1005 J/(kg*K) enters a heater with an inlet temperature of 300 K and has an outlet temperature of 350 K. Find the heat transfer rate." → 100500 W

Carnot efficiency of a reversible heat engine, `carnot_efficiency`

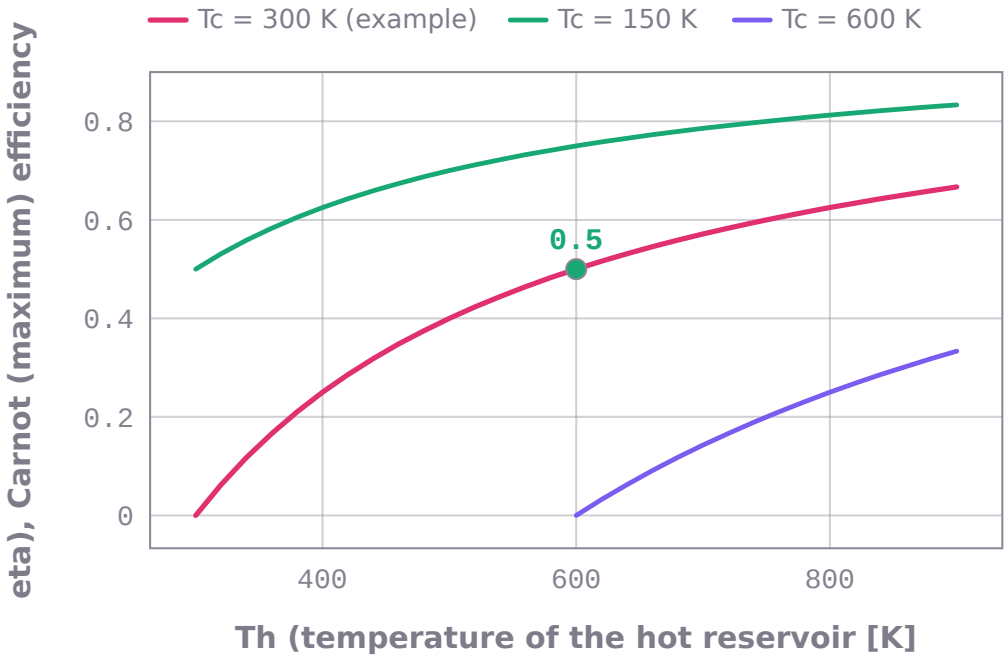
$$\eta = 1 - \frac{T_c}{T_h}$$

input	meaning	kind / role	unit	default
`eta`	Carnot (maximum) efficiency	efficiency / carnot	`1`	
`Tc`	temperature of the cold reservoir	temperature / cold	`K`	
`Th`	temperature of the hot reservoir	temperature / hot	`K`	

- **Conditions:** carnot
- **Not when:** refrigerator, heat_pump
- **Applies when:** the upper limit of any heat engine between two temperatures
- **Fails when:** real (irreversible) engines
- **Assumes:** reversible cycle between two reservoirs; absolute temperatures
- **Evidence:** theorem



Carnot cycle between Th = 600 K and Tc = 300 K (the worked example) for air: stated start 10 bar in 1 L (0.200454 mol, from the package's ideal-gas law), isothermal expansion to 2 L. The isentropes end at V3 = 11.3137 L and V4 = 5.65685 L (the package's isentropic relation solved for the volume); pressures from its ideal-gas law, entropy from its ideal-gas entropy change. P–V on logarithmic axes (top), T–s (bottom, a rectangle). Marked: the package's efficiency η = 0.5.



eta (Carnot (maximum) efficiency) against Th (temperature of the hot reservoir) from 300 K to 900 K. One curve for each value of Tc (temperature of the cold reservoir): 300 K, 150 K, 600 K. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Th = 600 K, eta = 0.5.

```
thermofluids_ai999x.solve('carnot_efficiency', Tc='300 K', Th='600 K', target='eta')
# -> 0.5 1   (SOLVED)
```

$$\eta = 1 - \frac{T_c}{T_h}$$

$$\eta = \frac{-T_c + T_h}{T_h} = 0.5\ 1$$

invalid call	outcome	message
without `Tc`	NEEDS_INPUT	carnot_efficiency needs Tc (temperature of the cold reservoir, K). Example: solve('carnot_efficiency', Tc='... K', Th='... K').
`Tc` in a wrong unit	INVALID_INPUT	Tc is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

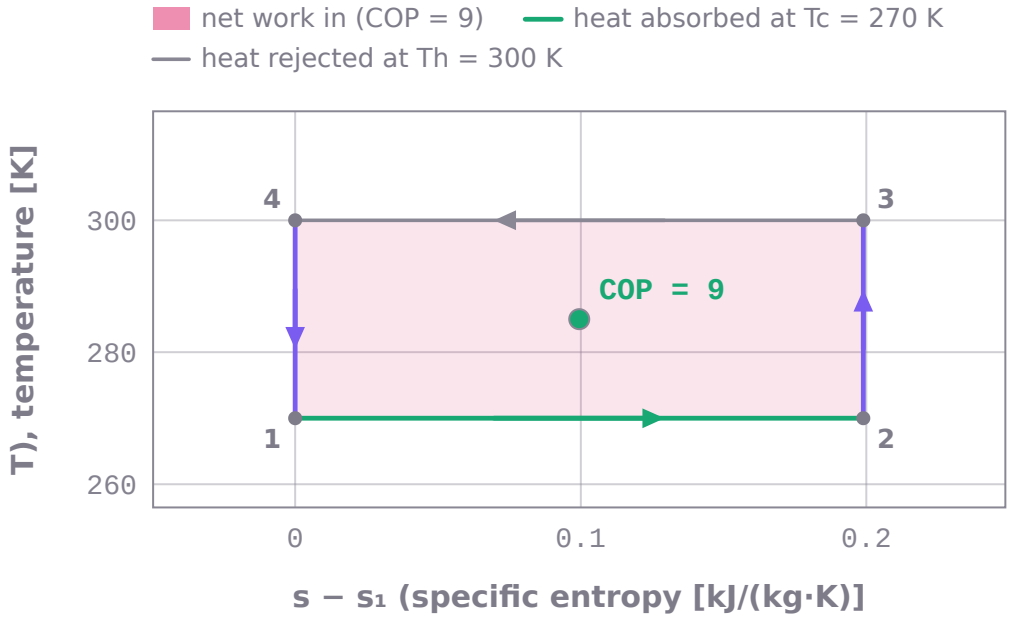
- "A Carnot engine operates between a hot reservoir temperature of 600 K and a cold reservoir temperature of 300 K. What is its efficiency?" → 0.5 1
- "Find the maximum possible efficiency of a reversible heat engine working between a hot reservoir at a temperature of the hot reservoir of 800 K and a cold reservoir temperature of 290 K." → 0.6375 1

Coefficient of performance of a Carnot refrigerator, `carnot_cop_refrigerator`

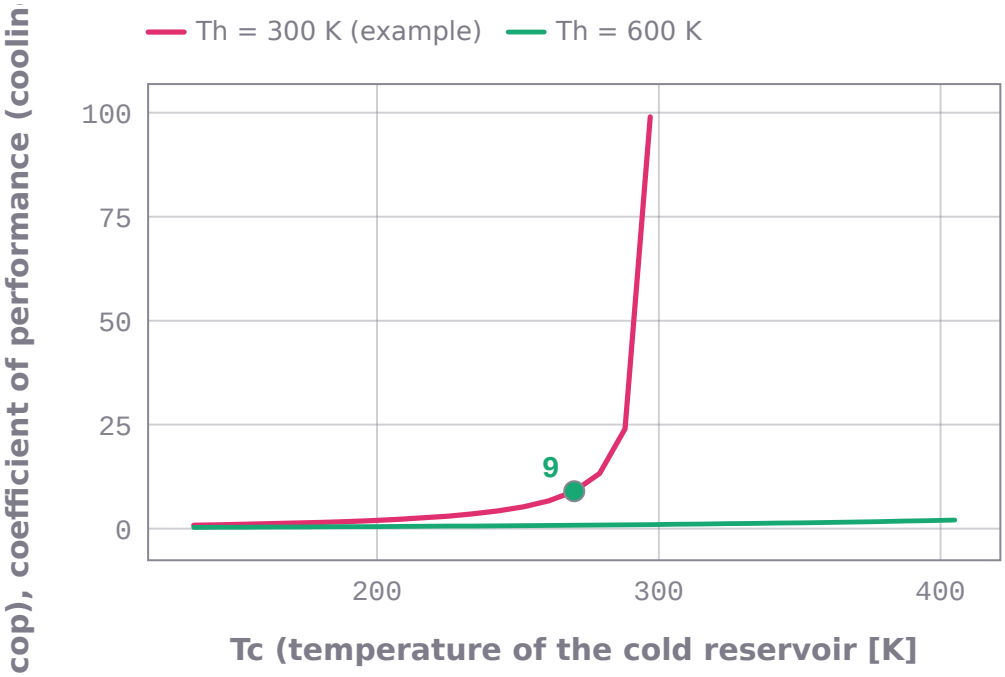
$$\mathrm{cop} = \frac{T_c}{T_h - T_c}$$

input	meaning	kind / role	unit	default
`cop`	coefficient of performance (cooling)	efficiency / cop	`1`	
`Tc`	temperature of the cold reservoir	temperature / cold	`K`	
`Th`	temperature of the hot reservoir	temperature / hot	`K`	

- **Conditions:** carnot, refrigerator
- **Not when:** heat_pump
- **Applies when:** the upper limit of a refrigerator's COP
- **Fails when:** real refrigerators
- **Assumes:** reversible refrigeration cycle; absolute temperatures
- **Evidence:** theorem



Reversed Carnot cycle (refrigerator) between Tc = 270 K and Th = 300 K (the worked example) on T–s axes, running counter-clockwise: heat taken in at Tc (1→2), isentropic compression (2→3), heat rejected at Th (3→4), isentropic expansion (4→1). The width is the entropy change of air at Tc for a stated pressure halving, 0.198933 kJ/(kg·K) from the package. Marked: the package's coefficient of performance 9.



cop (coefficient of performance (cooling)) against Tc (temperature of the cold reservoir) from 135 K to 405 K. One curve for each value of Th (temperature of the hot reservoir): 300 K, 600 K. 19 points per curve computed by thermofluids-ai999x; marked: the worked example, Tc = 270 K, cop = 9.

```
thermofluids_ai999x.solve('carnot_cop_refrigerator', Tc='270 K', Th='300 K', target='cop')
# -> 9 1   (SOLVED)
```

$$\mathrm{cop} = \frac{T_c}{T_h - T_c}$$

$$\mathrm{cop} = -\frac{T_c}{T_c - T_h} = 9\ 1$$

invalid call	outcome	message
without `Tc`	NEEDS_INPUT	carnot_cop_refrigerator needs Tc (temperature of the cold reservoir, K). Example: solve('carnot_cop_refrigerator', Tc='... K', Th='... K').
`Tc` in a wrong unit	INVALID_INPUT	Tc is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

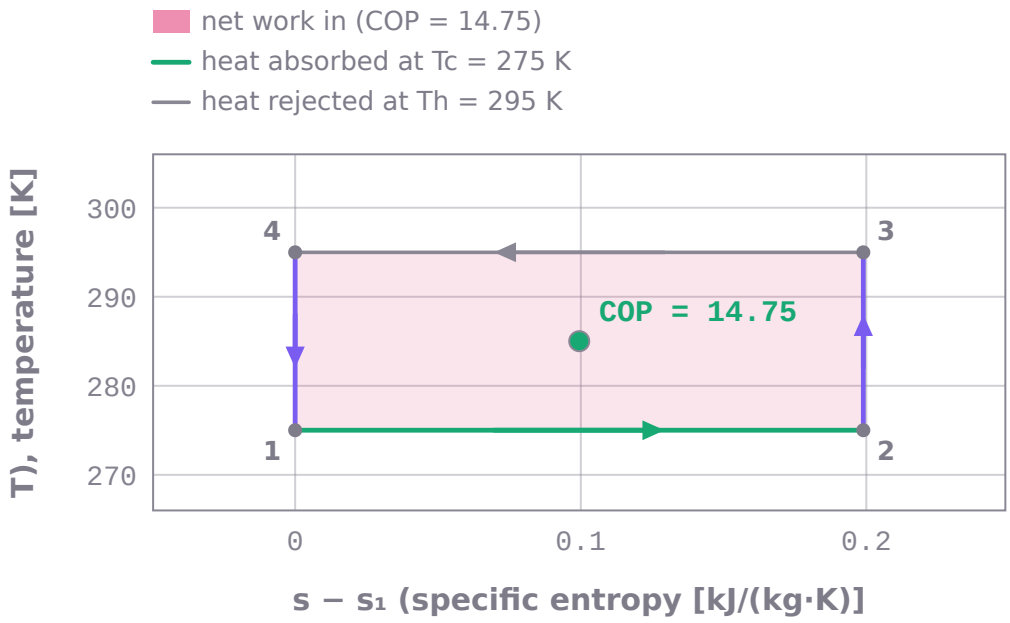
- "A Carnot refrigerator keeps a cold reservoir temperature of 270 K and rejects heat at a hot reservoir temperature of 300 K. What is its coefficient of performance?" → 9 1
- "Find the coefficient of performance of a reversible refrigerator with a cold reservoir temperature of 258 K and a hot reservoir temperature of 303 K." → 5.73333 1

Coefficient of performance of a Carnot heat pump, `carnot_cop_heat_pump`

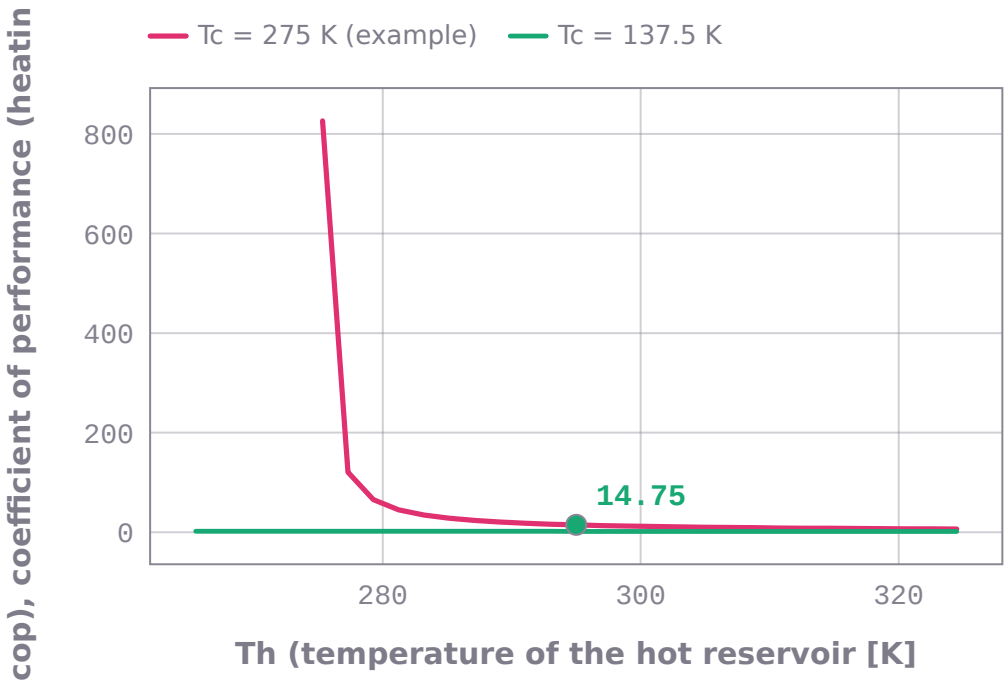
$$\mathrm{cop} = \frac{T_h}{T_h - T_c}$$

input	meaning	kind / role	unit	default
`cop`	coefficient of performance (heating)	efficiency / cop	`1`	
`Tc`	temperature of the cold reservoir	temperature / cold	`K`	
`Th`	temperature of the hot reservoir	temperature / hot	`K`	

- **Conditions:** carnot, heat_pump
- **Not when:** refrigerator
- **Applies when:** the upper limit of a heat pump's COP
- **Fails when:** real heat pumps
- **Assumes:** reversible heat-pump cycle; absolute temperatures
- **Evidence:** theorem



Reversed Carnot cycle (heat pump) between Tc = 275 K and Th = 295 K (the worked example) on T–s axes, running counter-clockwise: heat taken in at Tc (1→2), isentropic compression (2→3), heat rejected at Th (3→4), isentropic expansion (4→1). The width is the entropy change of air at Tc for a stated pressure halving, 0.198933 kJ/(kg·K) from the package. Marked: the package's coefficient of performance 14.75.



cop (coefficient of performance (heating)) against Th (temperature of the hot reservoir) from 265.5 K to 324.5 K. One curve for each value of Tc (temperature of the cold reservoir): 275 K, 137.5 K. 26 points per curve computed by thermofluids-ai999x; marked: the worked example, Th = 295 K, cop = 14.75.

```
thermofluids_ai999x.solve('carnot_cop_heat_pump', Tc='275 K', Th='295 K', target='cop')
# -> 14.75 1   (SOLVED)
```

$$\mathrm{cop} = \frac{T_h}{T_h - T_c}$$

$$\mathrm{cop} = -\frac{T_h}{T_c - T_h} = 14.75\ 1$$

invalid call	outcome	message
without `Tc`	NEEDS_INPUT	carnot_cop_heat_pump needs Tc (temperature of the cold reservoir, K). Example: solve('carnot_cop_heat_pump', Tc='... K', Th='... K').
`Tc` in a wrong unit	INVALID_INPUT	Tc is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

- "A Carnot heat pump delivers heat at a hot reservoir temperature of 295 K while extracting heat from a cold reservoir temperature of 275 K. What is its coefficient of performance?" → 14.75 1
- "Find the coefficient of performance of a reversible heat pump between a cold reservoir temperature of 268 K and a hot reservoir temperature of 313 K." → 6.95556 1

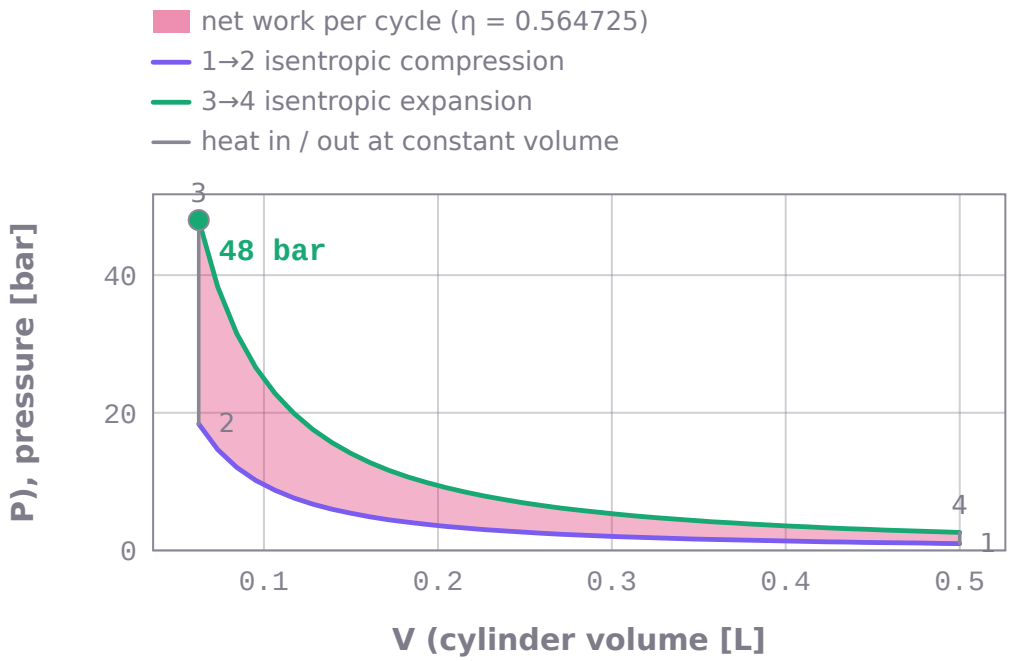
Thermal efficiency of the air-standard Otto cycle, `otto_efficiency_air`

$$\eta = 1 - r^{\frac{-2}{5}}$$

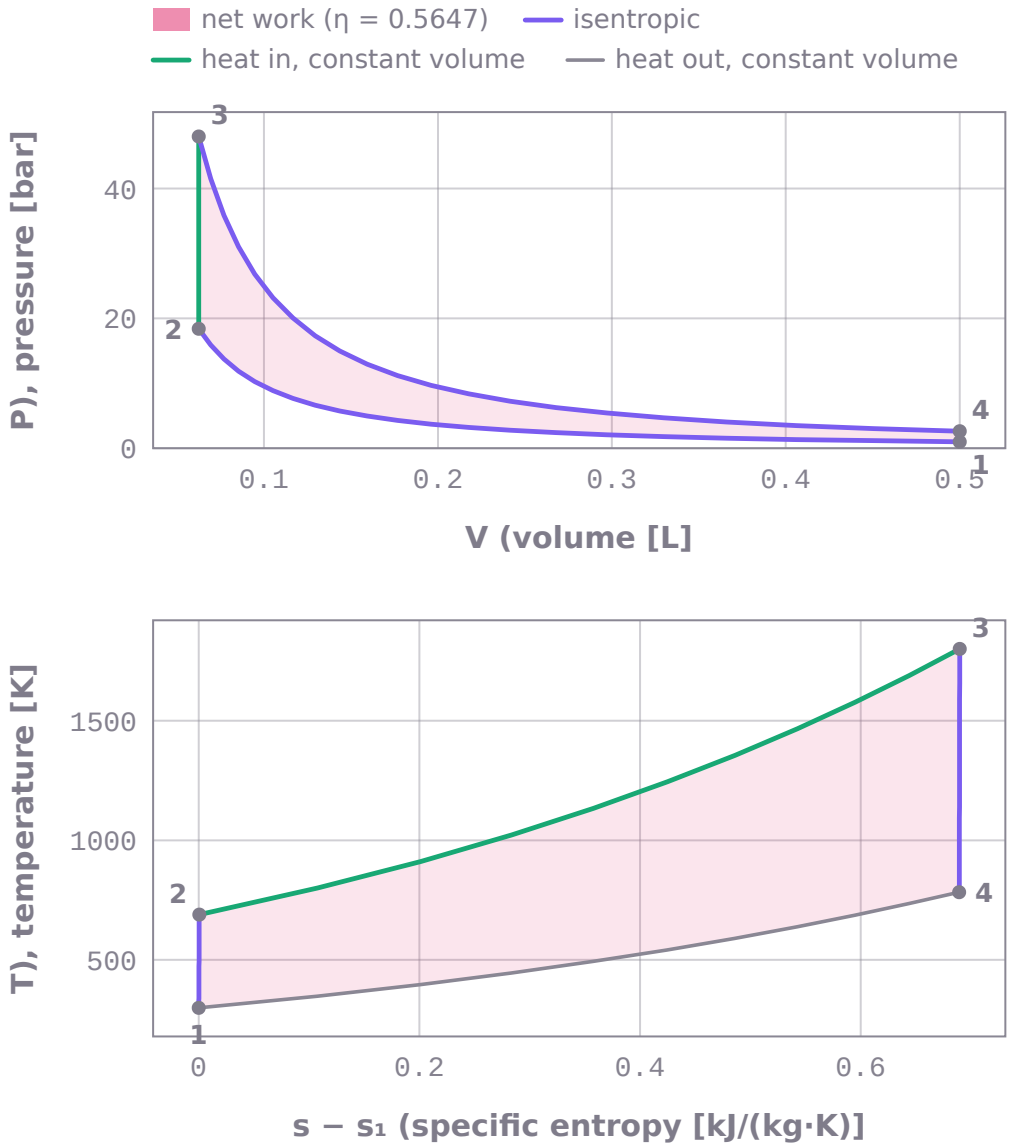
input	meaning	kind / role	unit	default
`eta`	thermal efficiency of the cycle	efficiency / cycle	`1`	

input	meaning	kind / role	unit	default
`r`	compression ratio V_1/V_2	compression_ratio / plain	`1`	

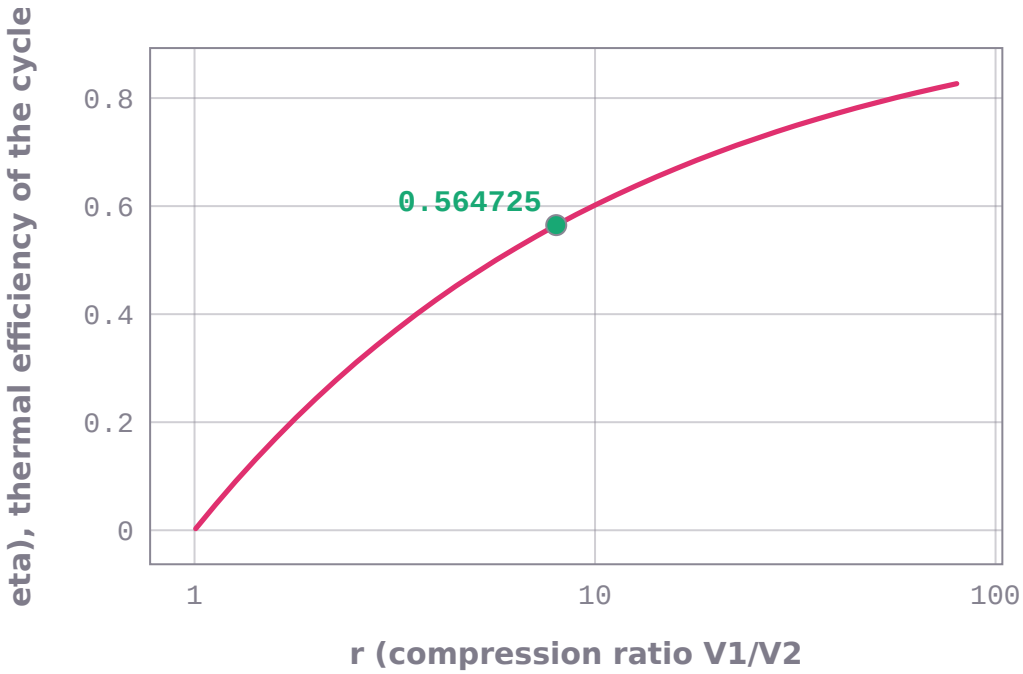
- **Conditions:** otto, air
- **Not when:** non_air
- **Applies when:** idealised spark-ignition engines
- **Fails when:** working fluids other than air; real engine losses
- **Assumes:** cold-air-standard: air as an ideal gas with $\gamma = 1.4$; reversible adiabatic compression and expansion
- **Evidence:** theorem



An air-standard Otto cycle drawn on pressure–volume axes: compression ratio 8, intake 1 bar and 300 K in 0.5 L, peak temperature 1800 K (stated). Each point is a solve of the package's isentropic and ideal-gas relations (amount of air 0.0200454 mol): T2 = 689.219 K, T4 = 783.496 K; the enclosed area is the net work. The package's Otto efficiency for r = 8 is 0.564725; marked: state 3, P = 48 bar.



Air-standard Otto cycle, compression ratio r = 8 (the worked example) with a stated intake of 1 bar, 300 K in 0.5 L and a peak temperature of 1800 K, on P–V (top) and T–s (bottom) axes. Every point is a run of the package: temperatures on the isentropes from its isentropic temperature–volume relation, pressures from the ideal-gas law (0.0200454 mol), entropy from its ideal-gas entropy change (cp = 1005, R = 287 J/(kg·K)). Marked: T2 = 689.219 K, T4 = 783.496 K, peak pressure 48 bar, the package's efficiency η = 0.564725.



eta (thermal efficiency of the cycle) against r (compression ratio V1/V2) from 0.8 to 80 on a logarithmic axis. 39 points computed by thermofluids-ai999x; marked: the worked example, r = 8, eta = 0.564725.

```
thermofluids_ai999x.solve('otto_efficiency_air', r='8 1', target='eta')
# -> 0.564725 1   (SOLVED)
```

$$\eta = 1 - r^{\frac{-2}{5}}$$

$$\eta = 1 - \frac{1}{r^{\frac{2}{5}}} = 0.564725\ 1$$

invalid call	outcome	message
without `r`	NEEDS_INPUT	otto_efficiency_air needs r (compression ratio V1/V2, 1). Example: solve('otto_efficiency_air', r='... 1').
`r` in a wrong unit	INVALID_INPUT	r is a compression ratio: its unit must be like 1, not 'kg'.

Questions in words that work:

- "An air-standard Otto cycle has a compression ratio of 8. What is its thermal efficiency?" → 0.564725 1
- "Find the thermal efficiency of an ideal Otto cycle using air with a compression ratio of 10." → 0.601893 1

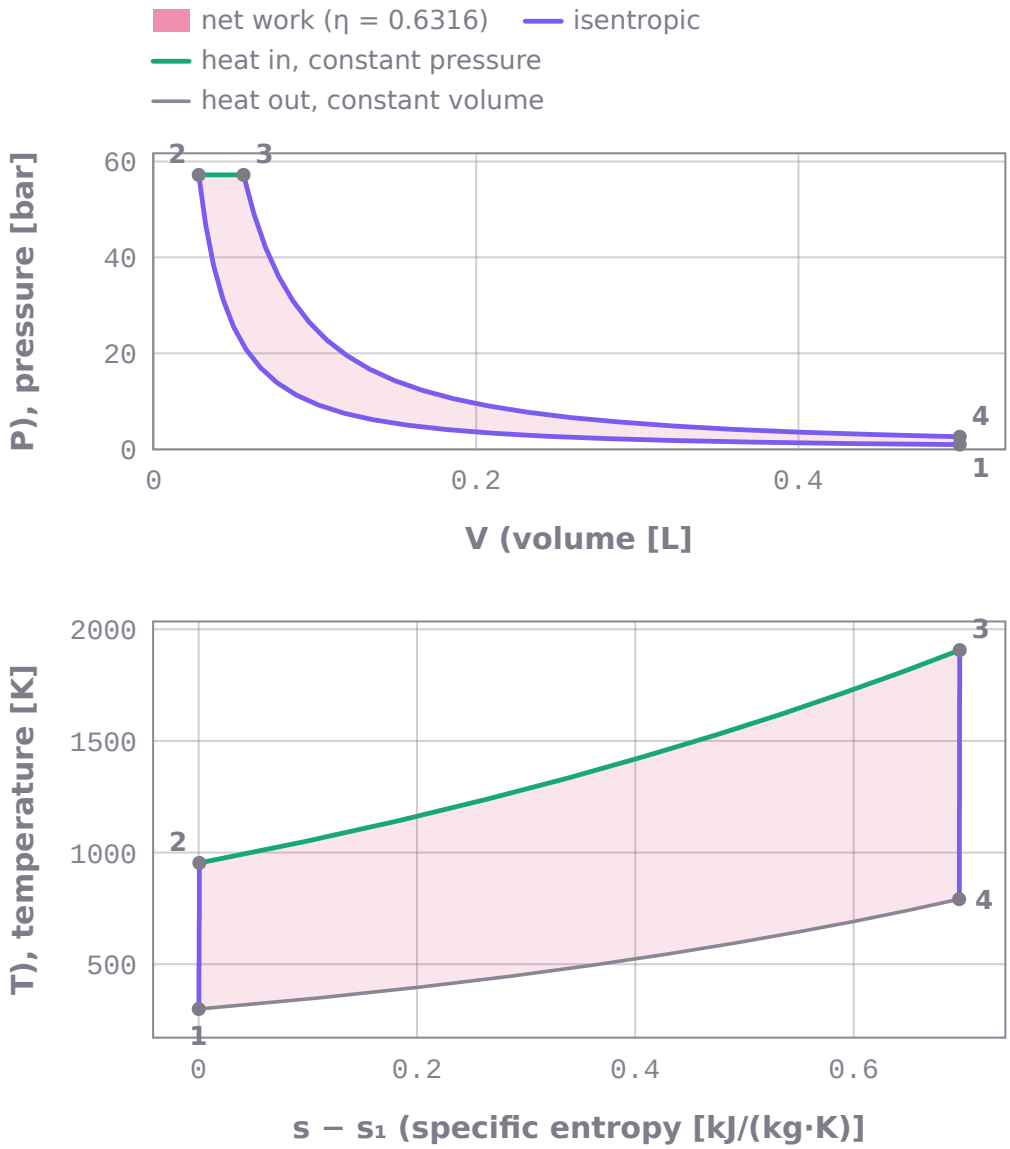
Thermal efficiency of the air-standard Diesel cycle, `diesel_efficiency_air`

$$\eta = 1 - \frac{r^{\frac{-2}{5}}\left(r_c^{\frac{7}{5}} - 1\right)}{\frac{7(r_c - 1)}{5}}$$

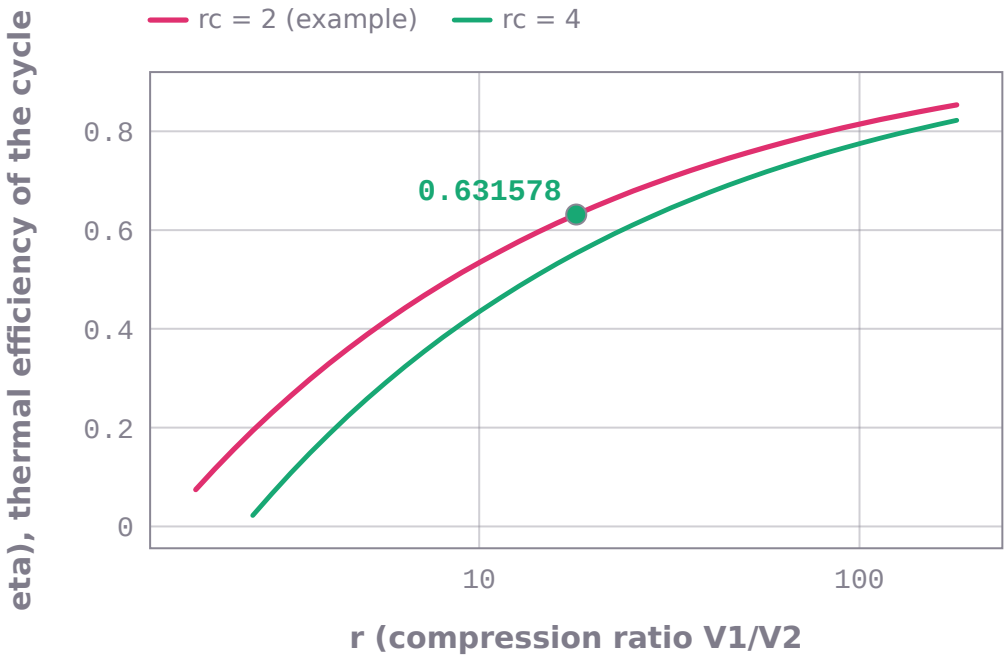
input	meaning	kind / role	unit	default
`eta`	thermal efficiency of the cycle	efficiency / cycle	`1`	

input	meaning	kind / role	unit	default
`r`	compression ratio V_1/V_2	compression_ratio / plain	`1`	
`rc`	cutoff ratio V_3/V_2	cutoff_ratio / plain	`1`	

- **Conditions:** diesel, air
- **Not when:** non_air
- **Applies when:** idealised compression-ignition engines
- **Fails when:** working fluids other than air
- **Assumes:** cold-air-standard: air as an ideal gas with $\gamma = 1.4$; heat added at constant pressure
- **Evidence:** theorem



Air-standard Diesel cycle, compression ratio r = 18, cut-off ratio rc = 2 (the worked example), stated intake 1 bar, 300 K in 0.5 L; P–V (top) and T–s (bottom). Isentropes from the package's isentropic relation, the constant-pressure heating from its ideal-gas law (0.0200454 mol), entropy from its ideal-gas entropy change (cp = 1005, R = 287 J/(kg·K)). Marked: T2 = 953.301 K, T3 = 1906.6 K, T4 = 791.705 K, P2 = 57.1981 bar, the package's efficiency η = 0.631578.



eta (thermal efficiency of the cycle) against r (compression ratio V1/V2) from 1.8 to 180 on a logarithmic axis. One curve for each value of rc (cutoff ratio V3/V2): 2, 4. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, r = 18, eta = 0.631578.

```
thermofluids_ai999x.solve('diesel_efficiency_air', r='18 1', rc='2 1', target='eta')
# -> 0.631578 1   (SOLVED)
```

$$\eta = 1 - \frac{r^{\frac{-2}{5}}\left(r_c^{\frac{7}{5}} - 1\right)}{\frac{7(r_c - 1)}{5}}$$

$$\eta = \frac{r^{\frac{2}{5}} r_c - r^{\frac{2}{5}} - \frac{5 r_c^{\frac{7}{5}}}{7} + \frac{5}{7}}{r^{\frac{2}{5}}\left(r_c - 1\right)} = 0.631578\ 1$$

invalid call	outcome	message
without `r`	NEEDS_INPUT	diesel_efficiency_air needs r (compression ratio V1/V2, 1). Example: solve('diesel_efficiency_air', r='... 1', rc='... 1').
`r` in a wrong unit	INVALID_INPUT	r is a compression ratio: its unit must be like 1, not 'kg'.

Questions in words that work:

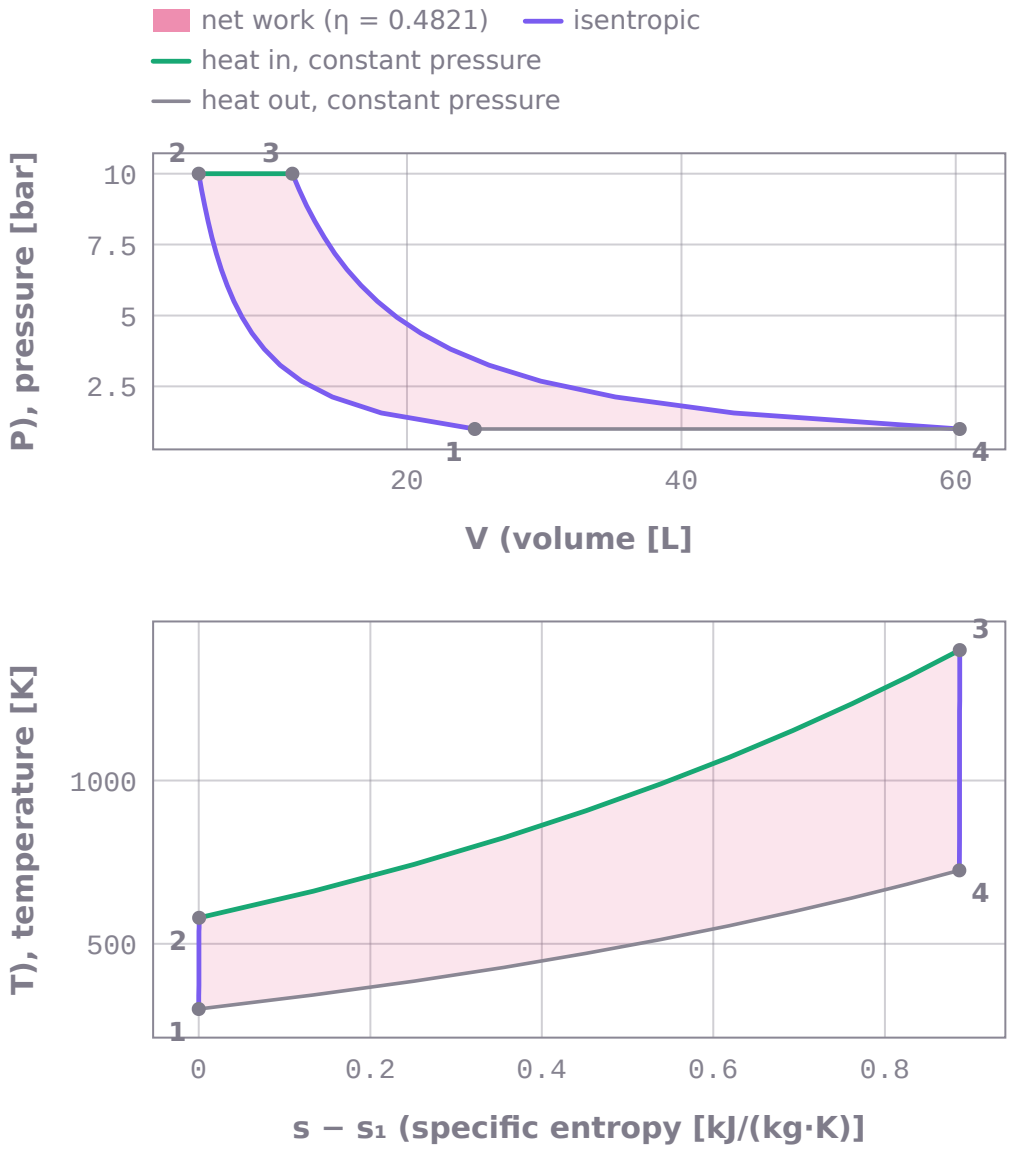
- "An air-standard Diesel cycle has a compression ratio of 18 and a cutoff ratio of 2. What is its thermal efficiency?" → 0.631578 1
- "Find the thermal efficiency of an ideal Diesel cycle with air, a compression ratio of 16 and a cut-off ratio of 2.5." → 0.590521 1

Thermal efficiency of the air-standard Brayton cycle, `brayton_efficiency_air`

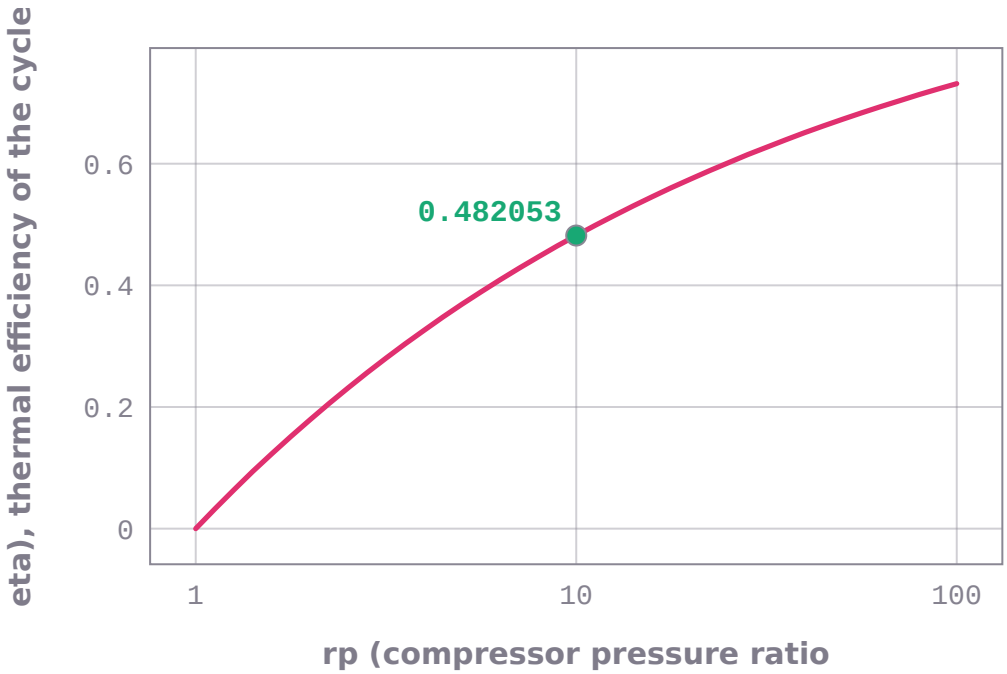
$$\eta = 1 - r_p^{\frac{-2}{7}}$$

input	meaning	kind / role	unit	default
`eta`	thermal efficiency of the cycle	efficiency / cycle	`1`	
`rp`	compressor pressure ratio	pressure_ratio / plain	`1`	

- **Conditions:** brayton, air
- **Not when:** non_air
- **Applies when:** idealised gas-turbine cycles
- **Fails when:** component inefficiencies; regeneration
- **Assumes:** cold-air-standard: air as an ideal gas with $\gamma = 1.4$; isentropic compressor and turbine
- **Evidence:** theorem



Air-standard Brayton cycle, pressure ratio rp = 10 (the worked example), stated compressor inlet 1 bar and 300 K, turbine inlet 1400 K, for 1 mol of air; P–V (top) and T–s (bottom). Temperatures on the isentropes from the package's isentropic temperature–pressure relation, volumes from its ideal-gas law, entropy from its ideal-gas entropy change (cp = 1005, R = 287 J/(kg·K)). Marked: T2 = 579.209 K, T4 = 725.126 K, the package's efficiency η = 0.482053.



eta (thermal efficiency of the cycle) against rp (compressor pressure ratio) from 1 to 100 on a logarithmic axis. 41 points computed by thermofluids-ai999x; marked: the worked example, rp = 10, eta = 0.482053.

```
thermofluids_ai999x.solve('brayton_efficiency_air', rp='10 1', target='eta')
# -> 0.482053 1   (SOLVED)
```

$$\eta = 1 - r_p^{\frac{-2}{7}}$$

$$\eta = 1 - \frac{1}{r_p^{\frac{2}{7}}} = 0.482053\ 1$$

invalid call	outcome	message
without `rp`	NEEDS_INPUT	brayton_efficiency_air needs rp (compressor pressure ratio, 1). Example: solve('brayton_efficiency_air', rp='... 1').
`rp` in a wrong unit	INVALID_INPUT	rp is a pressure ratio: its unit must be like 1, not 'kg'.

Questions in words that work:

- "An air-standard Brayton cycle operates with a pressure ratio of 10. What is its thermal efficiency?" → 0.482053 1
- "Find the thermal efficiency of an ideal gas turbine cycle using air with a compressor pressure ratio of 6." → 0.400663 1

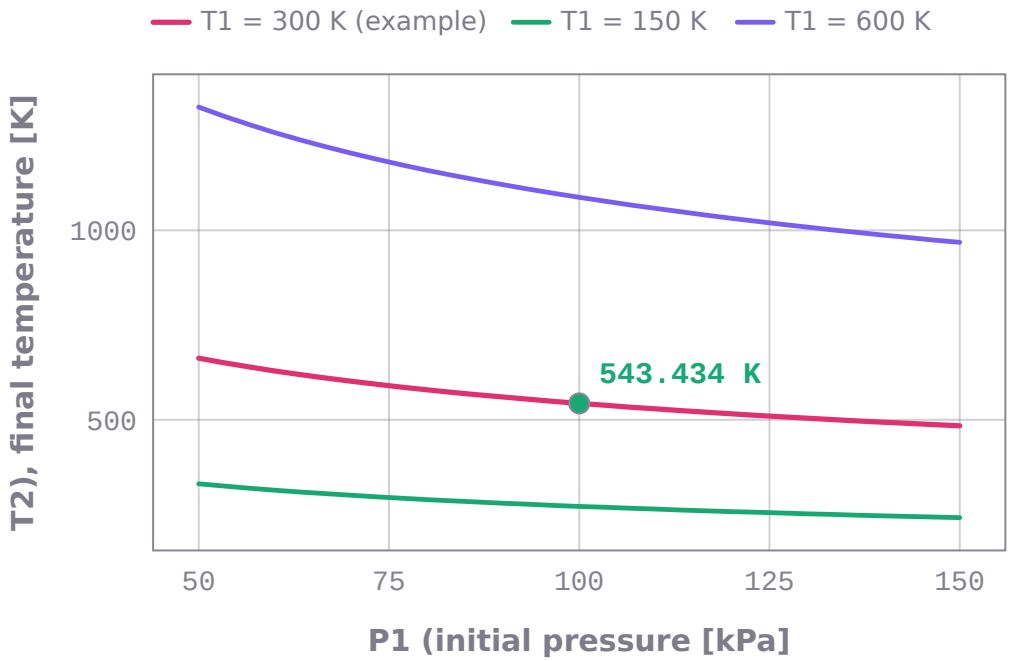
Temperature after an isentropic pressure change of air, `isentropic_temperature_pressure_air`

$$T_2 = T_1 \left(\frac{P_2}{P_1} \right)^{\frac{2}{7}}$$

input	meaning	kind / role	unit	default
`T2`	final temperature	temperature / final	`K`	

input	meaning	kind / role	unit	default
`T1`	initial temperature	temperature / initial	`K`	
`P2`	final pressure	pressure / final	`Pa`	
`P1`	initial pressure	pressure / initial	`Pa`	

- **Conditions:** adiabatic_process, air
- **Not when:** non_air, has_efficiency
- **Applies when:** ideal compressors, turbines and nozzles with air
- **Fails when:** irreversible processes (use an isentropic efficiency)
- **Assumes:** air as an ideal gas with $\gamma = 1.4$; reversible adiabatic process
- **Evidence:** theorem



T2 (final temperature) against P1 (initial pressure) from 50 kPa to 150 kPa. One curve for each value of T1 (initial temperature): 300 K, 150 K, 600 K. Other inputs as in the worked example: P2 = 800 kPa. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, P1 = 100 kPa, T2 = 543.434 K.

```
thermofluids_ai999x.solve('isentropic_temperature_pressure_air', T1='300 K', P1='100 kPa',
P2='800 kPa', target='T2')
# -> 543.434 K   (SOLVED)
```

$$T_2 = T_1 \left(\frac{P_2}{P_1} \right)^{\frac{2}{7}}$$

$$T_2 = \frac{P_2^{\frac{2}{7}} T_1}{P_1^{\frac{2}{7}}} = 543.434\ \mathrm{K}$$

invalid call	outcome	message
without `T1`	NEEDS_INPUT	isentropic_temperature_pressure_air needs T1 (initial temperature, K). Example: solve('isentropic_temperature_pressure_air', T1='... K', P2=
`T1` in a wrong unit	INVALID_INPUT	T1 is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

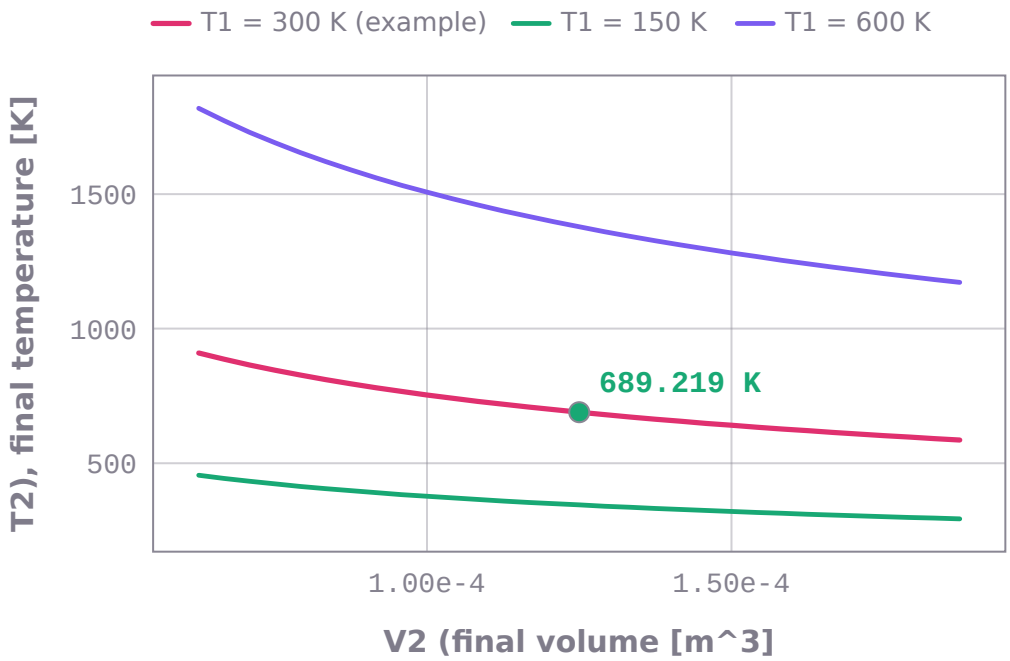
- "Air at an initial temperature of 300 K and an initial pressure of 100 kPa is compressed isentropically to a final pressure of 800 kPa. What is the final temperature?" → 543.434 K
- "Air is compressed adiabatically and reversibly from an initial pressure of 1 bar to a final pressure of 5 bar. If the initial temperature is 293 K, find the final temperature." → 464.059 K

Temperature after an isentropic volume change of air, `isentropic_temperature_volume_air`

$$T_2 = T_1 \left(\frac{V_1}{V_2} \right)^{\frac{2}{5}}$$

input	meaning	kind / role	unit	default
`T2`	final temperature	temperature / final	`K`	
`T1`	initial temperature	temperature / initial	`K`	
`V1`	initial volume	volume / original	`m^3`	
`V2`	final volume	volume / final	`m^3`	

- **Conditions:** adiabatic_process, air
- **Not when:** non_air, has_efficiency
- **Applies when:** ideal piston-cylinder compression or expansion of air
- **Fails when:** irreversible processes
- **Assumes:** air as an ideal gas with $\gamma = 1.4$; reversible adiabatic process
- **Evidence:** theorem



T2 (final temperature) against V2 (final volume) from 6.2500e-05 m^3 to 0.0001875 m^3. One curve for each value of T1 (initial temperature): 300 K, 150 K, 600 K. Other inputs as in the worked example: V1 = 0.001 m^3. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, V2 = 0.000125 m^3, T2 = 689.219 K.

```
thermofluids_ai999x.solve('isentropic_temperature_volume_air', T1='300 K', V1='0.001 m^3',
V2='0.000125 m^3', target='T2')
# -> 689.219 K   (SOLVED)
```

$$T_2 = T_1 \left(\frac{V_1}{V_2} \right)^{\frac{2}{5}}$$

$$T_2 = \frac{T_1 V_1^{\frac{2}{5}}}{V_2^{\frac{2}{5}}} = 689.219 \text{ K}$$

invalid call	outcome	message
without `T1`	NEEDS_INPUT	isentropic_temperature_volume_air needs T1 (initial temperature, K). Example: solve('isentropic_temperature_volume_air', T1='... K', V1='...
`T1` in a wrong unit	INVALID_INPUT	T1 is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

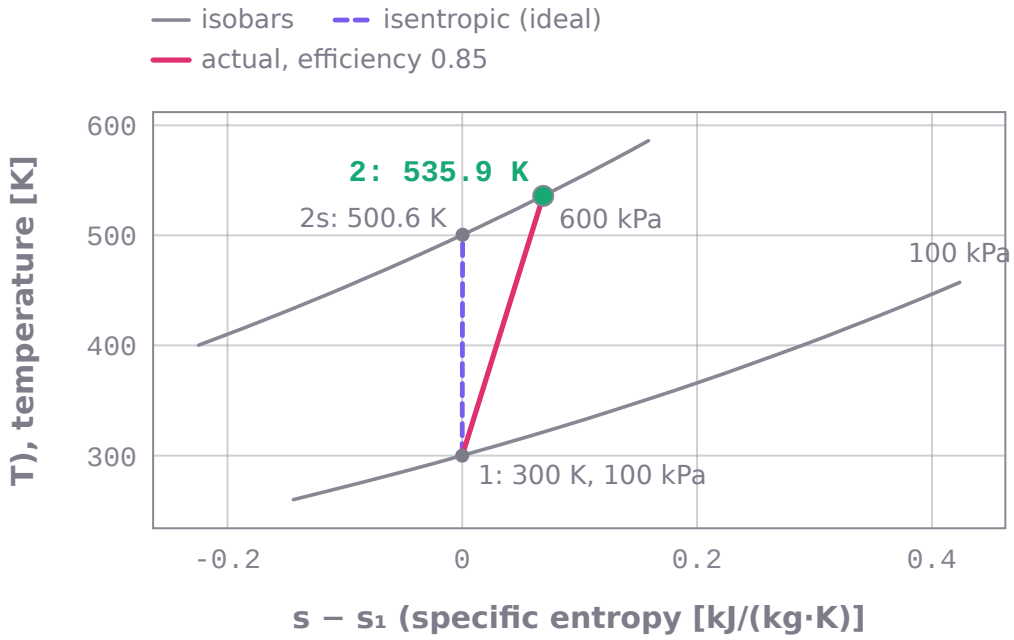
- "Air at an initial temperature of 300 K is compressed isentropically from an initial volume of 0.001 m^3 to a final volume of 0.000125 m^3. What is the final temperature?" → 689.219 K
- "Find the final temperature when air with an initial temperature of 350 K is compressed adiabatically and reversibly from an initial volume of 0.5 m^3 to a final volume of 0.1 m^3." → 666.279 K

Actual outlet temperature of an adiabatic air compressor, `compressor_outlet_temperature_air`

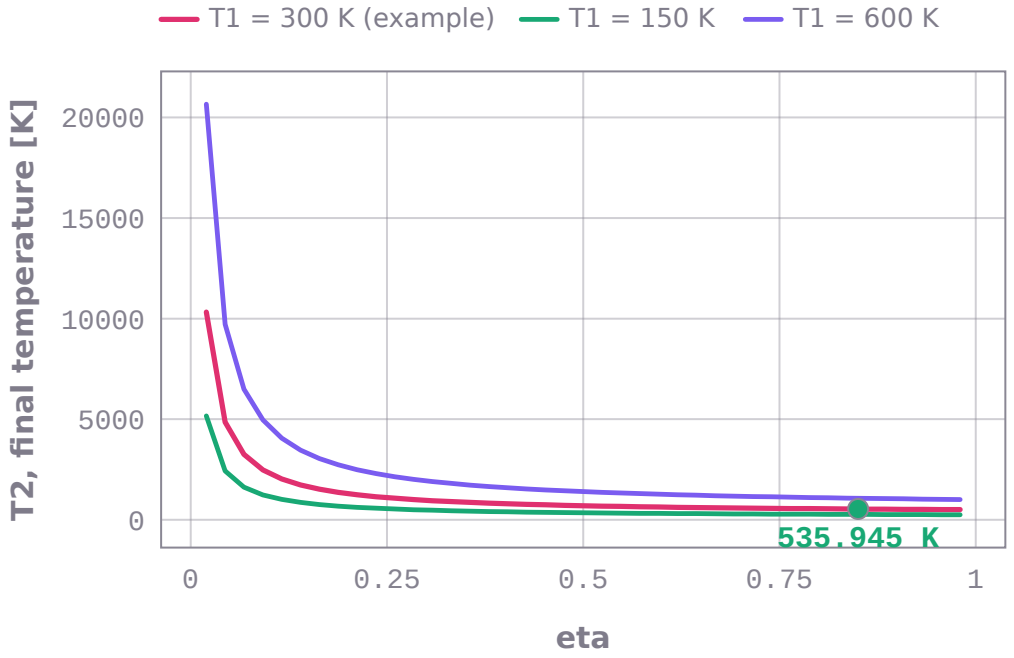
$$T_2 = T_1 \left(1 + \frac{\left(\frac{P_2}{P_1} \right)^{\frac{2}{7}} - 1}{\eta} \right)$$

input	meaning	kind / role	unit	default
`T2`	final temperature	temperature / final	`K`	
`T1`	initial temperature	temperature / initial	`K`	
`P2`	final pressure	pressure / final	`Pa`	
`P1`	initial pressure	pressure / initial	`Pa`	
`eta`	isentropic efficiency of the compressor	efficiency / isentropic	`1`	

- **Conditions:** compressor, air
- **Not when:** non_air
- **Applies when:** real compressors with a known isentropic efficiency
- **Fails when:** cooled compressors
- **Assumes:** air as an ideal gas with $\gamma = 1.4$; adiabatic compressor
- **Evidence:** theorem



Adiabatic compression of air from 300 K and 100 kPa to 600 kPa with an isentropic efficiency of 0.85 (the worked example) on T–s axes, with the two isobars from the package's ideal-gas entropy change (cp = 1005, R = 287 J/(kg·K)). The ideal outlet 2s comes from its isentropic temperature–pressure relation, the actual outlet 2 from the compressor capability. Marked: T2s = 500.553 K, T2 = 535.945 K, entropy generated 0.0689151 kJ/(kg·K).



T2 (final temperature) against eta (isentropic efficiency of the compressor) from 0.02 to 0.98. One curve for each value of T1 (initial temperature): 300 K, 150 K, 600 K. Other inputs as in the worked example: P2 = 600 kPa, P1 = 100 kPa. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, eta = 0.85, T2 = 535.945 K.

```
thermofluids_ai999x.solve('compressor_outlet_temperature_air', T1='300 K', P1='100 kPa',
P2='600 kPa', eta='0.85 1', target='T2')
# -> 535.945 K   (SOLVED)
```

$$T_2 = T_1 \left(1 + \frac{\left(\frac{P_2}{P_1}\right)^{\frac{2}{7}} - 1}{\eta} \right)$$

$$T_2 = T_1 - \frac{T_1}{\eta} + \frac{P_2^{\frac{2}{7}} T_1}{P_1^{\frac{2}{7}} \eta} = 535.945\ \mathrm{K}$$

invalid call	outcome	message
without `T1`	NEEDS_INPUT	compressor_outlet_temperature_air needs T1 (initial temperature, K). Example: solve('compressor_outlet_temperature_air', T1='... K', P2='...
`T1` in a wrong unit	INVALID_INPUT	T1 is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

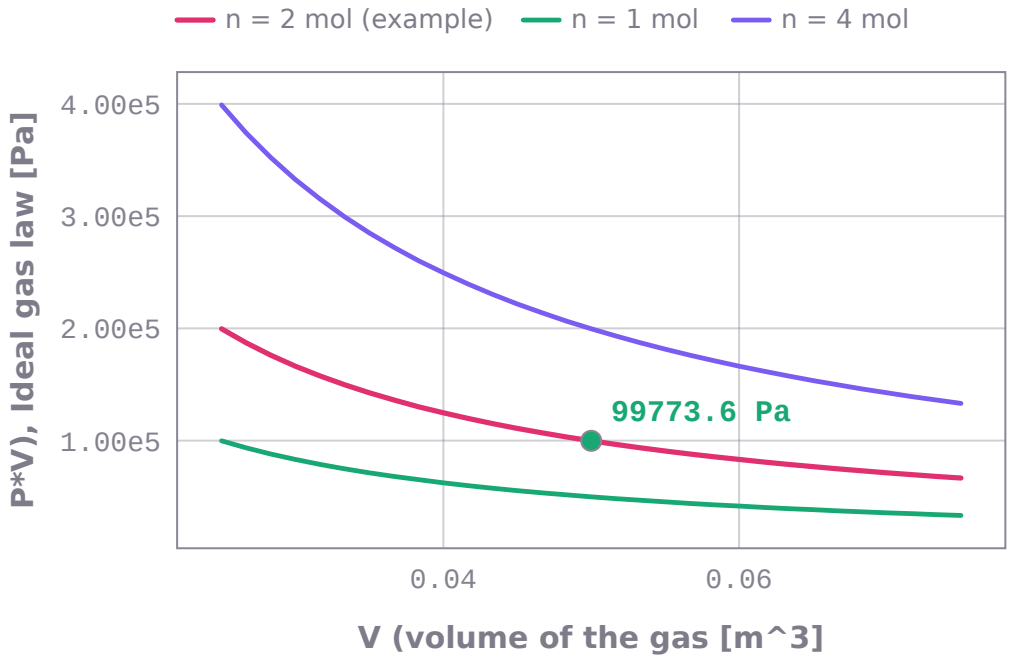
- "An adiabatic compressor takes in air at an initial temperature of 300 K and an initial pressure of 100 kPa and delivers it at a final pressure of 600 kPa. Its isentropic efficiency is 0.85. What is the final temperature?" → 535.945 K
- "Air is compressed from an initial pressure of 1 bar to a final pressure of 10 bar in a compressor with an isentropic efficiency of 0.8. If the initial temperature is 290 K, find the final temperature." → 627.378 K

Ideal gas law, `ideal_gas_law`

$$PV = nRT$$

input	meaning	kind / role	unit	default
`P`	absolute pressure of the gas	pressure / plain	`Pa`	
`V`	volume of the gas	volume / plain	`m^3`	
`n`	amount of gas	amount / plain	`mol`	
`R`		gas_constant / constant	`J/(mol*K)`	8.314462618 J/(mol*K)
`T`	absolute temperature	temperature / plain	`K`	

- **Conditions:** ideal_gas
- **Applies when:** gases at moderate pressure
- **Fails when:** high pressures or near condensation
- **Assumes:** ideal gas; absolute pressure and temperature
- **Evidence:** theorem



P*V (Ideal gas law) against V (volume of the gas) from 0.025 m^3 to 0.075 m^3. One curve for each value of n (amount of gas): 2 mol, 1 mol, 4 mol. Other inputs as in the worked example: T = 300 K. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, V = 0.05 m^3, P*V = 99773.6 Pa.

```
thermofluids_ai999x.solve('ideal_gas_law', V='0.05 m^3', n='2 mol', T='300 K', target='P')
# -> 99773.6 Pa   (SOLVED)
```

$$PV = nRT$$
$$P = \frac{RTn}{V} = 99773.6\ \text{Pa}$$

invalid call	outcome	message
without `V`	NEEDS_INPUT	ideal_gas_law needs V (volume of the gas, m^3). Example: solve('ideal_gas_law', V='... m^3', n='... mol', R='... J/(mol*K)', T='... K').
`V` in a wrong unit	INVALID_INPUT	V is a volume: its unit must be like m^3, not 'kg'.

Questions in words that work:

- "An ideal gas of 2 mol is held in a volume of 0.05 m^3 at a temperature of 300 K. What is its pressure?" → 99773.6 Pa
- "What volume does 0.5 mol of an ideal gas occupy at a pressure of 101325 Pa and a temperature of 273.15 K?" → 0.011207 m^3

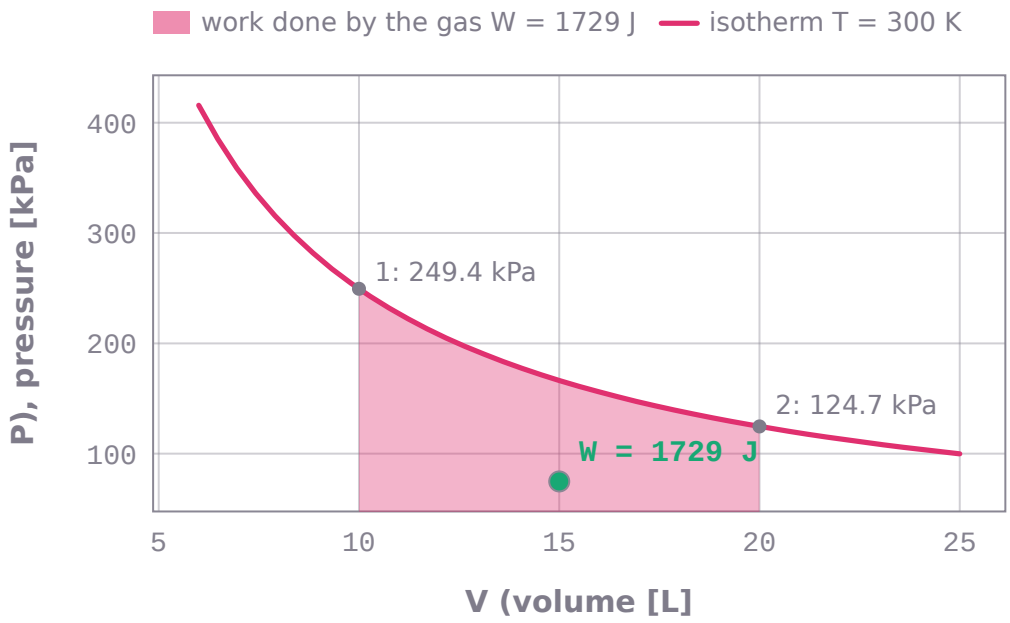
Work done by an ideal gas in a reversible isothermal process, `isothermal_work_ideal_gas`

$$W = nRT \ln\left(\frac{V_2}{V_1}\right)$$

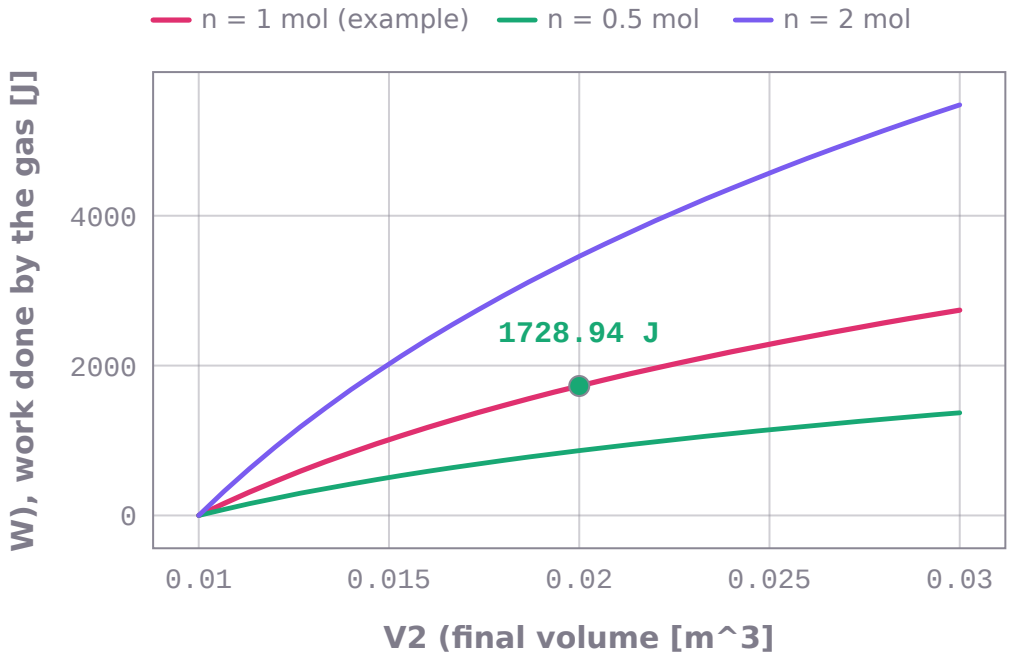
input	meaning	kind / role	unit	default
`W`	work done by the gas	energy / work	`J`	

input	meaning	kind / role	unit	default
`n`	amount of gas	amount / plain	`mol`	
`R`		gas_constant / constant	`J/(mol*K)`	8.314462618 J/(mol*K)
`T`	constant absolute temperature	temperature / plain	`K`	
`V2`	final volume	volume / final	`m^3`	
`V1`	initial volume	volume / original	`m^3`	

- **Conditions:** isothermal
- **Applies when:** slow expansion or compression at constant temperature
- **Fails when:** irreversible expansion against a constant external pressure
- **Assumes:** ideal gas; reversible (quasi-static) process
- **Evidence:** theorem



Isothermal expansion of 1 mol of ideal gas at 300 K from 10 L to 20 L (the worked example) on P–V axes; pressures from the package's ideal-gas law at 41 volumes. The shaded area under the isotherm is the work; marked: the package's W = 1728.94 J, P1 = 249.434 kPa, P2 = 124.717 kPa.



W (work done by the gas) against V2 (final volume) from 0.01 m^3 to 0.03 m^3. One curve for each value of n (amount of gas): 1 mol, 0.5 mol, 2 mol. Other inputs as in the worked example: T = 300 K, V1 = 0.01 m^3. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, V2 = 0.02 m^3, W = 1728.94 J.

```
thermofluids_ai999x.solve('isothermal_work_ideal_gas', n='1 mol', T='300 K', V2='0.02 m^3',
V1='0.01 m^3', target='W')
# -> 1728.94 J   (SOLVED)
```

$$W = nRT \ln\left(\frac{V_2}{V_1}\right)$$

$$W = \ln\left(\left(\frac{V_2}{V_1}\right)^{RTn}\right) = 1728.94 \text{ J}$$

invalid call	outcome	message
without `n`	NEEDS_INPUT	isothermal_work_ideal_gas needs n (amount of gas, mol). Example: solve('isothermal_work_ideal_gas', n='... mol', R='... J/(mol*K)', T='... K
`n` in a wrong unit	INVALID_INPUT	n is a amount: its unit must be like mol, not 'kg'.

Questions in words that work:

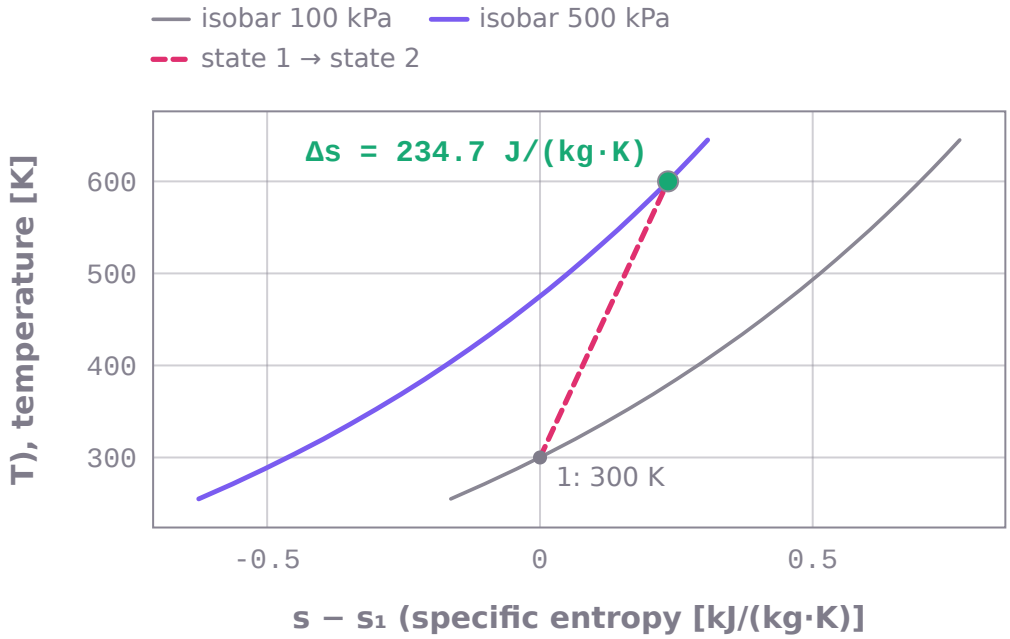
- "An ideal gas of 1 mol expands isothermally at a temperature of 300 K from an initial volume of 0.01 m^3 to a final volume of 0.02 m^3. How much work is done by the gas?" → 1728.94 J
- "Find the work done when 2 mol of an ideal gas at a temperature of 350 K is compressed isothermally from an initial volume of 0.05 m^3 to a final volume of 0.01 m^3." → -9367.13 J

Specific entropy change of an ideal gas, `entropy_change_ideal_gas`

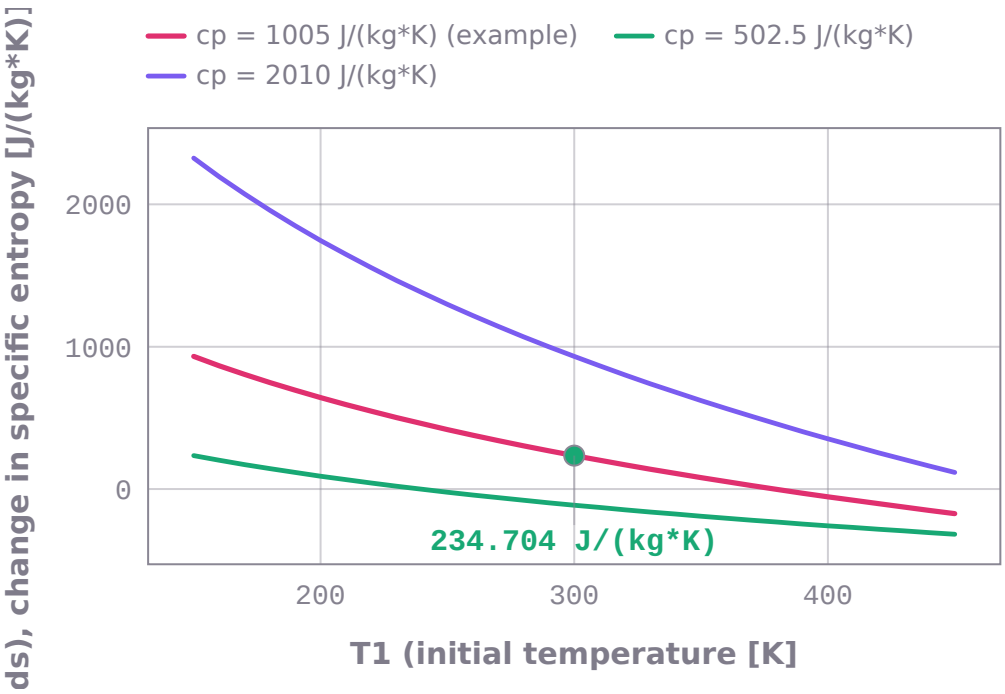
$$d_s = c_p \ln\left(\frac{T_2}{T_1}\right) - R \ln\left(\frac{P_2}{P_1}\right)$$

input	meaning	kind / role	unit	default
`ds`	change in specific entropy	specific_entropy / change	`J/(kg*K)`	
`cp`	specific heat at constant pressure	specific_heat / plain	`J/(kg*K)`	
`R`	specific gas constant	specific_gas_constant / plain	`J/(kg*K)`	
`T1`	initial temperature	temperature / initial	`K`	
`T2`	final temperature	temperature / final	`K`	
`P1`	initial pressure	pressure / initial	`Pa`	
`P2`	final pressure	pressure / final	`Pa`	

- **Conditions:** entropy, ideal_gas
- **Applies when:** any process between two equilibrium states
- **Fails when:** temperature-dependent cp over wide ranges; real gases
- **Assumes:** ideal gas with constant specific heats
- **Evidence:** theorem



Entropy change of an ideal gas (cp = 1005, R = 287 J/(kg·K)) from 300 K, 100 kPa to 600 K, 500 kPa (the worked example) on T–s axes; both isobars are 25 runs each of the package's entropy relation from state 1. Marked: state 2, Δs = 234.704 J/(kg·K).



ds (change in specific entropy) against T1 (initial temperature) from 150 K to 450 K. One curve for each value of cp (specific heat at constant pressure): 1005 J/(kg*K), 502.5 J/(kg*K), 2010 J/(kg*K). Other inputs as in the worked example: R = 287 J/(kg*K), T2 = 600 K, P1 = 100 kPa, P2 = 500 kPa. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, T1 = 300 K, ds = 234.704 J/(kg*K).

```
thermofluids_ai999x.solve('entropy_change_ideal_gas', cp='1005 J/(kg*K)', R='287 J/(kg*K)',
T1='300 K', T2='600 K', P1='100 kPa', P2='500 kPa', target='ds')
# -> 234.704 J/(kg*K)   (SOLVED)
```

$$d_s = c_p \ln\left(\frac{T_2}{T_1}\right) - R \ln\left(\frac{P_2}{P_1}\right)$$

$$d_s = \ln\left(\left(\frac{P_2}{P_1}\right)^{-R}\left(\frac{T_2}{T_1}\right)^{c_p}\right) = 234.704\ \mathrm{J/(kg\,K)}$$

invalid call	outcome	message
without `cp`	NEEDS_INPUT	entropy_change_ideal_gas needs cp (specific heat at constant pressure, J/(kg*K)). Example: solve('entropy_change_ideal_gas', cp='... J/(kg*K
`cp` in a wrong unit	INVALID_INPUT	cp is a specific heat: its unit must be like J/(kg*K), not 's'.

Questions in words that work:

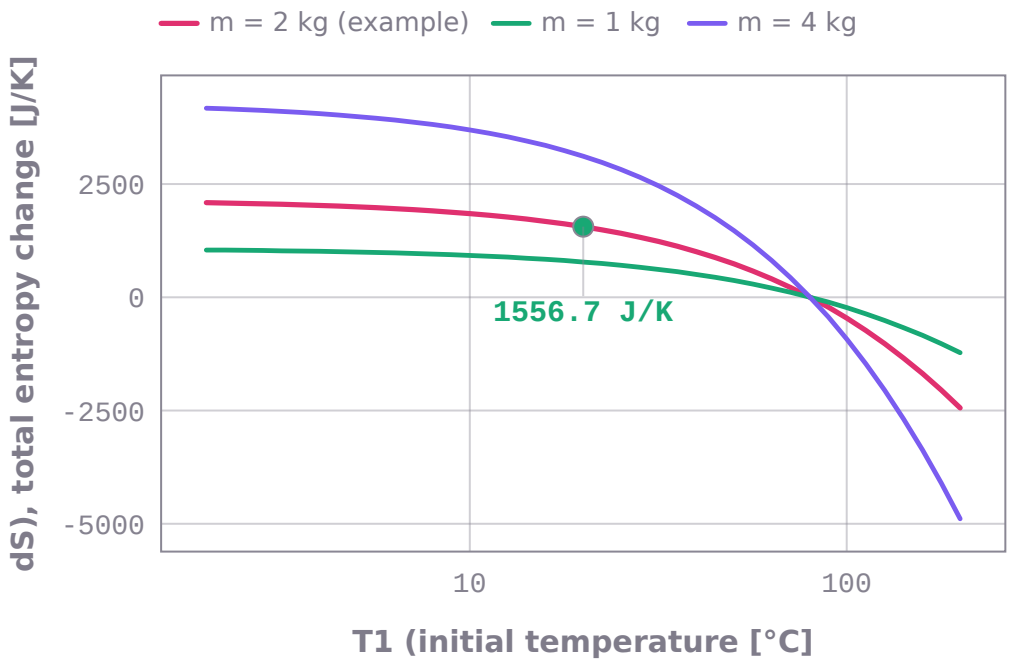
- "An ideal gas with a specific heat at constant pressure of 1005 J/(kg*K) and a gas constant of 287 J/(kg*K) goes from an initial temperature of 300 K and an initial pressure of 100 kPa to a final temperature of 600 K and a final pressure of 500 kPa. What is the specific entropy change?" → 234.704 J/(kg*K)
- "Find the change in specific entropy of nitrogen gas, treated as an ideal gas with specific heat 1040 J/(kg*K) and specific gas constant 297 J/(kg*K), heated from an initial temperature of 290 K at an initial pressure of 1 bar to a final temperature of 450 K at a final pressure of 3 bar." → 130.653 J/(kg*K)

Entropy change of an incompressible substance, `entropy_change_incompressible`

$$d_S = mc \ln\left(\frac{T_2}{T_1}\right)$$

input	meaning	kind / role	unit	default
`dS`	total entropy change	entropy / change	`J/K`	
`m`	mass	mass / plain	`kg`	
`c`	specific heat	specific_heat / plain	`J/(kg*K)`	
`T2`	final temperature	temperature / final	`K`	
`T1`	initial temperature	temperature / initial	`K`	

- **Conditions:** entropy, incompressible_substance
- **Not when:** ideal_gas
- **Applies when:** liquids and solids heated or cooled
- **Fails when:** phase change; gases
- **Assumes:** incompressible substance with constant specific heat
- **Evidence:** theorem



dS (total entropy change) against T1 (initial temperature) from 2 °C to 200 °C on a logarithmic axis. One curve for each value of m (mass): 2 kg, 1 kg, 4 kg. Other inputs as in the worked example: c = 4180 J/(kg*K), T2 = 80 °C. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, T1 = 20 °C, dS = 1556.7 J/K.

```
thermofluids_ai999x.solve('entropy_change_incompressible', m='2 kg', c='4180 J/(kg*K)',
T2='80 °C', T1='20 °C', target='dS')
# -> 1556.7 J/K   (SOLVED)
```

$$d_S = mc \ln\left(\frac{T_2}{T_1}\right)$$
$$d_S = \ln\left(\left(\frac{T_2}{T_1}\right)^{cm}\right) = 1556.7\ \mathrm{J/K}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	entropy_change_incompressible needs m (mass, kg). Example: solve('entropy_change_incompressible', m='... kg', c='... J/(kg*K)', T2='... K',
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A mass of 2 kg of liquid water with specific heat 4180 J/(kg*K) is heated from an initial temperature of 20 °C to a final temperature of 80 °C. What is its entropy change?" → 1556.7 J/K
- "Find the change in entropy of a 5 kg aluminium block with specific heat 900 J/(kg*K) cooled from an initial temperature of 400 K to a final temperature of 300 K." → -1294.57 J/K

Humidity ratio from the partial pressure of water vapour, `humidity_ratio_vapour_pressure`

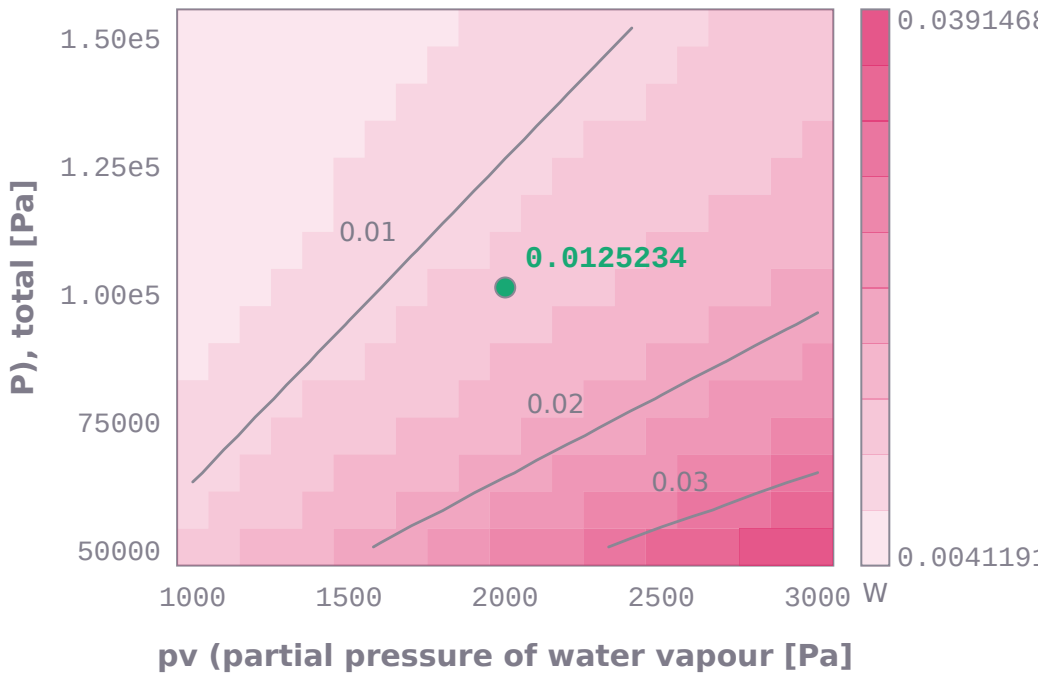
$$W = \frac{0.621945 p_v}{P - p_v}$$

input	meaning	kind / role	unit	default
`W`	humidity ratio (kg water vapour per kg dry air)	humidity_ratio / plain	`1`	
`pv`	partial pressure of water vapour	pressure / vapour	`Pa`	
`P`	total (barometric) pressure of the moist air	pressure / atmospheric	`Pa`	

- **Conditions:** moist_air
- **Applies when:** atmospheric air-conditioning and drying calculations
- **Fails when:** high pressures (enhancement factor)
- **Assumes:** moist air as an ideal-gas mixture; ratio of molar masses 0.621945
- **Evidence:** theorem



W (humidity ratio (kg water vapour per kg dry air)) against P (total (barometric) pressure of the moist air) from 50662.5 Pa to 151988 Pa. One curve for each value of pv (partial pressure of water vapour): 2000 Pa, 1000 Pa, 4000 Pa. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, P = 101325 Pa, W = 0.0125234.



W (humidity ratio (kg water vapour per kg dry air)) over pv (partial pressure of water vapour) from 1000 Pa to 3000 Pa and P (total (barometric) pressure of the moist air) from 50662.5 Pa to 151988 Pa: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 0.00411918 to dark 0.0391468); iso-lines at W = 0.01, 0.02, 0.03. Marked: the worked example, pv = 2000 Pa, P = 101325 Pa, W = 0.0125234.

```
thermofluids_ai999x.solve('humidity_ratio_vapour_pressure', pv='2000 Pa', P='101325 Pa',
target='W')
# -> 0.0125234 1   (SOLVED)
```

$$W = \frac{0.621945 p_v}{P - p_v}$$
$$W = \frac{124389 p_v}{200000 \left(P - p_v\right)} = 0.0125234\ 1$$

invalid call	outcome	message
without `pv`	NEEDS_INPUT	humidity_ratio_vapour_pressure needs pv (partial pressure of water vapour, Pa). Example: solve('humidity_ratio_vapour_pressure', pv='... Pa'
`pv` in a wrong unit	INVALID_INPUT	pv is a pressure: its unit must be like Pa, not 'kg'.

Questions in words that work:

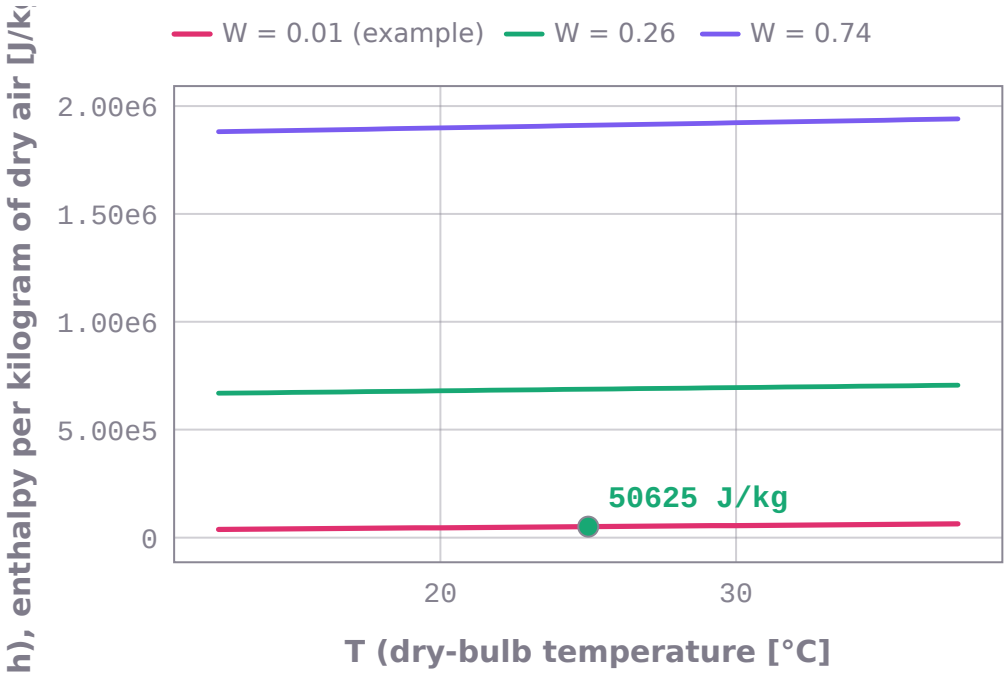
- "Moist air at a barometric pressure of 101325 Pa has a partial pressure of water vapour of 2000 Pa. What is the humidity ratio?" → 0.0125234 1
- "Find the humidity ratio of humid air at an atmospheric pressure of 90 kPa when the vapour pressure is 1.5 kPa." → 0.0105414 1

Specific enthalpy of moist air, `moist_air_enthalpy`

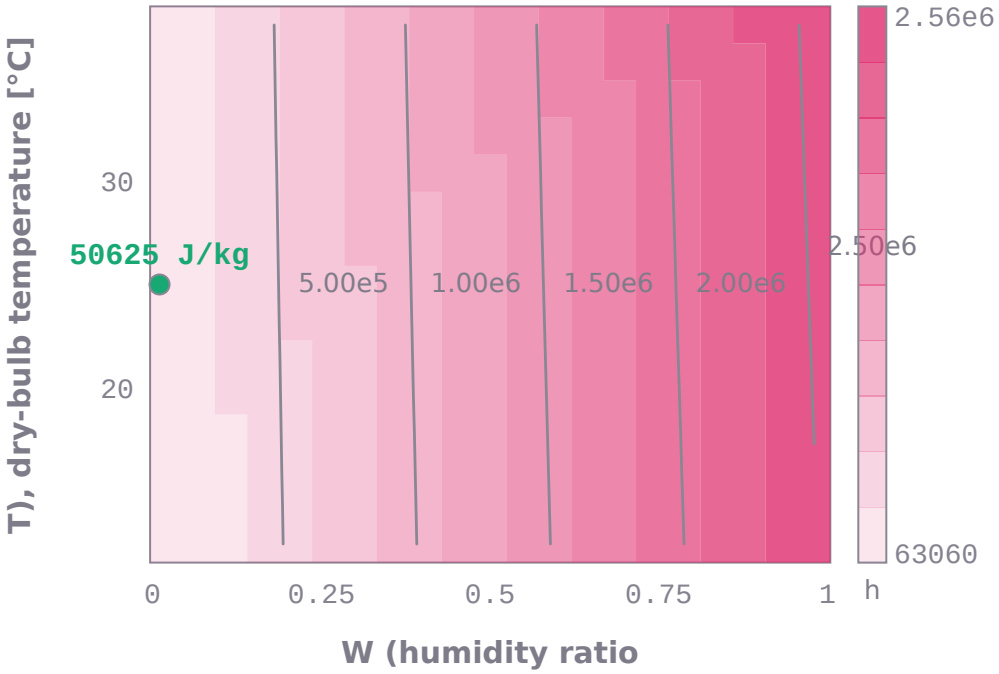
$$h = \mathrm{cpa} \left(T - T_0\right) + W \left(h_g + \mathrm{cpv} \left(T - T_0\right)\right)$$

input	meaning	kind / role	unit	default
`h`	enthalpy per kilogram of dry air	specific_enthalpy / plain	`J/kg`	
`T`	dry-bulb temperature	temperature / dry_bulb	`K`	
`W`	humidity ratio (kg water vapour per kg dry air)	humidity_ratio / plain	`1`	
`cpa`		specific_heat / dry_air	`J/(kg*K)`	1006 J/(kg*K)
`cpv`		specific_heat / water_vapour	`J/(kg*K)`	1860 J/(kg*K)
`hg`		latent_heat / reference	`J/kg`	2501000 J/kg
`T0`		temperature / reference	`K`	273.15 K

- **Conditions:** moist_air
- **Applies when:** air-conditioning processes between about -40 and 60 °C
- **Fails when:** very high temperatures
- **Assumes:** reference state: dry air and liquid water at 0 °C; constant specific heats
- **Evidence:** theorem



h (enthalpy per kilogram of dry air) against T (dry-bulb temperature) from 12.5 °C to 37.5 °C. One curve for each value of W (humidity ratio (kg water vapour per kg dry air)): 0.01, 0.26, 0.74. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, T = 25 °C, h = 50625 J/kg.



h (enthalpy per kilogram of dry air) over W (humidity ratio (kg water vapour per kg dry air)) from 0.02 to 0.98 and T (dry-bulb temperature) from 12.5 °C to 37.5 °C: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 63060 J/kg to dark 2.55706e+06 J/kg); iso-lines at h = 5.00e5, 1.00e6, 1.50e6, 2.00e6, 2.50e6 J/kg. Marked: the worked example, W = 0.01, T = 25 °C, h = 50625 J/kg.

```
thermofluids_ai999x.solve('moist_air_enthalpy', T='25 °C', W='0.01 1', target='h')
# -> 50625 J/kg   (SOLVED)
```

$$h = \mathrm{cpa}\,(T - T_0) + W\,(h_g + \mathrm{cpv}\,(T - T_0))$$
$$h = TW\mathrm{cpv} + T\mathrm{cpa} - T_0 W\mathrm{cpv} - T_0\mathrm{cpa} + Wh_g = 50625\ \mathrm{J/kg}$$

invalid call	outcome	message
without `T`	NEEDS_INPUT	moist_air_enthalpy needs T (dry-bulb temperature, K). Example: solve('moist_air_enthalpy', T='... K', W='... 1', cpa='... J/(kg*K)', cpv='..
`T` in a wrong unit	INVALID_INPUT	T is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

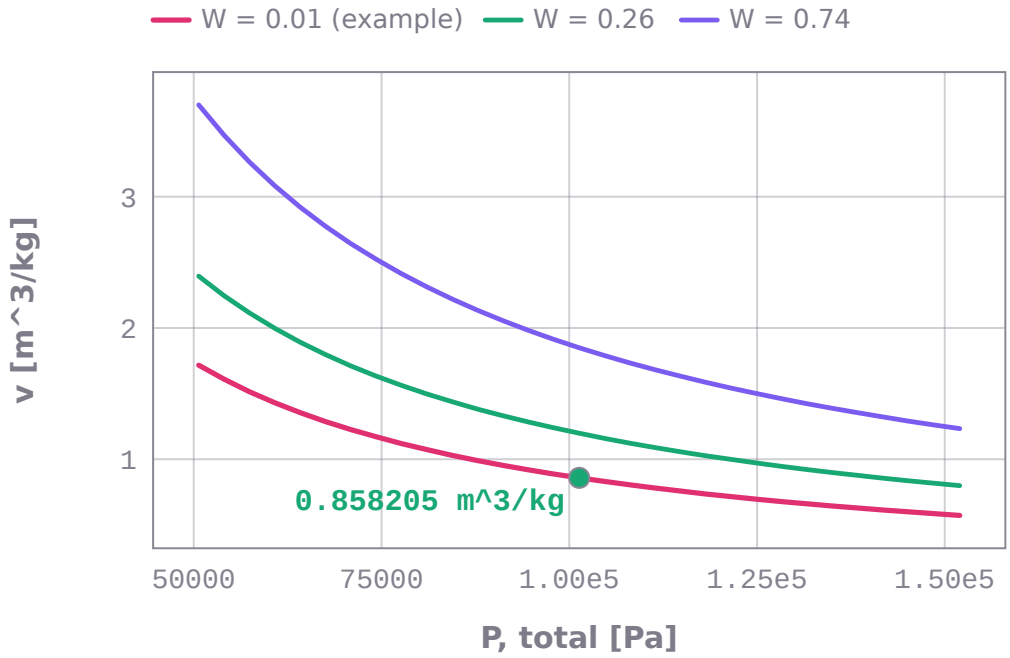
- "Moist air has a dry-bulb temperature of 25 °C and a humidity ratio of 0.01. What is its specific enthalpy?" → 50625 J/kg
- "Find the enthalpy of humid air at a dry bulb temperature of 35 °C with a humidity ratio of 0.02." → 86532 J/kg

Specific volume of moist air per kilogram of dry air, `moist_air_specific_volume`

$$v = \frac{\mathrm{Ra} T \left(1 + 1.607858 W\right)}{P}$$

input	meaning	kind / role	unit	default
`v`	specific volume per kilogram of dry air	specific_volume / plain	`m^3/kg`	
`T`	dry-bulb temperature	temperature / dry_bulb	`K`	
`W`	humidity ratio (kg water vapour per kg dry air)	humidity_ratio / plain	`1`	
`P`	total (barometric) pressure of the moist air	pressure / atmospheric	`Pa`	
`Ra`		specific_gas_constant / dry_air	`J/(kg*K)`	287.042 J/(kg*K)

- **Conditions:** moist_air
- **Applies when:** fan and duct sizing in air conditioning
- **Fails when:** high pressures
- **Assumes:** moist air as an ideal-gas mixture
- **Evidence:** theorem



v (specific volume per kilogram of dry air) against P (total (barometric) pressure of the moist air) from 50662.5 Pa to 151988 Pa. One curve for each value of W (humidity ratio (kg water vapour per kg dry air)): 0.01, 0.26, 0.74. Other inputs as in the worked example: T = 25 °C. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, P = 101325 Pa, v = 0.858205 m^3/kg.

```
thermofluids_ai999x.solve('moist_air_specific_volume', T='25 °C', W='0.01 1', P='101325
Pa', target='v')
# -> 0.858205 m^3/kg   (SOLVED)
```

$$v = \frac{\mathrm{Ra} T \left(1 + 1.607858 W\right)}{P}$$

$$v = \frac{\mathrm{Ra} T \left(803929 W + 500000\right)}{500000 P} = 0.858205\ \mathrm{m^3/kg}$$

invalid call	outcome	message
without `T`	NEEDS_INPUT	moist_air_specific_volume needs T (dry-bulb temperature, K). Example: solve('moist_air_specific_volume', T='... K', W='... 1', P='... Pa', R
`T` in a wrong unit	INVALID_INPUT	T is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

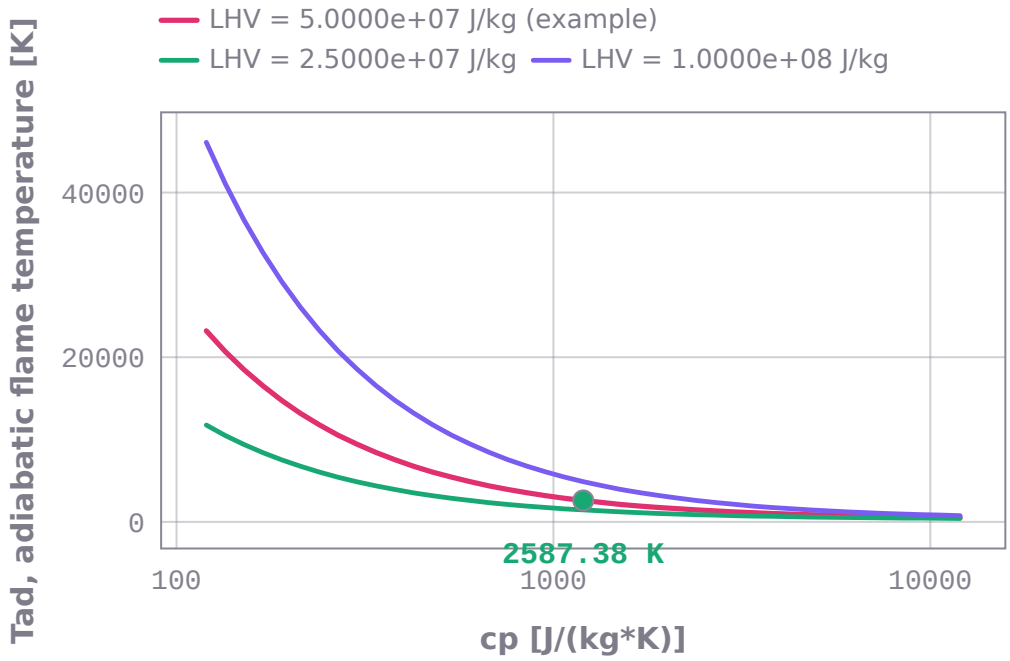
- "Moist air at a dry-bulb temperature of 25 °C, a humidity ratio of 0.01 and a barometric pressure of 101325 Pa. What is its specific volume?" → 0.858205 m^3/kg
- "Find the specific volume of humid air with a dry bulb temperature of 30 °C, a humidity ratio of 0.015 and an atmospheric pressure of 95 kPa." → 0.938057 m^3/kg

Adiabatic flame temperature with constant product specific heat, `adiabatic_flame_temperature_simple`

$$\mathrm{Tad} = T_0 + \frac{\mathrm{LHV}}{\left(1 + \mathrm{AFR}\right) c_p}$$

input	meaning	kind / role	unit	default
`Tad`	adiabatic flame temperature	temperature / flame	`K`	
`T0`	temperature of the reactants	temperature / reactant	`K`	
`LHV`	lower heating value of the fuel	heating_value / plain	`J/kg`	
`AFR`	air-fuel ratio by mass	air_fuel_ratio / plain	`1`	
`cp`	mean specific heat of the products	specific_heat / plain	`J/(kg*K)`	

- **Conditions:** combustion
- **Applies when:** first estimates of flame temperature for lean or stoichiometric mixtures
- **Fails when:** near-stoichiometric flames above about 2000 K (dissociation)
- **Assumes:** complete combustion; no dissociation; adiabatic, constant-pressure burning; constant mean cp of the products
- **Evidence:** theorem



Tad (adiabatic flame temperature) against cp (mean specific heat of the products) from 120 J/(kg*K) to 12000 J/(kg*K) on a logarithmic axis. One curve for each value of LHV (lower heating value of the fuel): 5.0000e+07 J/kg, 2.5000e+07 J/kg, 1.0000e+08 J/kg. Other inputs as in the worked example: T0 = 298 K, AFR = 17.2 1. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, cp = 1200 J/(kg*K), Tad = 2587.38 K.

```
thermofluids_ai999x.solve('adiabatic_flame_temperature_simple', T0='298 K', LHV='5e+07
J/kg', AFR='17.2 1', cp='1200 J/(kg*K)', target='Tad')
# -> 2587.38 K   (SOLVED)
```

$$\mathrm{Tad} = T_0 + \frac{\mathrm{LHV}}{(1+\mathrm{AFR})c_p}$$

$$\mathrm{Tad} = \frac{\mathrm{AFR}T_0c_p + \mathrm{LHV} + T_0c_p}{c_p\,(\mathrm{AFR}+1)} = 2587.38\ \mathrm{K}$$

invalid call	outcome	message
without `T0`	NEEDS_INPUT	adiabatic_flame_temperature_simple needs T0 (temperature of the reactants, K). Example: solve('adiabatic_flame_temperature_simple', T0='...

invalid call	outcome	message
`T0` in a wrong unit	INVALID_INPUT	T0 is a temperature: its unit must be like K, not 'kg'.

Questions in words that work:

- "Methane with a lower heating value of 50000000 J/kg burns at an air-fuel ratio of 17.2 with a reactant temperature of 298 K. Taking the specific heat of the products as 1200 J/(kg*K), what is the adiabatic flame temperature?" → 2587.38 K
- "Find the adiabatic flame temperature of a fuel with heating value 44000 kJ/kg burned with an air-fuel ratio of 20, if the temperature of the reactants is 300 K and the specific heat of the products is 1100 J/(kg*K)." → 2204.76 K

Procedures: algorithms with several outputs

Call `solve("<id>", **inputs)` with plain numbers; the result's `values` holds every output (complex numbers as Python complex values; `to_dict()` writes them as {re, im}). Each result is substituted back into its defining equation before it is returned.

id	operation	inputs	outputs
colebrook_friction_factor	Darcy friction factor from the Colebrook equation	Re, eD	f
lmtd	Log-mean temperature difference of a heat exchanger	Th_in, Th_out, Tc_in, Tc_out, arrangement	lmtd, dT1, dT2
hx_effectiveness_ntu	Heat exchanger rating by the effectiveness-NTU method	C_hot, C_cold, UA, Th_in, Tc_in, arrangement	Q, Th_out, Tc_out, effectiveness, NTU
saturation_vapour_pressure	Saturation pressure of water vapour	T	p_sat
dew_point_temperature	Dew-point temperature from dry-bulb temperature and relative humidity	T, RH	T_dew, p_v
humidity_ratio_from_relative_humidity	Humidity ratio from temperature, relative humidity and pressure	T, RH, P	W, p_v
wet_bulb_temperature	Thermodynamic wet-bulb temperature	T, RH, P	T_wb, W
stoichiometric_air_fuel_ratio	Stoichiometric air-fuel ratio of a gaseous fuel	fuel	afr_stoich, o2_moles, air_moles
equivalence_ratio	Equivalence ratio of a fuel-air mixture	fuel, afr	phi, afr_stoich, excess_air
gnielinski_nusselt	Gnielinski correlation for turbulent pipe flow	Re, Pr, f	Nu
churchill_bernstein_cylinder	Churchill-Bernstein correlation for a cylinder in cross flow	Re, Pr	Nu
churchill_chu_vertical_plate	Churchill-Chu correlation for natural convection on a vertical plate	Ra, Pr	Nu

id	operation	inputs	outputs
pin_fin_efficiency	Efficiency of a pin fin with an insulated tip	h, k, D, L	efficiency, mL
first_law_internal_energy_change	First law of thermodynamics: change of internal energy	Q, W	dU
isobaric_boundary_work	Boundary work of a constant-pressure (isobaric) process	P, V1, V2	W, dV
entropy_change_reservoir	Entropy change for heat exchanged at a constant temperature	Q, T	dS
entropy_generation_heat_transfer	Entropy generated when heat flows between two reservoirs	Q, T_hot, T_cold	dS_total, dS_hot, dS_cold
hydraulic_press_pascal	Hydraulic press (Pascal's principle)	F1, A1, A2	F2, P, MA, d_ratio
bernoulli_with_height	Bernoulli's equation with a change of height	P1, v1, h1, v2, h2, rho, g	P2
wet_steam_mixture	Wet steam: quality and mixture properties from saturated values	vf, vg, hf, hfg, sf, sfg, x, v, h, s	x, moisture, v, h, s, hg, sg

Darcy friction factor from the Colebrook equation, `colebrook_friction_factor`

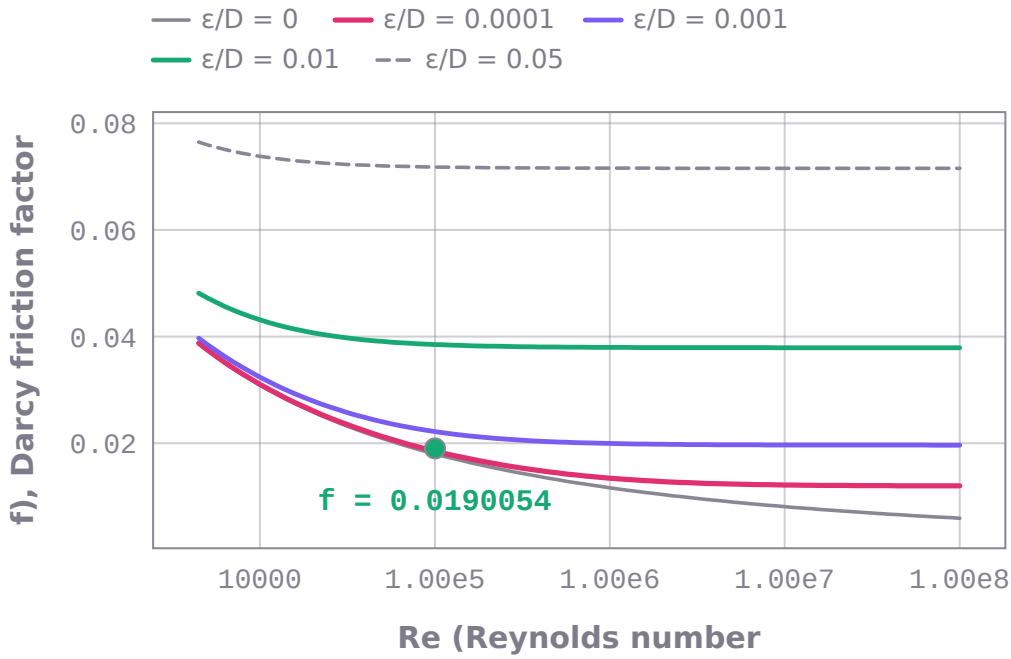
Solve the implicit Colebrook-White equation for the Darcy friction factor of turbulent pipe flow.

$$\frac{1}{\sqrt{f}} = -2\log_{10}\left(\frac{\varepsilon/D}{3.7} + \frac{2.51}{Re\sqrt{f}}\right)$$

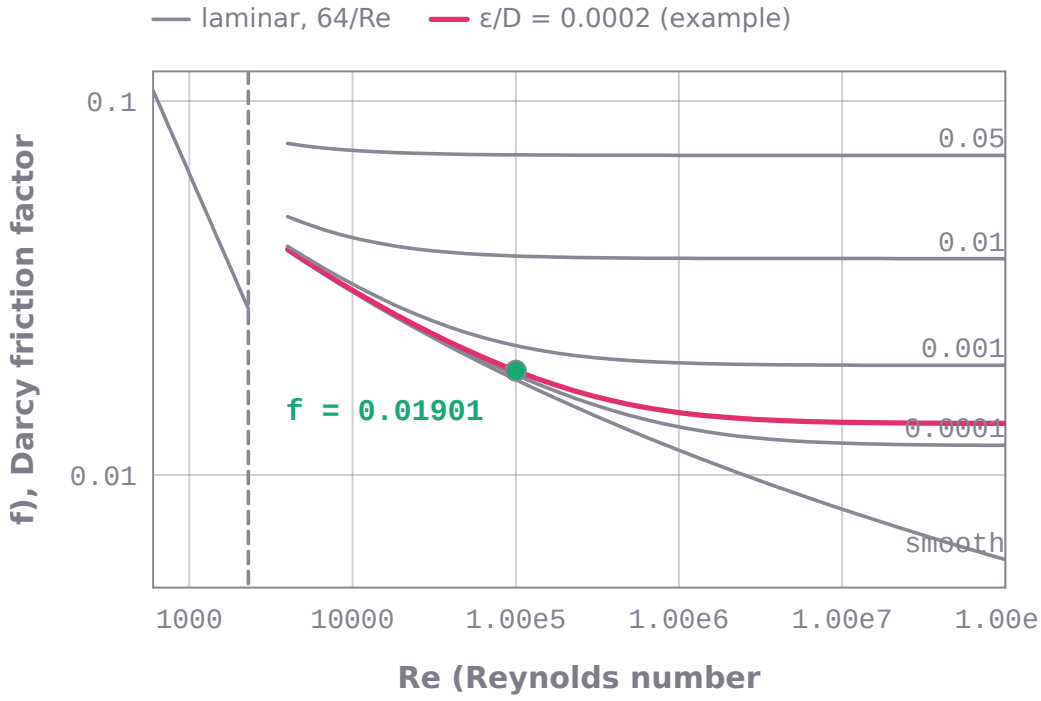
input	meaning	condition
`Re`	Reynolds number (turbulent, at least 4000)	
`eD`	relative roughness (roughness / diameter)	

output	meaning
`f`	Darcy friction factor

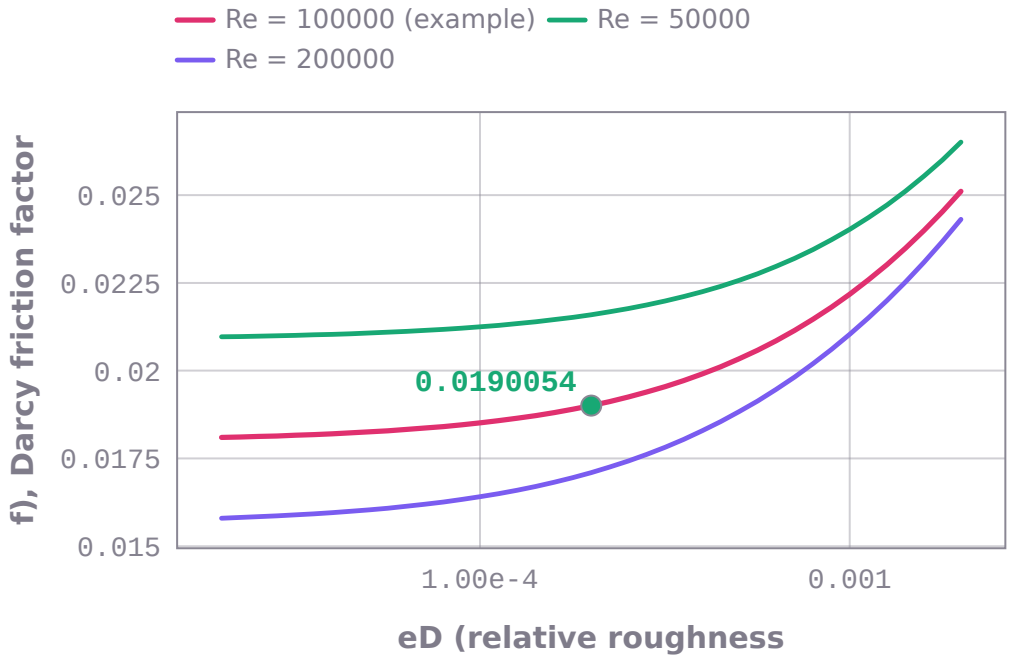
- **Method:** Newton iteration on $1/sqrt$(f) from a Swamee-Jain start, to machine precision
- **Assumes:** $Re \geq 4000$; $0 \leq eD \leq 0.05$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Darcy friction factor of turbulent pipe flow against Reynolds number (logarithmic axis) for five relative roughnesses from smooth to ε/D = 0.05, each from the Colebrook equation solved by the package at up to 89 Reynolds numbers between 4000 and 10^8. Marked: Re = 100000, ε/D = 0.0002, f = 0.0190054.



Friction-factor (Moody) chart: the package's Colebrook friction factor at 41 Reynolds numbers from 4000 to 10^8 for relative roughnesses 0, 0.0001, 0.0002, 0.001, 0.01, 0.05 (labelled at the right), with its laminar 64/Re line below Re = 2300; log–log axes. Marked: the worked example, Re = 100000, ε/D = 0.0002, f = 0.0190054.



f (Darcy friction factor) against eD (relative roughness (roughness / diameter)) from 2.0000e-05 to 0.002 on a logarithmic axis. One curve for each value of Re (Reynolds number (turbulent, at least 4000)): 100000, 50000, 200000. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, eD = 0.0002, f = 0.0190054.

```
thermofluids_ai999x.solve('colebrook_friction_factor', Re=100000, eD=0.0002)
# -> f = 0.0190054 1   (SOLVED)
```

```
thermofluids_ai999x.solve('colebrook_friction_factor', Re=1000000.0, eD=0.0)
# -> f = 0.011645 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Colebrook friction factor: Re = 100000, eD = 0.0002	SOLVED	f = 0.0190054 1
Colebrook friction factor: Re = 1500, eD = 0.001	INVALID_INPUT	Darcy friction factor from the Colebrook equation: the Colebrook equation is for turbulent flow: Re must be at least 400
Colebrook friction factor: Re = 100000	NEEDS_INPUT	colebrook_friction_factor needs eD (relative roughness (roughness / diameter)). Example: solve('colebrook_friction_facto

Log-mean temperature difference of a heat exchanger, `lmtd`

The LMTD of a counterflow or parallel-flow exchanger from the four terminal temperatures.

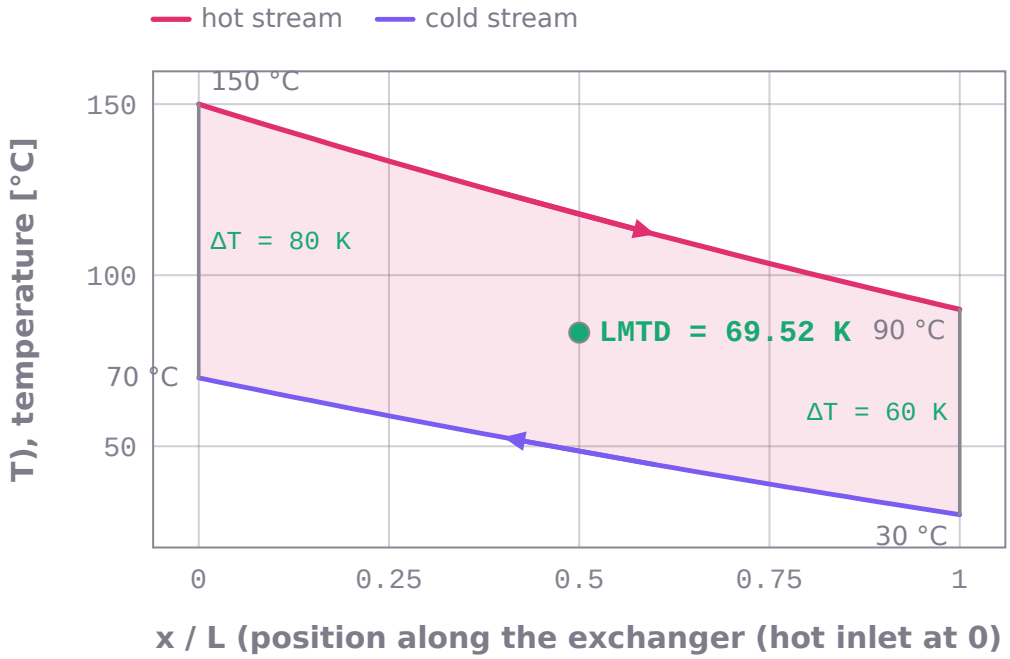
$$\Delta T_{lm} = \frac{\Delta T_1 - \Delta T_2}{\ln(\Delta T_1 / \Delta T_2)}$$

input	meaning	condition
`Th_in`	hot stream inlet temperature	
`Th_out`	hot stream outlet temperature	
`Tc_in`	cold stream inlet temperature	

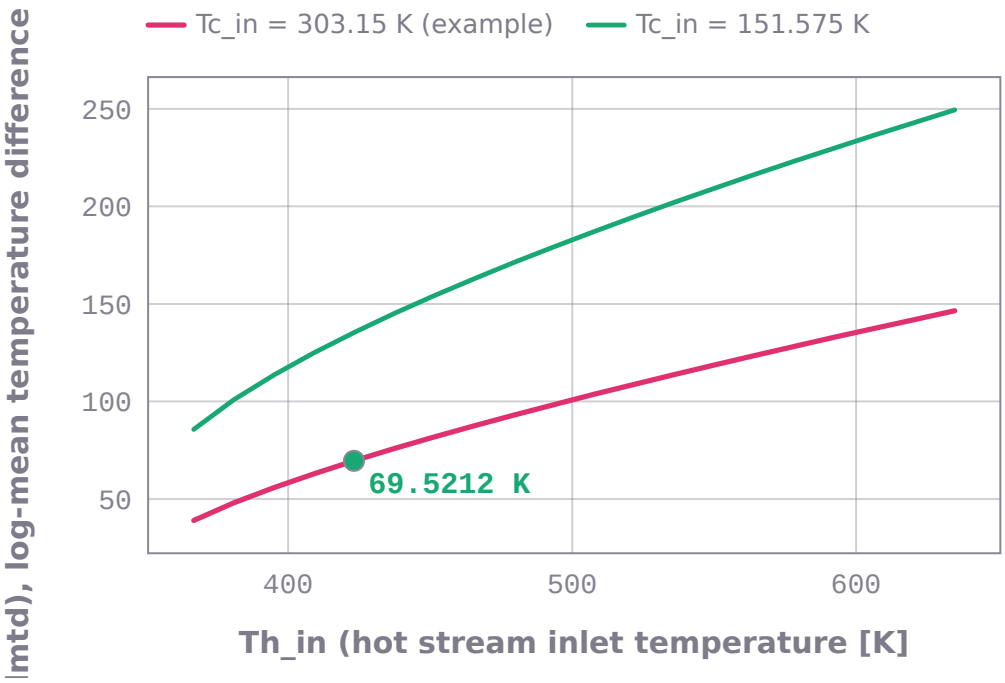
input	meaning	condition
`Tc_out`	cold stream outlet temperature	
`arrangement`	flow arrangement	

output	meaning
`lmtd`	log-mean temperature difference
`dT1`	temperature difference at the hot-inlet end
`dT2`	temperature difference at the other end

- **Method:** $(dT1 - dT2)/ln$(dT1 / dT2) from the terminal differences of the chosen arrangement
- **Assumes:** both terminal temperature differences positive; the hot stream cools and the cold stream warms
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Stream temperatures along a counterflow heat exchanger with the worked example's ends: hot 150 → 90 °C, cold 30 → 70 °C. The package returns the end differences ΔT1 = 80 K, ΔT2 = 60 K and the log-mean difference; the curves between the ends are derived from them (the local difference changes exponentially along a constant-U exchanger). Marked: LMTD = 69.5212 K.



lmtd (log-mean temperature difference) against Th_in (hot stream inlet temperature) from 211.575 K to 634.725 K. One curve for each value of Tc_in (cold stream inlet temperature): 303.15 K, 151.575 K. Other inputs as in the worked example: Th_out = 363.15 K, Tc_out = 343.15 K, arrangement = counterflow. 20 points per curve computed by thermofluids-ai999x; marked: the worked example, Th_in = 423.15 K, lmtd = 69.5212 K.

```
thermofluids_ai999x.solve('lmtd', Th_in=423.15, Th_out=363.15, Tc_in=303.15, Tc_out=343.15,
arrangement='counterflow')
# -> lmtd = 69.5212 K, dT1 = 80 K, dT2 = 60 K   (SOLVED)
```

```
thermofluids_ai999x.solve('lmtd', Th_in=423.15, Th_out=363.15, Tc_in=303.15, Tc_out=343.15,
arrangement='parallel')
# -> lmtd = 55.8111 K, dT1 = 120 K, dT2 = 20 K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Counterflow LMTD: Th_in = 150 °C, Th_out = 90 °C, Tc_in = 30 °C, Tc_out = 70 °C	SOLVED	lmtd = 69.5212 K, dT1 = 80 K, dT2 = 60 K
Parallel flow LMTD: Th_in = 150 °C, Th_out = 90 °C, Tc_in = 30 °C, Tc_out = 70 °C	SOLVED	lmtd = 55.8111 K, dT1 = 120 K, dT2 = 20 K
Parallel flow LMTD: Th_in = 150 °C, Th_out = 60 °C, Tc_in = 30 °C, Tc_out = 70 °C	INVALID_INPUT	Log-mean temperature difference of a heat exchanger: each terminal temperature difference must be positive (the streams'
Counterflow LMTD: Th_in = 150 °C, Th_out = 90 °C, Tc_in = 30 °C	NEEDS_INPUT	lmtd needs Tc_out (cold stream outlet temperature). Example: solve('lmtd', Th_in='423.15 K', Th_out='363.15 K', Tc_in='3

Heat exchanger rating by the effectiveness-NTU method, `hx_effectiveness_ntu`

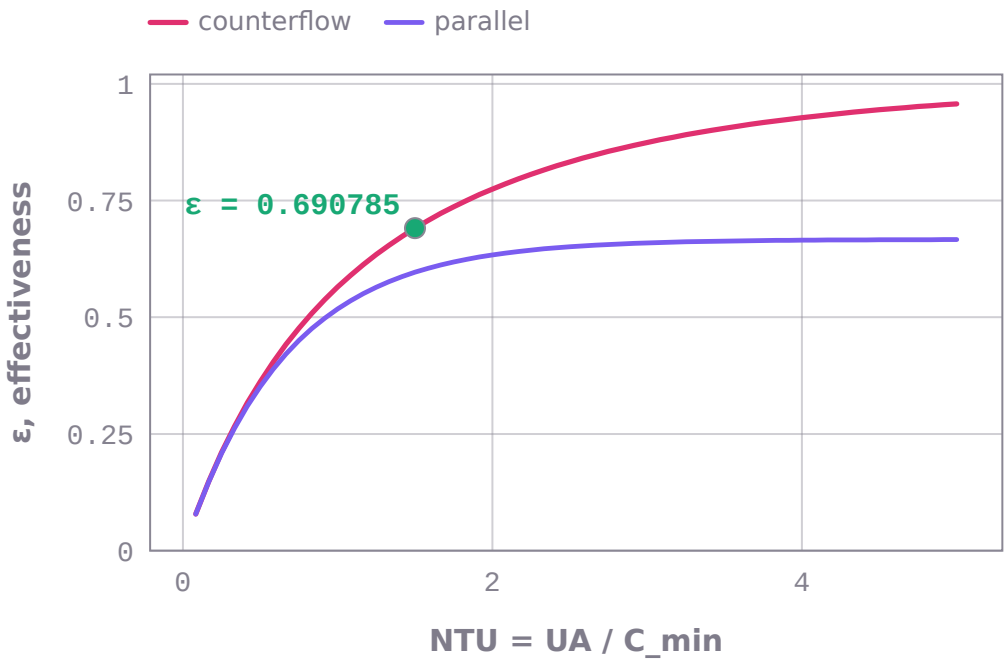
Heat duty, outlet temperatures, effectiveness and NTU of a counterflow or parallel-flow exchanger.

$$Q = \varepsilon(NTU, C_r)\, C_{min}(T_{h,i} - T_{c,i})$$

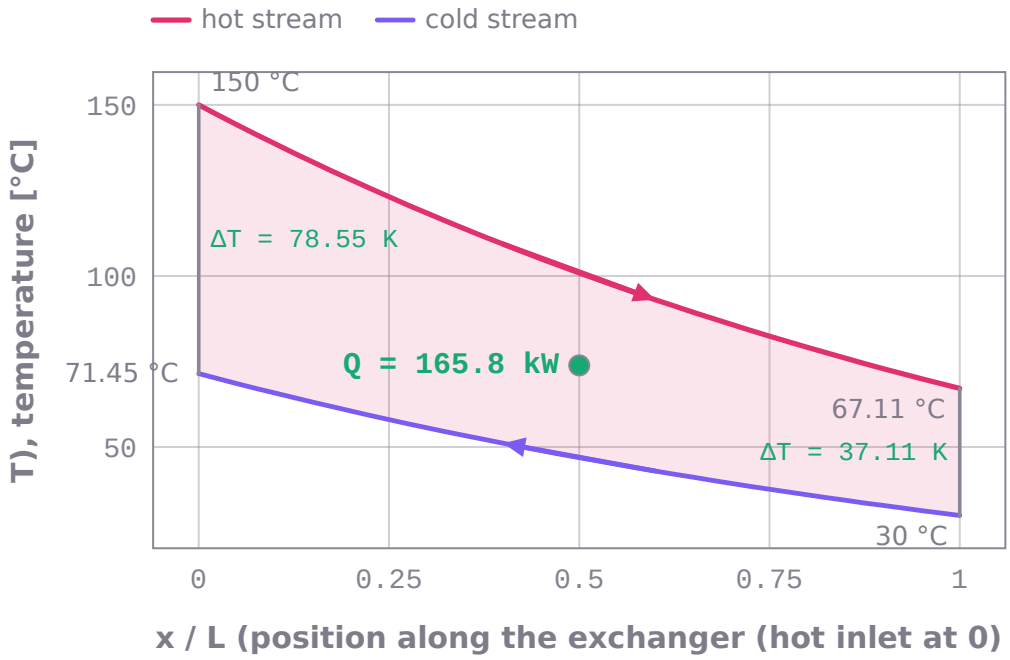
input	meaning	condition
`C_hot`	heat capacity rate of the hot stream (m cp)	
`C_cold`	heat capacity rate of the cold stream (m cp)	
`UA`	overall conductance U A	
`Th_in`	hot inlet temperature	
`Tc_in`	cold inlet temperature	
`arrangement`	flow arrangement	

output	meaning
`Q`	heat transfer rate
`Th_out`	hot outlet temperature
`Tc_out`	cold outlet temperature
`effectiveness`	effectiveness
`NTU`	number of transfer units

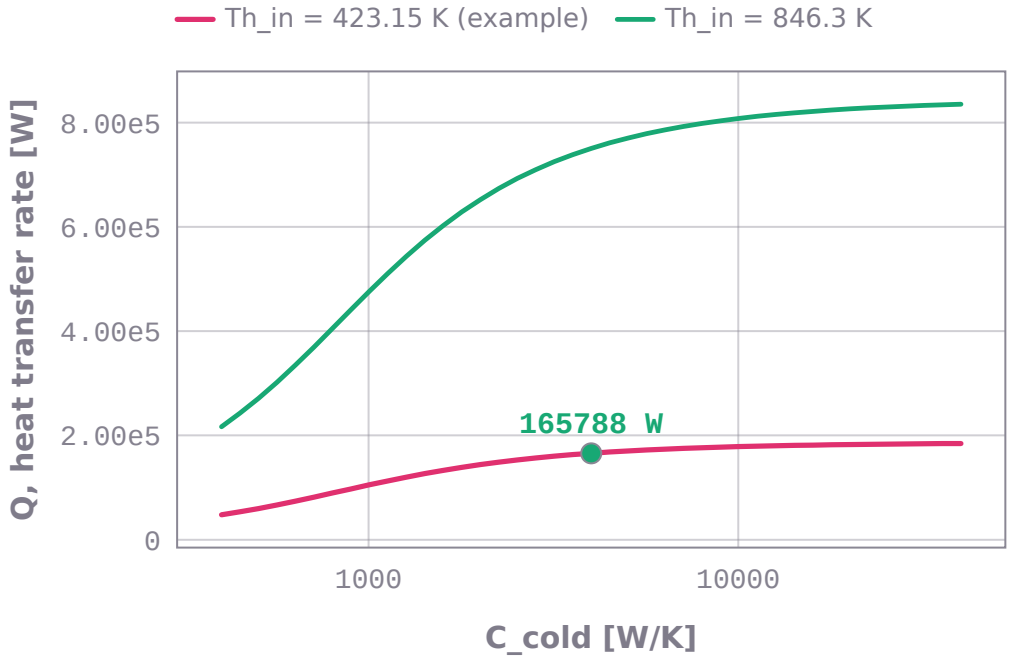
- **Method:** NTU = UA/Cmin, Cr = Cmin/Cmax, effectiveness of the arrangement, Q = eps Cmin (Th,in - Tc,in), energy balances
- **Assumes:** positive capacity rates and UA; hot inlet hotter than cold inlet
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Heat-exchanger effectiveness against the number of transfer units for counterflow and parallel flow with heat-capacity rates 2000 and 4000 W/K (ratio 0.5), computed by the package at 60 values of UA. Marked: counterflow, UA = 3000 W/K, NTU = 1.5, ε = 0.690785, Q = 165788 W.



Stream temperatures along a counterflow exchanger: hot stream C = 2000 W/K entering at 150 °C, cold stream C = 4000 W/K entering at 30 °C, UA = 3000 W/K (the worked example). The outlets are the package's: hot 67.1058 °C, cold 71.4471 °C; the curves between the ends are derived from the four end temperatures (exponential change of the local difference). Marked: Q = 165788 W, ε = 0.690785, NTU = 1.5.



Q (heat transfer rate) against C_cold (heat capacity rate of the cold stream (m cp)) from 400 W/K to 40000 W/K on a logarithmic axis. One curve for each value of Th_in (hot inlet temperature): 423.15 K, 846.3 K. Other inputs as in the worked example: C_hot = 2000 W/K, UA = 3000 W/K, Tc_in = 303.15 K, arrangement = counterflow. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, C_cold = 4000 W/K, Q = 165788 W.

```
thermofluids_ai999x.solve('hx_effectiveness_ntu', C_hot=2000, C_cold=4000, UA=3000,
Th_in=423.15, Tc_in=303.15, arrangement='counterflow')
# -> Q = 165788 W, Th_out = 340.256 K, Tc_out = 344.597 K, effectiveness = 0.690785 1, NTU
= 1.5 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Effectiveness-NTU rating, counterflow: C_hot = 2000 W/K, C_cold = 4000 W/K, UA = 3000 W/K, Th_in = 150 °C, Tc_in = 30 °C	SOLVED	Q = 165788 W, Th_out = 340.256 K, Tc_out = 344.597 K, effectiveness = 0.690785 1, NTU = 1.5 1
Effectiveness-NTU rating, parallel flow: C_hot = 2000 W/K, C_cold = 4000 W/K, UA = 3000 W/K, Th_in = 20 °C, Tc_in = 30 °C	INVALID_INPUT	Heat exchanger rating by the effectiveness-NTU method: the hot inlet must be hotter than the cold inlet.

Saturation pressure of water vapour, `saturation_vapour_pressure`

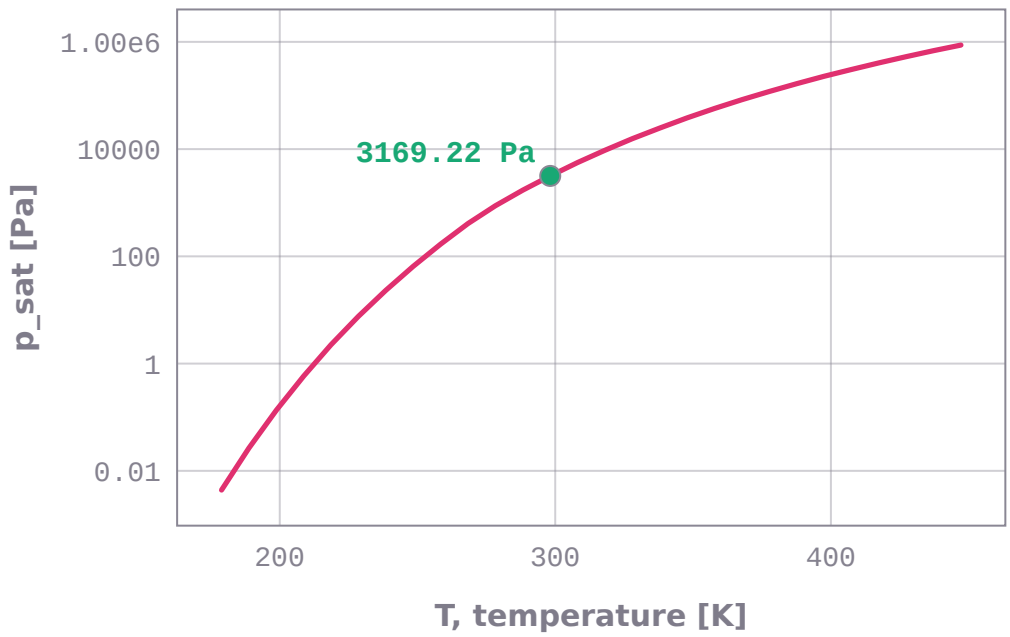
The saturation pressure of water vapour over liquid water (above 0.01 °C) or ice, -100 °C to 200 °C.

$$\ln p_{ws} = C_8/T + C_9 + C_{10}T + C_{11}T^2 + C_{12}T^3 + C_{13}\ln T$$

input	meaning	condition
`T`	temperature	

output	meaning
`p_sat`	saturation pressure of water vapour

- **Method:** ASHRAE (Hyland-Wexler) correlation for ln p_ws
- **Assumes:** -100 °$C \le T \le 200$ °C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



p_sat (saturation pressure of water vapour) against T (temperature) from 149.075 K to 447.225 K, logarithmic output axis. 28 points computed by thermofluids-ai999x; marked: the worked example, T = 298.15 K, p_sat = 3169.22 Pa.

```
thermofluids_ai999x.solve('saturation_vapour_pressure', T=298.15)
# -> p_sat = 3169.22 Pa   (SOLVED)
```

```
thermofluids_ai999x.solve('saturation_vapour_pressure', T=373.15)
# -> p_sat = 101419 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Saturation vapour pressure of water: T = 25 °C	SOLVED	p_sat = 3169.22 Pa
Saturation vapour pressure of water: T = 300 °C	INVALID_INPUT	Saturation pressure of water vapour: temperature must lie between -100 °C and 200 °C.
Saturation vapour pressure of water at 25 °C and 50 kPa	UNRECOGNIZED_FORMAT	I could not find what the question asks for. A supported way to ask: "Moist air at a barometric pressure of 101325 Pa ha

Dew-point temperature from dry-bulb temperature and relative humidity, `dew_point_temperature`

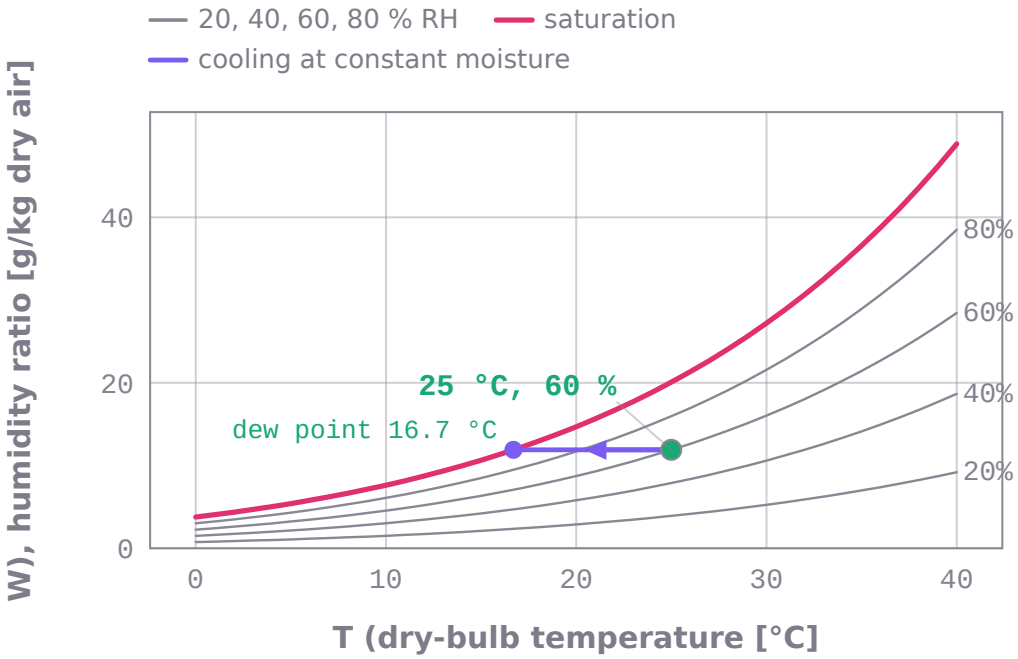
The temperature at which moist air of a given relative humidity becomes saturated on cooling.

$$p_{ws}(T_{dp}) = \phi\, p_{ws}(T)$$

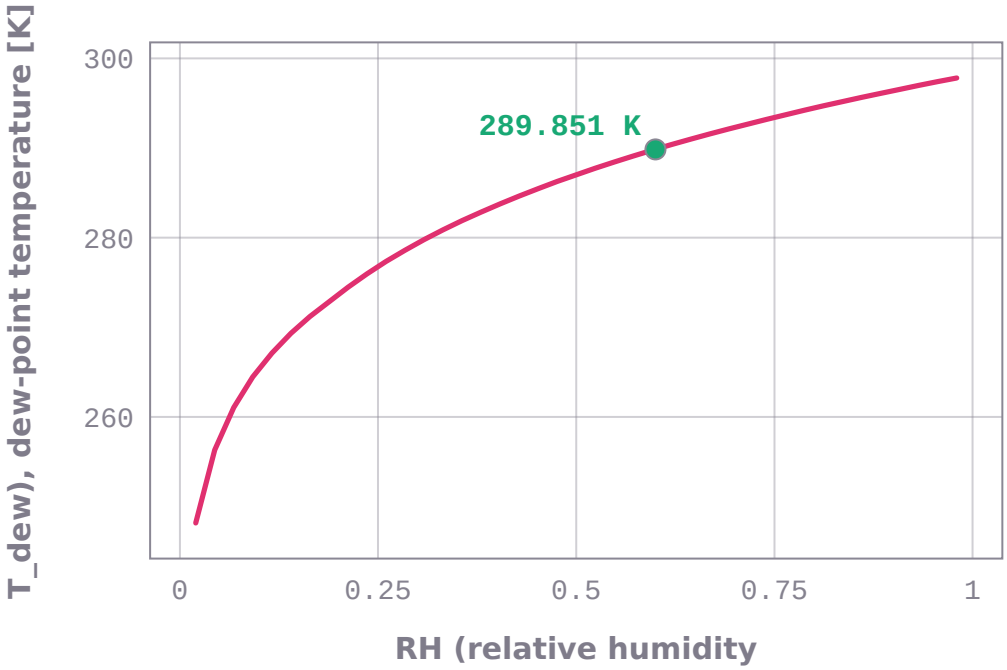
input	meaning	condition
`T`	dry-bulb temperature	
`RH`	relative humidity (fraction, or % written)	

output	meaning
`T_dew`	dew-point temperature
`p_v`	partial pressure of the water vapour

- **Method:** $p_v = RH$ p_ws(T), then p_ws(T_dew) = p_v solved by bisection to the last digit
- **Assumes:** $0 < RH \leq 1$; -100 °$C \leq T \leq 200$ °C
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Psychrometric chart at a stated 101325 Pa (the package's humidity ratio at 20–80 % and 100 % relative humidity) with the worked example: air at 25 °C and 60 % (11.895 g/kg) cooled at constant moisture until it saturates. Marked: the package's dew point 16.7011 °C and vapour pressure 1901.53 Pa.



T_dew (dew-point temperature) against RH (relative humidity (fraction, or % written)) from 0.02 to 0.98. Other inputs as in the worked example: T = 298.15 K. 41 points computed by thermofluids-ai999x; marked: the worked example, RH = 0.6, T_dew = 289.851 K.

```
thermofluids_ai999x.solve('dew_point_temperature', T=298.15, RH=0.6)
# -> T_dew = 289.851 K, p_v = 1901.53 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Dew point temperature: T = 25 °C, RH = 60%	SOLVED	T_dew = 289.851 K, p_v = 1901.53 Pa

question	outcome	result
Dew point temperature: T = 25 °C, RH = 0.6	SOLVED	T_dew = 289.851 K, p_v = 1901.53 Pa
Dew point temperature: T = 25 °C, RH = 60	INVALID_INPUT	Dew-point temperature from dry-bulb temperature and relative humidity: relative humidity must be a fraction between 0 an
Dew point temperature: T = 25 °C	NEEDS_INPUT	dew_point_temperature needs RH (relative humidity (fraction, or % written)). Example: solve('dew_point_temperature', T='

Humidity ratio from temperature, relative humidity and pressure,

`humidity_ratio_from_relative_humidity`

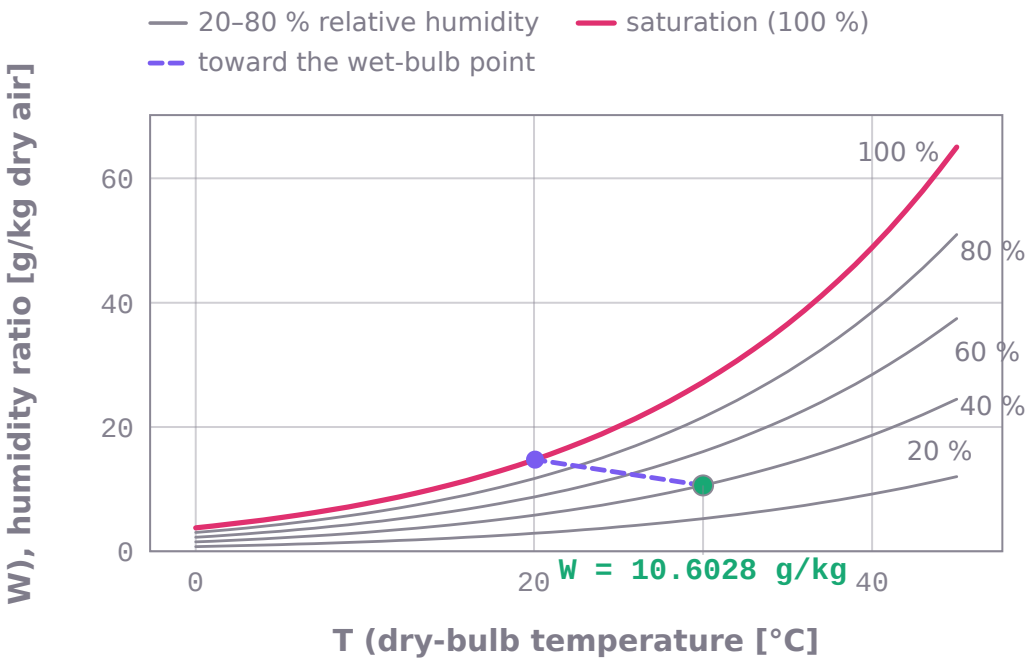
Mass of water vapour per kilogram of dry air in moist air of a given relative humidity.

$$W = 0.621945 \frac{\phi p_{ws}}{P - \phi p_{ws}}$$

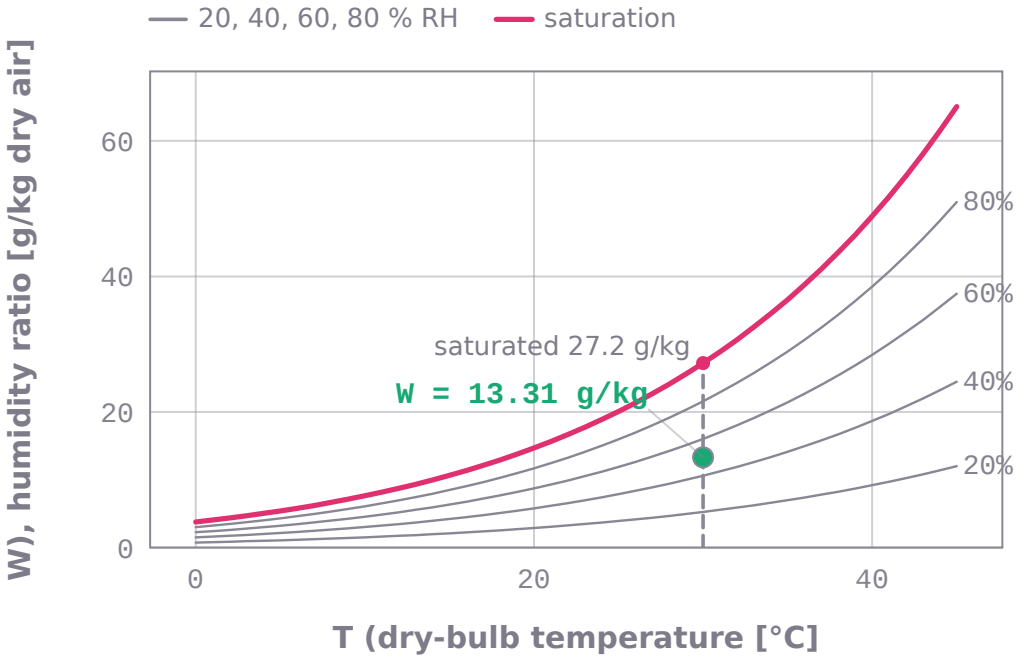
input	meaning	condition
`T`	dry-bulb temperature	
`RH`	relative humidity (fraction, or % written)	
`P`	total (barometric) pressure	

output	meaning
`W`	humidity ratio (kg/kg dry air)
`p_v`	partial pressure of the water vapour

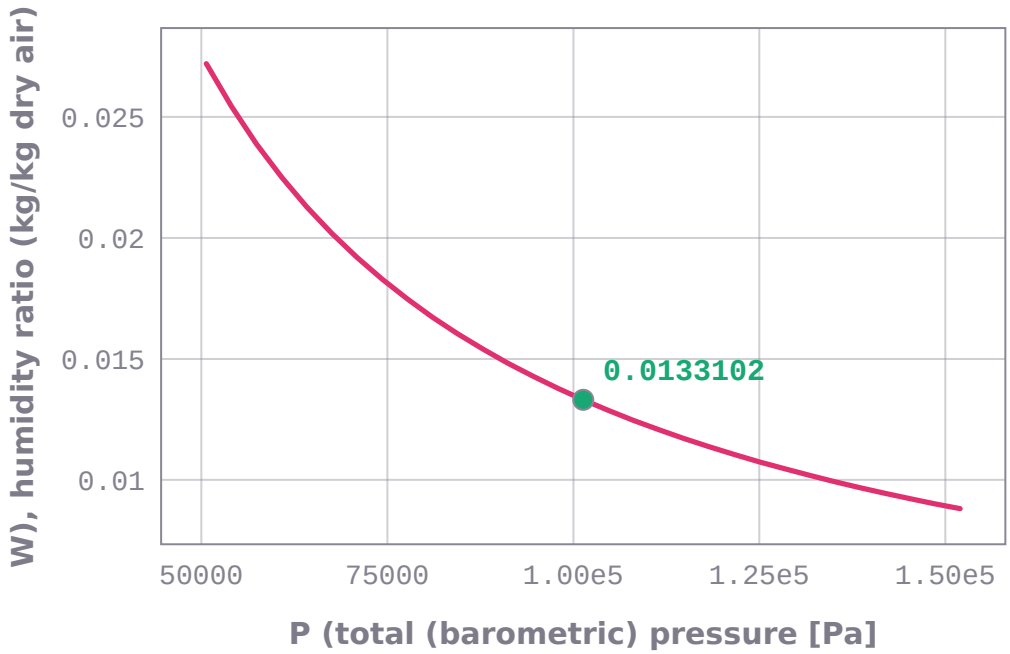
- **Method:** $p_v = RH$ p_ws(T); $W = 0.621945$ p_v / (P - p_v)
- **Assumes:** $0 < RH \leq 1$; P greater than the vapour pressure
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Psychrometric chart at 101325 Pa: humidity ratio against dry-bulb temperature at 20, 40, 60, 80 and 100 % relative humidity, computed by the package every 1 °C. Air at 30 °C and 40 % has a wet-bulb temperature of 20.0643 °C; marked: T = 30 °C, W = 10.6028 g/kg.



Psychrometric chart at 101325 Pa (humidity ratio against dry-bulb temperature at 20–80 % relative humidity and saturation, the package's humidity-ratio capability every 1 °C) with the worked example: air at 30 °C and 50 % relative humidity. Marked: W = 13.3102 g/kg, vapour pressure 2123.02 Pa; saturated air at the same temperature holds 27.2026 g/kg.



W (humidity ratio (kg/kg dry air)) against P (total (barometric) pressure) from 50662.5 Pa to 151988 Pa. Other inputs as in the worked example: T = 303.15 K, RH = 0.5. 31 points computed by thermofluids-ai999x; marked: the worked example, P = 101325 Pa, W = 0.0133102.

```
thermofluids_ai999x.solve('humidity_ratio_from_relative_humidity', T=303.15, RH=0.5,
P=101325)
# -> W = 0.0133102 1, p_v = 2123.02 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Humidity ratio: T = 30 °C, RH = 50%, P = 101325 Pa	SOLVED	W = 0.0133102 1, p_v = 2123.02 Pa
Humidity ratio: T = 30 °C, RH = 50%	NEEDS_INPUT	humidity_ratio_from_relative_humidity needs P (total (barometric) pressure). Example: solve('humidity_ratio_from_relativ

Thermodynamic wet-bulb temperature, `wet_bulb_temperature`

The wet-bulb temperature of moist air from its dry-bulb temperature, relative humidity and pressure.

$$W = \frac{(2501 - 2.326\,t^*)W_s^* - 1.006(t - t^*)}{2501 + 1.86\,t - 4.186\,t^*}$$

input	meaning	condition
`T`	dry-bulb temperature	
`RH`	relative humidity (fraction, or % written)	
`P`	total (barometric) pressure	

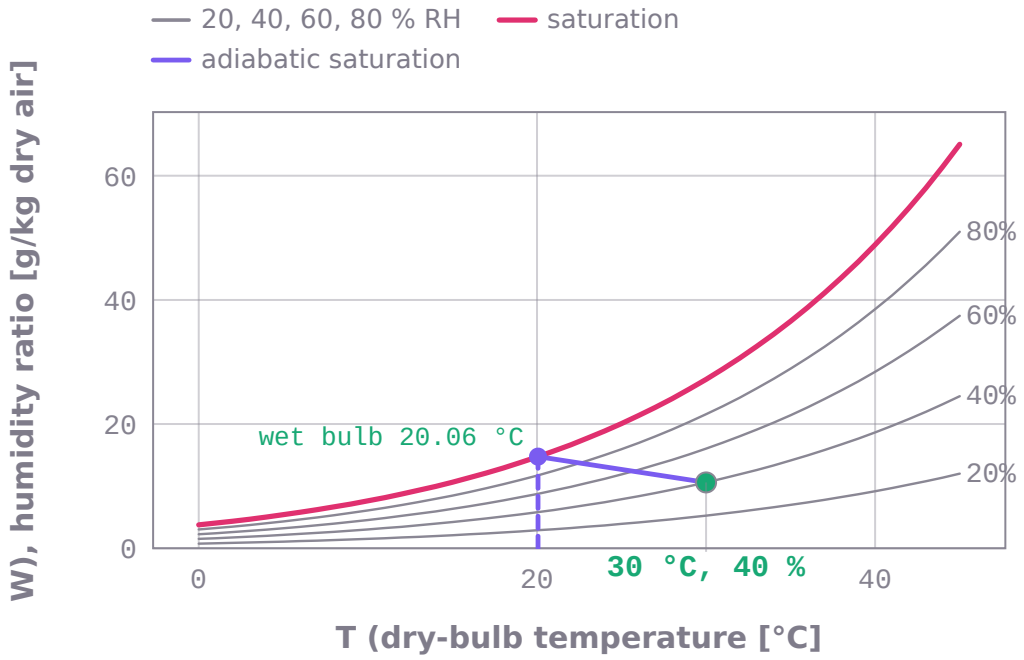
output	meaning
`T_wb`	thermodynamic wet-bulb temperature
`W`	humidity ratio

- **Method:** the ASHRAE psychrometric equation W(T, T_wb, P) = W solved for T_wb by bisection
- **Assumes:** $0 < RH \leq 1$; 0 °$C \leq T \leq 50$ °C for the liquid-water branch
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)

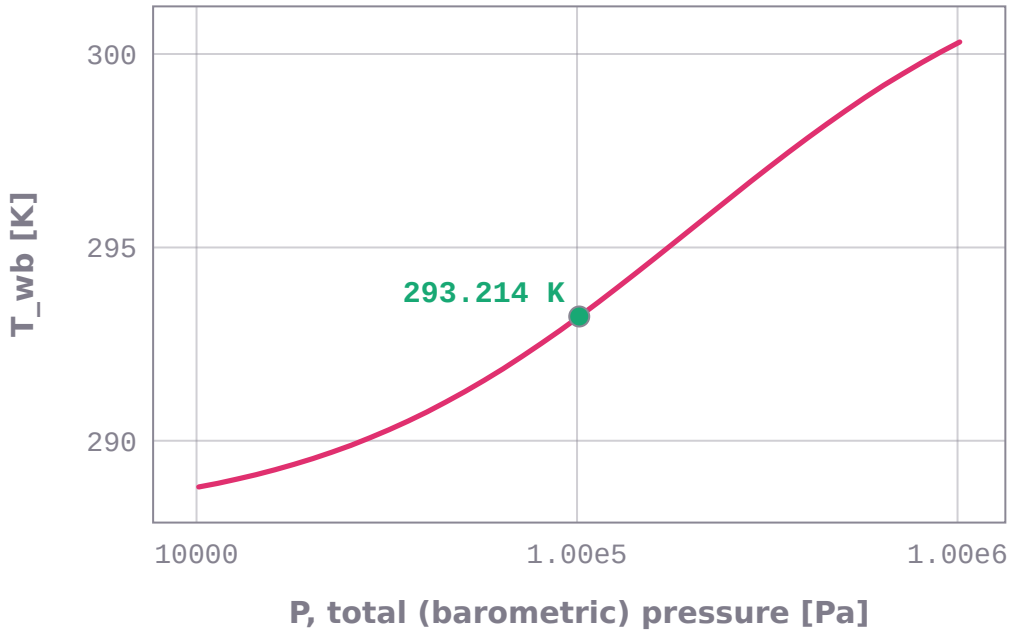
	1	2	3	4	5
1	2.5	4.6	6.5	8.3	10
2	6	8.5	10.8	13	15
3	9.3	12.4	15.1	17.7	20
4	12.5	16.2	19.5	22.4	25
5	15.7	20.1	23.8	27.1	30
6	18.9	23.9	28.2	31.8	35
7	22	27.8	32.6	36.6	40

wet bulb °C · rows dry bulb 10 → 40 °C · columns RH 20 → 100

Wet-bulb temperature at 101325 Pa for dry-bulb temperatures 10, 15, 20, 25, 30, 35, 40 °C (rows) and relative humidities 20, 40, 60, 80 and 100 % (columns), each cell computed by the package (°C, one decimal). At 100 % the wet bulb equals the dry bulb; at 40 °C and 20 % it is 22 °C.



Psychrometric chart at 101325 Pa (the package's humidity ratio at 20–80 % and 100 % relative humidity) with the worked example: air at 30 °C and 40 % (10.6028 g/kg, returned) followed toward saturation at its wet-bulb temperature. Marked: the package's T_wb = 20.0643 °C and the saturated humidity ratio there, 14.7551 g/kg.



T_wb (thermodynamic wet-bulb temperature) against P (total (barometric) pressure) from 10132.5 Pa to 1.01325e+06 Pa on a logarithmic axis. Other inputs as in the worked example: T = 303.15 K, RH = 0.4. 41 points computed by thermofluids-ai999x; marked: the worked example, P = 101325 Pa, T_wb = 293.214 K.

```
thermofluids_ai999x.solve('wet_bulb_temperature', T=303.15, RH=0.4, P=101325)
# -> T_wb = 293.214 K, W = 0.0106028 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wet bulb temperature: T = 30 °C, RH = 40%, P = 101325 Pa	SOLVED	T_wb = 293.214 K, W = 0.0106028 1

question	outcome	result
Wet bulb temperature: T = 30 °C, RH = 140%, P = 101325 Pa	INVALID_INPUT	Thermodynamic wet-bulb temperature: relative humidity must be a fraction between 0 and 1 (or a percentage written with %

Stoichiometric air-fuel ratio of a gaseous fuel, `stoichiometric_air_fuel_ratio`

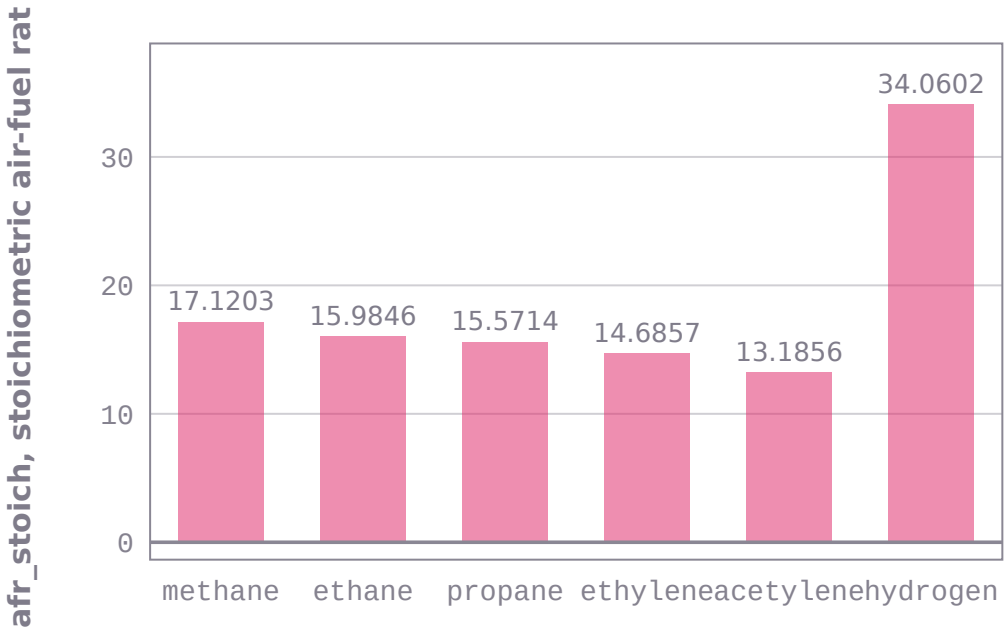
Mass of air per mass of fuel for complete combustion, with the moles of O2 and air per mole of fuel.

$$AFR_s = \frac{(x + y/4)(M_{O_2} + 3.76 M_{N_2})}{x M_C + y M_H}$$

input	meaning	condition
`fuel`	the fuel	

output	meaning
`afr_stoich`	stoichiometric air-fuel ratio (mass)
`o2_moles`	moles of O2 per mole of fuel
`air_moles`	moles of air per mole of fuel

- **Method:** atom balance of CxHy + (x + y/4)(O2 + 3.76 N2), then the mass ratio with standard atomic weights
- **Assumes:** complete combustion to CO2 and H2O; air = O2 + 3.76 N2
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



afr_stoich (stoichiometric air-fuel ratio (mass)) for each option of fuel (the fuel), one run of thermofluids-ai999x per option: methane: 17.1203, ethane: 15.9846, propane: 15.5714, ethylene: 14.6857, acetylene: 13.1856, hydrogen: 34.0602. The worked example uses methane.

```
thermofluids_ai999x.solve('stoichiometric_air_fuel_ratio', fuel='methane')
# -> afr_stoich = 17.1203 1, o2_moles = 2, air_moles = 9.52   (SOLVED)
```

```
thermofluids_ai999x.solve('stoichiometric_air_fuel_ratio', fuel='propane')
# -> afr_stoich = 15.5714 1, o2_moles = 5, air_moles = 23.8   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
What is the stoichiometric air-fuel ratio of methane?	SOLVED	afr_stoich = 17.1203 1, o2_moles = 2, air_moles = 9.52
What is the stoichiometric air-fuel ratio of propane C3H8?	SOLVED	afr_stoich = 15.5714 1, o2_moles = 5, air_moles = 23.8
What is the stoichiometric air-fuel ratio of methane at 300 K?	OUT_OF_SCOPE	No certified capability computes the asked quantity in this situation, so there is no missing input to ask for. See capa

Equivalence ratio of a fuel-air mixture, `equivalence_ratio`

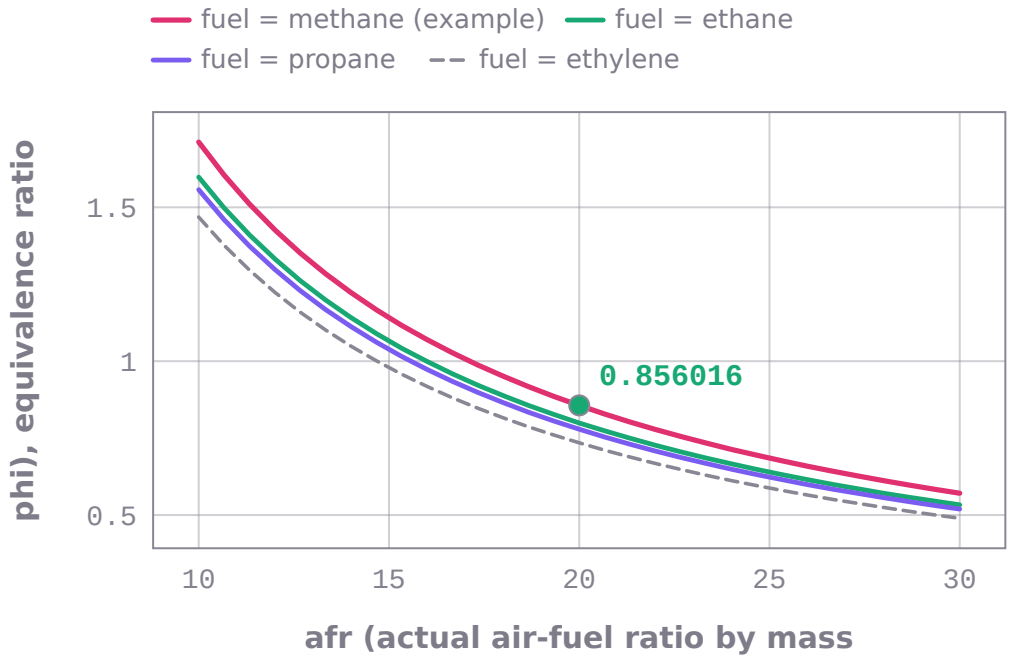
Equivalence ratio and excess air of a gaseous fuel burned at a given air-fuel ratio by mass.

$$\phi = \frac{AFR_s}{AFR}$$

input	meaning	condition
`fuel`	the fuel	
`afr`	actual air-fuel ratio by mass	

output	meaning
`phi`	equivalence ratio (fuel-air ratio over stoichiometric)
`afr_stoich`	stoichiometric air-fuel ratio
`excess_air`	excess air as a fraction

- **Method:** phi = AFR_s / AFR with the stoichiometric ratio from the atom balance
- **Assumes:** afr > 0
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



phi (equivalence ratio (fuel-air ratio over stoichiometric)) against afr (actual air-fuel ratio by mass) from 10 to 30. One curve for each value of fuel (the fuel): methane, ethane, propane, ethylene. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, afr = 20, phi = 0.856016.

```
thermofluids_ai999x.solve('equivalence_ratio', fuel='methane', afr=20)
# -> phi = 0.856016, afr_stoich = 17.1203 1, excess_air = 0.168203   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Equivalence ratio of methane: afr = 20	SOLVED	phi = 0.856016, afr_stoich = 17.1203 1, excess_air = 0.168203
Equivalence ratio of propane: afr = 0	INVALID_INPUT	Equivalence ratio of a fuel-air mixture: the air-fuel ratio must be positive.
Equivalence ratio of methane	NEEDS_INPUT	equivalence_ratio needs afr (actual air-fuel ratio by mass). Example: solve('equivalence_ratio', fuel='methane', afr=20)

Gnielinski correlation for turbulent pipe flow, `gnielinski_nusselt`

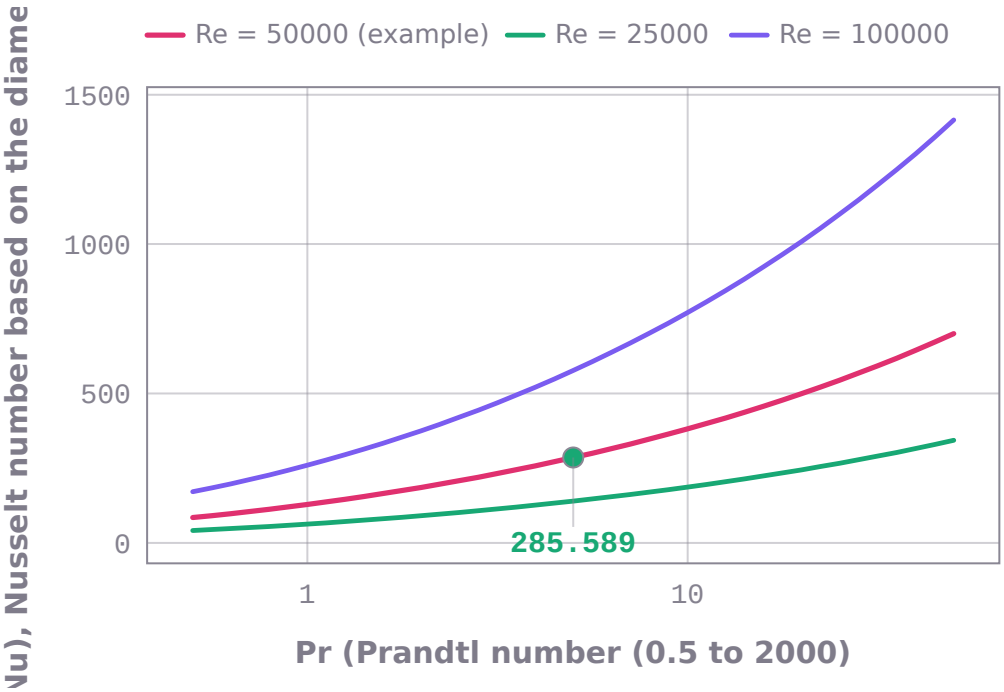
Nusselt number of fully developed turbulent or transitional flow in a tube.

$$Nu = \frac{(f/8)(Re - 1000)Pr}{1 + 12.7\sqrt{f/8}\,(Pr^{2/3} - 1)}$$

input	meaning	condition
`Re`	Reynolds number (3000 to 5e6)	
`Pr`	Prandtl number (0.5 to 2000)	
`f`	Darcy friction factor	

output	meaning
`Nu`	Nusselt number based on the diameter

- **Method:** $Nu = (f/8)$(Re - 1000)Pr / (1 + 12.7 sqrt(f/8)$(Pr^{2/3} - 1)$)
- **Assumes:** $3000 \le Re \le 5e6$; $0.5 \le Pr \le 2000$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Nu (Nusselt number based on the diameter) against Pr (Prandtl number (0.5 to 2000)) from 0.5 to 50 on a logarithmic axis. One curve for each value of Re (Reynolds number (3000 to 5e6)): 50000, 25000, 100000. Other inputs as in the worked example: f = 0.021. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, Pr = 5, Nu = 285.589.

```
thermofluids_ai999x.solve('gnielinski_nusselt', Re=50000, Pr=5, f=0.021)
# -> Nu = 285.589 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Gnielinski correlation: Re = 50000, Pr = 5, f = 0.021	SOLVED	Nu = 285.589 1
Gnielinski correlation: Re = 1000, Pr = 5, f = 0.06	INVALID_INPUT	Gnielinski correlation for turbulent pipe flow: Gnielinski holds for 3000 <= Re <= 5e6, 0.5 <= Pr <= 2000 and a Darcy fr

Churchill-Bernstein correlation for a cylinder in cross flow, `churchill_bernstein_cylinder`

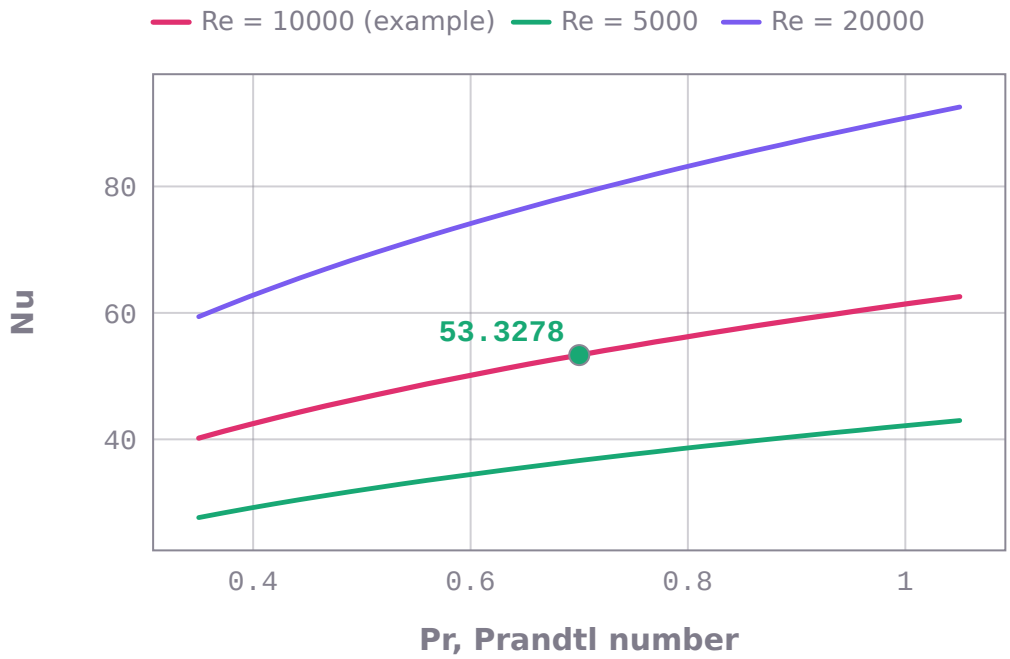
Average Nusselt number of a circular cylinder in cross flow over the whole Reynolds range.

$$Nu = 0.3 + \frac{0.62\,Re^{1/2}Pr^{1/3}}{[1+(0.4/Pr)^{2/3}]^{1/4}}\left[1+\left(\frac{Re}{282000}\right)^{5/8}\right]^{4/5}$$

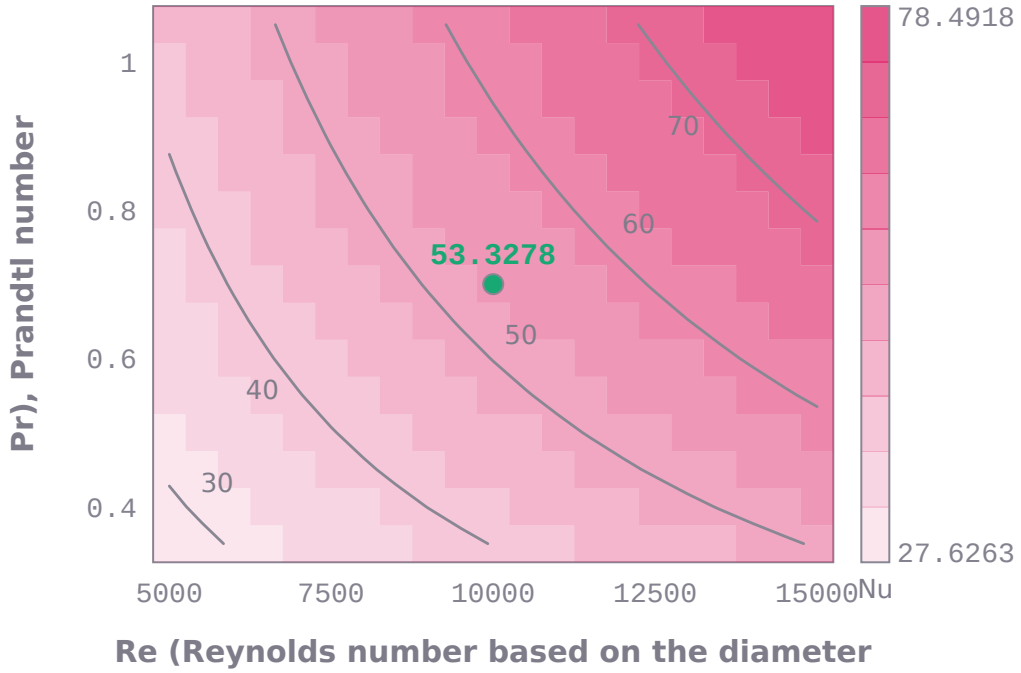
input	meaning	condition
`Re`	Reynolds number based on the diameter	
`Pr`	Prandtl number	

output	meaning
`Nu`	average Nusselt number based on the diameter

- **Method:** $Nu = 0.3 + 0.62\ Re^{1/2}$ Pr^(1/3) / (1 + (0.4/Pr)^(2/3))^$(1/4)$ (1 + (Re/282000)^(5/8))^$(4/5)$
- **Assumes:** $RePr \geq 0.2$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Nu (average Nusselt number based on the diameter) against Pr (Prandtl number) from 0.35 to 1.05. One curve for each value of Re (Reynolds number based on the diameter): 10000, 5000, 20000. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Pr = 0.7, Nu = 53.3278.



Nu (average Nusselt number based on the diameter) over Re (Reynolds number based on the diameter) from 5000 to 15000 and Pr (Prandtl number) from 0.35 to 1.05: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 27.6263 to dark 78.4918); iso-lines at Nu = 30, 40, 50, 60, 70. Marked: the worked example, Re = 10000, Pr = 0.7, Nu = 53.3278.

```
thermofluids_ai999x.solve('churchill_bernstein_cylinder', Re=10000, Pr=0.7)
# -> Nu = 53.3278 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Churchill-Bernstein correlation: Re = 10000, Pr = 0.7	SOLVED	Nu = 53.3278 1
Churchill-Bernstein correlation: Re = 0.1, Pr = 0.7	INVALID_INPUT	Churchill-Bernstein correlation for a cylinder in cross flow: Churchill-Bernstein holds for Re * Pr >= 0.2.
Churchill-Bernstein correlation: Re = 10000	NEEDS_INPUT	churchill_bernstein_cylinder needs Pr (Prandtl number). Example: solve('churchill_bernstein_cylinder', Re=10000, Pr=0.7)

Churchill-Chu correlation for natural convection on a vertical plate, `churchill_chu_vertical_plate`

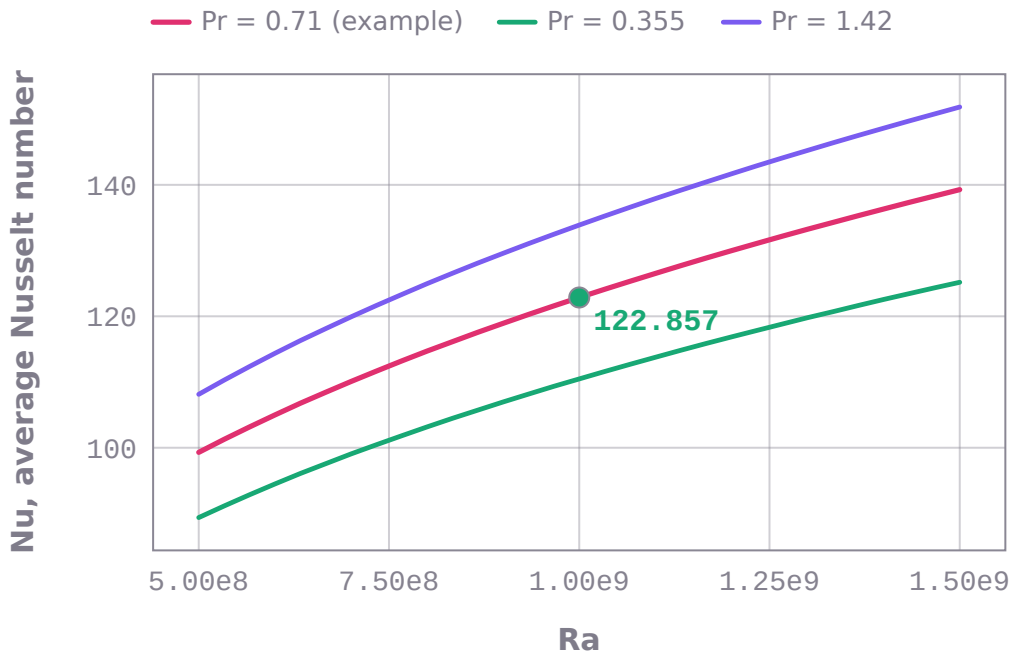
Average Nusselt number of free convection from an isothermal vertical plate, laminar and turbulent.

$$Nu = \left(0.825 + \frac{0.387\,Ra^{1/6}}{\left[1 + (0.492/Pr)^{9/16}\right]^{8/27}}\right)^2$$

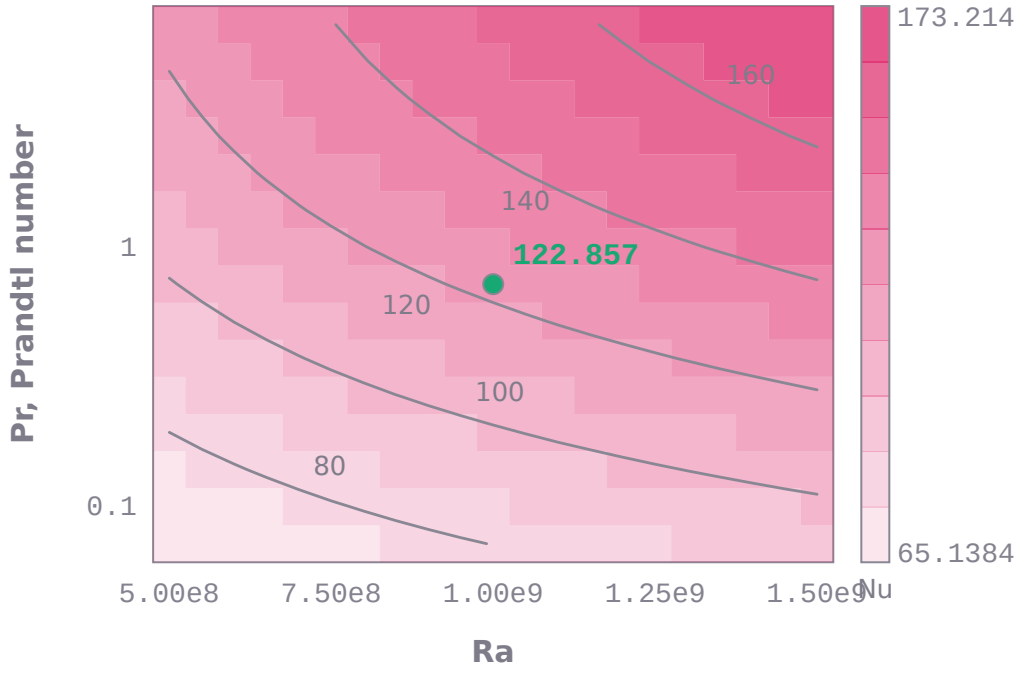
input	meaning	condition
`Ra`	Rayleigh number based on the plate height	
`Pr`	Prandtl number	

output	meaning
`Nu`	average Nusselt number

- **Method:** Nu = (0.825 + 0.387 Ra^(1/6) / (1 + (0.492/Pr)^(9/16))^$(8/27)$)^2
- **Assumes:** $0 < Ra \leq 1e13$
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



Nu (average Nusselt number) against Ra (Rayleigh number based on the plate height) from 5.0000e+08 to 1.5000e+09. One curve for each value of Pr (Prandtl number): 0.71, 0.355, 1.42. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Ra = 1.0000e+09, Nu = 122.857.



Nu (average Nusselt number) over Ra (Rayleigh number based on the plate height) from 5.0000e+08 to 1.5000e+09 and Pr (Prandtl number) from 0.071 to 7.1 (logarithmic vertical axis): each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 65.1384 to dark 173.214); iso-lines at Nu = 80, 100, 120, 140, 160. Marked: the worked example, Ra = 1.0000e+09, Pr = 0.71, Nu = 122.857.

```
thermofluids_ai999x.solve('churchill_chu_vertical_plate', Ra=1000000000.0, Pr=0.71)
# -> Nu = 122.857 1   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Churchill-Chu correlation: Ra = 1000000000, Pr = 0.71	SOLVED	Nu = 122.857 1
Churchill-Chu correlation: Ra = 1000000000	NEEDS_INPUT	churchill_chu_vertical_plate needs Pr (Prandtl number). Example: solve('churchill_chu_vertical_plate', Ra=1e+09, Pr=0.71

Efficiency of a pin fin with an insulated tip, `pin_fin_efficiency`

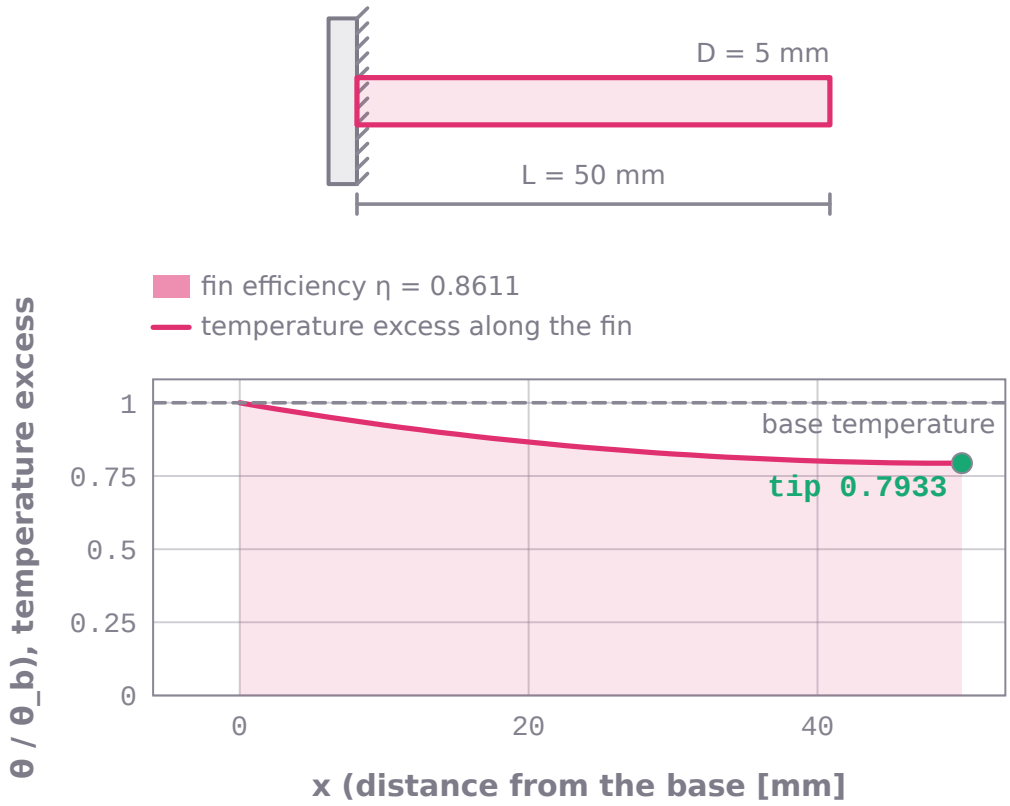
Fin efficiency tanh(mL)/(mL) of a cylindrical pin fin with $m = sqrt(4h/(kD))$.

$$\eta_f = \frac{\tanh(mL)}{mL}, \quad m = \sqrt{\frac{4h}{kD}}$$

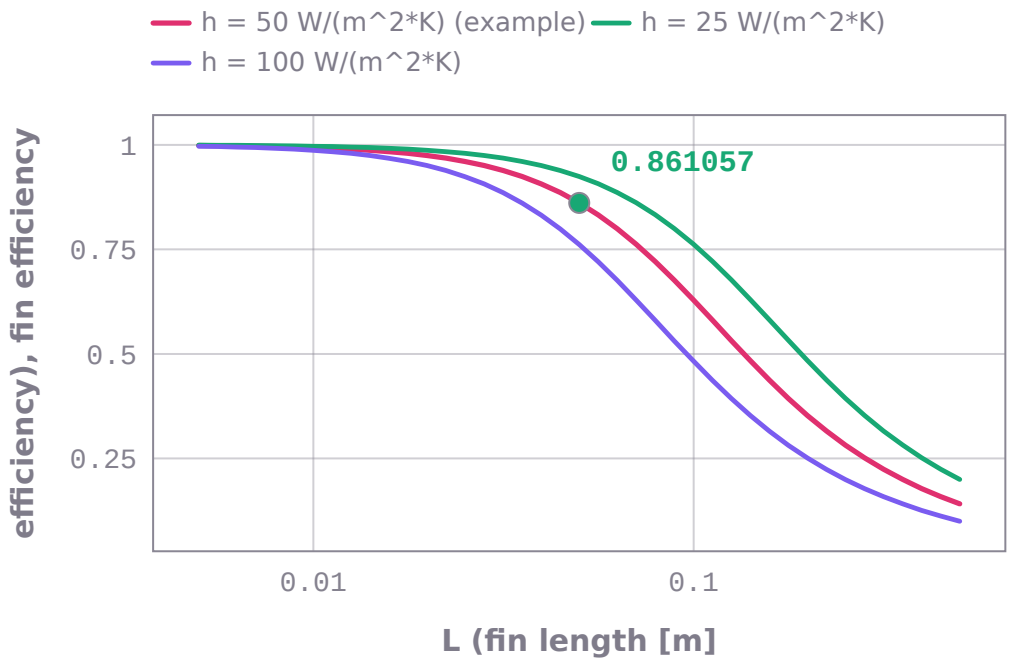
input	meaning	condition
`h`	heat transfer coefficient	
`k`	fin thermal conductivity	
`D`	pin diameter	
`L`	fin length	

output	meaning
`efficiency`	fin efficiency
`mL`	fin parameter m L

- **Method:** eta = tanh(mL)/(mL), $m = sqrt(4h/(kD))$
- **Assumes:** h, k, D, $L > 0$; one-dimensional conduction, uniform h, insulated tip
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



A pin fin (D = 5 mm, L = 50 mm, k = 200 W/(m·K), h = 50 W/(m²·K); the worked example) drawn to scale, over its temperature excess θ/θ_b = cosh(m(L−x))/cosh(mL) (adiabatic tip), derived from the package's mL = 0.707107. The efficiency is the mean excess over the base excess. Marked: the package's η = 0.861057, tip excess 0.793278 (derived).



efficiency (fin efficiency) against L (fin length) from 0.005 m to 0.5 m on a logarithmic axis. One curve for each value of h (heat transfer coefficient): 50 W/(m^2*K), 25 W/(m^2*K), 100 W/(m^2*K). Other inputs as in the worked example: k = 200 W/(m*K), D = 0.005 m. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, L = 0.05 m, efficiency = 0.861057.

```
thermofluids_ai999x.solve('pin_fin_efficiency', h=50, k=200, D=0.005, L=0.05)
# -> efficiency = 0.861057, mL = 0.707107   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Pin fin efficiency: h = 50 W/(m^2*K), k = 200 W/(m*K), D = 5 mm, L = 50 mm	SOLVED	efficiency = 0.861057, mL = 0.707107
Pin fin efficiency: h = 50 W/(m^2*K), k = 200 W/(m*K), D = 5 mm	NEEDS_INPUT	pin_fin_efficiency needs L (fin length). Example: solve('pin_fin_efficiency', h='50 W/(m^2*K)', k='200 W/(m*K)', D='0.00

First law of thermodynamics: change of internal energy, `first_law_internal_energy_change`

The change of internal energy of a closed system from the heat added to it and the work it does.

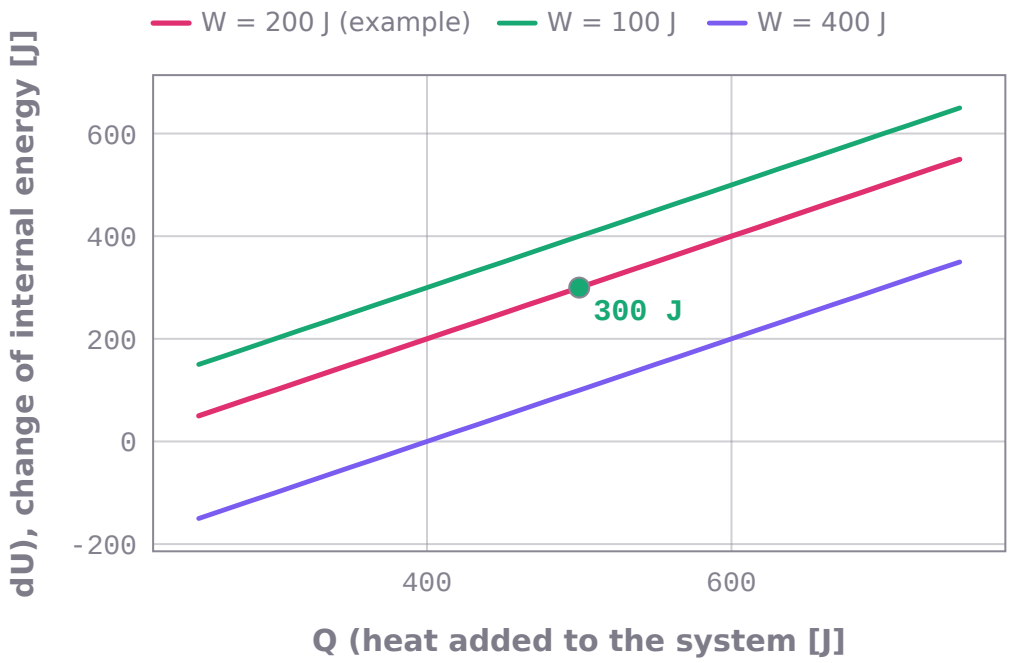
$$\Delta U = Q - W$$

input	meaning	condition
`Q`	heat added to the system (negative if heat leaves it)	
`W`	work done BY the system (negative if work is done on it)	

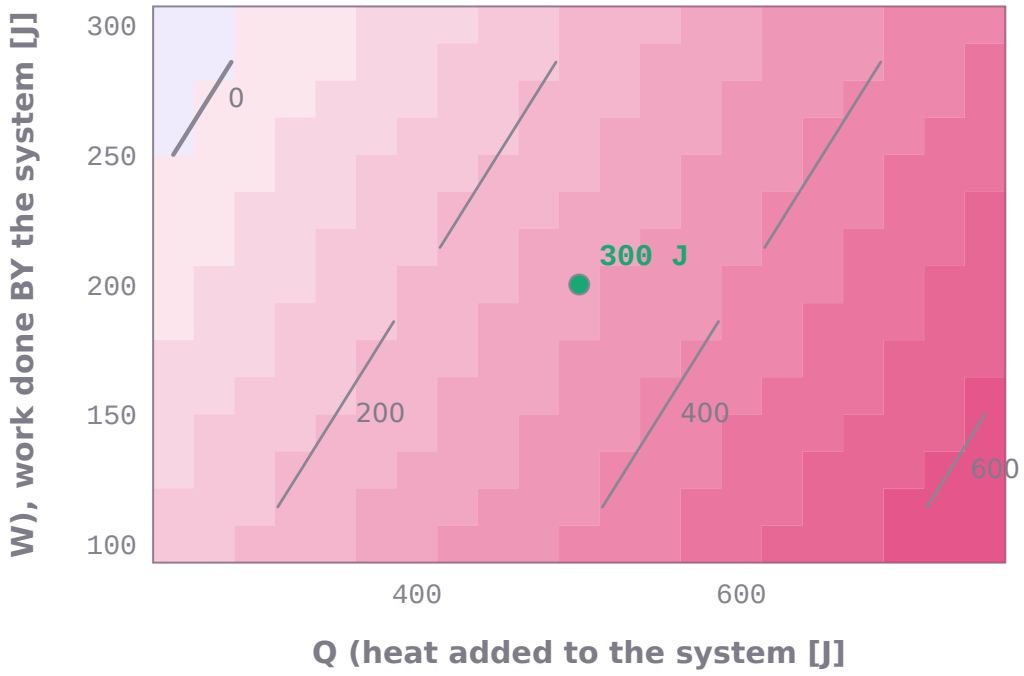
output	meaning
`dU`	change of internal energy

- **Method:** $dU = Q - W$
- **Assumes:** closed system; Q added to the system, W done by the system

- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dU (change of internal energy) against Q (heat added to the system (negative if heat leaves it)) from 250 J to 750 J. One curve for each value of W (work done BY the system (negative if work is done on it)): 200 J, 100 J, 400 J. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, Q = 500 J, dU = 300 J.



dU (change of internal energy) over Q (heat added to the system (negative if heat leaves it)) from 250 J to 750 J and W (work done BY the system (negative if work is done on it)) from 100 J to 300 J: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its sign and magnitude (second colour negative); iso-lines at dU = 0, 200, 400, 600 J. Marked: the worked example, Q = 500 J, W = 200 J, dU = 300 J.

```
thermofluids_ai999x.solve('first_law_internal_energy_change', Q=500.0, W=200.0)
# -> dU = 300 J   (SOLVED)
```

```
thermofluids_ai999x.solve('first_law_internal_energy_change', Q=-300.0, W=-100.0)
# -> dU = -200 J   (SOLVED)
```

```
thermofluids_ai999x.solve('first_law_internal_energy_change', Q=1200.0, W=1500.0)
# -> dU = -300 J   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
First law of thermodynamics: Q = 500 J, W = 200 J	SOLVED	dU = 300 J
First law of thermodynamics: Q = -300 J, W = -100 J	SOLVED	dU = -200 J
First law: Q = 1.2 kJ, W = 1500 J	SOLVED	dU = -300 J
First law of thermodynamics: Q = 1e400 J, W = 200 J	INVALID_INPUT	First law of thermodynamics: change of internal energy: the heat and the work must be finite numbers.
First law of thermodynamics: Q = 500 J	NEEDS_INPUT	first_law_internal_energy_change needs W (work done BY the system (negative if work is done on it)). Example: solve('fir

Boundary work of a constant-pressure (isobaric) process, `isobaric_boundary_work`

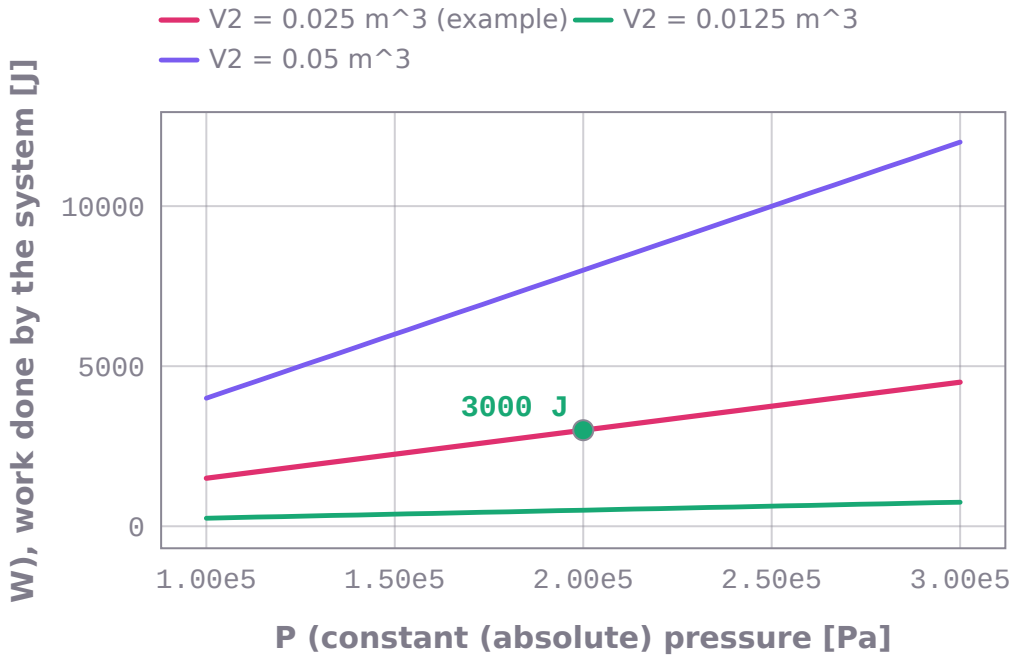
The work done by a gas that expands or is compressed at constant pressure from V1 to V2.

$$W = \int_{V_1}^{V_2} P\,dV = P\,(V_2 - V_1)$$

input	meaning	condition
`P`	constant (absolute) pressure	
`V1`	initial volume	
`V2`	final volume	

output	meaning
`W`	work done by the system (negative for a compression)
`dV`	change of volume $V_2 - V_1$

- **Method:** $W = P\,(V_2 - V_1)$
- **Assumes:** $P > 0$ and constant; V1, $V_2 > 0$; quasi-static process
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



W (work done by the system (negative for a compression)) against P (constant (absolute) pressure) from 100000 Pa to 300000 Pa. One curve for each value of V2 (final volume): 0.025 m^3, 0.0125 m^3, 0.05 m^3. Other inputs as in the worked example: V1 = 0.01 m^3. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, P = 200000 Pa, W = 3000 J.

```
thermofluids_ai999x.solve('isobaric_boundary_work', P=200000.0, V1=0.01, V2=0.025)
# -> W = 3000 J, dV = 0.015 m^3   (SOLVED)
```

```
thermofluids_ai999x.solve('isobaric_boundary_work', P=101325.0, V1=0.002, V2=0.001)
# -> W = -101.325 J, dV = -0.001 m^3   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Isobaric work: P = 200 kPa, V1 = 0.010 m^3, V2 = 0.025 m^3	SOLVED	W = 3000 J, dV = 0.015 m^3
Isobaric work: P = 2 bar, V1 = 10 L, V2 = 25 L	SOLVED	W = 3000 J, dV = 0.015 m^3
Isobaric work: P = 200 kPa, V1 = -0.010 m^3, V2 = 0.025 m^3	INVALID_INPUT	Boundary work of a constant-pressure (isobaric) process: the initial and final volumes must be positive.
Isobaric work: P = 200 kPa, V1 = 0.010 m^3	NEEDS_INPUT	isobaric_boundary_work needs V2 (final volume). Example: solve('isobaric_boundary_work', P='200000 Pa', V1='0.01 m^3', V

Entropy change for heat exchanged at a constant temperature, `entropy_change_reservoir`

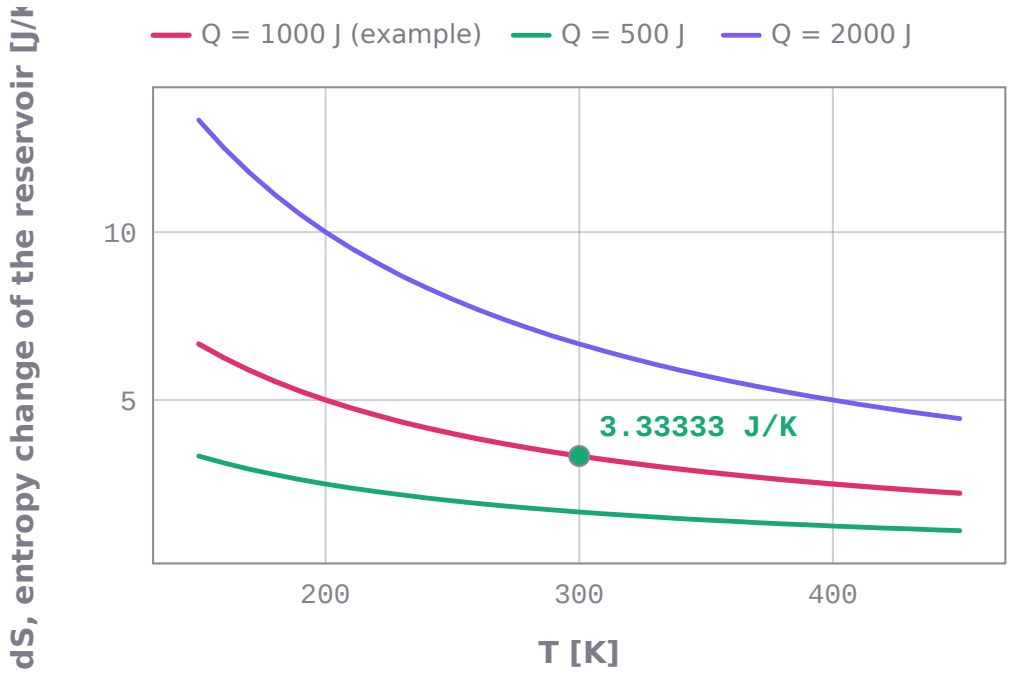
The entropy change $dS = Q/T$ of a reservoir (or any system) that takes in heat Q reversibly at a fixed absolute temperature T.

$$\Delta S = \frac{Q}{T}$$

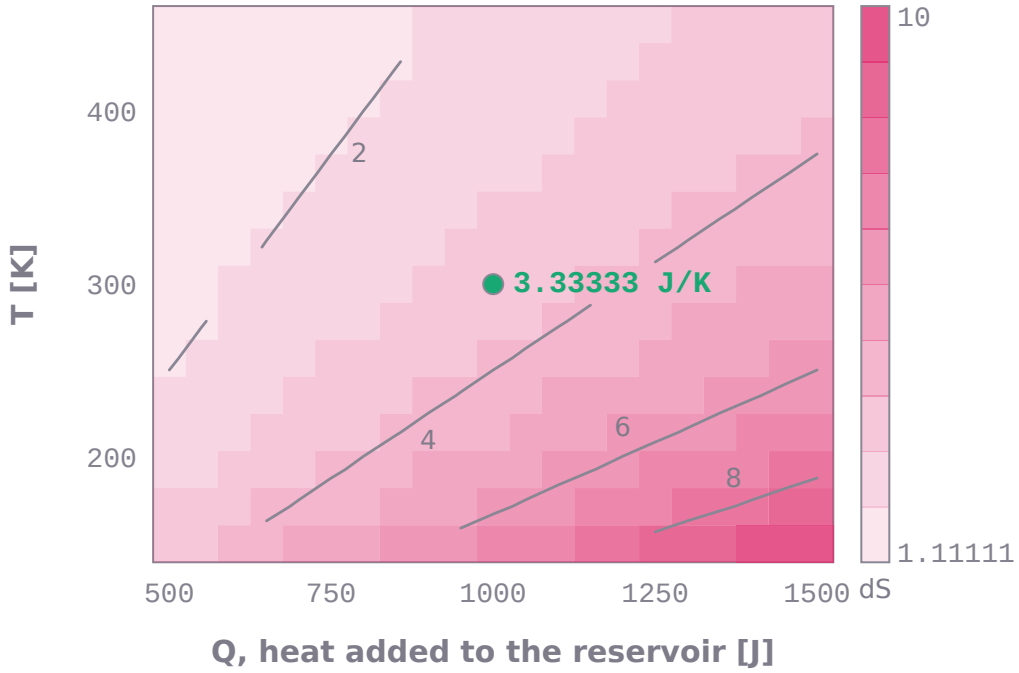
input	meaning	condition
`Q`	heat added to the reservoir (negative if heat leaves it)	
`T`	absolute temperature of the reservoir	

output	meaning
`dS`	entropy change of the reservoir

- **Method:** $dS = Q/T$
- **Assumes:** $T > 0$ (absolute); the temperature stays constant while Q is exchanged
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dS (entropy change of the reservoir) against T (absolute temperature of the reservoir) from 150 K to 450 K. One curve for each value of Q (heat added to the reservoir (negative if heat leaves it)): 1000 J, 500 J, 2000 J. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, T = 300 K, dS = 3.33333 J/K.



dS (entropy change of the reservoir) over Q (heat added to the reservoir (negative if heat leaves it)) from 500 J to 1500 J and T (absolute temperature of the reservoir) from 150 K to 450 K: each of the 21 × 15 cells is one run of thermofluids-ai999x, shaded by its value (light 1.11111 J/K to dark 10 J/K); iso-lines at dS = 2, 4, 6, 8 J/K. Marked: the worked example, Q = 1000 J, T = 300 K, dS = 3.33333 J/K.

```
thermofluids_ai999x.solve('entropy_change_reservoir', Q=1000.0, T=300.0)
# -> dS = 3.33333 J/K   (SOLVED)
```

```
thermofluids_ai999x.solve('entropy_change_reservoir', Q=-5000.0, T=500.0)
# -> dS = -10 J/K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Entropy change of a reservoir: Q = 1000 J, T = 300 K	SOLVED	dS = 3.33333 J/K
Entropy change of a reservoir: Q = 2 kJ, T = 27 °C	SOLVED	dS = 6.66333 J/K
Entropy change of a reservoir: Q = 1000 J, T = 0 K	INVALID_INPUT	Entropy change for heat exchanged at a constant temperature: the absolute temperature must be positive.
Entropy change of a reservoir: Q = 1000 J	NEEDS_INPUT	entropy_change_reservoir needs T (absolute temperature of the reservoir). Example: solve('entropy_change_reservoir', Q='

Entropy generated when heat flows between two reservoirs,

`entropy_generation_heat_transfer`

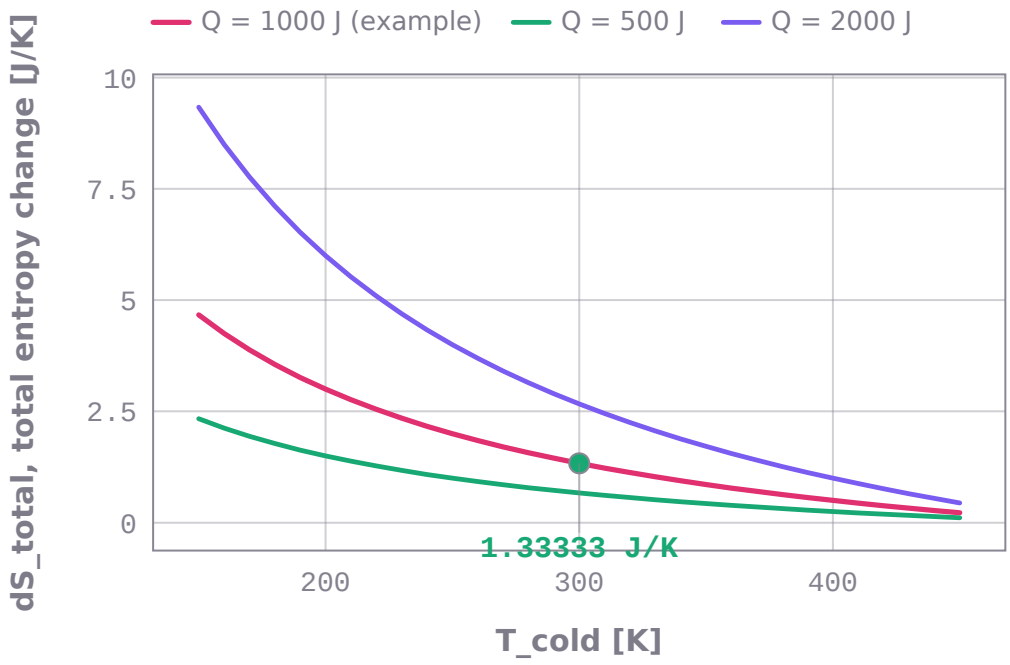
The entropy changes of a hot and a cold reservoir when heat Q flows from the hot one to the cold one, and the total (generated) entropy.

$$\Delta S_{tot} = -\frac{Q}{T_h} + \frac{Q}{T_c} = Q\,\frac{T_h - T_c}{T_h T_c}$$

input	meaning	condition
`Q`	heat that flows from the hot to the cold reservoir	
`T_hot`	absolute temperature of the hot reservoir	
`T_cold`	absolute temperature of the cold reservoir	

output	meaning
`dS_total`	total entropy change (entropy generated)
`dS_hot`	entropy change of the hot reservoir
`dS_cold`	entropy change of the cold reservoir

- **Method:** dS_hot = -Q/T_hot, dS_cold = Q/T_cold, dS_total = Q (T_hot - T_cold)/(T_hot T_cold) >= 0
- **Assumes:** $Q > 0$; T_hot >= T_cold > 0 (absolute); both temperatures stay constant
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



dS_total (total entropy change (entropy generated)) against T_cold (absolute temperature of the cold reservoir) from 150 K to 450 K. One curve for each value of Q (heat that flows from the hot to the cold reservoir): 1000 J, 500 J, 2000 J. Other inputs as in the worked example: T_hot = 500.0 K. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, T_cold = 300 K, dS_total = 1.33333 J/K.

```
thermofluids_ai999x.solve('entropy_generation_heat_transfer', Q=1000.0, T_hot=500.0,
T_cold=300.0)
# -> dS_total = 1.33333 J/K, dS_hot = -2 J/K, dS_cold = 3.33333 J/K   (SOLVED)
```

```
thermofluids_ai999x.solve('entropy_generation_heat_transfer', Q=1200.0, T_hot=600.0,
T_cold=400.0)
# -> dS_total = 1 J/K, dS_hot = -2 J/K, dS_cold = 3 J/K   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Entropy generated by heat transfer: Q = 1000 J, T_hot = 500 K, T_cold = 300 K	SOLVED	dS_total = 1.33333 J/K, dS_hot = -2 J/K, dS_cold = 3.33333 J/K
Entropy generated by heat transfer: Q = 1.2 kJ, T_hot = 327 °C, T_cold = 127 °C	SOLVED	dS_total = 0.999375 J/K, dS_hot = -1.9995 J/K, dS_cold = 2.99888 J/K
Entropy generated by heat transfer: Q = 1000 J, T_hot = 300 K, T_cold = 500 K	INVALID_INPUT	Entropy generated when heat flows between two reservoirs: heat cannot flow by itself from the colder to the hotter reser
Entropy generated by heat transfer: Q = 1000 J, T_hot = 500 K	NEEDS_INPUT	entropy_generation_heat_transfer needs T_cold (absolute temperature of the cold reservoir). Example: solve('entropy_gene

Hydraulic press (Pascal's principle), `hydraulic_press_pascal`

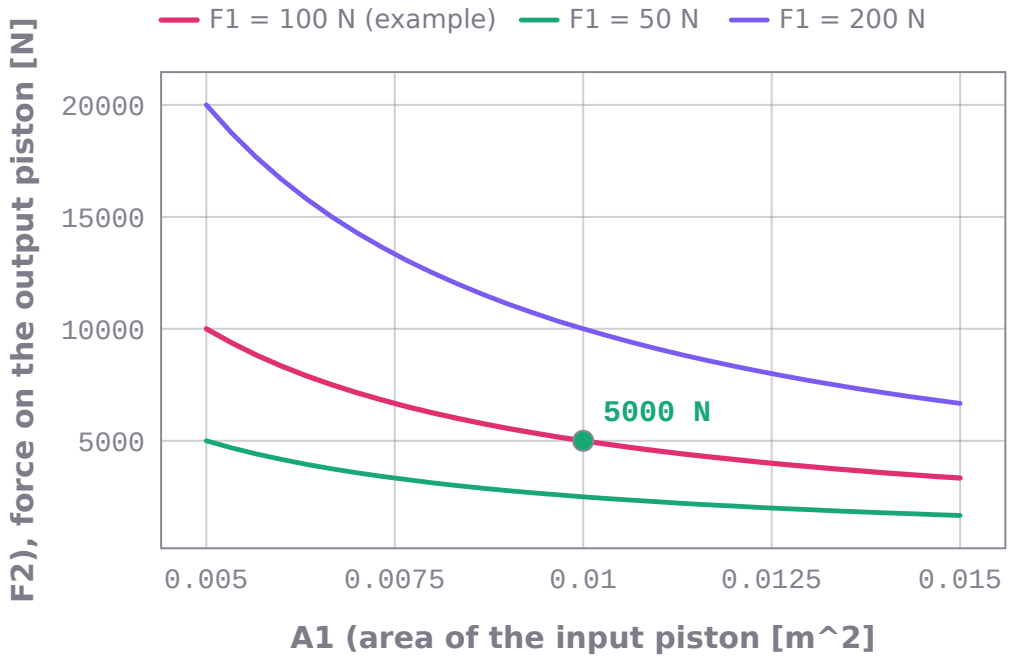
Output force, pressure, mechanical advantage and piston-travel ratio of an ideal hydraulic press or lift.

$$\frac{F_1}{A_1} = \frac{F_2}{A_2}, \quad \frac{d_2}{d_1} = \frac{A_1}{A_2}$$

input	meaning	condition
`F1`	force on the input (small) piston	
`A1`	area of the input piston	
`A2`	area of the output piston	

output	meaning
`F2`	force on the output piston
`P`	pressure transmitted through the liquid (above ambient)
`MA`	mechanical advantage $F_2/F_1 = A_2/A_1$
`d_ratio`	output piston travel / input piston travel $(d_2/d_1 = A_1/A_2)$

- **Method:** $P = F_1/A_1$ is transmitted undiminished, $F_2 = P$ A2; the displaced volume is shared, $d_2/d_1 = A_1/A_2$
- **Assumes:** F1, A1, $A_2 > 0$; incompressible liquid, pistons at the same level, no friction
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



F2 (force on the output piston) against A1 (area of the input piston) from 0.005 m^2 to 0.015 m^2. One curve for each value of F1 (force on the input (small) piston): 100 N, 50 N, 200 N. Other inputs as in the worked example: A2 = 0.5 m^2. 31 points per curve computed by thermofluids-ai999x; marked: the worked example, A1 = 0.01 m^2, F2 = 5000 N.

```
thermofluids_ai999x.solve('hydraulic_press_pascal', F1=100.0, A1=0.01, A2=0.5)
# -> F2 = 5000 N, P = 10000 Pa, MA = 50, d_ratio = 0.02   (SOLVED)
```

```
thermofluids_ai999x.solve('hydraulic_press_pascal', F1=50.0, A1=0.001, A2=0.02)
# -> F2 = 1000 N, P = 50000 Pa, MA = 20, d_ratio = 0.05   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Hydraulic press: F1 = 100 N, A1 = 0.01 m^2, A2 = 0.5 m^2	SOLVED	F2 = 5000 N, P = 10000 Pa, MA = 50, d_ratio = 0.02
Hydraulic press: F1 = 50 N, A1 = 10 cm^2, A2 = 200 cm^2	SOLVED	F2 = 1000 N, P = 50000 Pa, MA = 20, d_ratio = 0.05
Hydraulic press: F1 = 100 N, A1 = 0 m^2, A2 = 0.5 m^2	INVALID_INPUT	Hydraulic press (Pascal's principle): the piston areas must be positive.
Hydraulic press: F1 = 100 N, A1 = 0.01 m^2	NEEDS_INPUT	hydraulic_press_pascal needs A2 (area of the output piston). Example: solve('hydraulic_press_pascal', F1='100 N', A1='0.

Bernoulli's equation with a change of height, `bernoulli_with_height`

The absolute pressure at a second point of a streamline of steady, inviscid, incompressible flow from the pressure, speed and height at the first and the speed and height at the second.

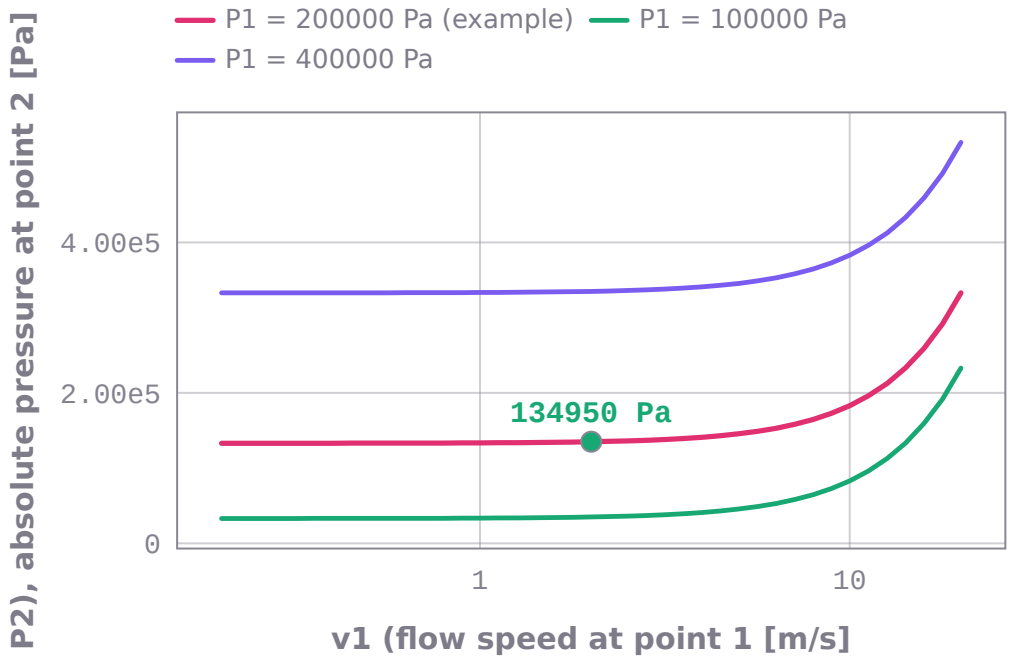
$$P_1 + \tfrac{1}{2}\rho v_1^2 + \rho g h_1 = P_2 + \tfrac{1}{2}\rho v_2^2 + \rho g h_2$$

input	meaning	condition
`P1`	absolute pressure at point 1	

input	meaning	condition
`v1`	flow speed at point 1	
`h1`	height of point 1 above a datum	
`v2`	flow speed at point 2	
`h2`	height of point 2 above the same datum	
`rho`	density of the fluid	
`g`	gravitational acceleration	

output	meaning
`P2`	absolute pressure at point 2

- **Method:** $P_2 = P_1 + \rho\,(v_1^2 - v_2^2)/2 + \rho\,\mathrm{g}\,(h_1 - h_2)$
- **Assumes:** steady, inviscid, incompressible flow along one streamline; rho, $g > 0$; P1 and P2 absolute and positive
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



P2 (absolute pressure at point 2) against v1 (flow speed at point 1) from 0.2 m/s to 20 m/s on a logarithmic axis. One curve for each value of P1 (absolute pressure at point 1): 200000 Pa, 100000 Pa, 400000 Pa. Other inputs as in the worked example: h1 = 0.0 m, v2 = 6.0 m/s, h2 = 5.0 m, rho = 1000.0 kg/m^3, g = 9.81 m/s^2. 41 points per curve computed by thermofluids-ai999x; marked: the worked example, v1 = 2 m/s, P2 = 134950 Pa.

```
thermofluids_ai999x.solve('bernoulli_with_height', P1=200000.0, v1=2.0, h1=0.0, v2=6.0,
h2=5.0, rho=1000.0, g=9.81)
# -> P2 = 134950 Pa   (SOLVED)
```

```
thermofluids_ai999x.solve('bernoulli_with_height', P1=150000.0, v1=3.0, h1=10.0, v2=3.0,
h2=0.0, rho=1000.0, g=9.80665)
# -> P2 = 248066 Pa   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Bernoulli equation with height: P1 = 200 kPa, v1 = 2 m/s, h1 = 0 m, v2 = 6 m/s, h2 = 5 m, rho = 1000 kg/m^3, g = 9.81 m/s^2	SOLVED	P2 = 134950 Pa
Bernoulli equation with height: P1 = 150 kPa, v1 = 3 m/s, h1 = 10 m, v2 = 3 m/s, h2 = 0 m, rho = 1000 kg/m^3, g = 9.80665 m/s^2	SOLVED	P2 = 248066 Pa
Bernoulli equation with height: P1 = 100 kPa, v1 = 1 m/s, h1 = 0 m, v2 = 1 m/s, h2 = 20 m, rho = 1000 kg/m^3, g = 9.81 m/s^2	INVALID_INPUT	Bernoulli's equation with a change of height: the computed absolute pressure P2 is not positive: the flow cannot exist s
Bernoulli equation with height: P1 = 200 kPa, v1 = 2 m/s, h1 = 0 m, v2 = 6 m/s, h2 = 5 m, rho = 1000 kg/m^3	NEEDS_INPUT	bernoulli_with_height needs g (gravitational acceleration). Example: solve('bernoulli_with_height', P1='200000 Pa', v1='

Wet steam: quality and mixture properties from saturated values, `wet_steam_mixture`

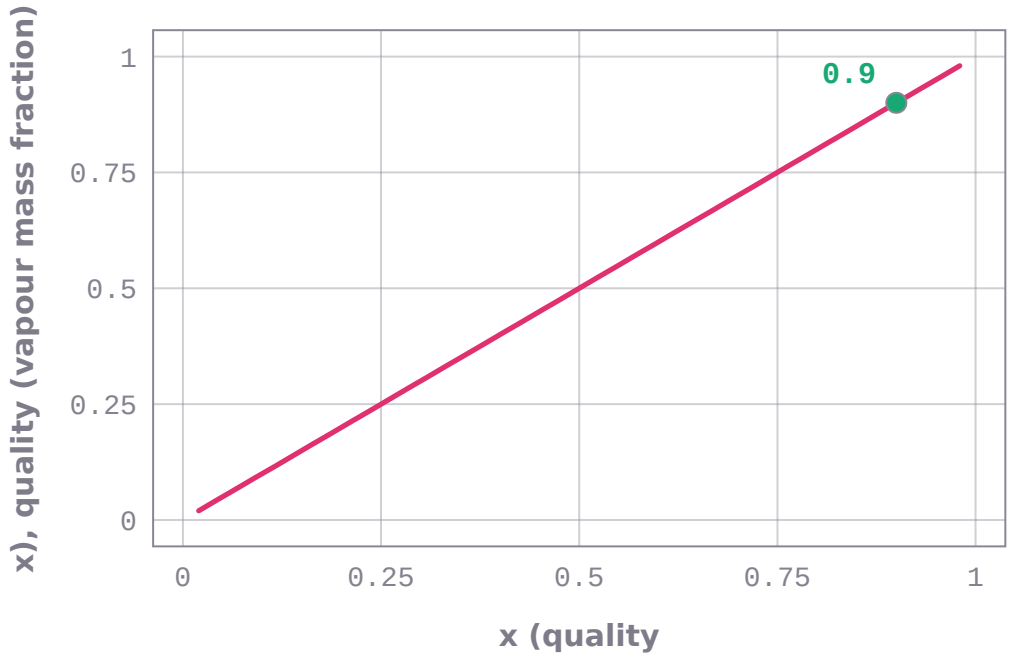
The quality x of a saturated liquid-vapour mixture and its specific volume, enthalpy and entropy by the lever rule, from the saturated-liquid and saturated-vapour values the user supplies (vf, vg; hf, hfg; sf, sfg) and either the quality or one mixture property (v, h or s).

$$x = \frac{h - h_f}{h_{fg}}, \quad v = v_f + x\,(v_g - v_f), \quad h = h_f + x\,h_{fg}, \quad s = s_f + x\,s_{fg}$$

input	meaning	condition
`vf`	specific volume of the saturated liquid	
`vg`	specific volume of the saturated vapour	
`hf`	specific enthalpy of the saturated liquid	
`hfg`	enthalpy of vaporization hg - hf	
`sf`	specific entropy of the saturated liquid	
`sfg`	entropy of vaporization sg - sf	
`x`	quality (vapour mass fraction), a fraction or %	
`v`	specific volume of the mixture (instead of x)	
`h`	specific enthalpy of the mixture (instead of x)	
`s`	specific entropy of the mixture (instead of x)	

output	meaning
`x`	quality (vapour mass fraction)
`moisture`	moisture content $1 - x$ (liquid mass fraction)
`v`	specific volume of the mixture
`h`	specific enthalpy of the mixture
`s`	specific entropy of the mixture
`hg`	specific enthalpy of the saturated vapour hf + hfg
`sg`	specific entropy of the saturated vapour sf + sfg

- **Method:** lever rule: $y = yf + x$ (yg - yf) for $y = v$, h, s; $x = (y - yf)/(yg - yf)$ from a stated mixture property
- **Assumes:** saturated values at the mixture's own pressure (or temperature), supplied by the user; each pair whole; $0 < vf < vg$, hfg > 0, sfg > 0; exactly one of x, v, h, s; $0 \le x \le 1$ (otherwise not a wet mixture)
- **Evidence:** certified procedure (gates: conditions, control, examples, fast_enough, questions, reference, sensitivity, templates)



x (quality (vapour mass fraction)) against x (quality (vapour mass fraction), a fraction or %) from 0.02 to 0.98. Other inputs as in the worked example: vf = 0.001043 m^3/kg, vg = 1.694 m^3/kg, hf = 417500.0 J/kg, hfg = 2257500.0 J/kg, sf = 1302.8 J/(kg*K), sfg = 6056.2 J/(kg*K). 41 points computed by thermofluids-ai999x; marked: the worked example, x = 0.9, x = 0.9.

```
thermofluids_ai999x.solve('wet_steam_mixture', vf=0.001043, vg=1.694, hf=417500.0,
hfg=2257500.0, sf=1302.8, sfg=6056.2, x=0.9)
# -> x = 0.9, moisture = 0.1, v = 1.5247 m^3/kg, h = 2.44925e+06 J/kg, hg = 2.675e+06 J/kg,
s = 6753.38 J/(kg*K), sg = 7359 J/(kg*K)   (SOLVED)
```

```
thermofluids_ai999x.solve('wet_steam_mixture', hf=417500.0, hfg=2257500.0, h=2000000.0)
# -> x = 0.700997, moisture = 0.299003, h = 2e+06 J/kg, hg = 2.675e+06 J/kg   (SOLVED)
```

```
thermofluids_ai999x.solve('wet_steam_mixture', vf=0.001043, vg=1.694, sf=1302.8,
sfg=6056.2, v=0.5)
# -> x = 0.294725, moisture = 0.705275, v = 0.5 m^3/kg, s = 3087.71 J/(kg*K), sg = 7359
J/(kg*K)   (SOLVED)
```

Questions in words (documented forms), as executed:

question	outcome	result
Wet steam: vf = 0.001043 m^3/kg, vg = 1.694 m^3/kg, hf = 417.5 kJ/kg, hfg = 2257.5 kJ/kg, sf = 1.3028 kJ/(kg*K), sfg = 6.0562 kJ/(kg*K), x = 90%	SOLVED	x = 0.9, moisture = 0.1, v = 1.5247 m^3/kg, h = 2.44925e+06 J/kg, hg = 2.675e+06 J/kg, s = 6753.38 J/(kg*K), sg = 7359 J/(kg*K)
Wet steam: hf = 417.5 kJ/kg, hfg = 2257.5 kJ/kg, h = 2000 kJ/kg	SOLVED	x = 0.700997, moisture = 0.299003, h = 2e+06 J/kg, hg = 2.675e+06 J/kg

question	outcome	result
Steam quality: vf = 0.001043 m^3/kg, vg = 1.694 m^3/kg, sf = 1.3028 kJ/(kg*K), sfg = 6.0562 kJ/(kg*K), v = 0.5 m^3/kg	SOLVED	x = 0.294725, moisture = 0.705275, v = 0.5 m^3/kg, s = 3087.71 J/(kg*K), sg = 7359 J/(kg*K)
Wet steam: vf = 0.001043 m^3/kg, vg = 1.694 m^3/kg, v = 2 m^3/kg	INVALID_INPUT	Wet steam: quality and mixture properties from saturated values: the state is not a wet mixture: the property lies outsi
Wet steam: hf = 417.5 kJ/kg, hfg = 2257.5 kJ/kg, x = 1.2	INVALID_INPUT	Wet steam: quality and mixture properties from saturated values: the quality x must be a fraction from 0 to 1.
Wet steam: hf = 417.5 kJ/kg, hfg = 2257.5 kJ/kg, x = 0.9, h = 2449.25 kJ/kg	INVALID_INPUT	Wet steam: quality and mixture properties from saturated values: state the mixture by exactly one of the quality x, or i

Proofs and derivations

Each derivation below was certified step by step: every step follows mechanically from the ones before and from what it rests on (a certified relation, an identity, a definition or a stated hypothesis), and a wrong variant of the result is refused. Ask in one of the documented forms, or call `thermofluids_ai999x.prove("<id>")`. A statement no certified derivation proves is OUT_OF_SCOPE; a conditional result asked without its hypotheses is NEEDS_INPUT and names them.

documented question (executed)	id	steps
Prove that P2 = P1 + rho*(v1**2 - v2**2)/2.	`derive_bernoulli_horizontal`	1
Prove that cf = 0.6641146724303926/sqrt(Rex).	`derive_blasius_local_skin_friction`	1
Prove that th = 0.6641146724303926*x/sqrt(Rex).	`derive_blasius_momentum_thickness`	1
Prove that eta = 1 - rp**(-2/7).	`derive_brayton_efficiency_air`	1
Prove that F = rho*g*V.	`derive_buoyant_force`	1
Prove that v2 = v1*(D1/D2)**2.	`derive_continuity_speed`	1
Prove that hL = f*L*v**2/(2*g*D).	`derive_darcy_weisbach_head_loss`	2
Prove that if h = hL then dP = f*L*rho*v**2/(2*D).	`derive_darcy_weisbach_pressure_drop`	4
Prove that eta = 1 - r**(-2/5)*(rc**(7/5) - 1)/(7*(rc - 1)/5).	`derive_diesel_efficiency_air`	1
Prove that q = rho*v**2/2.	`derive_dynamic_pressure`	1
Prove that if Cr = 0 then eps = 1 - exp(-N).	`derive_effectiveness_phase_change`	3
Prove that W = 0.621945*pv/(P - pv).	`derive_humidity_ratio_vapour_pressure`	1
Prove that P = rho*g*Q*H.	`derive_hydraulic_power`	1
Prove that F = rho*g*hc*A.	`derive_hydrostatic_force_plane`	1
Prove that hL = K*v**2/(2*g).	`derive_minor_loss_head`	1
Prove that h = Nu*k/Lc.	`derive_nusselt_heat_transfer_coefficient`	2
Prove that eta = 1 - r**(-2/5).	`derive_otto_efficiency_air`	1
Prove that q = U*A*(Ti - To).	`derive_overall_heat_rate`	1
Prove that U = 1/(1/h1 + L/k + 1/h2).	`derive_overall_u_plane_wall`	1
Prove that q = k*A*(T1 - T2)/L.	`derive_plane_wall_conduction`	1

documented question (executed)	id	steps
Prove that P = rho*g*Q*H/eta.	`derive_pump_shaft_power`	1
Prove that q = eps*sigma*A*(Ts**4 - Tsur**4).	`derive_radiation_net_exchange`	1
Prove that qf = sigma*(T1**4 - T2**4)/(1/e1 + 1/e2 - 1).	`derive_radiation_parallel_plates`	1
Prove that Re = rho*v*D/mu.	`derive_reynolds_number`	1
Prove that Re = v*D/nu.	`derive_reynolds_number_kinematic`	3
Prove that a = sqrt(gamma*R*T).	`derive_speed_of_sound_ideal_gas`	1
Prove that q = 4*pi*k*(T1 - T2)/(1/r1 - 1/r2).	`derive_sphere_conduction`	1